

Extension of a RISC-V Core with Custom Matrix Multiplication Instructions

Erklärung zur Abschlussarbeit

Hiermit versichere ich, dass die eingereichte Ausarbeitung von mir persönlich verfasst und meine eigene individuelle Prüfungsleistung ist.

Ich versichere, dass die Ausarbeitung und auch keine Teile davon durch künstliche Intelligenz (KI) dergestalt erstellt wurden, dass das KI-Werk bzw. KI-Werkteile meine eigene Prüfungsleistung ersetzen. Ich versichere, KI allenfalls eingesetzt zu haben, um einen von KI für meine Aufgabenstellung ausgearbeiteten Lösungsvorschlag kritisch zu beurteilen und/oder einen Überblick über Aspekte zu erhalten, die für die von mir in Eigenleistung zu erbringende Prüfungsleistung relevant sein könnten. Soweit durch die Aufgabenstellung bzw. Hinweise der Prüfenden der Einsatz von KI vorgegeben ist, sind die von KI erzeugten Werkteile von mir in der Arbeit entsprechend gekennzeichnet. Mir ist bewusst, dass die von KI erzeugten Werkteile auf ihre Validität zu überprüfen und nicht zitierfähig sind.

Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden.

Wörtliche oder sinngemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Literatur und sonstige Quellen sind nachgewiesen und im Literatur- und Quellenverzeichnis aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann.

Stuttgart, 7.2.2026 *Wolfgang Boll*

Ort, Datum

Unterschrift des Verfassers / der Verfasserin

Veröffentlichung der Arbeit in der Bibliothek der Technischen Hochschule Aschaffenburg

Der Veröffentlichung der Bachelorarbeit in der Bibliothek der Technischen Hochschule Aschaffenburg wird zugestimmt.

Stuttgart, 7.2.2026

Wolfgang Beul

Ort, Datum

Unterschrift des Verfassers / der Verfasserin

Extension of a RISC-V Core with Custom Matrix Multiplication Instructions

Bachelorarbeit von
Wolfgang Bischoff

12. Februar 2026

Extension of a RISC-V Core with Custom Matrix Multiplication Instructions

Extension of a RISC-V Core with Custom Matrix Multiplication Instructions

Autor:
Wolfgang Bischoff
2228538

Erstprüfer:
Prof. Dr. Christian Jakob HDA



TECHNISCHE HOCHSCHULE ASCHAFFENBURG

FAKULTÄT INGENIEURSWISSENSCHAFTEN

WÜRZBURGER STRASSE 45

D-63743 ASCHAFFENBURG

Vorwort

Ich bedanke mich bei Herrn Prof. Dr. Christian Jakob sehr herzlich für die Betreuung der Arbeit auf einem Themengebiet, das mich persönlich sehr interessiert und mir viel Freude bereitet.

Ich bedanke mich bei der TH Aschaffenburg und allen beteiligten MitarbeiterInnen und ProfessorenInnen für die Organisation eines berufsbegleitenden Studiums. Ich schätze es sehr, dass es ermöglicht wird, sich Chancen und Möglichkeiten zu erarbeiten, die man ohne den Studiengang nicht hätte.

Ein Dank geht schließlich auch an Stephan Nolting, der seine Arbeit, die NEORV32 CPU, so großzügig mit der Öffentlichkeit teilt.

Inhaltsverzeichnis

Abbildungsverzeichnis	X
Quellcodeauszüge	XI
Glossar	XII
1 Einleitung	1
1.1 Motivation	1
1.2 Aufgabenstellung	4
1.3 Stand der Technik	5
1.4 Struktur der Arbeit	8
2 Theoretische Grundlagen	9
2.1 Vektoren und Matrizen	9
2.2 Matrixmultiplikation mit SISD	10
2.3 Matrixmultiplikation mit Strip-Mining und SIMD	11
2.4 Die RISC-V Vector Extension	12
2.5 Das Hardware-Software-Co-Design	14
2.6 Übergang zu einer Matrix Extension	18
2.7 Blockweise Matrixmultiplikation	20
2.8 Dyadisches Produkt	22
2.9 Zusammenfassung	24
3 Durchführung und Methodik	25
3.1 Schrittweises Vorgehen	25
3.2 Der NEORV32 Core und dessen Einsatz	26
3.3 Test-Software	30
3.4 Evaluierung der Performance	32
3.5 Zusammenfassung	32
4 Analyse der NEORV32 CPU	33
4.1 Architektur des NEORV32 Core	33
4.2 Die Fetch-Engine	34
4.3 CPU-Control und Execution-Engine	34
4.4 Die Load-Store-Unit	37
4.5 Zusammenfassung	38
5 Erweiterung der NEORV32-CPU	39
5.1 Die Matrix-Register	39
5.2 Die Instruktionen	39
5.3 Erweiterung der Execution-Engine	41
5.4 Erweiterung der Load-Store-Unit	46
5.5 Matrix-Engine	47
5.6 Zusammenfassung	51

6	Auswertung und Evaluierung	53
6.1	Testdaten	53
6.2	Messung und Vergleichbarkeit	54
6.3	Prüfung mit Matrix-Erweiterung	56
6.4	Prüfung ohne Matrix-Erweiterung	57
6.5	Auswertung	59
7	Zusammenfassung und Ausblick	61
7.1	Zusammenfassung	61
7.2	Ausblick	62
	Quellenverzeichnis	65
	Anhang	I

Abbildungsverzeichnis

1.1	View Frustum	2
1.2	Perspective Projection Matrix	2
1.3	Einfaches Neuronales Netz	3
1.4	Mehrere kombinierte Eingaben	4
1.5	Strip Mining	6
2.1	SISD (links) und SIMD (rechts)	11
2.2	RVV-Register	13
2.3	Generierung der RVV-Anweisungen	14
2.4	Strip-Mining Ablauf	15
2.5	Ansätze zur Matrix-Extension	18
2.6	Matrix in Blöcken	20
2.7	Matrixmultiplikation blockweise	21
2.8	Matrixmultiplikation Ergebnis	21
2.9	Matrixmultiplikation Berechnungsvorschrift	21
2.10	Matrix Multiplikation Blöcke A, E, B und G	22
2.11	Matrix Accumulator (MAC)	23
3.1	Entwicklungsschritte	26
3.2	Waveforms	27
3.3	Schaltplan nach der Synthese	28
3.4	Floorplan	29
4.1	NEORV32-Architektur	33
4.2	NEORV32 Gültige Instruktion	34
4.3	Design eines RISC-V CPU mit Pipeline	35
4.4	Struktur der Execution-Engine	36
4.5	Die Load-Store-Unit und ihre Schnittstellen	37
5.1	VHDL Entities	42
5.2	Matrix Engine	47
6.1	Waveforms der Matrixmultiplikation	57

Quellcodeauszüge

2.1	Matrixmultiplikation Iterativ	10
2.2	Vektoraddition Strip-Mining	17
5.1	Die Matrix-Register	39
5.2	Hardwarebeschleunigte Multiplikation	40
5.3	Test auf illegale Opcodes	43
5.4	Erweiterung der Sensitivitätsliste	43
5.5	Übernahme der Immediate-Parameter	43
5.6	Matrixmultiplikation beenden	43
5.7	Erweiterung des EX_EXECUTE Zustands	44
5.8	Erweiterung des EX_MATRIX_MEM_REQ Zustands	44
5.9	Ende der Speichertransaktionen	45
5.10	Warten auf das Ende der Multiplikation	45
5.11	Datentransfer der Matrixengine	48
5.12	Befüllen der Arbeitsregister	50
5.13	Multiply Accumulate Schritt	50
5.14	Matrixengine Transfer Flag	50
6.1	Verwendung der Timing-Register	54
6.2	Verwendung der Intrinsics	56
6.3	Matrixmultiplikation ohne Hardwarebeschleunigung	57
6.4	Ausgabe der Zyklen, beschleunigte Multiplikation	59
6.5	Ausgabe der Zyklen, einfache Multiplikation	59

Glossar

AME

Attached Matrix Extension

AVL Application Vector Length

CSR

Control and Status Register

IME Integrated Matrix Extension

LSU Load Store Unit

LLM

Large Language Model

LMUL

Group Modifier

MAC

Matrix Accumulator

MLEN

Matrix-Registerlänge in Bits

RVV

RISC-V Vector Extensions

SISD

Single Instruction Single Data

SIMD

Single Instruction Multiple Data

XLEN

Standardregisterlänge in Bits

SEW

Selected Element Width

VLEN

Vector-Registerlänge in Bits

VME

Vector Matrix Extension

1 Einleitung

Die Matrixmultiplikation findet in Gebieten Anwendung, die auf massiver Datenverarbeitung und linearer Algebra basieren. Beispiele sind Neuronale Netze und Large Language Models (LLM) als eine Anwendung von neuronalen Netzen, sowie die 3D-Computergrafik. Dadurch sind Matrixmultiplikationen ein extrem relevantes Leistungsmerkmal für moderne CPU Architekturen.

In dieser Arbeit soll ein bestehendes RISC-V CPU Design um Anweisungen zur Berechnung der Matrixmultiplikation erweitert werden.

Zunächst wird vertieft dargestellt, weshalb die RISC-V Architektur und die Matrixmultiplikation relevant sind. Dann wird die Aufgabenstellung an sich beschrieben. Es folgt eine Betrachtung des allgemeinen Entwicklungsstandes, in dem die Arbeit ausgeführt wird. Der Stand der Technik wird beschrieben. Das Kapitel wird mit einer Erläuterung zum Aufbau der Arbeit abgeschlossen.

1.1 Motivation

In dieser Arbeit soll eine bestehende RISC-V CPU um Anweisungen zur Durchführung von Matrixmultiplikationen erweitert werden.

RISC-V ist eine lizenzfreie Beschreibung eines Prozessors und der Anweisungen, die der Prozessor verarbeiten können muss. Durch diese Lizenzfreiheit ist die RISC-V Architektur sehr interessant, sowohl für den akademischen als auch für den kommerziellen Einsatz.

Die RISC-V Architektur ist die fünfte Iteration des Berkeley RISC Projekts der Universität von Kalifornien. Die Arbeit an RISC-V begann im Jahre 2010 und ist inzwischen so ausgereift, dass seit einigen Jahren kommerzielle RISC-V Chips auf dem Markt verfügbar sind.

Die ersten verfügbaren Chips waren aufgrund der fehlenden Lizenzkosten und einfachen Umsetzbarkeit für wenige Cent verfügbar. Im Jahre 2026 gibt es RISC-V Chips, die erweiterte Funktionen bieten und daher sowohl Desktop-Betriebssysteme als auch LLMs ausführen können.

Die Firma Qualcomm beispielsweise, als eines der weltweit führenden Unternehmen auf dem Gebiet der Halbleiter und der Telekommunikation, hat eine RISC-V Erweiterung namens Xqci zu RISC-V und auch zu dem Compiler-Framework LLVM beigesteuert [12]. Qualcomm führt auch Käufe von Firmen

mit RISC-V Expertise durch. Dies deutet daraufhin, dass Qualcomm den Einsatz von RISC-V in Produkten ernsthaft in Erwägung zieht.

Die Matrixmultiplikation ist die Grundlage für perspektivische Projektionen, Rotationen und Bewegung von Objekten in der 3D Computer-Grafik. Dort ist es notwendig 4x4 Matrizen zu verwenden und algebraische Berechnungen so schnell wie möglich auf diesen Matrizen ausführen zu können.

94

4. Transforms

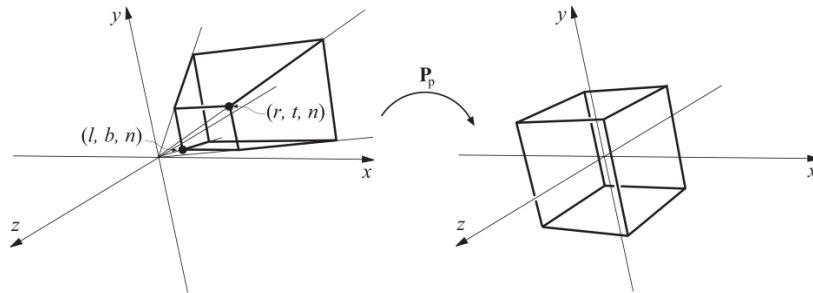


Abbildung 1.1: View Frustum (Quelle: [1], S.94)

In Abbildung 1.1 aus [1] wird auf der linken Seite das Sichtfeld (View Frustum) einer Kamera in einer Szene dargestellt. Durch die Matrix P_p wird dieses Sichtfeld perspektivisch korrekt auf den Einheitswürfel auf der rechten Seite abgebildet. Durch das Umformen des Sichtfelds erscheinen weiter entfernte Objekte z.B. kleiner.

Diese Projektion muss auf jedes Objekt einer Szene angewendet werden. Aus mathematischer Sicht werden also viele Vektoren mit der Projektionsmatrix multipliziert um die transformierten Punkte zu erhalten, die später auf einem Computerbildschirm erscheinen sollen.

Berücksichtigt man die modernen 4K- oder 8K-Auflösungen und die hohen Bildwiederholraten von bis zu 200 Hz, wird deutlich, dass die Matrixmultiplikation extrem schnell und womöglich stark parallelisiert ablaufen muss um eine hohe Bildgenerierungsrate zu erreichen.

Die Projektionsmatrix ist in Abbildung 1.2 aus [1] dargestellt.

$$P_p = \begin{pmatrix} \frac{2n}{r-l} & 0 & -\frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & -\frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Abbildung 1.2: Perspective Projection Matrix (Quelle: [1], S. 95)

Der Einsatz von 3D-Computergrafik ist seit mehreren Jahren nicht mehr aus

dem Alltag, aber auch aus vielen professionellen Bereichen wegzudenken. Man denke dabei an die Computerspiel- und Film-Industrie sowie Modellierungssoftware für Ingenieursanwendungen aller Art.

Seit Google im Jahr 2017 die Transformer-Architektur erfunden und daraufhin OpenAI ChatGPT 1 programmiert hat, setzt sich die Erfolgsgeschichte von Large Language Models bis heute fort. Generative künstliche Intelligenz wird als revolutionäre Erfindung betrachtet und führt schon jetzt dazu, dass sich Softwareentwickler Gedanken um ihren zukünftigen Wert auf dem Arbeitsmarkt machen. Es ist spätestens im Jahre 2026 klar, dass eine KI bestimmte Aufgaben bedeutend viel schneller erledigt, als es ein Mensch jemals könnte.

Firmen haben diesen Vorteil der KI erkannt und haben vor allem die hohen Lohnkosten menschlicher Arbeit im Blick. Hardware, die LLMs trainieren und ausführen kann, wird damit zum relevanten Posten in der Bilanz großer Unternehmen. Anstatt fortlaufende Personalkosten zu bezahlen, möchten Unternehmen KI-fähige Hardware einsetzen, die naiv betrachtet nur einmalige Anschaffungskosten verursacht.

Es soll verdeutlicht werden, welcher Zusammenhang zwischen der Matrixmultiplikation und Neuronalen Netzen bzw. LLMs besteht.

Ein LLM basiert auf Neuronalen Netzen. Zwei Neuronen in einem sehr einfachen Netz sind in Abbildung 1.3 aus [17] dargestellt. Dabei sind $w_{i,j}$ die Gewichte und b_i die Biases, deren Werte durch Training über einer Datenbasis bestimmt werden. Nach dem Training soll das Netz auf Eingaben x_i mit sinnvollen Ausgabewerten y_i reagieren.

Zusammen werden Gewichte und Biases als Parameter bezeichnet. Das kleinste frei verfügbare LLM verwendet 10 Millionen Parameter, große LLM verwenden Billionen von Parametern.

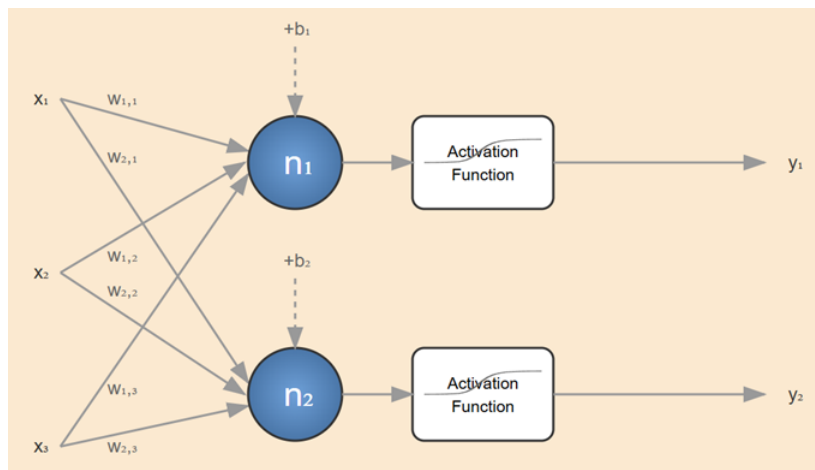


Abbildung 1.3: Einfaches Neuronales Netz (Quelle: [17])

Ein Neuronales Netz lässt sich als Matrix beschreiben. Eingaben an ein Neuronales Netz erfolgen in Form von Werten $x_{i,j}$. Werden mehrere Eingaben x_i

gleichzeitig an das neuronale Netz gegeben, wird aus den einzelnen Eingabevektoren eine Matrix. Dieser Umstand ist in Abbildung 1.4 aus [17] dargestellt.

$$\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} \begin{bmatrix} x_1^1 & x_1^2 \\ x_2^1 & x_2^2 \\ x_3^1 & x_3^2 \end{bmatrix} = \begin{bmatrix} w_{1,1} \cdot x_1^1 + w_{1,2} \cdot x_2^1 + w_{1,3} \cdot x_3^1 & w_{1,1} \cdot x_1^2 + w_{1,2} \cdot x_2^2 + w_{1,3} \cdot x_3^2 \\ w_{2,1} \cdot x_1^1 + w_{2,2} \cdot x_2^1 + w_{2,3} \cdot x_3^1 & w_{2,1} \cdot x_1^2 + w_{2,2} \cdot x_2^2 + w_{2,3} \cdot x_3^2 \end{bmatrix}$$

Abbildung 1.4: Mehrere kombinierte Eingaben (Quelle: [17])

Die rasante Verbreitung von LLMs hat Auswirkungen auf die Hardwarehersteller und den Handel der Hardware. Die Firma Micron hat beispielsweise einen Rückzug aus dem Endkundengeschäft vollzogen und verkauft RAM-Bausteine nur noch an Firmenkunden, die den RAM-Speicher dann bevorzugt für die Berechnung in Rechenzentren einsetzen. Es liegt nahe, dass LLMs in den Rechenzentren trainiert und betrieben werden. Die RAM-Preise für Endkunden sind aus diesem Grund innerhalb der Jahre 2025 und 2026 um ein Vielfaches angestiegen.

Gleiches gilt für Grafikkbeschleuniger der Firma NVIDIA. Auch hier konkurriert der Endkunde mit dem Firmenkunden. NVIDIA kann die Preise beinahe beliebig festlegen. Das Top-Produkt für Endkunden der Firma NVIDIA ist eine 5090 RTX Grafikkarte. Sie ist im Januar 2026 für etwa 3600 Euro erhältlich und kostet damit mehr als der gesamte Rest eines Computers. Eine Grafikkarte ist in der Lage Matrixmultiplikationen in großer Parallelität und extrem schnell zu berechnen. Grafikkarten werden auch zum Training von LLMs verwendet.

Getrieben durch den Erfolg von LLMs kann sich eine Spezifikation für CPUs, wie z.B. RISC-V, nicht mehr durchsetzen, wenn sie keine Anweisungen für den effizienten Umgang mit Vektoren und Matrizen besitzt. Die Endkunden werden Produkte, die nicht in der Lage sind, Matrizen hardwarebeschleunigt zu berechnen, nicht mehr kaufen, z.B. aus Gründen der Computergrafik. Das gleiche gilt für Firmenkunden, die den Betrieb eines LLM planen. Ohne beschleunigte Matrixberechnung kann sich eine CPU nicht mehr auf dem Markt behaupten.

Die CPUs der Firmen AMD und Intel besitzen bereits die Erweiterungen AVX2 (Advanced Vector Extensions 2), AMX (Advanced Matrix Extensions) und FMA (Fused Multiply Add) zum Rechnen mit Vektoren und Matrizen.

All diese Punkte zusammengenommen ergeben eine große Motivation für den Einsatz von RISC-V und den Einsatz der Matrixmultiplikation innerhalb einer RISC-V-CPU.

1.2 Aufgabenstellung

Die NEORV32-CPU wird um Anweisungen erweitert, die die hardwarebeschleunigte Matrixmultiplikation ermöglichen. Die NEORV32 CPU liegt in Form einer textuellen Beschreibung vor. Diese Beschreibung wird ergänzt,

durch ein Werkzeug übersetzt und kann dann auf einem FPGA ausgeführt werden.

Ein Field Programmable Gate Array (FPGA) ist ein Chip, der eine große Menge an logischen Gattern bereitstellt, die zunächst nicht miteinander verbunden sind, also zunächst keine Funktion ausführen. Das Innere des FPGA muss programmiert werden. Damit kann ein FPGA unterschiedliche Hardware abbilden, je nachdem was die textuelle Beschreibung vorgibt.

Die neu hinzugefügten Anweisungen orientieren sich von der Kleinteiligkeit her an den Anweisungen, wie sie bereits für die RVV-Vektor-Extension in RISC-V spezifiziert worden sind. Die Vektor-Extension wird im Abschnitt 1.3 beschrieben. Im Speziellen wird das Konzept des Strip-Mining ermöglicht. Dabei handelt es sich um die Aufteilung des Problems in Teile, die der CPU, aufgrund von begrenzten Ressourcen, schrittweise zugeführt werden, bis die Aufgabe als Ganzes gelöst ist.

Die Erweiterung soll durch Entwickler mit geringer Erfahrung auf dem Feld der Hardware-Beschreibung durchgeführt werden können. Die Matrizenberechnungen erfolgen daher lediglich auf Matrizen, deren Dimensionen ein Vielfaches der Zahl vier sind (also z.B. 4×4 , 8×8 , 16×16) und es werden nur ganzzahlige Multiplikationen ohne Vorzeichen durchgeführt. Ohne Beschränkung der Allgemeinheit lässt sich eine Matrix an den Rändern mit Nullen auffüllen um auf ein Vielfaches der Dimension vier zu kommen, ohne das Ergebnis der Multiplikation zu verändern.

Da es viele Ansätze gibt Matrizen zu multiplizieren, wird eine Einschränkung vorgegeben. Die Multiplikation erfolgt mit dem Outer Product, da dieser Ansatz auch in dem maßgebenden Dokument zu dieser Arbeit beschrieben ist [11]. Der deutsche Begriff für das Outer Product ist das dyadische Produkt.

1.3 Stand der Technik

Ein RISC-V-Komitee hat im Jahre 2015 mit der ersten Version der RISC-V Vector Extension (RVV) begonnen. Im Jahre 2021 wurde das Release 1.0 ratifiziert und ist jetzt in einen finalen Zustand (Frozen) überführt worden. RISC-V wird durch die RVV um Register für Vektoren und explizite Anweisungen zur Manipulation dieser Vektoren erweitert.

In der RISC-V Vector Extension Spezifikation [2] wird das Konzept des Strip-Mining beschrieben und hervorgehoben. Eine grafische Darstellung ist in Abbildung 1.5 aus [19] zu sehen.

Die Software lädt den Cache mit einem Teil einer oder mehrerer Vektoren voll und beginnt den Cache dann abzuarbeiten. Dabei wird ein Teil der Vektoren dann wiederum teilweise aus dem Cache in die CPU geladen, verarbeitet und danach wieder aus der CPU zurück in den Cache bzw. den Speicher übertragen. In Abbildung 1.5 aus [19] wird einer dieser Schritte als "vector iteration" bezeichnet.

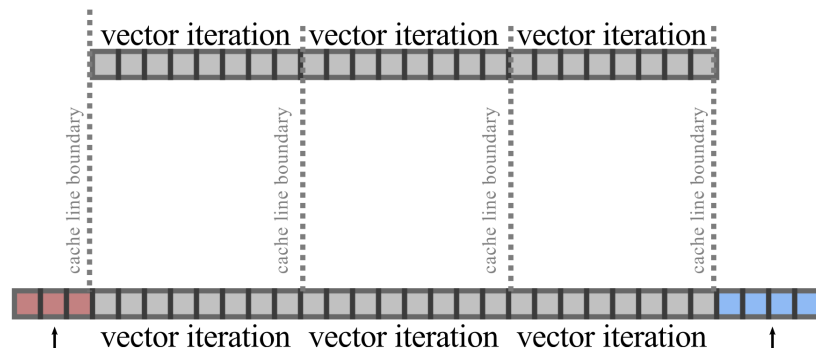


Abbildung 1.5: Strip Mining (Quelle: [19], S.10)

Das Strip-Mining wiederholt diese Einzelschritte, bis die Gesamtaufgabe erledigt ist, also alle Elemente der Vektoren bearbeitet sind. Das Beispiel in Kapitel 6.4. namens "Example of stripmining and changes to SEW" aus [2] verdeutlicht dieses Vorgehen.

RVV fügt einer RISC-V CPU neue Register hinzu. Der Vorteil der RVV-Register ist es, dass diese Vektorregister mehrere Elemente parallel auf eine Instruktion hin zusammenrechnen können, anstatt jedes Element einzeln zu berechnen. Die CPU verfährt dann in einem echten SIMD-Betriebsmodus (Single Instruction Multiple Data) und spart ein Vielfaches an Taktzyklen im Vergleich zur sequentiellen Ausführung. SIMD bedeutet, dass eine einzige Instruktion mehrere Datensätze auf einmal verarbeitet.

Das Prinzip des Strip-Mining wird auch in mathematischen Bibliotheken verwendet. Eine bekannte Bibliothek ist blis [21]. blis verwendet den Begriff Kernel oder Microkernel. Ein Kernel ist ein Code, der auf einem Cache-Frame ausgeführt wird, also den Code darstellt, den die CPU sehr schnell ausführen kann. Optimierte Anweisungen für verschiedene CPU-Architekturen werden hauptsächlich in dem Code verwendet, der dem Kernel zuzurechnen ist um Optimierungen dort durchzuführen, wo sie den meisten Nutzen bringen.

Um die Berechnungen im Kernel herum gibt es weiteren Code, der über Schleifen nacheinander alle Daten in den Kernel lädt, bis die Berechnung vollständig abgeschlossen ist.

Es gibt sehr starke Parallelen zwischen dem Strip-Mining und dem Kernel-Ansatz der blis-Bibliothek. Sollte blis für RISC-V portiert werden, ist es sinnvoll, dass RISC-V das Prinzip des Strip-Mining ermöglicht. Dies wurde durch die RISC-V Vector Extension für Vektoren bereits erreicht.

Für die RISC-V-Matrixberechnung gibt es zum jetzigen Zeitpunkt keine ratifizierte Extension. Innerhalb der RISC-V-Arbeitsgruppen werden unterschiedliche Ansätze diskutiert. Die Ansätze werden im Kapitel zu den theoretischen Grundlagen beschrieben.

Der Status-Quo in der kommerziellen Entwicklung von RISC-V-Chips ist, dass bisher keine Chips mit Matrix-Erweiterungen produziert worden sind, da die Matrix-Spezifikation noch nicht ratifiziert worden ist. Einzelne Firmen haben

allerdings mit der Entwicklung eigener Designs für die Matrixberechnung begonnen.

Zu nennen ist die chinesische Firma Stream Computing, die sich zur Mission gesetzt hat, high-end RISC-V-Chips zum Einsatz in Rechenzentren zu entwickeln. Bereits im Jahre 2022 konnte Stream Computing ein eigenes Arbeitsergebnis vorweisen [7]. Dieses Arbeitsergebnis wurde von Stream Computing selbst "RISC-V Matrix ISA Specification" in der Version v0.1 genannt. Die Version v0.5 wurde im Oktober 2024 erstellt.

Es handelt sich aber trotz des Namens nicht um die offizielle Version, die das RISC-V-Gremium veröffentlichen wird. Stream Computing hat ebenfalls die notwendigen Anpassungen zum LLVM Compiler-Framework beigetragen. Die Änderung des LLVM Framework stellt eine große Investition in Form von Arbeitsstunden dar, die Stream Computing aufgebracht hat, ohne zu wissen, ob ihre Ergebnisse jemals in der Breite zum Tragen kommen werden.

Wie tragbar die Ergebnisse sind, lässt sich schwer einschätzen, ohne eine eingehende Evaluierung des Quellcodes durchgeführt zu haben. Es ist jedoch interessant zu sehen, welche Innovationskraft, Arbeitsgeschwindigkeit und auch welchen Mut und welche Risikobereitschaft die chinesische Industrie an den Tag legt.

Die Onlinezeitung Heise berichtet jüngst über die Strategien, die die großen Player der Chip-Industrie bezüglich Matrix-Rechenwerken verfolgen um KI auf Ihren CPUs effizient ausführbar zu machen [20]. Apple, als Vorreiter, hat bereits Matrixrechenwerke in die M-Prozessoren integriert. Die Firma Qualcomm möchte diesen Schritt im Jahre 2026 vollziehen, während AMD und Intel die Erweiterung um eine Matrixengine im Jahre 2027 angehen.

In dem Heise-Bericht [20] wird angesprochen, dass die grundlegende Operation der Matrixrechenwerke die Matrix-Matrix-Operation ist, also die Multiplikation von zwei Matrizen. Insbesondere sollen von den Matrixrechenwerken die speziellen, neuartigen Datentypen unterstützt werden, die von modernen Neuronalen Netzen verwendet werden und die sich von den traditionellen Datentypen herkömmlicher CPU-Designs unterscheiden.

Innerhalb des NEORV32 CPU-Umfeldes gibt es eine Arbeit von Qizal Arsalan [3], bei der die NEORV32 CPU um Matrixberechnungen erweitert wird. Diese Bachelorarbeit unterscheidet sich allerdings hinsichtlich der Herangehensweise stark von der Arbeit aus [3].

Die NEORV32 CPU besitzt im Design vorgesehene Erweiterungspunkte namens CFS und CFU. Das CFS (Custom Functions Subsystem) erlaubt es Hardware-Beschleuniger in die NEORV32 CPU einzubringen, die über Speicheradressen (Memory Mapping) aus einer Softwareanwendung mit Daten versorgt werden können. CFU (Custom Function Unit) erlaubt es einzelne Instruktionen zu ergänzen um die CPU um kleinschrittige Operationen zu erweitern.

Qizal Arsalan untersucht den Ansatz der Matrixmultiplikation mit CFS und

CFU. In dieser Bachelorarbeit werden Matrix-Instruktionen jedoch in den NEORV32-Chip eingebracht, die nicht über das CFS oder CFU Feature eingebunden werden, sondern über die Execution-Engine, also der State-Machine, des NEORV32-Chips. Der Grund ist, dass die Speicheradressabhängigkeit von CFS und CFU nicht konform mit einer Matrix-Extension-Spezifikation des RISC-V Gremiums sein können und nicht sein werden.

Bei CFS und CFU handelt es sich um Systeme, die nicht Teil der RISC-V Spezifikation sind. Diese Bachelorarbeit versucht die Ergebnisse einer Matrix-Extension im RISC-V Umfeld zu antizipieren und kann daher CFS und CFU nicht verwenden.

1.4 Struktur der Arbeit

Die Matrixmultiplikation wird im Kapitel "Theoretische Grundlagen" beschrieben. Es wird verdeutlicht, dass sich die Matrixmultiplikation durch Aufteilung der Matrizen in Blöcke aufteilen und damit schrittweise abarbeiten lässt. Damit wird das Strip-Mining ermöglicht.

Danach werden die momentan ablaufenden Anstrengungen des RISC-V Gremiums zur Erstellung einer Extension zur Matrixmultiplikation beschrieben. Dabei gibt es drei Ansätze. Der in dieser Bachelorarbeit untersuchte Ansatz wird zusammen mit den anderen Ansätzen vorgestellt. Dabei wird der Strip-Mining-Ansatz erläutert und es wird eine Softwareanwendung besprochen, die diesen Ansatz als Computerprogramm implementiert.

Es folgt eine Beschreibung des Vorgehens bei der Erstellung dieser Bachelorarbeit.

Nachdem der Ansatz beschrieben ist, wird die NEORV32 CPU als das Ziel der Implementierung beschrieben. Das Design wird herausgearbeitet und es wird diskutiert, welche Änderungen an welchen Stellen vorgenommen werden, um die CPU um Matrixbefehle zu erweitern.

Danach wird der VHDL Quellcode der NEORV32 CPU erweitert und auf einen FPGA programmiert. Auf dem FPGA wird eine Software-Anwendung ausgeführt, die die Matrix-Instruktionen verwendet um eine Matrixmultiplikation durchzuführen.

Schließlich wird das Ergebnis mit einer normalen Matrixberechnung verglichen.

Die Arbeit endet mit einer Zusammenfassung und einem Ausblick auf mögliche Verbesserungen.

Der Anhang enthält die Design-Dateien aus dem NEORV32 Projekt, die im Zuge dieser Arbeit geändert worden sind.

2 Theoretische Grundlagen

Dieses Kapitel beschreibt Vektoren und Matrizen aus der Mathematik und deren Berechnung durch ein Computersystem.

Bei der Berechnung ist auf die Geschwindigkeit zu achten. Ausgehend von dem naiven Berechnungsverfahren werden die Schwächen dieses Verfahrens diskutiert und beschrieben, welche Antworten das RISC-V Ökosystem auf diese Probleme hat.

Die ersten Hardwarebeschleunigungen sind mit der RISC-V Vector Extension für Vektoren eingeführt worden. Damit kann eine RISC-V CPU SIMD für die Vektorberechnung verwenden. Aufbauend auf der Vector-Extension wird diskutiert, welche Ansätze für die Berechnung von Matrizen verwendet werden können.

Der in dieser Arbeit gewählte Ansatz wird beschrieben.

2.1 Vektoren und Matrizen

Eine einzelne Zahl wird als Skalar bezeichnet. Ein Vektor ist eine eindimensionale Gruppierung von skalaren Werten. Es handelt sich dabei um eine Fortführung von Skalaren zu neuen Objekten der Mathematik, wie sie auch z.B. bei komplexen Zahlen stattfindet.

Auf Vektoren sind mathematische Operationen wie die Addition definiert, die wiederum Vektoren erzeugen. Es sind auch Operationen wie das Skalarprodukt oder der Betrag definiert, die aus Vektoren wieder Skalare erzeugen.

Eine Matrix wird in [18] als rechteckiges Zahlenschema mit m Zeilen und n Spalten beschrieben.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \dots & a_{mn} \end{bmatrix}$$

Die Multiplikation zwischen zwei Matrizen A und B ist nicht kommutativ und nur definiert, wenn die inneren Dimensionen der Matrizen übereinstimmen, wenn also die Spaltenzahl der ersten Matrix mit der Zeilenzahl der zweiten Matrix übereinstimmt.

Zur Berechnung der Matrixmultiplikation

$$C = A * B \quad (2.1)$$

muss zur Berechnung einer Zelle der Matrix C ein Skalarprodukt aus einer Zeile von A und einer Spalte von B berechnet werden. Dabei wird zunächst aus der Spalte von B durch Transponieren eine Zeile gemacht und dann der Zeilenvektor aus A mit dem Zeilenvektor aus B durch das Skalarprodukt in einen Skalar umgerechnet. Dieser Skalar wird dann in die Matrix C eingesetzt.

Ein anschaulichere Beschreibung bietet das Falksche Schema [6].

2.2 Matrixmultiplikation mit SISD

Eine Implementierung, die sich direkt an der mathematischen Rechenvorschrift orientiert, verwendet drei For-Schleifen. Eine Schleife für alle Zeilen von A. Eine weitere für alle Spalten von B und die dritte Schleife für das Skalarprodukt.

```
1 C = zero_matrix() // Starte mit Nullwerten
2
3 for int i = 1 to l // ueber die Zeilen von C
4   for int k = 1 to n // ueber die Spalten von C
5     for int j = 1 to m // ueber die Spalten von A
6       // und Zeilen von B
7       C(i,k) = C(i,k) + A(i,j) * B(j,k)
8     end
9   end
10 end
```

Quellcode 2.1: Matrixmultiplikation Iterativ

Ganz abgesehen von einer Komplexität von $\mathcal{O}(n^3)$ ist der Nachteil dieser Beschreibung nicht sichtbar, wenn man sich lediglich den Pseudo-Code oder eine Implementierung in einer realen, höheren Programmiersprache ansieht.

Wenn der Compiler das Optimierungspotential für den Einsatz hardwarebeschleunigter Instruktionen nicht erkennt oder die CPU keine speziellen Anweisungen anbietet, generiert der Compiler SISD (Single Instruction Single Data)-Anweisungen, die jede Multiplikation und Addition einzeln ausführen. Im ungünstigsten Fall besteht der generierte Code aus einer langen Folge aus einer Load-Anweisung, gefolgt von einer Operation, gefolgt von einer Store-Anweisung. Dieses ständige Laden und Speichern führt zu einer langen Laufzeit.

In den beiden Architekturen Van-Neumann und Harvard gilt gleichermaßen, dass eine Berechnung nicht im Speicher, sondern nur innerhalb der CPU ausgeführt wird. Da die Daten im Speicher stehen, muss also immer zunächst

eine Übertragung zwischen Speicher und CPU erfolgen. Danach kann die Berechnung innerhalb der CPU durchgeführt werden und abschließend muss das Ergebnis zurück in den Speicher übertragen werden. Man nennt dies die Load-Store-Architektur. Dabei sind die beiden Schritte Load und Store aufwändig, da sie im schlimmsten Fall durch die gesamte Speicherhierarchie kaskadieren, wenn die Daten nicht im Cache vorliegen.

Der Nachteil der naiven Implementierung liegt also in der Tatsache begründet, dass Werte nicht in den Registern der CPU verbleiben können, da eine CPU nur wenige Register besitzt und diese Register wiederverwendet werden müssen. Um die Register wiederzuverwenden, muss der enthaltene Wert zurück in den Speicher geschrieben werden. Dies ist in der höheren Programmiersprache nicht ersichtlich und wird erst sichtbar, wenn man sich den Assembler-Quellcode ansieht, den der Compiler erzeugt.

Es muss also nach Möglichkeit ein Algorithmus gefunden werden, der sich die Cache-Lokalität zunutze macht und ebenfalls Hardwarebeschleunigung per SIMD Instruktionen ermöglicht.

2.3 Matrixmultiplikation mit Strip-Mining und SIMD

Ein Ansatz, diesen Verlust, der durch Load-Store entsteht zu minimieren, ist das Strip-Mining in Kombination mit der Verwendung von SIMD.

SIMD (Single Instruction Multiple Data) erlaubt es, mehr als ein Datum gleichzeitig durch eine einzige Instruktion verarbeiten zu können. Die Addition zweier Vektoren geschieht aus mathematischer Sicht elementweise und jedes Elementpaar kann unabhängig von allen anderen Paaren in jeglicher Reihenfolge addiert werden. Damit eignet sich die Vektoraddition für SIMD Anweisungen. Dazu muss die CPU die passende Hardware bereitstellen. Dann ist es möglich, dass die CPU, wenn eine Vektoradditionsanweisung aufgeführt wird, gleich mehrere Elemente auf einmal miteinander addiert.

Abbildung 2.1 stellt SISD auf der linken und SIMD auf der rechten Seite schematisch dar. SISD addiert pro Aufruf ein Elementpaar, wobei SIMD mehrere Paare pro Aufruf addiert.

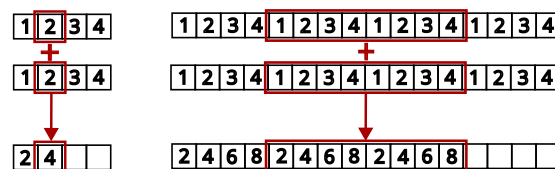


Abbildung 2.1: SISD (links) und SIMD (rechts)

Des Weiteren erlaubt das Strip-Mining auch Datenobjekte zu verarbeiten, die so groß sind, dass sie nicht als Ganzes in die Register der CPU passen.

Beim Strip-Mining wird von einer Ressourcen-Hierarchie ausgegangen. Die Ressourcen sind dabei die Register und der Speicher einer CPU, wobei Regis-

ter letztendlich auch nur sehr kleine Speicherzellen sind. Der Grundgedanke ist, dass, je näher der Speicher an der CPU liegt, dessen Preis steigt und daher kleiner in der verfügbaren Größe ausfallen muss. Gleichzeitig ist Speicher aber umso schneller, je näher er an der CPU liegt. Die Hierarchie besteht aus den Registern in der CPU, gefolgt von einem oder mehreren Cache-Leveln und schließlich den RAM-Bausteinen.

Die Vektor- oder Matrix-Daten kommen aus dem Speicher und müssen der CPU in Form seiner Register zugeführt werden. Die Addition zweier Vektoren beispielsweise kann nur durchgeführt werden, wenn Vektordaten in den Registern der CPU stehen. Die Kunst besteht also darin, die Daten in effizienter Form an den CPU heranzuführen. Beispielsweise ist es ineffizient ständig im Speicher hin- und herzuspringen. Ein lineares Laden ist effizienter um die Lokalität der Daten innerhalb des Cache auszunutzen.

Eine Cache-Hierarchie legt nahe, Vektoren blockweise in den Speicher zu legen und von dort auch wieder blockweise zu laden, so dass ein Block jeweils genau in den verfügbaren Cache passt. Einmal geladen, dient der Cache der CPU als schneller Speicher und ein Durchgriff auf den langsameren RAM-Speicher erfolgt nur, wenn die Daten im Cache verarbeitet worden sind.

Die Register innerhalb der CPU sind von begrenzter Größe. Eine sinnvolle Annahme ist etwa 1024 bit Registerbreite. Damit passen dort dann 32 Elemente eines Vektors hinein, wenn jedes Element eine 32-Bit Zahl darstellt.

Wenn nun sehr große Vektoren addiert werden sollen, dann kommt Strip-Mining zum Einsatz. Mit groß ist hier gemeint, dass die Vektoren weder im Ganzen in die CPU-Register passen, noch passen die Vektoren im Ganzen in einen Cache-Frame. Die Software verwendet eine Instruktion um zu ermitteln, wie viel Platz innerhalb der CPU-internen Register verfügbar ist. Die CPU antwortet mit der Registerbreite.

Für die Berechnung von Vektoren wurde die RISC-V Vector Extension (RVV) [2] erstellt und ratifiziert. Die Vektor Extension ist für diese Arbeit relevant, da sie zwar Vektoren verarbeitet, aber damit auch die Grundlage für die Verarbeitung von Matrizen legt.

2.4 Die RISC-V Vector Extension

Eine RISC-V CPU besitzt die sogenannte X-Register-File. Es handelt sich dabei um alle Register, die eine RISC-V CPU standardmäßig, ohne Extensions hat. Dies könnten beispielsweise die Register x0 bis x31 sein. Die Länge der Register in bits wird durch die Konstante XLEN festgelegt.

Die RVV erweitert eine RISC-V CPU um zusätzliche Vektorregister v0 bis v31. Die Länge der Vektorregister wird durch die Konstante VLEN (in Anlehnung an die Konstante XLEN) vorgegeben. VLEN wird bei der Synthese des Designs auf einen Wert festgelegt und ist zur Laufzeit nicht änderbar. VLEN

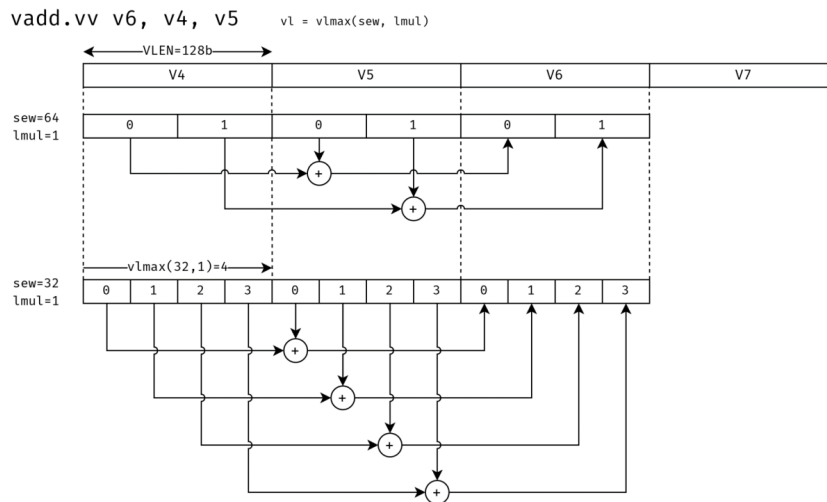
stellt somit eine Eigenschaft der CPU dar. Die RVV Spezifikation [2] macht die Vorgabe:

“The number of bits in a single vector register, $VLEN \geq ELEN$, which must be a power of 2, and must be no greater than 2^{16} ”.

Das Besondere an den Vektorregistern ist, dass sie konfiguriert werden können. Es lässt sich festlegen, welcher Datentyp in den Registern erwartet wird, also 8-bit Bytes, 16-bit Halbwörter oder 32- oder 64-bit Wörter. Zusätzlich lassen sich die Vektorregister gruppieren, also aneinander hängen, um noch größere Register zu bilden.

Der Vorteil ist, dass man beispielsweise 8-bit Bytes als Datentyp festlegen kann und dann vier Vektorregister gruppieren kann. Unter der Annahme, dass ein RISC-V Chip mit einer $VLEN$ von 2^{16} synthetisiert worden ist, ergibt sich damit Speicher für beispielsweise 262144 bytes. Werden zwei Vektoren dieser Länge in den CPU geladen, kann eine Vektoraddition all dieser Elemente mit nur einer einzigen Instruktion erfolgen. Dies ergibt einen Speedup um den Faktor 262144 gegenüber einem naiven Load-Store-Ansatz, bei dem jedes 8-Bit Elementpaar einzeln geladen und verrechnet wird.

Die Abbildung 2.2 aus [9] zeigt zwei mögliche Konfigurationen. Beide Konfigurationen verwenden die drei Vektorregister V4 bis V6. Die obere Konfiguration addiert 64 Bit Datentypen, während die untere Konfiguration 32 Bit Datentypen addiert. Die Vektor-Engine wird basierend auf der aktuellen Konfiguration automatisch die korrekten Stellen der Register auslesen, miteinander addieren und die Ergebnisse an die richtigen Stellen in das Zielregister ablegen.



Die Abbildung 2.2 bedarf einer Korrektur. Die Ergebnisse der Additionen, die jeweils in den Zielregistern V6 dargestellt sind, sind in der Abbildung nicht korrekt. Der Informationsgehalt der Abbildung ist aber trotzdem hilfreich für das Verständnis.

2.5 Das Hardware-Software-Co-Design

Die Verwendung der Vektor-Extension erfolgt durch ein Hardware-Software-Co-Design. Damit ist gemeint, dass eine Anwendung mit der Hardware zusammenarbeiten muss um von der Hardware-Beschleunigung zu profitieren.

Moderne, optimierende Compiler wie der GCC oder die LLVM-Infrastruktur wurden um das Wissen um die RISC-V-Vektor-Extension erweitert und können den Anwendungsfall automatisch erkennen und das gewünschte Hardware-Software Co-Design automatisch generieren.

Dies lässt sich testen, indem man Vektoroperationen mit der neuesten Version des GCC für RISC-V übersetzt. Im generierten Assembler-Code findet man die RVV-Instruktionen wieder. Der Compiler hat den Anwendungsfall erkannt und die entsprechenden, hardware-beschleunigten Anweisungen ausgewählt.

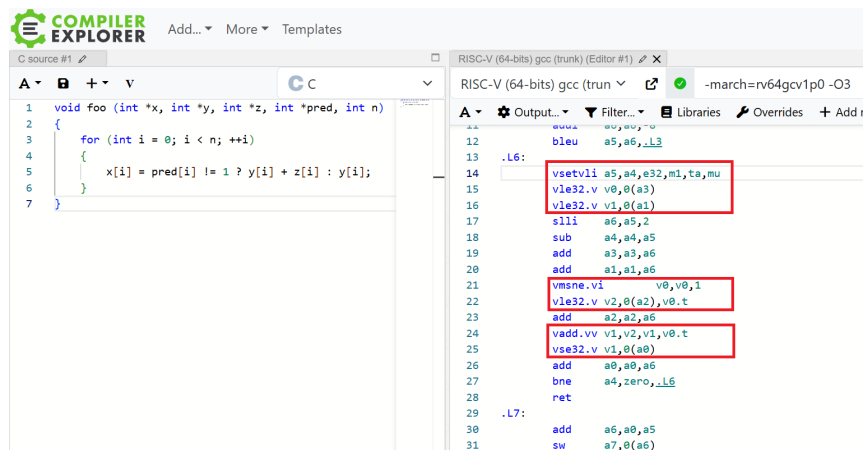


Abbildung 2.3: Generierung der RVV-Anweisungen

In der Abbildung 2.3 wird gezeigt, dass der GCC-Compiler für den 64-Bit RISC-V Target bereits für die RVV angepasst worden ist. Auf der linken Seite ist eine Vektoroperation dargestellt. Auf der rechten Seite der Abbildung ist der generierte Code dargestellt. Die speziellen Anweisungen aus der RVV-Extension sind mit einem roten Rahmen kenntlich gemacht. Der GCC-Compiler hat diese Anweisungen automatisch ausgewählt, da er das Optimierungspotential erkannt hat.

Das Hardware-Software Co-Design sieht das iterative Vorgehen Strip-Mining vor, bei dem die Software die Hardware in jeder Iteration aufs neue konfiguriert und dann Daten zur parallelen Verarbeitung zuführt.

Ein Ablaufdiagramm ist in Abbildung 2.4 dargestellt:

Die Schritte sind:

1. Die erste bzw. nächste Iteration beginnt.

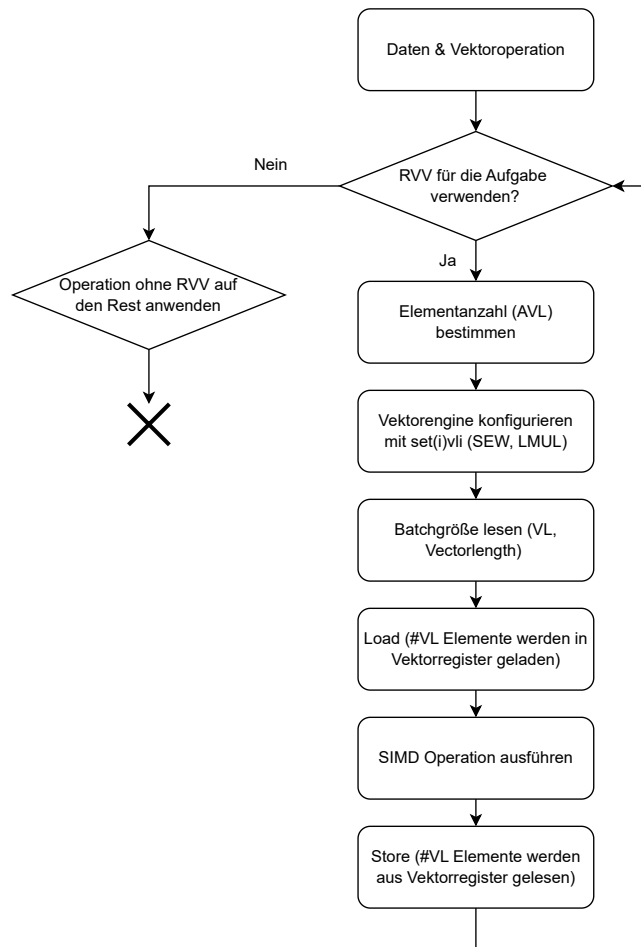


Abbildung 2.4: Strip-Mining Ablauf

2. Die Applikation entscheidet, ob es effizienter ist die Vektoroperation mit oder ohne Vektor-Extension auszuführen. Wenn die Vektorlänge beispielsweise nur aus zwei Elementen besteht, ist es schneller, diese beiden Elemente einfach so zu verrechnen.
3. Wenn sich die Anwendung dazu entscheidet die Vektor-Extension zu verwenden, dann beginnt die Strip-Mining-Schleife. Wenn es zu wenige Elemente gibt, dann springt die Anwendung zu einem Label, das die Verarbeitung terminiert. Der verbleibende Rest der Elemente wird dann manuell ohne Hardwarebeschleunigung berechnet.
4. Die Anwendung bestimmt den Anteil der Vektorelemente, die noch verarbeitet werden müssen. Dieser Wert wird Application Vector Length (AVL) genannt. Beispielsweise entspricht die AVL in der ersten Iteration der gesamten Länge des Vektors. Mit jedem Schleifendurchlauf

werden Teile des Vektors verarbeitet und die AVL nimmt immer mehr ab, bis schließlich keine Elemente zur Verarbeitung mehr übrig sind und das Verfahren terminiert.

5. Neben der AVL muss die Anwendung auch in jeder Iteration den Datentyp bzw. die Größe der einzelnen Vektorelemente konfigurieren. Die Größe wird (= Selected Element Width, SEW) genannt. Es wird auch bestimmt, wie die Vektorelemente in den Registern angeordnet werden und wie die Vektorregister gruppiert werden. Diese Konfiguration wird Group Modifier oder LMUL genannt. Um die Konfiguration vorzunehmen ruft die Anwendung die Instruktion `set(i)vli` auf.
6. Die Vector-Engine verwendet die konfigurierten Werte AVL, SEW und LMUL und berechnet die Batch-Größe. Ein Batch ist die Menge an Vektorelementen, die in einer einzigen SIMD-Operation verrechnet werden. Hier kommt der Hardware-Teil des Hardware-Software Co-Designs zum tragen, denn die RISC-V CPU und die enthaltene Vector-Engine müssen entscheiden, wie viele Elemente jetzt tatsächlich parallel verarbeitet werden können. Dabei berücksichtigt die Vector-Engine, wie viele Rechenknoten sie zur Verfügung hat um parallel zu rechnen. Die ermittelte Anzahl an Elementen wird Vector Length (VL) genannt und von der Hardware an die Anwendung zurückgegeben. Dazu gibt die Instruktion `set(i)vli` diesen Wert in ein Rückgaberegister aus, aus dem sich die Anwendung den Wert holen kann.
7. Die Anwendung kennt jetzt die Vector Length (VL), die für diese Iteration gilt. Damit kennt die Software die Batch-Size. Die Software wird nun die Vector-Load Instruktionen (`vle8`, `vle16`, ...) verwenden um einen Anteil des Vektors oder der Vektoren, der genau der Vector Length (VL) entspricht, in die Vektorregister zu laden. Die Anwendung wird auch einen Zeiger nach vorne verschieben, um sich zu merken, von welcher Stelle in der nächsten Iteration geladen werden muss.
8. Die Vector-Engine verarbeitet die Vector-Load-Anweisungen und schreibt die geladenen Daten in die Vektorregister.
9. Die Anwendung führt nun eine Instruktion aus um die Vektoren zu verrechnen. Es können beispielsweise eine elementweise Addition (`vadd`), Subtraktion (`vsub`), Multiplikation oder sonstige Operationen ausgeführt werden.
10. Die Vector-Engine führt nun die Instruktion auf allen Elementen parallel aus. Es handelt sich also um eine SIMD-Operation, bei der mehrere Daten aufgrund einer einzigen Anweisung verarbeitet werden. Damit ist der aktuelle Batch der Iteration abgearbeitet.
11. Die Anwendung führt eine Store-Anweisung aus um die Ergebnisdaten aus dem Ergebnisregister in den Speicher zurückzuschreiben und Platz für neue Ergebnisdaten zu schaffen.
12. Die Vector Engine verarbeitet die Store-Anweisung und die Daten werden in den Cache bzw. den RAM übertragen.

13. Die nächste Iteration beginnt. Im Grunde springt die Anwendung von hier zurück zum ersten Punkt dieser Auflistung.

Es folgt ein Beispiel in RISC-V Assembly, das das obige Vorgehen umsetzt. Die Anweisungen aus der Vektor-Extension sind weniger eingerückt als die normalen RISC-V Anweisungen.

```

1 // vector-vector add routine of 32-bit integers
2 //
3 // void vvaddint32(size_t n,
4 //     const int*x, const int*y, int*z)
5 // {
6 //     for (size_t i=0; i<n; i++)
7 //     {
8 //         z[i] = x[i] + y[i];
9 //     }
10 // }
11 //
12 // Mapping parameters to argument register (a0 .. a7)
13 // a0 = n, a1 = x, a2 = y, a3 = z
14 //
15 // Non-vector instructions are indented
16 vvaddint32:
17
18     // Set vector length based on 32-bit vectors.
19     // t0 contains the amount of elements that will be
20     // processed
21     vsetvli t0, a0, e32, ta, ma
22
23     // Load for first vector
24     vle32.v v0, (a1)
25
26     // Decrement number of elements that
27     // still need processing.
28     // t0 is the result of vsetvli. It
29     // is the amount of elements processed
30     // this iteration
31     sub a0, a0, t0
32
33     // Multiply number done by 4 bytes (= 32 bit
34     // integer) because of SEW=e32
35     slli t0, t0, 2
36
37     // Increment/Advance pointer for first vector
38     add a1, a1, t0
39
40     // Load for second vector
41     vle32.v v1, (a2)
42     // Increment/Advance pointer for second vector
43     add a2, a2, t0
44
45     // Sum vectors (with hardware acceleration)
46     vadd.vv v2, v0, v1
47
48     // Store result back to memory
49     vse32.v v2, (a3)
50
51     // Increment/Advance pointer for destination
52     add a3, a3, t0
53
54     // Loop back to the beginning of the for loop
55     // if the element count is not zero. Otherwise
56     // continue with the ret instruction
57     bnez a0, vvaddint32
58

```

```
59         // Finished with the function
60     ret
```

Quellcode 2.2: Vektoraddition Strip-Mining

2.6 Übergang zu einer Matrix Extension

Jetzt, da die Wirkungsweise der RVV-Extension für Vektoren beschrieben worden ist, kann nachvollzogen werden, in welchem Spannungsfeld die RISC-V-Gremien Entscheidungen treffen müssen, wenn es um den nächsten Schritt hin zu einer Matrix-Extension geht.

Da die RVV-Extension ratifiziert worden ist, gibt es bereits Chips, die auf dem Markt verfügbar sind und die RVV-Extension implementieren. Ein Beispiel ist der OrangePi RV2 Single Board Computer, der einen Spacemit-K1 RISC-V-Chip verwendet.

Das bedeutet, Firmen haben bereits Designs erstellt und damit Zeit und Geld in die RISC-V RVV Strategie investiert. Wenn eine Matrix-Extension Vektorregister wiederverwenden würde, hätten solche Firmen einen Vorteil. Auf der anderen Seite möchte man das bestmögliche Verfahren ermöglichen um effizient mit Matrizen rechnen zu können und sich damit von der Konkurrenz abzuheben. Eventuell ist der Ansatz, der für Vektoren gewählt wurde, für Matrizen nicht optimal und es ist lohnenswerter komplett neue Wege zu gehen und auf die Wiederverwendung von Hardware zu verzichten.

Es gibt laut Krste Asanovic [4] bei der Arbeit an einer Extension für Matrizen drei große Strömungen innerhalb der RISV-V-Organisation. Für jeden der Ansätze wurde ein Charter gebildet und alle Charter treiben ihre Arbeit eigenständig voran.

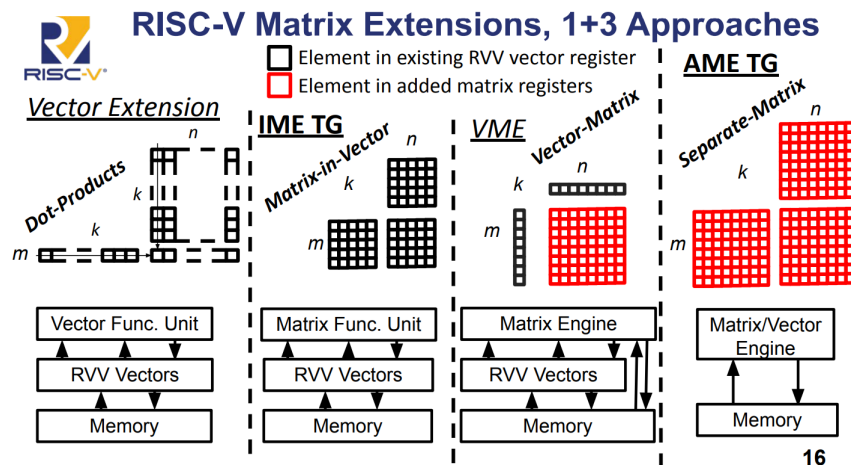


Abbildung 2.5: Ansätze zur Matrix-Extension (Quelle: [4], S.16)

Zunächst wird in der linken Spalte der Abbildung 2.6 aus [4] gezeigt, dass sich eine Matrix theoretisch elementweise berechnen lässt, wenn für jedes

Element der Matrix individuell ein Skalarprodukt mit Hilfe der RVV-Extension berechnet wird. Dies ist nur möglich, wenn die CPU die RVV besitzt und gleichzeitig, wenn die Matrix spalten- oder zeilenweise in ein Vektorregister passt.

Die Integrated Matrix Extension IME [10] verwendet die RVV-Register wieder um darin Matrix-Berechnungen auszuführen. Es wird keine spezielle Hardware für die Matrix-Berechnung hinzugefügt. Die erklärten Ziele des IME-Charter sind es, einen bedeutenden Geschwindigkeitsvorteil zu erzielen, bei vergleichbaren Kosten, wie sie bereits für die RVV anfallen.

Die Vektor-Matrix-Extension VME baut auf die RVV-Extension auf und führt zusätzlich einen sogenannten Matrix-State bzw. eine Matrix-Engine hinzu. Mit dem Matrix-State ist gemeint, dass die CPU um Komponenten erweitert wird, die in der Lage sind Zwischenergebnisse bei einer Matrix-Multiplikation zu speichern. Es kommt bei der VME also weitere Hardware für die Matrix-Berechnung hinzu. Diese zusätzliche Hardware ist in der Abbildung in roter Farbe markiert.

Die Attached Matrix-Extension AME geht noch einen Schritt weiter und verwendet dabei keine Vektorregister aus der RVV-Extension mehr, sondern fügt Architected State sogar für komplette Matrix-Register hinzu. Sowohl die Eingabedaten als auch die Zwischen- und Endergebnisse werden in speziellen Matrix-Registern gespeichert. Damit ist die AME als einziger Ansatz unabhängig von der RVV-Extension und könnte auch komplett ohne RVV-Extension angeboten werden. Auch hier ist neue Hardware in roter Farbe dargestellt.

Für die offizielle Ratifizierung einer RISC-V Matrix-Extension ist zum jetzigen Zeitpunkt nicht klar, welcher Ansatz verwendet wird oder ob eventuell sogar mehr als eine Spezifikation ratifiziert werden wird.

In dieser Bachelor-Arbeit wird ein Ansatz verfolgt, der sich vom Architected State her am ehesten an der Attached Matrix Extension (AME) orientiert. Die Entscheidung fiel zugunsten der AME orientierten Lösung, da die NEORV32 CPU im jetzigen Zustand keine RVV-Extension implementiert und daher zunächst RVV Register erstellt werden müssten um IME oder VME zu implementieren.

Ein weiterer Grund ist, dass sich die zusätzliche Matrix-Hardware der AME sehr logisch aus der Berechnung des dyadischen Produkts ergibt und daher ein einfaches Verständnis der zu erledigenden Aufgaben möglich ist. Im begrenzten Zeitrahmen dieser Bachelorarbeit wurde daher zugunsten einer Lösung entschieden, die intellektuell auch unter Kontrolle gehalten werden kann und für die ein Abschluss der Arbeit absehbar ist.

Es werden Instruktionen für die Matrixberechnung vorgesehen, wie sie auch bei der Vektor-Extension zu finden sind. Damit ist gemeint, dass die hinzugefügten Instruktionen den Strip-Mining Ansatz ermöglichen. Daher werden Load- und Store-Anweisungen und eine Anweisung für die Operation der Matrixmultiplikation vorgesehen. Damit kann eine Matrixmultiplikation auch für sehr große Matrixen, die nicht als Ganzes in die CPU-Register passen, erfol-

gen. Außerdem können so auch Kernel in der blis-Bibliothek implementiert werden.

2.7 Blockweise Matrixmultiplikation

Im Folgenden wird das blockweise Vorgehen erläutert, mit dem die Matrixmultiplikation ausgeführt werden wird.

Bei der herkömmlichen Matrixmultiplikation über das Skalarprodukt von Zeilen und Spalten werden die beiden Matrizen A und B in ihrer kompletten Dimension betrachtet, da jeweils komplette Zeilen und Spalten verrechnet werden müssen um ein Matrix-Element zu bestimmen.

Bei sehr großen Matrizen ist dies nicht zielführend, da die CPU die Matrix nur schrittweise erfassen und damit nur sehr ineffizient berechnen kann. Die angestrebte Verbesserung besteht darin, eine Matrix blockweise zu berechnen. Die Abbildung 2.6 visualisiert das blockweise Vorgehen.

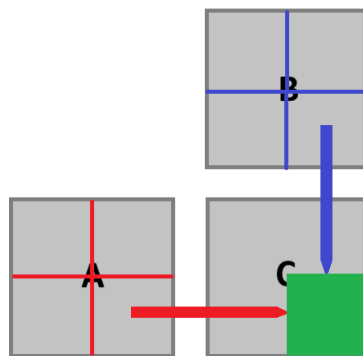


Abbildung 2.6: Matrix in Blöcken

Um einen Teil der Matrix C zu berechnen, reicht es aus in mehreren Schritten nur entsprechende Teile der Matrix A und Teile der Matrix B zu betrachten. Die Matrizen müssen nicht mehr im Ganzen in die CPU oder den Cache passen sondern es können in jedem Schritt nur Teile der Matrizen betrachtet werden. Die Abbildung 2.6 zeigt einen Schritt in diesem Prozess.

Zusätzlich eröffnet sich ebenfalls das Potential der parallelen Berechnung. Wie bei Vektoren ist es durch das blockweise Vorgehen auch für die Berechnung von Matrizen egal, in welcher Reihenfolge die einzelnen Ergebnisse berechnet werden. Damit ist auch eine vollkommene Parallelität denkbar.

Als Ausblick würde das für ein RISC-V Design bedeuten, dass eine RISC-V CPU aus mehreren Harts synthetisiert wird, wobei jeder Hart eine Matrix-Engine erhält, also in der Lage ist, die Matrixmultiplikation hardwarebeschleunigt zu berechnen. Hart ist der RISC-V Begriff für CPU-Cores. Dann wird eine große Matrix auf mehrere Cores verteilt um das Ergebnis in einzelnen Blöcken zu berechnen und die Teilergebnisse zurück in den RAM-Speicher zu schreiben.

Zunächst wird anhand eines Beispiels verdeutlicht, was es bedeutet, wenn von blockweiser Berechnung gesprochen wird:

A	B		E	F	
C	D	G	H		
1	2	3	4	17	18
5	6	7	8	21	22
9	10	11	12	25	26
13	14	15	16	29	30
				19	20
				23	24
				27	28
				31	32

Abbildung 2.7: Matrixmultiplikation blockweise

In Abbildung 2.7 wird die grüne Matrix mit der blauen Matrix multipliziert. Es werden einzelne Blöcke A bis H innerhalb der beiden Matrizen definiert.

Zur Gegenprüfung ist das Ergebnis in Abbildung 2.8 dargestellt.

250	260	270	280
618	644	670	696
986	1028	1070	1112
1354	1412	1470	1528

Abbildung 2.8: Matrixmultiplikation Ergebnis

Dieses Ergebnis wird den Berechnungsvorschriften aus 2.9 zufolge berechnet. Beispielsweise gilt für den grauen Block die Vorschrift $A * E + B * G$

A	B	E	F	$A * E + B * G$	$A * F + B * H$
C	D	G	H	$C * E + D * G$	$C * F + D * H$
nach links		nach unten			

Abbildung 2.9: Matrixmultiplikation Berechnungsvorschrift

Der grau markierte Block ist das Ergebnis der Multiplikation der Blöcke A, E, B und G. Es ist zu prüfen, ob die Multiplikation der Blöcke tatsächlich den grauen Bereich der Ergebnismatrix ergibt.

In Abbildung 2.10 wird der grau hervorgehobene Bereich der Berechnungsvorschrift zufolge berechnet. Zunächst werden die Blöcke A mit E und B mit G zusammen sortiert. Das Ergebnis der beiden Multiplikationen ist in der folgenden Zeile zu finden. Das Ergebnis der Addition der Zwischenergebnisse ist in der untersten Zeile. Die unterste Zeile enthält den grauen Block.

Mit dem Wissen über die blockweise Aufteilung ist es nun möglich per SIMD zu parallelisieren oder Strip-Mining auszuführen, wenn keine parallelen Rechenknoten verfügbar sind. In dieser Arbeit werden Instruktionen analog zur RVV bereitgestellt um das Strip-Mining für Matrizen zu ermöglichen und dabei blockweise vorzugehen.

Es ist noch zu klären, wie die einzelnen, inneren Blöcke miteinander multipliziert und addiert werden. Die Instruktionen, die im Rahmen dieser Arbeit in die NEORV32 CPU eingebracht werden, ermöglichen genau die Berechnung dieser inneren Blöcke. Der Rest muss durch ein Hardware-Software Co-Design organisiert werden.

Die inneren Blöcke werden durch die wiederholte Anwendung des dyadischen Produkts berechnet. Das dyadische Produkt wird im folgenden Abschnitt definiert.

A	E	B	G
1	2	17	18
5	6	21	22
		3	4
		7	8
		25	26
		29	30

A*E		B*G	
59	62	191	198
211	222	407	422

A*E + B*G	
250	260
618	644

Abbildung 2.10: Matrix Multiplikation Blöcke A, E, B und G

2.8 Dyadisches Produkt

Das Outer Product, oder auch dyadisches Produkt, wird verwendet um die Matrixmultiplikation von Blöcken auszuführen. Die Eigenschaft, die das dyadische Produkt nützlich macht ist, dass es die Matrixmultiplikation auf die Berechnung von Additionen und Multiplikationen in einer gleichförmigen Form reduziert. Damit lässt sich das Verfahren einfach in einer Hardwarebeschreibungssprache formulieren. Die Synthese-Werkzeuge können dann parallelisierte Hardware zur Berechnung generieren.

Das dyadische Produkt erhöht auch die Parallelität im Vergleich zur Berechnung einzelner Zellen der Ergebnismatrix über einzelne Skalarprodukte. Beim dyadischen Produkt wird ein gesamtes Feld pro Durchgang berechnet. Nach allen Durchgängen enthält das Feld dann die Ergebnismatrix der Matrixmultiplikation.

In der Übersicht [11] von Earl A. Killian wird das dyadische Produkt auf Seite 53 beschrieben.

"The outer product is a fully parallel computation of mn multiplications."

Das dyadische Produkt wird aus zwei Vektoren u und v berechnet.

$$u = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_m \end{bmatrix}, v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

Das Symbol ist $\otimes$ und das Ergebnis ist eine m x n Matrix C. Die Rechenvorschrift ist:

$$C = u \otimes v = \begin{pmatrix} u_1 v_1 & u_1 v_2 & \cdots & u_1 v_n \\ u_2 v_1 & u_2 v_2 & \cdots & u_2 v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_m v_1 & u_m v_2 & \cdots & u_m v_n \end{pmatrix} \quad (2.2)$$

Nachdem das dyadische Produkt definiert ist, lässt sich das dyadische Produkt aufaddieren, um die Matrixmultiplikation zu beschreiben.

“Using this formulation, the matrix product can be expressed as the sum of K outer products of the columns of A with the rows of B”

$$C = AB = A_1^* \otimes B_*^1 + A_2^* \otimes B_*^2 + \dots + A_M^* \otimes B_*^N = \sum_{p=1}^k A_p^* \otimes B_*^p \quad (2.3)$$

In der Formel (2.3) sind A und B zwei Matrizen, die miteinander multipliziert werden. Man kann erkennen, dass es nicht ausreicht lediglich die Multiplikationen aus Formel (2.2) auszuführen, vielmehr müssen die Ergebnisse auch aufaddiert werden. Es handelt sich um eine sogenannte Multiply-Accumulate-Operation. Daher ist es notwendig im Design Speicherzellen vorzusehen, deren Inhalt schrittweise durch Addition vervollständigt wird, bis das Endergebnis vorliegt.

Der “Matrix-Kern” aus Speicherzellen für die Multiplikation ist in Abbildung 2.11 aus [11] dargestellt. Dieses Konstrukt wird auch Matrix Accumulator (MAC) genannt, da es Zwischenergebnisse aufsummiert, also kummuliert.

Figure 1 4×4 Outer Product MAC Array with Local Accumulators

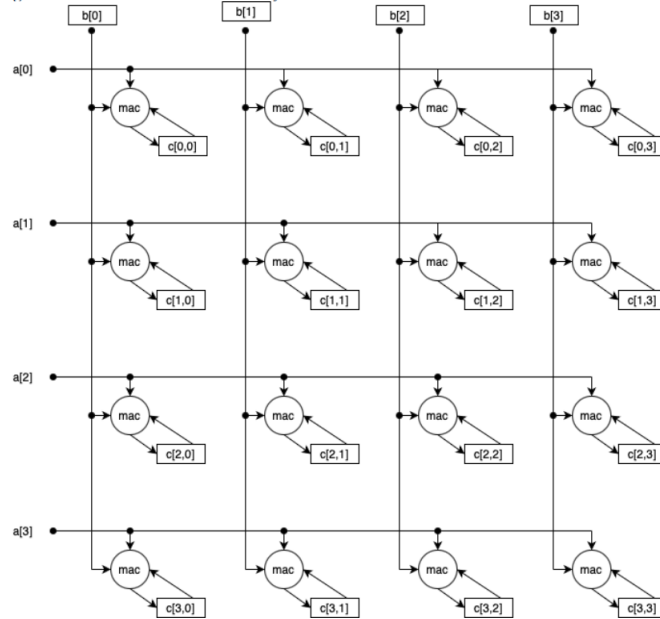


Abbildung 2.11: Matrix Accumulator (MAC) (Quelle: [11], S.23)

Wenn zwei Vektoren außen an den MAC angelegt werden, dann berechnet er ein Zwischenergebnis durch das dyadische Produkt aus den beiden Vektoren (siehe Formel (2.2)) und akkumuliert das Zwischenergebnis zum Rest auf (siehe Formel (2.3)). Wenn alle Vektoren angelegt worden sind, enthält der MAC somit das komplette Ergebnis der Multiplikation.

Die Kombination aus dem blockweisen Ansatz und dem dyadischen Produkt mittels MAC ermöglicht es ein Design zu erstellen, das dem angestrebten RISC-V AME Ansatz entspricht.

2.9 Zusammenfassung

Dieses Kapitel über die theoretischen Grundlagen beginnt bei der mathematischen Definition von Matrizen und führt hin zu einer konkreten Umsetzung der Matrixmultiplikation in einem Computersystem. Die Gedankengänge aus diesem Kapitel stellen die Grundlagen für die Implementierung dar.

Da die streng mathematische Multiplikation auf einem Load-Store-Computersystem zu langsam ist, wurde die SIMD-Verarbeitung in Kombination mit der Cache-Hierarchie besprochen. Auf einem RISC-V-System gibt es bereits SIMD-Instruktionen für die Berechnung von Vektoren in Form der Vektor-Extension. Es wurde beschrieben, wie das Hardware-Software Co-Design des Strip-Mining für die RVV-Extension aufgebaut ist.

Ausgehend von diesem Standpunkt sind die drei großen Strömungen IME, VME und AME der RISC-V-Charter zur Handhabung der Matrixberechnung vorgestellt worden. Es wurde erläutert, welche Überlegungen zur Entscheidung zugunsten der AME für diese Bachelorarbeit zugrunde liegen.

Das Kapitel wurde mit der Beschreibung der blockweisen Berechnung der Matrixmultiplikation unter Verwendung des dyadischen Produkts geschlossen. Damit ist das Strip-Mining auch für Matrizen möglich.

Gerüstet mit einem dokumentierten Zielbild kann als nächstes eine Implementierung methodisch geplant und dann durchgeführt werden.

3 Durchführung und Methodik

Das Ziel ist beschrieben. Es geht um die Erweiterung eines RISC-V-Kerns um eine hardwarebeschleunigte Matrixmultiplikation. Dabei soll die Arbeit möglichst nahe an einer der möglichen Erweiterungen liegen, die momentan tatsächlich von den RISC-V-Gremien vorbereitet werden.

Es wurde besprochen, mit welcher Technik die Matrixmultiplikation ausgeführt werden soll und welche theoretischen Grundlagen hinter den Überlegungen stehen.

In diesem Kapitel wird beschrieben, welches Vorgehen gewählt wird um die Arbeit durchzuführen.

3.1 Schrittweises Vorgehen

Das Vorgehen besteht aus folgenden Schritten:

1. Analyse der theoretischen Grundlagen zur Matrixmultiplikation. Es soll eine Möglichkeit gefunden werden, die die Berechnung auf einem FPGA ermöglicht und mit VHDL beschrieben werden kann. Dieser Schritt erfolgte im Kapitel 2.
2. Analyse der NEORV32 CPU. Es sollen die Komponenten der CPU identifiziert werden mit dem Ziel, die Stellen zu finden, die geändert bzw. erweitert werden müssen. Dieser Schritt erfolgt in Kapitel 4.
3. Ein Compiler (selbst erstellt oder der GCC Compiler) wird verwendet um Testprogramme zu übersetzen.
4. Die Änderungen werden in den NEORV32 eingebracht.
5. Das geänderte Design wird simuliert bis die gewünschten Signale vorliegen.
6. Das geänderte Design wird synthetisiert und auf dem FPGA ausgeführt. Das Design wird angepasst, bis es auf dem FPGA funktioniert.
7. Die Testprogramme werden ausgeführt. Die Zyklenzahl wird gemessen und verglichen.

Das gewählte Vorgehen zur Programmierung ist in Abbildung 3.1 dargestellt.

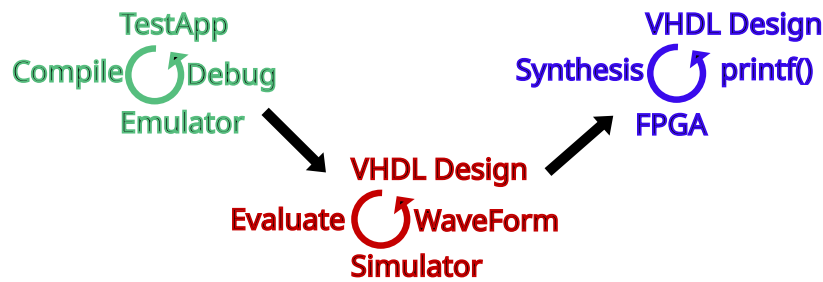


Abbildung 3.1: Entwicklungsschritte

Es gibt drei große Phasen, die in einem Wasserfallprozess streng linear abgearbeitet werden. Bei den drei Phasen handelt es sich um die Erstellung der Testanwendungen, der Änderung des VHDL-Quellcodes während der Simulation und der Erstellung des VHDL-Quellcodes bei der Ausführung auf dem FPGA. Innerhalb jedes Schritts findet ein iteratives Vorgehen statt, das durch den Kreis dargestellt werden soll.

Links des Kreises ist der Name des Vorgehens aufgeführt, das den Quellcode in den kompilierten Zustand überführt, also der Compiler, das Evaluieren oder Synthetisieren. Unterhalb des Kreises ist das Werkzeug aufgelistet, mit dem das Kompilat ausgeführt wird. Rechts des Kreises ist das Mittel aufgelistet, über das die Ausführung bewertet wird.

Im ersten Schritt, bei der Ausführung auf dem Emulator, kann der Quellcode debugged werden, oder es können printf()-Ausgaben bewertet werden. Im Simulator wird die WaveForm über einen WaveForm Viewer ausgewertet. Wenn das Design im letzten Schritt auf dem FPGA läuft, gibt es keinen Debugger mehr für das Design, es werden auch keine Waveforms mehr aufgezeichnet. Es lassen sich lediglich indirekte Schlüsse über die Funktion ziehen, indem man die printf()-Ausgaben bewertet, die ein Programm macht, das auf dem Design ausgeführt wird.

Wie bei jeder Softwareentwicklung ist ein Fehler umso kostspieliger, je später er in der Entwicklung entdeckt wird. Fehler im letzten Schritt bei der Synthese zu finden ist extrem zeitintensiv, da es wenig Werkzeugunterstützung gibt um den Fehler zu lokalisieren.

Der Quellcode wird bearbeitet und getestet, bis er das gewünschte Ergebnis erzielt. In diesem Fall wird in die nächste Phase übergegangen. Dies wird durch die Pfeile zwischen den Phasen verdeutlicht.

3.2 Der NEORV32 Core und dessen Einsatz

Der NEORV32-Chip ist ein VHDL- und auch ein Verilog-Design von Stephan Nolting und anderen Programmierern¹, welches als Open-Source-Projekt unter der BSD-3 Lizenz veröffentlicht ist [14]. Es gibt zum einen sehr gute Do-

¹<https://github.com/stnolting/neorv32/graphs/contributors>

kumentation zu dem Projekt in Form von textueller Dokumentation, dem sogenannten Datenblatt [15]. Zum anderen ist auch der Quellcode selbst sehr gut dokumentiert, einheitlich formatiert und damit gut lesbar.

Am Anfang der Entwicklung wurden zunächst zwei Anwendungen erstellt, die auf dem NEORV32 nach der Erweiterung um eine Matrix-Engine laufen sollen. Eine der Anwendungen verwendet die neu hinzugefügten Anweisungen, die andere nicht. Durch die beiden Anwendungen wird die Laufzeit einer Matrixmultiplikation mit und ohne Matrixerweiterung mess- und vergleichbar.

Die Entwicklung der Anwendungen erfolgt mit Hilfe eines eigenen Compilers und auch des GCC-Compilers. Die Ausführung findet auf einem eigens entwickelten Emulator statt. Dieser Schritt ist in Abbildung 3.1 auf der linken Seite dargestellt. Wenn beide Testprogramme im Emulator funktionieren, geht die Entwicklung in die nächste Phase, bei der das VHDL-Design angepasst wird.

Zunächst ist zu sagen, dass man den VHDL- oder Verilog-Quellcode auf unterschiedliche Arten nutzen kann. Dabei sind die unterschiedlichen Möglichkeiten interessant für unterschiedliche Phasen eines Projekts.

Im Zuge der Durchführung dieser Arbeit wird der NEORV32 Quellcode zunächst durch einen Simulator evaluiert. Dieser Schritt ist in Abbildung 3.1 mittig dargestellt. Das Anwendungsprogramm aus dem ersten Schritt wird in den VHDL-Quellcode in Form eines Speicherdumps eingebettet, damit der Simulator die Testanwendungen ausführt. Als Simulator kommt GHDL in der Version 5.1.1 zum Einsatz. Die Evaluierung ermöglicht dann die Simulation des Designs für einen bestimmten Zeitschritt, beispielsweise für wenige Mikrosekunden.

Die Simulation verwendet Signale, die das Design "stimulieren". Dabei werden die Signale so gewählt, dass ein bestimmter Ablauf provoziert wird. Beispielsweise wird eine Clock an das Design angelegt, damit die NEORV32 CPU mit jedem Clock-Tick einen Schritt ausführt. Der vorbereitete Ablauf kann dann durch die Simulation getestet werden.

Das Design verarbeitet die eingehenden Signale und produziert Ausgangssignale. Der Simulator speichert alle Signaländerungen, sowohl die Eingaben an das Design, als auch dessen Ausgaben, in Value Change Dump (VCD)-Dateien. Diese VCD-Dateien können dann in einem separaten Schritt ausgewertet werden.

Ein sogenannter Waveform-Viewer kann die VCD-Dateien laden und die Signale dann grafisch darstellen, wie es der Abbildung 3.2 zu entnehmen ist.

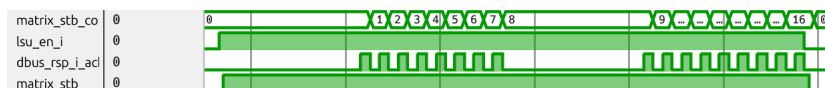


Abbildung 3.2: Waveforms

In Abbildung 3.2 lässt sich erkennen, dass die Load-Store-Unit aktiviert wird,

da das Signal `Isu_en_i` auf einen neuen Zustand wechselt. Dann werden Daten geladen und man sieht die 16 einzelnen Rückmeldungen des Datenbus. Durch diese einzelnen Ereignisse kann ein Rückschluss auf die Funktion des Designs gezogen werden.

Durch visuelle Inspektion der Waveform kann nun geprüft werden, ob das Design so auf die Eingabedaten reagiert wie vom Designer erwartet. Das wichtigste Signal ist die Clocksource, die die CPU taktet, damit die sequentielle Logik Schritte ausführt.

Das NEORV32-Projekt bietet ein vorgefertigtes Skript, welches den Simulator GHDL ausführt. Als solches ist das NEORV32-Projekt für Anfänger sehr gut geeignet, da es diese Funktion bereits automatisch bereitstellt.

Wenn die Simulation die erwarteten Ergebnisse zeigt, dann kann zum finalen Schritt übergegangen werden, der in Abbildung 3.1 auf der rechten Seite dargestellt ist. Das Design wird jetzt synthetisiert statt evaluiert.

Synthese-Werkzeuge führen das sogenannte „Lowering“ durch. „Lowering“ im Sinne des Wortes soll andeuten, dass die Synthese-Werkzeuge eine Transformation von den abstrakten, also hohen Beschreibungen im Quellcode in Designs aus physischen Logikzellen durchführen. Von abstrakt zu physisch findet also ein „Lowering“ statt.

Ein Ausschnitt aus dem Ergebnis der Synthese ist in Abbildung 3.3 zusehen.

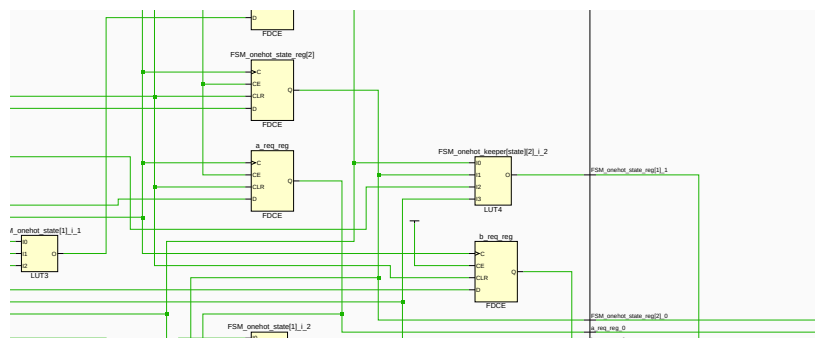


Abbildung 3.3: Schaltplan nach der Synthese

In Abbildung 3.3 sind einzelne Look-Up-Tables zu sehen, die mit LUT3 bzw. LUT4 beschrieben sind. Eine LUT ist eine Tabelle, die das gewünschte Ergebnis einer logischen Funktionen ausgibt, wenn man die passenden Eingangssignale anlegt. Die logischen Funktionen, aus denen das Design im Kern besteht, werden in diesen LUTs abgespeichert und so miteinander kombiniert, dass die Funktion des gesamten Designs realisiert wird. Es ergibt sich ein sehr großes Netz aus Elementen. Ein Mensch kann die Wirkungsweise des Designs auf dieser Detailebene nur sehr schwer verstehen.

Nach der Synthese folgt der Schritt der Implementierung. Die einzelnen LUTs und alle anderen Objekte werden konkret in die Ressourcen eingebettet, die der FPGA bietet. Wo möglich sollte die Toolchain Hardwarelemente nut-

zen, die der FPGA bereits bietet, anstatt Logikzellen für diese Aufgaben zu verschwenden. Wenn das Tool beispielsweise RAM-Speicher im Design erkennt und der FPGA RAM-Speicher in hardwareform besitzt, kann der RAM-Speicher an das Design angebunden werden.

Ein Floor-Plan entsteht. Das Vivado-Synthesewerkzeug kann also tatsächlich grafisch angeben, welche LUTs an welcher Stelle in den FPGA-Chip angeordnet worden sind. Die Implementierung ist exemplarisch in Abbildung 3.4 dargestellt. Man sieht, dass nur ein Teil des FPGA durch das Design belegt ist. Der belegte Teil ist in türkiser Farbe dargestellt.

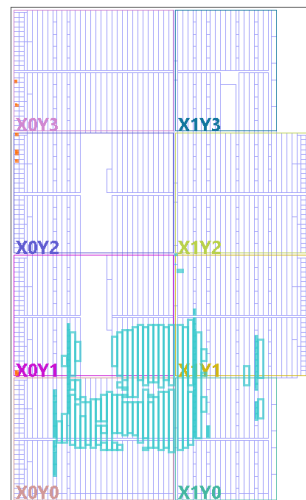


Abbildung 3.4: Floorplan

Sind die Logikzellen bestimmt, lokalisiert und miteinander verbunden, dann kann diese Aufteilung durch eine Bitstream-Datei auf einen FPGA übertragen werden. Dieser Bit-Stream wird von dem FPGA-Chip als Vorgabe verwendet, wie er sich zu konfigurieren hat. Die programmierbare Hardware nimmt jetzt die gewünschte Form an und führt die Funktion des Designs in Hardware aus.

Um die Synthese-Werkzeuge einzusetzen, gibt es ein Parallelprojekt zum NEORV32-Projekt namens neorv32-setups. TCL ist eine Skriptsprache, die zur Steuerung von Abläufen verwendet werden kann. Das neorv32-setups Projekt enthält ein TCL-Script, welches in der Entwicklungsumgebung Vivado in der Version 2025.1 ausgeführt werden kann und dann wiederum ein Vivado-Projekt erzeugt. Wenn dieses erzeugte Vivado-Projekt dann schließlich in Vivado 2025.1 geöffnet wird, dann kann der VHDL-Quellcode übersetzt werden.

Dabei führt Vivado seine gesamte Synthese-Toolchain durch und führt damit das Lowering konkret aus. Das Ergebnis ist die Bitstream-Datei. Vivado kann mit dem FPGA Board ARTY A7 100T sprechen und die Bistream-Datei über eine USB-Verbindung auf den FPGA übertragen. Der FPGA wird sich dem bistream zufolge konfigurieren und ab dann läuft die NEORV32 CPU auf dem FPGA, als ob sie auf einem Wafer produziert worden wäre.

Erfahrungsgemäß verhält sich das synthetisierte Design anders als das simulierte. Die Arbeit ist also nach der Simulation nicht getan. Die Umgestaltung des Designs nach der Simulation bis zur korrekten Funktion nach der Synthese wird nochmals als eigener Arbeitsschritt bei der Durchführung dieser Arbeit antizipiert. In Abbildung 3.1 ist dieser Schritt auf der rechten Seite dargestellt. Auch hier müssen mehrere Zyklen durchgeführt werden, bis die Funktion hergestellt ist.

In dieser Phase wird die Funktion über den UART des NEORV32 getestet, der verwendet wird, wenn die Testanwendung einen `printf()` Aufruf absetzt. Dies ist momentan die einfachste Möglichkeit die Extension zu testen.

3.3 Test-Software

Um zu evaluieren, ob das Design wirklich funktioniert, muss die CPU mit einer Anwendungssoftware betrieben werden, die die neuen Instruktionen wirklich ausführt.

Die Anwendung muss in Form von Maschinencode, also kodierten Befehlen vorliegen. Eine Anweisung ist bei RISC-V eine 32-bit Zahl oder eine 16-bit Zahl, wenn komprimierte Anweisungen unterstützt werden. Die Software kann durch einen Assembler in Maschinencode übersetzt werden. Tatsächlich ist die Kodierung in Maschinencode für RISC-V nicht kompliziert und dazu auch sehr gut dokumentiert.

Trotzdem wurde der Einsatz eines Assemblers für das Vorgehen bei dieser Arbeit nicht gewählt, da das Erstellen eines Assembler-Programms ziemlich aufwändig ist. Es bestand die Furcht davor viel Zeit zu verlieren, wenn das Assembler-Programm angepasst werden muss oder wenn mehr als eine Testsoftware erstellt werden soll. Dann ist für jede Anpassung wieder Assembler-Entwicklung notwendig. Direkte Assembler-Entwicklung wird daher nicht verwendet.

Der erste Ansatz an ein lauffähiges Programm zu kommen, war es, einen C-Compiler für eine Untermenge der Programmiersprache C zu erstellen. Der Vorteil, der sich aus diesem Ansatz ergibt, ist es zum einen volle Kontrolle darüber zu haben, welche RISC-V-Anweisungen der Compiler ausgibt. Damit lässt sich ein RISC-V-Programm erstellen, das tatsächlich auf der NEORV32 CPU ausgeführt werden kann, wenn man nur Anweisungen generiert, die der NEORV32 auch unterstützt.

Zum anderen lassen sich in den Compiler dann auch neue Anweisungen einpflegen, die im RISC-V-Standard noch nicht definiert sind. Zwangsweise kennt ein GCC Compiler diese Anweisungen folglich auch nicht. Die Erweiterung des GCC Compiler ist um ein Vielfaches komplizierter als die Erweiterung eines simplen, selbsterstellten Compilers. Die Unterstützung eigener Anweisungen wird in dieser Arbeit benötigt um die Matrix-Anweisungen verwenden zu können, die im Zuge dieser Arbeit hinzugefügt werden und daher von GCC nicht unterstützt werden.

Da die C-Programmiersprache viele syntaktische Elemente und eine Vielzahl an Datentypen enthält, wurde der unterstützte Umfang der Sprache für den eigenen Compiler sehr stark eingeschränkt. Es gibt nur einen einzigen Datentyp, einen 32 bit Integer. Damit entfällt jegliche Prüfung von Datentypen. Unterstützt werden Funktionsdefinitionen mit Parametern und Aufrufe dieser Funktionen. Dies hat zur Folge, dass der Compiler Stack-Frames erzeugen und auch wieder entfernen können muss. Zusätzlich gibt es for-Schleifen, aber keine if-Anweisung, da für eine Matrixmultiplikation mit dem dyadischen Produkt keine if-Anweisung benötigt wird. Zur Beschreibung der Matrizen unterstützt der Compiler Arrays vom Integer-Datentyp. Es gibt außerdem einen einfachen Präprozessor, der Define-Anweisungen ersetzen und include-Anweisungen ausführen kann.

Der Compiler erzeugt ein Listing aus Assembler-Befehlen, die dann von einem Assembler in Maschinen-Code übersetzt werden müssen. Es gibt keine Ausgabe von Objekt-Dateien und keine Linker-Infrastruktur und damit auch keine Bibliotheken. Alle Programme müssen komplett in Form von Quellcode vorliegen und können nicht aus vorübersetzten Bibliotheken stammen.

Es wurde ein beträchtlicher Zeitaufwand in diesen Compiler gesteckt und die Teilaufgabe wurde tatsächlich gelöst. Eine Testsoftware zur Matrixmultiplikation lässt sich mit dem Compiler aus der Untermenge von C in RISC-V Assembler übersetzen. Die Erstellung mehrerer Testanwendungen und deren schnelle Anpassung ist effizient möglich.

Zu einem späteren Zeitpunkt wurde zusätzlich auch das Verfahren von Stephan Nolting angewandt. Stephan Nolting hat das Problem von neuartigen Anweisungen auch unter Verwendung des bestehenden GCC Compilers gelöst, indem er Inline-Assembly verwendet um damit Maschinen-Code direkt aus dem Compiler ausgeben zu können. Damit hat er die Möglichkeit geschaffen auch Maschinencode für Anweisungen mit dem GCC zu erstellen, obwohl der GCC diese Anweisungen nicht kennt.

Der Nachteil ist, dass die Verwendung etwas kryptisch ist und viel Erfahrung mit dem GCC voraussetzt. Ein weiterer Nachteil des Inline-Assembly Ansatzes ist, dass der Compiler jetzt nicht mehr autonom arbeiten kann, sondern von einem Menschen gesteuert wird.

Damit wird der Compiler davon abgehalten den Quellcode zu optimieren. Des weiteren kann ein Compiler der die Anweisungen nicht kennt, die zum Strip-Mining benötigt werden, auch keinen Code für das Hardware-Software Co-Design generieren.

Für Testfälle ist das Vorgehen zielführend. Für einen industriellen Einsatz muss der Compiler jedoch auf längere Sicht um die Instruktionen erweitert werden um die Optimierung wieder zu ermöglichen.

Das gewählte Vorgehen ist es, die Matrix-Erweiterung mit einer kleinen Testanwendung zu testen, die mit der Strategie von Stephan Nolting und dem GCC übersetzt wird.

3.4 Evaluierung der Performance

Die Hardware-beschleunigte Matrix-Berechnung ergibt nur Sinn, wenn sich ein Vorteil gegenüber einer Matrixmultiplikation ohne die Beschleunigung ergibt. Um zu bestimmen, ob sich die Matrix-Erweiterung positiv auswirkt, soll ein Vergleich zwischen zwei Anwendungen durchgeführt werden.

Die Anwendung A berechnet eine Matrixmultiplikation unter Verwendung von drei For-Schleifen. Die Anwendung B führt dieselbe Matrixmultiplikation unter Verwendung der neu hinzugefügten Anweisungen aus. Für beide Anwendungen wird gezählt wie viele CPU-Zyklen die Ausführung benötigt. Es wird selbstverständlich erwartet, dass die Anwendung B die Aufgabe mit weniger CPU-Zyklen erledigt.

Die Voraussetzungen für dieses Vorgehen ist, dass sowohl Anwendung A als auch Anwendung B in den Instruktions-Speicher des NEORV32 passen.

3.5 Zusammenfassung

In diesem Abschnitt wurde beschrieben, welche Schritte zur Durchführung dieser Arbeit ausgeführt werden müssen. Die Planung und Durchführung der Änderungen am NEORV32-Design, das Erstellen von Testsoftware und das Messen des Ergebnisses wurden als wichtige Punkte identifiziert und beschrieben. Es wird zwischen Simulation und Synthese bzw. Ausführung auf dem FPGA unterschieden.

Zusätzlich wurden die Werkzeuge aufgelistet, mit denen die Entwicklung durchgeführt wird und welche Bedeutung diese Werkzeuge in den einzelnen Schritten haben.

Nachdem das Vorgehen beschrieben ist, werden die Analyseergebnisse am NEORV32 im folgenden Kapitel beschrieben. Darauf folgt ein Kapitel, das identifizierten Änderungen ausführt, gefolgt von einem Kapitel, in dem die Änderungen dann gemessen und ausgewertet werden.

4 Analyse der NEORV32 CPU

Das Ziel dieses Kapitels ist die Analyse des bestehenden NEORV32 Cores. Die wichtigsten Elemente werden beschrieben. Durch die Analyse wird Klarheit darüber geschaffen, wie die Wirkungsweise der NEORV32 CPU ist und wie das System erweitert werden muss um die Matrixmultiplikation zu ergänzen.

4.1 Architektur des NEORV32 Core

Der NEORV32 besitzt den in Abbildung 4.1 dargestellten Aufbau. Diese Abbildung stammt aus dem NEORV32 Datenblatt [15].

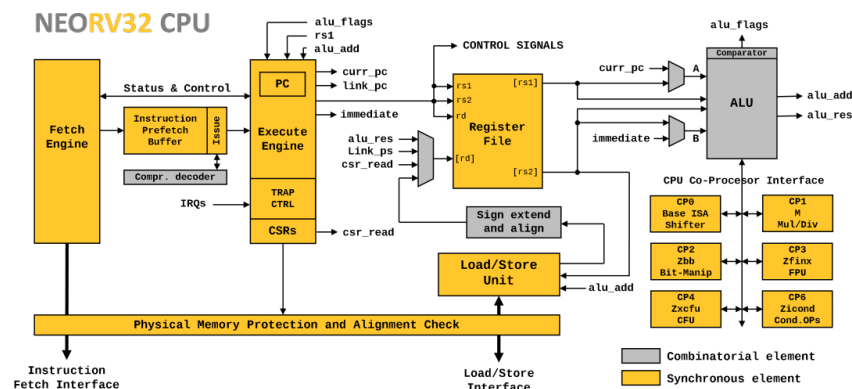


Abbildung 4.1: NEORV32-Architektur (Quelle: [15])

Er ist im Groben aus einem Frontend und einem Backend aufgebaut. Das Frontend ermittelt die nächste Anweisung und das Backend führt diese Anweisung dann aus.

Beide Teile arbeiten in gewisser Weise unabhängig voneinander. Damit kann das Frontend bereits neue Instruktionen puffern während das Backend die aktuelle Anweisung abarbeitet. Im Falle von Sprüngen kann sich der Puffer dann mit den neuen Anweisungen am Sprungziel füllen, während das Backend parallel arbeitet.

4.2 Die Fetch-Engine

Die Fetch-Engine steht am Anfang der Abarbeitung einer Anweisung. Sie ist für das Laden der Anweisung aus dem Speicher zuständig und lädt von der Adresse, die momentan im Program-Counter-Register steht.

Die Fetch-Engine arbeitet unabhängig vom Rest der CPU. Sie implementiert intern eine State-Machine, die Anweisungen puffern kann und sie kann intern Anweisungen laden ohne, dass die Anweisungen akut vom Rest des Systems gebraucht werden.

Aus diesem Grund wurden, lediglich zur Analyse, zwei weitere Signale hinzugefügt, welche in Abbildung 4.2 als "debug_valid_i" und "debug_instr_i" sichtbar gemacht wurden.

Das Signal "debug_instr_i" zeigt die geladene Instruktion an. Das Signal "debug_valid_i" zeigt an, ob die geladene Instruktion gültig ist und verwendet werden sollte oder nicht. In 4.2 sieht man, dass dem Backend Anweisungen angeboten werden während gleichzeitig das Signal "debug_valid_i" nicht aktiv ist. In diesem Fall muss die angebotene Anweisung ignoriert werden.



Abbildung 4.2: NEORV32 Gültige Instruktion

Dieses Wissen ist wichtig, wenn man das Verhalten der CPU anhand einer WaveForm-Ansicht diagnostizieren möchte. Dabei geht man oft von einem Assembler-Programm aus, das von der CPU ausgeführt wird. Es wird erwartet, dass die Assembler-Instruktionen in einer gewissen Reihenfolge abgearbeitet werden. Wenn die Fetch-Engine kreuz und quer Instruktionen lädt, muss man wissen, ob die momentan anliegende Instruktion gültig ist oder nicht um dem Programmablauf folgen zu können.

Es ist des Weiteren zu bemerken, dass die Fetch-Engine nur über einen Cache verfügt, wenn ein Cache konfiguriert wird. In der Standardkonfiguration ist kein Cache für Instruktionen konfiguriert, wie in der Abbildung 4.1 dargestellt. Die Fetch-Engine greift stattdessen direkt auf den Speicher zu.

4.3 CPU-Control und Execution-Engine

Bei einigen CPU-Designs für FPGAs findet man eine Kontrolllogik vor, die abhängig von der momentan geladenen Anweisung Steuersignale für den Datenpfad erzeugt. Der Begriff Datenpfad beschreibt zusammenfassend alle Hardware-Komponenten, die die Anweisungen wirklich berechnen. Beispiele für Komponenten im Datenpfad sind die Arithmetisch-Logische-Einheit oder der Sign-Extender.

Die Kontrolllogik besteht oft aus kombinatorischer Logik und gibt damit di-

stellt. Die Zustände, die in dieser Arbeit neu hinzugefügt werden, sind mit grüner Farbe markiert.

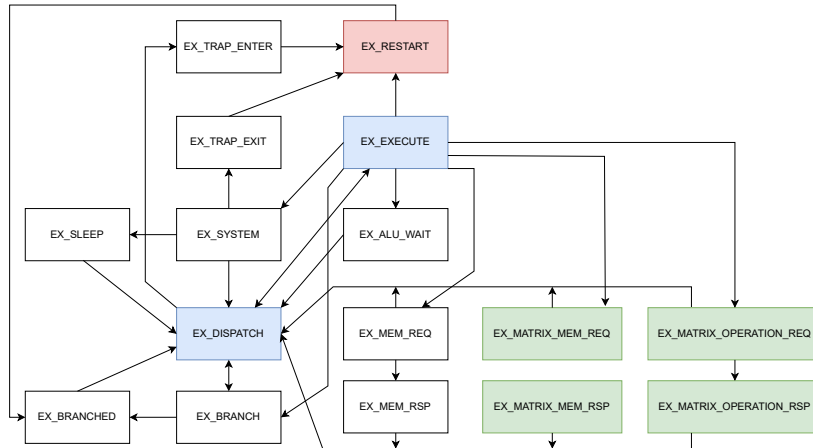


Abbildung 4.4: Struktur der Execution-Engine

Der Start-Zustand ist der EX_RESTART-Zustand und wird nach dem Reset eingenommen. Aus dem Reset und aus EX_RESTART geht die State-Machine in EX_BRANCHED und erst dann in den EX_DISPATCH Leerlaufzustand über. Der Umweg über EX_BRANCHED erlaubt es auf ein Instruction Fetch zu warten.

Im Leerlaufzustand EX_DISPATCH wartet die Execution-Engine, bis ein Trap oder eine Exception eintritt oder bis das Frontend eine gültige Instruktion meldet, die dann abgearbeitet werden muss. Für normale Anweisungen wird der Zustand EX_EXECUTE betreten.

In EX_EXECUTE wird der nächste Zustand abhängig von der Art der Anweisung gewählt. Das heißt EX_EXECUTE ist ein Switch-State, der in mehrere Äste der State-Machine verzweigen kann. Es gibt Zweige für ALU-Operationen, Sprünge und Speicherzugriffe (Load und Store).

Für länger laufende Befehle muss die Execution-Engine auf den Abschluss der Operation warten. Daher sind lange laufende Operationen jeweils durch ein Paar aus Request- und Response-Zuständen modelliert.

Der Request Zustand wird direkt in Richtung Response-Zustand verlassen. Der Response-Zustand wird erst in Richtung EX_DISPATCH verlassen, wenn das eingehende Response-Signal aus der Sensitivitätsliste aktiviert worden ist. Im Response-Zustand werden die erzeugten Daten betrachtet und Signale zurückgesetzt.

4.4 Die Load-Store-Unit

Die State-Machine der Execution-Engine produziert eine Vielzahl an Signalen, die andere Entitäten des VHDL-Designs aktivieren und mit Eingabedaten versorgen. Ein wichtiger Teil des Designs ist die sogenannte Load-Store-Unit.

Die Load-Store-Unit bildet das Bindeglied zwischen der Execution-Engine und dem Datenspeicher. Sie hat zur Aufgabe Bus-Requests zu erstellen und Bus-Responses zu verarbeiten. Diese Bus-Requests sind mit der Adresse des Datenspeichers versehen. Die Abbildung 4.5 stellt die Load-Store-Unit im Gesamtkontext dar.

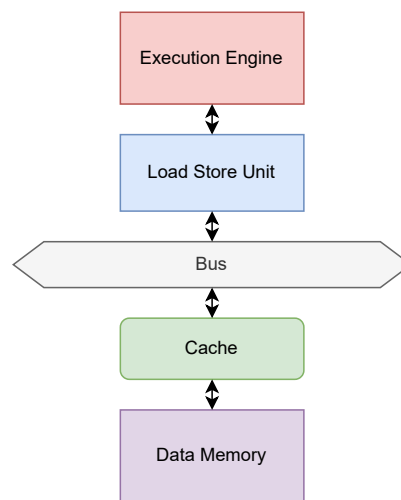


Abbildung 4.5: Die Load-Store-Unit und ihre Schnittstellen

Die Bus-Requests werden auf dem Bus innerhalb der NEORV32 CPU ausgeführt. Ein Teilnehmer am Bus ist z.B. der Datenspeicher. Mit Datenspeicher ist der Teil des Speichers gemeint, der die Werte der Variablen speichert, also konkret keinen Quellcode sondern eben Daten enthält.

Der Datenspeicher wird über Load-Anweisungen gelesen und produziert damit Daten für die Register oder er wird durch Store-Anweisungen geschrieben. Die Aufgabe der Load-Store-Unit besteht darin die Signale der Execution-Engine in passende Bus-Requests für Load- oder Store Anweisungen zu übersetzen.

Dabei ist folgende Besonderheit zu beachten. Die Bus-Requests, die von der Load-Store-Unit erzeugt werden sind in der Datei `neorv32_top.vhd` mit dem Data-Cache verbunden. Damit unterstützt die NEORV32 CPU eine Cache-Hierarchie für Daten. Anstatt direkt auf den Speicher zuzugreifen, greift die Load-Store-Unit immer zunächst in den Cache. Wenn ein Cache-Hit stattfindet, ergibt sich ein Geschwindigkeitszuwachs.

4.5 Zusammenfassung

In diesem Abschnitt wurde das Zielsystem, die NEORV32 CPU, analysiert. Das Ziel war es die Komponenten kennenzulernen, die erweitert werden müssen um die Matrixmultiplikation hinzuzufügen.

Die NEORV32 CPU weicht von einem traditionellen Design mit Kontrolllogik, Datenpfad und Pipeline ab. Das spezielle Design wurde beschrieben. Es besteht aus einem Frontend und einem Backend. Die geplanten Änderungen finden im Backend statt.

Das Backend wurde anhand seiner relevanten Bestandteile Execution-Engine, Load-Store-Unit, Datenbus, Cache und RAM-Speicher beschrieben. Die einzelnen Bestandteile wurden besprochen.

Gerüstet mit dem Wissen um den inneren Aufbau des Backend, kann nun im nächsten Kapitel definiert werden, welche neue Hardware für die Matrixmultiplikation ergänzt werden muss.

5 Erweiterung der NEORV32-CPU

In diesem Kapitel werden die Erweiterungen an den Entitäten des vorgegebenen Designs beschrieben, die gemacht wurden, um die Matrix-Multiplikation zu ermöglichen.

5.1 Die Matrix-Register

Die Matrixregister bestehen in der jetzigen Version aus 16 32-Bit-Elementen und sind nicht für andere Datentypen konfigurierbar und nicht für größere Längen miteinander kombinierbar, wie es bei der RVV der Fall ist.

Die Register sind wie folgt definiert.

```
type ram_type is array (0 to MLEN-1)
  of std_ulogic_vector(XLEN-1 downto 0);
subtype matrix_ram_addr is integer range 0 to MLEN-1;

signal matrix_ram_0 : ram_type;
signal matrix_ram_1 : ram_type;
signal matrix_ram_2 : ram_type;
```

Quellcode 5.1: Die Matrix-Register

5.2 Die Instruktionen

Dieser Abschnitt beschreibt, welche Instruktionen hinzugefügt werden sollen und warum diese Instruktionen ausgewählt worden sind. Es wird beschrieben, wie man diese neuen Instruktionen im Übersetzungsprozess eines existierenden Compilers verwenden kann.

Um die Matrixmultiplikation durchzuführen sollen neue Instruktionen hinzugefügt werden, die es noch nicht bereits in anderen Extensions für RISC-V gibt. Die Instruktionen sollen das Strip-Mining für Matrizen ermöglichen und sich dabei an der RVV-Extension orientieren. Daher folgt zunächst ein kurzer Rückblick zur Vektor-Extension.

Bei der Diskussion des Strip-Mining bei der RVV-Extension wurde die Instruktion `set(i)vl` besprochen, mit der die Vektor-Engine für Datentypen und gewünschte Datenlängen konfiguriert wird. Des Weiteren wurden die Instruktionen `vle32.v`, `vadd.vv` und `vse32.v` im Rahmen eines Beispiels verwendet.

vle32.v lädt Daten aus dem Daten-Speicher in die Vector-Register (Load-Operation), vadd.vv ist die SIMD-Instruktion, die zwei Vektoren elementweise parallel miteinander addiert hat und vse32.v speichert die Daten schließlich in den Daten-Speicher zurück (Store-Operation)

Für die Matrix-Extension werden die Instruktion mle32, mmul32 und mse32 hinzugefügt. Dabei ist mle32 die Load-Operation, mmul32 die SIMD-Instruktion und mse32 die Store-Operation. Ein Äquivalent zur set(i)vli ist nicht vorgesehen, da die Matrix-Engine zum jetzigen Stand keine Konfigurationsmöglichkeiten anbietet.

Die NEORV32 CPU besitzt eine Vielzahl an Beispielen, die sich im Software-Unterraster befinden. Jedes Beispiel kann von Funktionen Gebrauch machen, die über die neorv32.h Header-Datei eingebunden werden können.

Teil dieser Funktionen sind sogenannte Intrinsics. Ein Intrinsic ist eine Funktion, die in der Programmiersprache C geschrieben ist und die Programmiersprache C damit um Funktionalität erweitert, die thematisch zu einem anderen Themengebiet gehört.

Ein Beispiel dafür sind die Intrinsics the X-Window Systems [5]. Hier wird eine einfach zu verwendende Schnittstelle zur Softwareentwicklung mit dem X-Window System für die Programmiersprache erstellt.

Die Intrinsics des NEORV32 sind eine Erweiterung der Programmiersprache C um RISC-V Instruktionen aus C-Quellcode heraus auszugeben. Ein Beispiel für die Verwendung ist das folgende Testprogramm.

```

1  #include <neorv32.h>
2
3  uint32_t matrix_a[16]
4  = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
5
6  uint32_t matrix_b[16]
7  = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
8
9  int main() {
10
11     uint32_t *matrix_a_addr = &matrix_a[0];
12     uint32_t *matrix_b_addr = &matrix_b[0];
13
14     uint32_t matrix_unit_a = 0;
15     uint32_t matrix_unit_b = 0;
16
17     matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr, 0b010
18                                         , 0b1011011);
19     matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, matrix_b_addr, 0b010
20                                         , 0b1011011);
21
22     uint32_t matrix_mul = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr,
23                                               0b100, 0b1011011);
24
25     matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr, 0b001
26                                         , 0b1011011);
27     matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, matrix_b_addr, 0b001
28                                         , 0b1011011);
29
30     for (int i = 0; i < 16; i++) {
31         neorv32_uart0_printf("After Matrix A %d\r\n", matrix_a_addr[i
32                               ]);
33     }
34 }
```

```
27  }
28  for (int i = 0; i < 16; i++) {
29      neorv32_uart0_printf("After Matrix B %d\r\n", matrix_b_addr[i
30                          ]);
31  }
32  return 0;
33 }
```

Quellcode 5.2: Hardwarebeschleunigte Multiplikation

In der Zeile 17 ff wird als Intrinsic "CUSTOM_INSTR_I_TYPE" für I-Type RISC-V Instruktionen verwendet. Aus jeder Zeile mit einem "CUSTOM_INSTR_I_TYPE" Aufruf wird eine RISC-V Assembler Instruktion generiert.

Die Verwendung ist schwer zu verstehen, daher wird jetzt beschrieben, wie die Parameter zum Intrinsic zu interpretieren sind. Der letzte Parameter ist der Op-Code. Für die Matrix-Extension wurde in dieser Arbeit der Op-Code 0b1011011 gewählt. Innerhalb dieses Opcodes werden dann durch den vorletzten Parameter die drei Anweisungen mle32, mmul32 und mse32 kodiert.

Es ist noch abschließend zu erklären, weshalb Intrinsics verwendet werden müssen. Der grundsätzliche Ablauf bei der Verwendung der Programmiersprache C ist, dass ein Compiler den Quellcode der höheren Programmiersprache in Eigenregie in Assembler-Code übersetzt. Dabei wählt er die Anweisungen selbst aus. Optimierende Compiler führen diese Aufgabe in vielen Fällen besser aus, als es ein Mensch könnte.

Im Fall dieser Arbeit ist dieses Vorgehen nicht möglich. Der C-Compiler kennt die neu definierten Instruktionen mle32, mmul32 und mse32 schlicht nicht und wird sie daher nur ausgeben, wenn man den Compiler explizit dazu anweist. Diese explizite Anweisung passiert durch die Intrinsics.

Intern sind die Intrinsics durch Inline Assembly implementiert. Das bedeutet, dass dem C-Compiler exakt vorgegeben wird, welchen inline assembly Code er auszugeben hat. Im Beispiel-Code werden die Instruktionen mle32, mmul32 und mse32 per Opcode generiert.

Im Zuge dieser Arbeit ist ein einfacher Compiler erstellt worden, der auch dazu verwendet werden kann, die neuen Instruktionen auszugeben. Dieser Compiler wurde verwendet, bis der Weg über die Intrinsics des NEORV32 entdeckt wurde. Die Intrinsics erlauben es State-of-the-Art-Compiler wie den GCC zu verwenden. Dies ist sehr viel komfortabler als einen eigenen Compiler zu pflegen.

5.3 Erweiterung der Execution-Engine

Bis zu diesem Punkt liegt ein Beispiel-Programm mit den neuen Matrix-Instruktionen vor, welches in die NEORV32 CPU geladen und ausgeführt

werden kann. Damit wurde der Software-Anteil der Erweiterung beschrieben. Es folgt nun eine Diskussion der Erweiterungen der Hardware.

In diesem Abschnitt wird die Execution-Engine erweitert, so dass sie die neuen Instruktionen tatsächlich ausführt, wenn sie von dem Frontend präsentiert werden.

Eine Übersicht aller Komponenten ist in Abbildung 5.1 zu sehen.

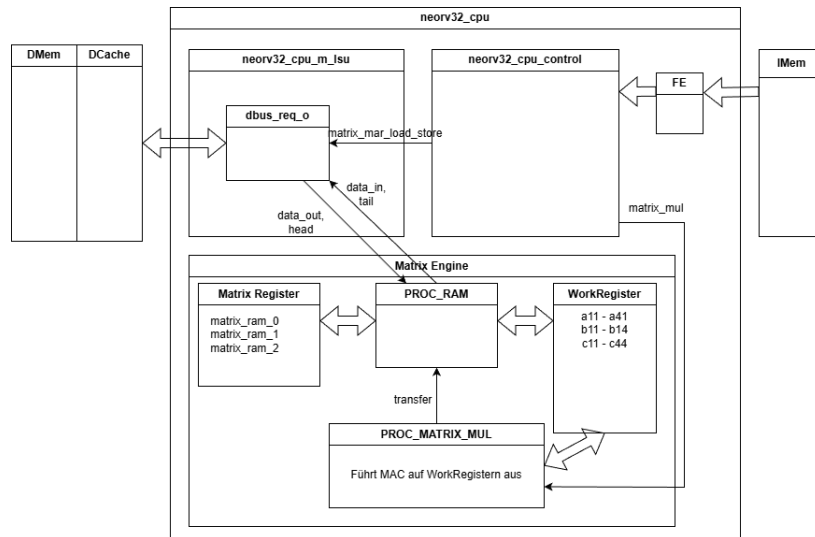


Abbildung 5.1: VHDL Entities

Die Instruktionen werden über das Frontend (FE) aus dem Anweisungsspeicher (IMem) geladen und an die Kontrolllogik weitergegeben. Die Daten der Matrix-Elemente kommen aus dem Datenspeicher (DMEM) und durchlaufen dabei den Daten-Cache (DCache).

Die Komponente `neorv32_cpu_control` gibt die Kontrollsignale `matrix_mar_load_store` und `matrix_mul` aus, wenn die Instruktionen `mle32`, `mse32` bzw. `mmul32` ausgeführt werden müssen.

Im Falle von `mle32` (Load) werden die Signale `data_out` und `head` verwendet um die 16 Matrixelemente vom Bus-Request an die richtige Stelle in die Matrix-Register zu übertragen. `PROC_RAM` übernimmt diese Koordinationsaufgabe.

Wird eine `mmul32` Anweisung ausgeführt und damit das `matrix_mul`-Signal von der Control-Logik gesetzt, dann beginnt der Prozess `PROC_MATRIX_MUL` mit der Multiplikation. Die Matrix-Register werden in die Arbeitsregister übertragen. Nach der Bearbeitung wird das Ergebnis aus den Arbeitsregistern wieder zurück in die Matrixregister übertragen.

Analog zu `mle32` werden die Signale `data_in` und `tail` für `mse32` (Store)-Anweisungen verwendet um Daten aus den Matrixregistern zurück in den Datenspeicher und den Cache zu schreiben.

Im Folgenden werden die Änderungen im Detail beschrieben.

Als erstes wird der neue Matrix-Opcode aus der Erkennung illegaler Opcodes ausgeschlossen.

```
-- MATRIX EXTENSION
when opcode_matrix_c =>
    illegal_cmd <= '0';
```

Quellcode 5.3: Test auf illegale Opcodes

Der Execution-Engine wird ein neues Signal "matrix_operation_wait_i" in die Sensitivitätsliste eingefügt. Dadurch kann sie das Ende der Matrixberechnung erkennen und in den nächsten Zustand übergehen.

```
execute_engine_fsm_comb: process(
    ...
    matrix_operation_wait_i
    ...
)
```

Quellcode 5.4: Erweiterung der Sensitivitätsliste

Der ALU wird beigebracht, dass sie den Operand B als Immediate verwenden muss, sobald sie den Matrix-Extension Opcode sieht. Der Grund ist, dass die Load- und Store-Matrix-Anweisungen die Register-Id als Immediate-Wert verwenden um zu ermitteln, welches Matrix-Register Sie lesen und schreiben sollen. Damit muss die ALU diesen Immediate-Wert weitergeben, sonst ist er verloren und kann nicht mehr verwendet werden.

```
-- ALU operand B: is immediate? --
case opcode is
    -- add matrix opcode here because the mle32, mse32 instructions
    -- contain an immediate
    -- that needs to be added to the register encoded in the
    -- instruction
    when opcode_matrix_c | opcode_alui_c | opcode_lui_c ...
        ctrl_nxt.alu_opb_mux <= '1';
    when others =>
        ctrl_nxt.alu_opb_mux <= '0';
end case;
```

Quellcode 5.5: Übernahme der Immediate-Parameter

Im EX_DISPATCH-Zustand wird die Matrixmultiplikation gestoppt.

```
matrix_mul_o <= '0';
```

Quellcode 5.6: Matrixmultiplikation beenden

Der EX_EXECUTE-Zustand wird um die Behandlung der Matrix-Instruktionen erweitert.

```

-- Matrix Extension --
when opcode_matrix_c =>

case funct3_v is

    -- load from memory into matrix unit --
    when funct3_mle32_c =>
        -- go to the specific states for the matrix extension.
        -- The states are inserted in order to load all the elements
        -- of a matrix in a loop
        exe_engine_nxt.state <= EX_MATRIX_MEM_REQ;

    when funct3_mse32_c =>
        exe_engine_nxt.state <= EX_MATRIX_MEM_REQ;

    when funct3_mmul32_c =>
        -- go to the state that processes matrix operations
        exe_engine_nxt.state <= EX_MATRIX_OPERATION_REQ;

    when others => -- undefined

end case;

```

Quellcode 5.7: Erweiterung des EX_EXECUTE Zustands

Für die drei Instruktionen mle32, mse32 und mmul32 werden neue Zustände in die Execution-Engine eingefügt, so dass diese Zustände aus EX_EXECUTE angesprungen werden können.

Für die Matrix Load- und Store-Instruktionen wird der Zustand EX_MATRIX_MEM_REQ hinzugefügt.

```

-- trigger a loop of memory requests for the matrix
when EX_MATRIX_MEM_REQ =>

    -- tell the matrix unit to come online and process the request
    ctrl_nxt.lsu_m_en    <= '1';

    if (or_reduce_f(trap_ctrl.exc_buf(exc_ialign_c downto
        exc_iaccess_c)) = '0') then

        -- memory request if no instruction exception

        -- tell the load store unit to process a request
        ctrl_nxt.lsu_req    <= '1';

        -- forward the immediate from the instruction
        matrix_immediate_o <= immediate;

        -- rs1 (source register 1) carries the address to load from
        / store to
        matrix_mar_load_store_o <= rf_rs1_i;

        exe_engine_nxt.state <= EX_MATRIX_MEM_RSP;

    else
        -- On Error
        exe_engine_nxt.state <= EX_DISPATCH;
    end if;

```

Quellcode 5.8: Erweiterung des EX_MATRIX_MEM_REQ Zustands

Im Zustand EX_MATRIX_MEM_RSP wird auf die Rückmeldung gewartet. Wenn die Rückmeldung anhand des Signals "lsu_wait_i" erfolgt ist, dann wird der geladene Wert über das write-back signal (wb) in die Register-File (RF) geschrieben. Dann wird die Matrix-Engine deaktiviert.

```
when EX_MATRIX_MEM_RSP => -- wait for memory response
    if (lsu_wait_i = '0') then
        -- write-back to RF if read operation (won't happen in case
        -- of exception)
        ctrl_nxt.rf_wb_en    <= (not ctrl.lsu_rw) or ctrl.lsu_rvs or
            ctrl.lsu_rmw;

        -- tell the matrix unit to stay offline
        ctrl_nxt.lsu_m_en    <= '0';

        exe_engine_nxt.state <= EX_DISPATCH;
    end if;
```

Quellcode 5.9: Ende der Speichertransaktionen

Für Matrixoperationen gibt es die beiden neuen Zustände EX_MATRIX_OPERATION_REQ und EX_MATRIX_OPERATION_RSP. Mit Matrix-Operationen ist die Matrixmultiplikation gemeint. Da die Matrixmultiplikation nicht in einem einzigen Schritt ausgeführt ist, muss die Execution-Engine auf die Matrix-Engine warten. Daher ergibt sich die Notwendigkeit für einen Request- und einen Response-State.

```
when EX_MATRIX_OPERATION_REQ => -- matrix add operation
    if (or_reduce_f(trap_ctrl.exc_buf(exc_ialign_c downto
        exc_iaccess_c)) = '0') then
        matrix_mul_o <= '0';

        case funct3_v is
            when funct3_mmul32_c =>
                matrix_mul_o <= '1';

            when others => -- undefined
        end case;

        matrix_immediate_o <= immediate;

        -- next state
        exe_engine_nxt.state <= EX_MATRIX_OPERATION_RSP;
    else
        exe_engine_nxt.state <= EX_DISPATCH;
    end if;

when EX_MATRIX_OPERATION_RSP =>

    -- next state
    if (matrix_operation_wait_i = '0') then
```

```
matrix_mul_o <= '0';  
  
exe_engine_nxt.state <= EX_DISPATCH;  
  
end if;
```

Quellcode 5.10: Warten auf das Ende der Multiplikation

Diese beiden Zustände sind sehr simpel. Im Request-State wird das Signal `matrix_mul_o` aktiviert und dann in den Response-State übergegangen. Im Response-State wird auf das Signal `matrix_operation_wait_i` gewartet und dann wird die Matrixmultiplikation deaktiviert.

5.4 Erweiterung der Load-Store-Unit

Der nächste Schritt ist die Erweiterung der Load-Store-Unit. Die Load-Store-Unit ist innerhalb der CPU-Entity definiert.

Die Aufgabe besteht darin, Daten aus dem Datenspeicher in die neuen Matrix-Register zu laden. Hier wäre es möglich die Matrix-Engine als eigenen Bus-Teilnehmer an den Bus zu binden. Die Matrix-Engine hätte dann die Möglichkeit Daten aus dem Datenspeicher über den Bus zu laden.

Der Nachteil an diesem Ansatz ist, dass die Matrix-Engine und die Load-Store-Unit nicht mehr synchronisiert sind. Die Matrix-Engine wird Daten direkt aus dem RAM laden ohne auf den Cache zuzugreifen. Die Load-Store Unit hingegen verwendet den Cache weiterhin. Da die Load-Store-Unit verantwortlich für den Rest der CPU ist, greift die CPU dann insgesamt auf veraltete Daten zu.

Um die Synchronisierung wieder herzustellen, müsste die Load-Store-Unit ihren Cache nach Ausführung einer Matrix- Load-Store-Operation invalidieren, da dieser veraltete Daten enthalten könnte. Daraufhin muss sich der Cache erst wieder aufwärmen und die Performance der CPU leidet folglich.

Aus diesem Grund wurde das Laden der Matrix-Engine direkt in die LSU integriert. Wenn eine Matrixoperation eine Matrix fertig berechnet hat und diese Daten dann wieder in den Datenspeicher übertragen werden, dann durchlaufen die Daten den Cache der LSU und wärmen den Cache damit für den Rest der CPU mit aktuellen Daten auf. Das bedeutet, dass der Rest der ausgeführten Anwendung dann direkt auf die gecachte Matrix zugreifen kann anstelle die Matrix erst wieder aus dem Datenspeicher laden zu müssen.

Damit werden Daten über die Load-Store-Unit geladen, damit die Daten im Cache landen. Die Load-Store-Unit erzeugt Bus-Anfragen wenn Sie Daten laden und speichern soll. Da ein Matrix-Register aus 16 Elementen besteht, werden beide Prozesse für Load und für Store erweitert um in einer Schleife 16 Elemente laden zu können.

Der Load-Prozess wird für Matrix-Load-Anweisungen um eine State-Machi-

ne erweitert. Der State ML_IDLE deaktiviert die Load-Operation. Der Zustand ML_INIT setzt den Zähler auf 0 und geht in den ML_PROCESS Status über. Im ML_PROCESS Status erfolgen 16 Load Operationen um das Matrix-Register zu füllen. Alle 16-Operation durchlaufen den Cache was positiv ist. Der ML_INCREMENT wird abwechselnd mit ML_PROCESS ausgeführt und zählt den Zähler nach oben.

Sind die 16 Operationen ausgeführt, geht die State-Machine zurück nach ML_IDLE.

Ein etwa gleichartiges Vorgehen wird auch für die Matrix-Store-Operation gewählt, welche 16 Elemente zurückschreiben muss.

Der neue PROC_RAM-Prozess produziert Steuersignale für den RAM-Speicher der Matrix-Engine. Abhängig von den Zählern, die in den Operationen Load und Store verändert werden, produziert der PROC_RAM Prozess Signale, die die Load-Store-Unit verlassen und in der CPU mit dem RAM für die Matrix-Engine verbunden sind. Damit wird der RAM der Matrix-Engine gesteuert und gibt entweder Daten für ein Store zurück oder speichert Daten für eine Load-Anweisung. Die eigentlichen Daten kommen aus dem Daten-Speicher der CPU und durchlaufen den Cache der LSU.

5.5 Matrix-Engine

In die Datei "neorv32_cpu.vhd" wird die Matrix-Engine eingebaut.

Die Matrix-Engine ist schematisch in Abbildung 5.2 dargestellt.

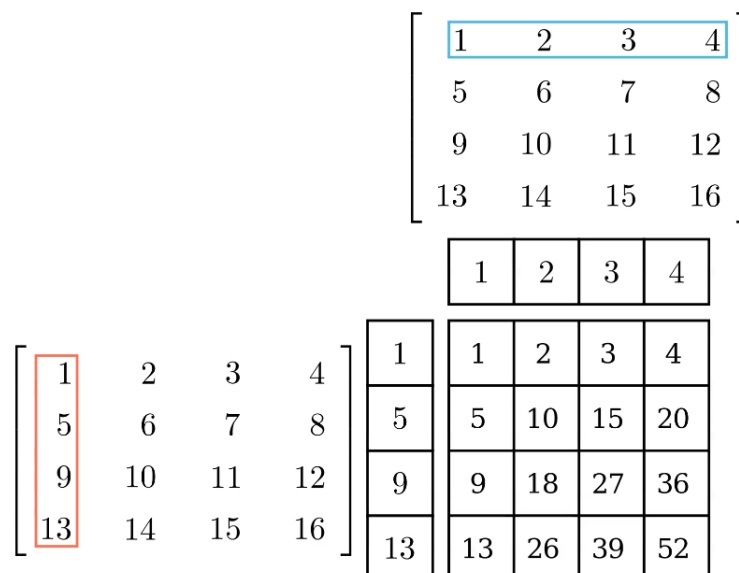


Abbildung 5.2: Matrix Engine

Auf der linken Seite und am oberen Bildrand befinden sich die beiden Matrixregister für die Matrizen A und B. Genau in der Mitte der Abbildung befindet sich die MAC-Einheit, in der die dyadischen Produkte aufaddiert werden und in der sich das Ergebnis, also die Matrix C bildet. Zwischen den Matrix-Registern und der MAC-Einheit befinden sich die Arbeitsregister.

Um die Matrix-Engine zu verwenden, sind folgende Schritte notwendig:

1. Die Matrix Register A und B müssen aus dem Speicher bzw. dem Cache mit Matrix-Daten geladen werden.
2. Die MAC-Einheit muss mit Nullen gefüllt werden.
3. Die Arbeitsregister müssen mit einer Spalte der Matrix A und einer Zeile der Matrix B gefüllt werden.
4. Die MAC-Einheit wird verwendet um das dyadische Produkt zu berechnen und zu speichern.
5. Die vorhergehenden zwei Schritte müssen insgesamt viermal wiederholt werden um zwei 4x4 Matrizen zu multiplizieren.
6. Der Inhalt der MAC-Einheit muss zurück in den Speicher und den Cache übertragen werden, damit er dann der Applikation zur Verfügung steht.

Die Engine besteht zunächst aus dem PROC_RAM Prozess.

```

PROC_RAM : process(clk_i)
  variable l : line;
begin
  if (rstn_i = '0') then
    ...
  elsif rising_edge(clk_i) then
    if (transfer = '1') then
      -- store the matrixmult result into the first matrix engine
      register
      matrix_ram_0(0) <= std_ulogic_vector(c11(31 downto 0));
      matrix_ram_0(1) <= std_ulogic_vector(c12(31 downto 0));
      ...
      matrix_ram_0(15) <= std_ulogic_vector(c44(31 downto 0));

      -- store the matrixmult result into the second matrix engine
      register
      matrix_ram_1(0) <= std_ulogic_vector(c11(31 downto 0));
      matrix_ram_1(1) <= std_ulogic_vector(c12(31 downto 0));
      ...
      matrix_ram_1(15) <= std_ulogic_vector(c44(31 downto 0));
    end if;
    if (load = '1') then
      -- data_out is the output of the LoadStoreUnit (LSU) and it
      contains the RAM/cache value
      -- which is loaded into the MatrixEngine
      if (to_integer(unsigned(matrix_immediate)) = 0) then
        matrix_ram_0(head) <= data_out;
      else
        matrix_ram_1(head) <= data_out;
      end if;
    end if;
  end if;

```

```

end if;
if (store = '1') then
  if (to_integer(unsigned(matrix_immediate)) = 0) then
    data_in <= matrix_ram_0(tail);
  else
    data_in <= matrix_ram_1(tail);
  end if;
end if;
end if;
end process PROC_RAM;

```

Quellcode 5.11: Datentransfer der Matrixengine

Dieser Prozess hat die Aufgabe die Matrixregister zu speichern und zu laden und die Rechenergebnisse der Matrixmultiplikation in (momentan beide) Zielmatrixregister zu übertragen (transfer). Beim Speichern und Laden der Matrixregister werden die Signale head und tail verwendet, die von der LSU ausgegeben werden. Diese Signale beschreiben das Element, das geschrieben werden soll.

Wenn die LSU Daten über den Bus aus dem RAM bzw. dem Cache erhält, dann wird gleichzeitig dazu das entsprechende Element im Register geschrieben oder gelesen. Die Synchronisation der LSU mit dem PROC_RAM Prozess geschieht über die Signale head und tail.

Der zweite Teil der Matrix-Engine besteht aus der Berechnung des dyadischen Produkts. Dazu sind bei 16 Elementen pro Matrixregister und damit mit 4x4 Matrizen vier Schritte notwendig. In den vier Schritten werden Zeilen aus der Matrix A und Spalten aus der Matrix B miteinander verrechnet. Mit vier Zeilen und vier Spalten müssen also vier Berechnungen pro Schritt ausgeführt werden.

Um die Berechnung auszuführen wird eine State-Machine verwendet. Die State-Machine durchläuft die Zustände MM_RESET, MM_INIT_1, und MM_PROCESS_1, bis MM_INIT_4, MM_PROCESS_4 und endet mit MM_STORE, MM_DONE.

Der MM_RESET Zustand setzt die Matrixregister auf 0, wenn das matrix_mul-Signal auf 1 gesetzt wird. Solange matrix_mul auf 0 steht, verbleibt der Wert der letzten durchgeführten Matrixmultiplikation in den Matrixregister stehen. Der Grund ist, dass man die Werte in den Registern halten muss um sie in den RAM zurückschreiben zu können und dies einige CPU-Zyklen dauert. Das matrix_mul-Signal wird von der Kontrolllogik der CPU gesetzt, wenn ein mmul32-Befehl ausgeführt wird und steht ansonsten auf 0.

Wenn das matrix_mul-Signal gesetzt wird, werden die Matrixregister mit 0 überschrieben und die State Machine geht in den nächsten Zustand über. Es folgen vier Schritte, die jeweils aus einem INIT und einem PROCESS-Schritt bestehen. Alle vier Schritte sind äquivalent, es wird daher nur ein Schritt beschrieben.

Im INIT-Schritt wird eine Zeile aus der A-Matrix und eine Spalte aus der B-Matrix in die Arbeitsregister geladen:

```

when MM_INIT_1 =>
  a11 <= unsigned(matrix_ram_0(0));
  a21 <= unsigned(matrix_ram_0(4));
  a31 <= unsigned(matrix_ram_0(8));
  a41 <= unsigned(matrix_ram_0(12));

  b11 <= unsigned(matrix_ram_1(0));
  b12 <= unsigned(matrix_ram_1(1));
  b13 <= unsigned(matrix_ram_1(2));
  b14 <= unsigned(matrix_ram_1(3));

  mat_mul_engine_state <= MM_PROCESS_1;

```

Quellcode 5.12: Befüllen der Arbeitsregister

Im folgenden MM_PROCESS Schritt erfolgt die sogenannte Multiply-Accumulate-Operation. Mit Multiply-Accumulate ist gemeint, dass eine Multiplikation als Kernoperation ausgeführt wird, aber das Ergebnis dieser Multiplikation nur ein Teilinkrement einer größeren Operation darstellt. In diesem Fall ist die größere Operation die Matrixmultiplikation, die sich aus vier addierten (kummulierten) dyadischen Produkten zusammensetzt. Die dyadischen Produkte werden in den Akkumulator-Speichern c11 bis c44 aufgenommen.

```

when MM_PROCESS_1 =>

  c11 <= c11 + (a11 * b11);
  c12 <= c12 + (a11 * b12);
  c13 <= c13 + (a11 * b13);
  c14 <= c14 + (a11 * b14);

  c21 <= c21 + (a21 * b11);
  c22 <= c22 + (a21 * b12);
  c23 <= c23 + (a21 * b13);
  c24 <= c24 + (a21 * b14);

  c31 <= c31 + (a31 * b11);
  c32 <= c32 + (a31 * b12);
  c33 <= c33 + (a31 * b13);
  c34 <= c34 + (a31 * b14);

  c41 <= c41 + (a41 * b11);
  c42 <= c42 + (a41 * b12);
  c43 <= c43 + (a41 * b13);
  c44 <= c44 + (a41 * b14);

  -- next state
  mat_mul_engine_state <= MM_INIT_2;

```

Quellcode 5.13: Multiply Accumulate Schritt

Nach vier MM_INIT- und vier MM_PROCESS-Zuständen ist die Matrixmultiplikation durchgeführt. Das Ergebnis wird schließlich im MM_STORE Schritt in das Zielregister übertragen. Dazu wird das Signal "transfer" auf 1 gesetzt.

```

when MM_STORE =>
  -- enable transfer in the RAM part
  transfer <= '1';

```

Quellcode 5.14: Matrixengine Transfer Flag

Das „transfer“-Signal wird im PROC_RAM Prozess gelesen und führt dort dazu, dass die MAC - Speicherzellen c11 bis c44 in das Ziel-Matrixregister kopiert werden. Dieser Sachverhalt wurde bereits weiter oben bei der Beschreibung des PROC_RAM Prozesses beschrieben.

Es soll an dieser Stelle kurz bewertet werden, ob die Berechnung der vier dyadischen Produkte auf diese Weise sinnvoll ist oder nicht. Als erstes ist diese Lösung funktional und diese Qualität ist sicherlich sehr wichtig. Ein VHDL Design zu erstellen, dass auch wirklich nach der Synthese auf einem FPGA funktioniert ist speziell für Anfänger immer ein großer Erfolg.

Als zweites ist der Quellcode vom Standpunkt der VHDL-Synthese aus zu betrachten. Aus Sicht einer höheren Programmiersprache ist die Implementierung nicht sehr sinnvoll, da man beispielsweise durch das Strassen-Verfahren einen Geschwindigkeitsvorteil erzielen kann. VHDL ist aber keine höhere Programmiersprache, sondern eine Hardwarebeschreibungssprache. Dinge können in Hardware tatsächlich parallel ablaufen, wenn Elemente mehrfach synthetisiert werden.

Die 16 Multiplikationen pro MM_PROCESS-Zustand werden hoffentlich von der Toolchain auf Hardware-Blöcke innerhalb des FPGA abgebildet und laufen damit effizient und wirklich parallel ab. Wenn der FPGA mehrere Hardware-Blöcke für die Multiplikation hat, dann können diese Multiplikationen tatsächlich parallel ausgeführt werden.

Zusätzlich wäre es denkbar die vier MM_INIT und MM_PROCESS-Schritte nicht sequentiell an eine State-Machine zu binden sondern sich über die parallele Ausführung der vier Schritte Gedanken zu machen. Dazu muss die Aufgabe gelöst sein eine parallele Akkumulation der Teilergebnisse in die Speicher c11 bis c44 zu bewerkstelligen. Ein vierfaches, gleichzeitiges Schreiben in die Zellen ist ohne Synchronisation vermutlich nicht möglich. Aus diesem Grund wurde im ersten Schritt das sequentielle Vorgehen gewählt. Hier kann weiter optimiert werden.

Damit sind die wichtigsten Erweiterungen an der NEORV32 CPU beschrieben.

5.6 Zusammenfassung

Die Umsetzung der Erweiterung wurde besprochen. Softwareseitig besteht die Umsetzung aus den neuen Anweisungen für den Umgang mit Matrizen. Es wurde beschrieben, wie die Anweisungen ausgewählt worden sind und wie sie über einen Compiler verfügbar gemacht wurden.

Auf der Hardwareseite wurde beschrieben, wie die Load-Store-Unit erweitert wird und wie die Matrix-Engine aussieht und mit Daten versorgt wird.

Im folgenden Kapitel soll die umgesetzte Erweiterung benutzt werden und

Laufzeiten sollen gemessen werden um entscheiden zu können, ob die Erweiterung tatsächlich einen Geschwindigkeitsvorteil bringt.

6 Auswertung und Evaluierung

In diesem Kapitel wird untersucht, ob der gewählte Ansatz zur Integration von Matrixoperationen in die NEORV32 CPU zielführend war. Zunächst wird die Funktionalität der Erweiterung geprüft. Das Prüfverfahren wird erläutert.

Danach wird untersucht, ob das erweiterte System eine Matrixmultiplikation schneller ausführen kann, als das unmodifizierte System. Um diesen Vergleich anzustellen wird die mathematische Vorschrift zur Matrixmultiplikation mit drei for-Schleifen auf zwei Matrizen ausgeführt. Die gleichen beiden Matrizen werden mit den hinzugefügten Anweisungen zur Matrixmultiplikation verrechnet.

Das Messverfahren wird beschrieben. Die Laufzeiten werden gemessen, ausgewertet und verglichen. Das Ergebnis dieses Vergleichs wird eingeordnet und diskutiert und es wird ein abschließendes Fazit gezogen.

6.1 Testdaten

Um eine Vergleichbarkeit der Verfahren und eine Überprüfung der Korrektheit zu erzielen, werden Testdaten mit erwartetem Ergebnis vorgegeben. In diesem Fall handelt es sich um zwei Matrizen A und B, von denen das Ergebnis der Multiplikation bekannt ist und welches durch Matrix C dokumentiert wird.

Es wird eine 4x4 Matrixmultiplikation mit den beiden Matrizen A und B durchgeführt, für die das Ergebnis bekannt ist.

$$C = A * B \quad (6.1)$$

$$\begin{bmatrix} 90 & 100 & 110 & 120 \\ 202 & 228 & 254 & 280 \\ 314 & 356 & 398 & 440 \\ 426 & 484 & 542 & 600 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

$$\begin{bmatrix} 5A & 64 & 6E & 78 \\ CA & E4 & FE & 118 \\ 13A & 164 & 18E & 1B8 \\ 1AA & 1E4 & 21E & 258 \end{bmatrix} = \begin{bmatrix} 90 & 100 & 110 & 120 \\ 202 & 228 & 254 & 280 \\ 314 & 356 & 398 & 440 \\ 426 & 484 & 542 & 600 \end{bmatrix}$$

Zur einfacheren Vergleichbarkeit der Matrix C mit der Waveform der Simulation wurde die Matrix C noch in hexadezimaler Schreibweise angegeben, da die Waveform Zahlen hexadezimal darstellt.

6.2 Messung und Vergleichbarkeit

Um Anweisungsfolgen miteinander zu vergleichen, muss zunächst eine Skala gewählt werden. Danach müssen die Anweisungsfolgen auf der Skala abgebildet werden. Die Abbildung muss dem Quellcode eine Zahl auf der Skala zuordnen. Als letztes muss für die Skala dann noch erwähnt werden, welcher Wert besser ist, also wie die Qualität von der Skala abzulesen ist.

Als Skala wird die Zahl der Clock-Ticks gewählt, die die CPU zur Ausführung der Anweisungsfolge benötigt. Die Abbildung erfolgt über ein Feature, das die NEORV32 CPU liefert. Es handelt sich hier um Register, die die Clock-Ticks zählen. Die Qualität auf der Skala ist dadurch bestimmt, dass weniger Clock-Zyklen besser sind. Die CPU löst eine Aufgabe dann schneller.

Es wird nun das Zählen der Clock-Ticks beschrieben. In RISC-V-Terminologie gibt es sogenannte CSR. CSR steht für Control- und Statusregister. Diese Register werden nicht bei der Abarbeitung von Anweisungen direkt verwendet. Sie können indirekt Einfluss auf die Arbeitsweise der CPU haben, da sie deren Zustand enthalten. Über bestimmte Befehle lassen sich die CSR ändern und damit lässt sich der Zustand der CPU konfigurieren.

Ein Paar von CSR, das bei der Zählung der Clock-Zyklen verwendet wird, sind die Register CSR_MCYCLE und CSR_MCYCLEH. Das erste Register (CSR_MCYCLE) speichert das Low-Word und das zweite Register (CSR_MCYCLEH) speichert das High-Word des Clock-Cycle-Zählers.

Es werden wirkliche Clock-Cycles gezählt und keine Anweisungen. Das ist ein Vorteil, da damit eine wirkliche Messung erzielt werden kann. Wenn Anweisungen gezählt werden, ist nicht klar, wieviel Zeit die CPU zur Bearbeitung der Anweisungen benötigt.

Ein Clock-Cycle beschreibt einen Ausschlag der externen Clock-Source. Die externe Clock-Source wird über einen Crystal-Oscillator erzeugt, optional über Phase-Locked-Loops oder Divider verändert und dann an das VHDL-Design als Input-Signal herangeführt.

Das Clock-Signal wird in sehr vielen Prozessen des Designs verwendet um sequentielle, getaktete Logik voranzutreiben. Die NEORV32 CPU kann beispielsweise mit einer Taktrate von 100 Mhz betrieben werden. Damit ist die externe Clock-Source dann auf 100 MHz einzustellen. CSR_MCYCLE und CSR_MCYCLEH werden dann mit einer Taktrate von 100 Mhz inkrementiert.

Hier ist ein Beispiel für die Verwendung der Abbildung auf die Cycle-Skala:

```
1 // start timing
```

```

2 neorv32_cpu_csr_write(CSR_MCYCLE, 0);
3
4 for (i=0; i<(DATA_NUM/2); i++) {
5     v[0] = input_data[i*2+0];
6     v[1] = input_data[i*2+1];
7     xtea_sw_encipher(XTEA_ROUNDS, v, key);
8     cypher_data_sw[i*2+0] = v[0];
9     cypher_data_sw[i*2+1] = v[1];
10 }
11
12 // stop timing
13 uint32_t time_enc_sw = neorv32_cpu_csr_read(CSR_MCYCLE);
14
15 // -----
16 // Print timing results
17 // -----
18 neorv32_uart0_printf("\nExecution timing:\n");
19 neorv32_uart0_printf("ENC SW=%u cycles\n", time_enc_sw);

```

Quellcode 6.1: Verwendung der Timing-Register

Im Beispiel wird nur das Low-Word und damit nur CSR_MCYCLE verwendet. Die Erwartungshaltung ist offensichtlich, dass die gemessene Befehlsfolge berechnet ist, bevor der Zähler in das High-Word überläuft.

Da das CSR_MCYCLE-CSR-Register einen beliebigen Wert enthält, muss es zunächst in Zeile 2 auf 0 zurückgesetzt werden. Dann kann sein Wert nach Abarbeitung der Befehlsfolge in Zeile 13 ausgelesen und schließlich über eine serielle Schnittstelle ausgegeben werden.

Dieses Verfahren wird im Folgenden zur Bewertung der unterschiedlichen Matrixmultiplikationen verwendet. Es wird dabei inklusive der Übertragung der Daten von und zum CPU-RAM gemessen, da der Zugriff auf den CPU-RAM bei der Hardwarebeschleunigung berücksichtigt werden kann und damit ein Geschwindigkeitsvorteil erzielt werden kann.

Das Vorgehen ist das Folgende:

1. In Vivado 2025.1 wird das Design synthetisiert. Dabei wird der Bootloader als erstes Anwendungsprogramm direkt in das Design synthetisiert. Damit ist der Bootloader-Maschinencode direkt im internen Instructionsspeicher der CPU enthalten und wird damit direkt beim Systemstart ausgeführt.
2. Der Bitstream wird über den in Vivado integrierten Hardware-Manager auf das FPGA-Board übertragen und dort ausgeführt. Der Bootloader ist jetzt über eine UART-Schnittstelle erreichbar.
3. Für jedes Testprogramm wird dieses Testprogramm mit dem GCC Compiler in der Version riscv-none-elf-gcc.exe (xPack GNU RISC-V Embedded GCC x86_64) 14.2.0 übersetzt. Für die Übersetzung der Beispiele und Testprogramme sind im NEORV32-Projekt Skripte und auch das sogenannte "image_gen" Werkzeug bereitgestellt. Mit "image_gen" wird aus den Ausgaben des GCC eine Datei erstellt, die vom Bootloader verstanden wird.

4. Die Ausgabe des "image_gen" Werkzeugs wird dem Bootloader über die UART-Schnittstelle zugeführt. Dazu wurde ein eigener Uploader erstellt. Die Firmware könnte aber auch über jeden beliebigen Terminal-Emulator übertragen werden.
5. Der Bootloader speichert das übertragene Programm in der NEORV32 CPU und diese führt das Programm aus, sobald der Uploader den Befehl zur Ausführung gibt.
6. In alle Testprogramme wird die Messung der Zyklen und die Ausgabe über einen UART eingefügt. Über die UART-Schnittstelle wird die gemessene Anzahl an Clock-Zyklen gelesen und notiert.
7. Die Ausgaben aller Testprogramme werden verglichen und bewertet.

6.3 Prüfung mit Matrix-Erweiterung

Für die Prüfung mit Hardwarebeschleunigung enthält das ausgeführte Anwendungsprogramm die mle32, mmul32 und mse32-Anweisungen als Intrinsics.

Die Matrix A und die Matrix B werden aus dem CPU-RAM in die Matrix-Engine geladen. Danach erfolgt die Multiplikation durch Aufruf der mmul32-Anweisung, danach erfolgt das Speichern der Matrix A und Matrix B zurück in den CPU RAM. Matrix A enthält in der momentanen Implementierung das Ergebnis der Multiplikation.

```
1 #include <neorv32.h>
2
3 uint32_t matrix_a[16]
4 = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
5
6 uint32_t matrix_b[16]
7 = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16 };
8
9 int main() {
10
11     uint32_t *matrix_a_addr = &matrix_a[0];
12     uint32_t *matrix_b_addr = &matrix_b[0];
13
14     uint32_t matrix_unit_a = 0;
15     uint32_t matrix_unit_b = 0;
16
17     neorv32_cpu_csr_write(CSR_MCYCLE, 0);
18
19     matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr, 0b010
20                                     , 0b1011011);
21     matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, matrix_b_addr, 0b010
22                                     , 0b1011011);
23
24     uint32_t matrix_mul = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr,
25                                     0b100, 0b1011011);
26
27     matrix_unit_a = CUSTOM_INSTR_I_TYPE(0x000, matrix_a_addr, 0b001
28                                     , 0b1011011);
```

```

25  matrix_unit_b = CUSTOM_INSTR_I_TYPE(0x001, matrix_b_addr, 0b001
    , 0b1011011);
26
27  uint32_t time_enc_sw = neorv32_cpu_csr_read(CSR_MCYCLE);
28
29  neorv32_uart0_printf("\nExecution timing:\n");
30  neorv32_uart0_printf("ENC SW = %u cycles\n", time_enc_sw);
31
32  return 0;
33 }

```

Quellcode 6.2: Verwendung der Intrinsics

Zunächst wird über den WaveForm-Viewer visuell geprüft, ob die korrekten Steuersignale ausgegeben werden. Danach wird geprüft, ob die Matrixmultiplikation in der Simulation mit den richtigen Daten abgearbeitet wird und ob die richtigen Werte in der Matrix C abgelegt werden.

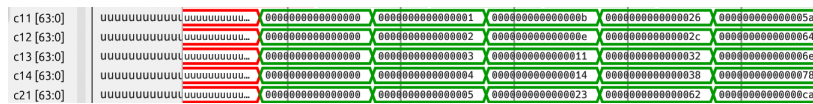


Abbildung 6.1: Waveforms der Matrixmultiplikation

In Abbildung 6.1 ist zu sehen, dass sich über die Zeit hinweg neue Daten in den Registern a und b befinden. Es ist auch zu sehen, dass die Matrix-Engine die Werte aus den Registern a und b über das dyadische Produkt multipliziert und auf die Accumulator-Register c aufaddiert. Schließlich ist nach den vier Schritten zu sehen, dass das korrekte Ergebnis in den Accumulator Registern c steht und dort auch für einige Zeit stehen bleibt, so dass die Daten weiterverarbeitet werden können und nicht direkt gelöscht werden.

Abschließend lässt sich sagen, dass die hardwarebeschleunigte Matrixmultiplikation mit dem erstellten Design wie geplant funktioniert.

6.4 Prüfung ohne Matrix-Erweiterung

Das Testprogramm für die Matrixmultiplikation ohne Hardwarebeschleunigung ist:

```

1  #include <stdio.h>
2
3  #define DIMENSION 4
4  #define ELEMENTS DIMENSION*DIMENSION
5
6  int standardMatrixMult(int* matrixA, int* matrixB, int* matrixC,
    int rows, int columns) {
7
8      int counter = 0;
9
10     // over row of matrix B
11     for (int i = 0; i < rows; i++) {
12
13         // over column of matrix A
14         for (int j = 0; j < columns; j++) {
15

```

```

16         // fuse row and column together into a single cell of
           matrix C
17     for (int k = 0; k < columns; k++) {
18
19         // int aldx = i * columns + k;
20         int temp_aldx = i * columns;
21         int aldx = temp_aldx + k;
22
23         // int bldx = k * columns + j;
24         int temp_bldx = k * columns;
25         int bldx = temp_bldx + j;
26
27         // int cldx = i * rows + j;
28         int temp_cldx = i * rows;
29         int cldx = temp_cldx + j;
30
31         int aVal = matrixA[aldx];
32         int bVal = matrixB[bldx];
33
34         int mult_temp = aVal * bVal;
35
36         int temp_2 = matrixC[cldx];
37         int temp_3 = temp_2 + mult_temp;
38         matrixC[cldx] = temp_3;
39
40     }
41
42 }
43
44 }
45
46 return 0;
47 }
48
49 int main()
50 {
51     neorv32_cpu_csr_write(CSR_MCYCLE, 0);
52
53     int matrixA[ELEMENTS] = {
54         1, 2, 3, 4,
55         5, 6, 7, 8,
56         9, 10, 11, 12,
57         13, 14, 15, 16
58     };
59     int matrixC[ELEMENTS] = {
60         0, 0, 0, 0,
61         0, 0, 0, 0,
62         0, 0, 0, 0,
63         0, 0, 0, 0
64     };
65     int matrixB[ELEMENTS] = {
66         1, 2, 3, 4,
67         5, 6, 7, 8,
68         9, 10, 11, 12,
69         13, 14, 15, 16
70     };
71
72     int a22 = standardMatrixMult(matrixA, matrixB, matrixC, 4, 4)
73     ;
74
75     uint32_t time_enc_sw = neorv32_cpu_csr_read(CSR_MCYCLE);
76
77     // -----
78     // Print timing results
79     // -----
80     neorv32_uart0_printf("\nExecution timing:\n");

```

```
80     neorv32_uart0_printf("ENC SW = %u cycles\n", time_enc_sw);
81
82     int resultPrettyPrintC = prettyPrintFormatMatrix(matrixC,
83                                     DIMENSION);
84     exit();
85     return 0;
86 }
```

Quellcode 6.3: Matrixmultiplikation ohne Hardwarebeschleunigung

Der C-Quellcode ist sehr außergewöhnlich gestaltet, einzig und alleine, weil der selbsterstellte C-Compiler diesen Quellcode in dieser Form auch übersetzen können soll, falls weitere Tests durchgeführt werden sollten. Da der selbsterstellte C-Compiler nur eine Untermenge der Programmiersprache C versteht, muss der Quellcode auf diese Weise erstellt werden.

6.5 Auswertung

Für die hardwarebeschleunigte Variante erfolgt folgende Ausgabe:

```
1 "\nBooting from 0x00000000...\r"
2 "\n\r"
3 "\nMatrix Multiplication Sample \r\r"
4 "\n\r"
5 "\nExecution timing:\r"
6 "\nENC SW=185 cycles\r"
```

Quellcode 6.4: Ausgabe der Zyklen, beschleunigte Multiplikation

Es werden 185 Zyklen notiert.

Für die nicht hardwarebeschleunigte Variante erfolgt folgende Ausgabe:

```
1 "\nBooting from 0x00000000...\r"
2 "\n\r"
3 "\nMatrix Multiplication Sample \r\r"
4 "\n\r"
5 "\nExecution timing:\r"
6 "\nENC SW=10385 cycles\r"
```

Quellcode 6.5: Ausgabe der Zyklen, einfache Multiplikation

Es werden 10385 Zyklen notiert.

Laut gemessenen Werten und rein mathematisch betrachtet wird die Matrixmultiplikation zweier 4x4, vorzeichenloser Integer-Matrizen unter Ausführung der Matrix-Extension um einen Faktor von etwa 56 beschleunigt.

Die Messung vergleicht im Grunde zwei Testanwendungen. Die Testanwendung mit den Anweisungen aus der Matrix-Extension lässt sich nicht mehr stark optimieren. Das Vergleichsprogramm mit der herkömmlichen Multiplikation ließe sich durch eine bessere Implementierung sicherlich stark vereinfachen.

Optimiert man die gewählten Testprogramme weiter oder wählt man komplett andere Testprogramme, so ergibt sich ein anderes Verhältnis zwischen den Zyklen, die die Testprogramme verbrauchen. Es ist jedoch nicht abzusehen, dass sich das Verhältnis zugunsten einer herkömmlichen Implementierung ohne Hardwarebeschleunigung wendet, auch wenn das Testprogramm stark optimiert wird.

Es lässt sich abschließend konstatieren, dass die Hardware-Beschleunigung einen positiven Einfluss auf die Laufzeit hat.

Dies ist hauptsächlich auf die effiziente Übertragung der Elemente zwischen CPU-RAM und dem Prozess-RAM der Matrix-Engine zurückzuführen. Die 16 Datenelemente werden als Stream zwischen dem Speicher und der Matrix-Engine übertragen und dies geschieht, ohne dass die Execution-Engine 16 einzelne Load- und Store-Befehle ausführen muss.

Anstatt die Matrixmultiplikation durch eine iterative Anwendung steuern zu lassen, bei der alle Instruktionen für die Schleifen durch die Execution-Engine geschleust werden, ist die NEORV32 CPU jetzt in der Lage die Matrixmultiplikation ohne äußere Einflüsse intern zu berechnen. Der zusätzliche Aufwand wird damit gegenüber einer naiven Implementierung um ein Vielfaches reduziert.

7 Zusammenfassung und Ausblick

Am Ende der Arbeit wird das Erreichte nochmals zusammengefasst um von diesem Standpunkt aus einen Blick in die Zukunft zu wagen.

Rückblickend lässt sich sagen, welche Entscheidungen zielführend und welche Entscheidungen hinderlich waren. Dies wird in der Zusammenfassung diskutiert.

Für die Zukunft werden im Ausblick Ideen angeführt, die während oder nach der Arbeit an der Lösung entsanden sind. Diese Gedanken sollen als Wegweiser für eine eventuelle Verbesserung oder Weiterführung der Arbeit dienen.

7.1 Zusammenfassung

Im Rahmen dieser Arbeit wurde die existierende NEORV32 CPU um Komponenten zur Matrix-Berechnung erweitert. Die Komponenten unterliegen starken Einschränkungen. Unter anderem können zum jetzigen Zeitpunkt lediglich 4×4 Matrizen mit vorzeichenlosen 32-Bit-Zahlen berechnet werden. Zum termingerechten Abschluss dieser Arbeit war es notwendig solche Einschränkungen vorzunehmen. Für die Zukunft wäre es durch zusätzlichen Arbeitseinsatz möglich das Design um weitere Datentypen und Konvertierungen zu erweitern.

Die durchgeführten Änderungen orientieren sich an der Befehlsmenge der bereits existierenden RISC-V Vektor-Extension und greifen damit auch das Konzept des Strip-Mining auf. Strip-Mining erlaubt die Erstellung eines Hardware-Software-Co-Designs, welches in der Lage ist Matrixmultiplikationen größerer Matrizen durch Multiplikationen kleinerer Dimensionen zusammenzusetzen.

Um die Softwareentwicklung für Anwendungen und Testprogramme zu ermöglichen, wurden exzessive Anstrengungen unternommen, die etwa den gleichen Zeitaufwand wie die VHDL-Entwicklung zur Folge hatten. Insbesondere wurde es durch einen Compiler ermöglicht die neuen Anweisungen auch unter Verwendung einer höheren Programmiersprache wie C in RISC-V-Maschinencode zu übersetzen. Im Nachhinein wäre der schnellstmögliche Einsatz der NEORV32 Intrinsics ein sehr großer Vorteil gewesen. Leider wurde dieser Weg erst spät gelernt.

Es wurde ein Vergleich zweier Testprogramme auf dem NEORV32 Chip durchgeführt. Die Messungen zeigen einen Geschwindigkeitszuwachs um

einen relevanten Faktor. Das Experiment ist geglückt und könnte in Zukunft durch die Erweiterung auf ein vollständiges Typsystem und weiter verbesserte Speicherzugriffe noch verbessert werden.

7.2 Ausblick

Am Ende dieser Arbeit drängt sich eine grundlegende Frage auf. In welchen Szenario ist es sinnvoll Hardwarebeschleuniger in ein bestehendes RISC-V-Design einzupflegen und ab wann sollte sich ein Unternehmen für die Erstellung eines komplett neuen Designs entscheiden. Diese Frage ist eng verknüpft mit der Frage, ob der in dieser Arbeit gewählte Ansatz richtig oder falsch ist.

Im Zuge dieser Arbeit wurde ein bestehendes Design erweitert. Der Zugriffsweg auf den Speicher über die Load-Store-Unit und den am Bus angeschlossenen Speicher wurde wiederverwendet. Der Vorteil ist es sicherlich Kosten und Zeit sparen zu können, da existierende Komponenten wiederverwendet werden. Ein Nachteil ist eventuell, dass die wiederverwendeten Komponenten nicht optimal auf eine Matrixmultiplikation ausgelegt sind, da die NEORV32 CPU für die Lösung allgemeiner Aufgaben gedacht ist. Für diese Arbeit ist dieser Umstand nicht zu einem Problem geworden, wie die Auswertung gezeigt hat. Die Matrixmultiplikation wurde beschleunigt. Gleichzeitig kann die NEORV32 CPU immer noch allgemeine Probleme lösen.

Die Alternative wäre es, ein grundlegend neues Design zu beginnen, welches dann die neuen Anforderungen besser abdeckt. Ein Beispiel für eine neue Anforderung ist es beispielsweise, mehr als einen Speicher-Zugriff parallel durchführen zu können. Die Hardware für die Berechnung des dyadischen Produkts könnte dann mehrmals synthetisiert werden. Jede Instanz der Berechnung des dyadischen Produkts könnte möglicherweise über ihren eigenen Zugriff auf den Speicher verfügen. Wenn der Speicher echt-parallel Daten über mehrere Ports gleichzeitig liefern kann, wird eine Matrix-Berechnung parallel statt sequentiell ausführbar.

Ein Nachteil eines solch spezialisierten Designs ist es, dass es seine Allgemeinheit verliert. Eine RISC-V CPU soll eine allgemeine Rechenmaschine bleiben und sich nicht auf einzelne Anwendungsfälle einschränken. CPU Designs von Intel und AMD sind um Instruktionserweiterungen wie AVX2, AMX und FMA für die Vektor- und Matrixberechnung erweitert worden, wobei sie immer noch allgemeines Computing erlauben. In Zukunft wird RISC-V diesen Weg mit einer Matrix-Extension auch gehen und hat mit der RVV bereits begonnen.

Unternehmen wie NVIDIA erstellen Beschleunigerkarten, die für High-Performance Computing im Sinne von großer Parallelität eingesetzt werden können und damit mehrere größere Felder abdecken wie beispielsweise die Grafikberechnung aber auch das Training von Neuronalen Netzen. Auch NVIDIA strebt also eine Gewisse Verallgemeinerung an.

Aus Sicht eines Kunden sollte der Chip möglichst auf den Anwendungsfall spezialisiert sein. Aus Sicht eines Herstellers sollte der Chip möglichst breit anwendbar sein um viele Kunden anzusprechen.

Auf der anderen Seite des Spektrums kann ein finanzstarkes Unternehmen sogar zu seinem eigenen Kunden werden und sich für die In-House Erstellung von anwendungsspezifischen Chips entscheiden, wie man es im Falle von Amazon und dem Trainium Chip sehen kann. Hier kann Amazon die eigenen Bedürfnisse perfekt abdecken. Der Chip ist hochspezialisiert und arbeitet damit schnell, lässt sich aber weniger allgemein einsetzen.

Es ist fallabhängig, welchen Weg ein Chip-Hersteller einschlagen sollte. Wahrscheinlich ist es aus wirtschaftlicher Sicht notwendig zu wissen, was der Markt fordert um Hardware erstellen zu können, die die Anforderung des Marktes erfüllen und damit Umsatz erzeugen.

Als nächster Schritt zur Fortführung dieser Arbeit ist es denkbar die Matrix-Extension intelligenter und damit performanter zu gestalten. Der Vergleich mit anderen Designs wie z.B. [13] und [16] führt zu neuen Erkenntnissen und einen Zuwachs an Erfahrung.

Zusätzlich ist das System aus dieser Arbeit um weitere Datentypen zu ergänzen. Es gibt traditionelle Datentypen wie beispielsweise 8-, 16-, 32- und 64-bit ganzzahlige Datentypen mit und ohne Vorzeichen und auch 32- und 64-bit Gleitkommatypen. Dazu kommen neuartige Aufteilungen von Daten, wie sie für Large Language Models verwendet werden. Ein Beispiel ist hier NF4, was für normalized float mit 4-Bit steht.

Sobald die Matrix-Extension mit unterschiedlichen Datentypen verwendet werden können soll, ist über die Konvertierung zwischen den Datentypen nachzudenken. Es müssen ebenfalls dazu passende Instruktionen ergänzt werden. Die `set(i)vi` Instruktion aus der RVV-Extension benötigt dann ein Äquivalent in der Matrix-Extension um die Matrix-Extension für Datentypen konfigurierbar zu machen.

Es ist zu prüfen, welche Ergebnisse das Lowering hervorgebracht hat. Wurden wirklich Hardware-Bausteine innerhalb des FPGA verwendet oder sind Anteile unnötigerweise durch Logikzellen synthetisiert worden? Hier lässt sich in Kleinstarbeit noch eine Optimierung durchführen, falls Logikzellen anstatt von vorhandener Hardware verwendet werden. Wird der VHDL Code verändert, erkennt das Synthese-Werkzeug dann eventuell eine Struktur, die es auf vorhandene Hardware abbilden kann und nicht in Logikzellen synthetisiert werden muss. Wertvolle Logikzellen bleiben damit unbenutzt und stehen für wirklich einzigartige Design zur Verfügung.

Ein weiterer Schritt ist es sicherlich eine Umgebung für die Validierung des Systems bereitzustellen. Aufgrund zeitlicher Beschränkungen wurde das Design nur visuell mit einem Waveform-Viewer geprüft. Die Sprache System-Verilog beispielsweise bietet weitreichendere Möglichkeiten zur Validierung.

Im Zuge dieser Arbeit wurde die NEORV32 CPU lediglich um die Instruktio-

nen für die Matrixmultiplikation erweitert. Es ist noch zu erwähnen, dass das Hardware-Software Co-Design auch die Bereitstellung eines Compilers notwendig macht, der den Anwendungsfall erkennt und die notwendigen Anweisungen generiert. Um die Ergebnisse dieser Arbeit anderen Benutzer zur Verfügung zustellen, müsste eine Erweiterung des GCC oder der LLVM Infrastruktur erfolgen, damit die Erweiterung nicht nur mit einem Assembler genutzt werden kann.

Quellenverzeichnis

- [1] AKENINE-MÖLLER, Tomas ; HAINES, Eric ; HOFFMAN, Naty: Real-Time Rendering 3rd Edition. Natick, MA, USA : A. K. Peters, Ltd., 2008. – 1045 S. – ISBN 987-1-56881-424-7
- [2] ALON AMID, et. a. Krste Asanovic A. Krste Asanovic: RISC-V "V"Vector Extension. <https://github.com/riscvarchive/riscv-v-spec/blob/master/v-spec.adoc>. Version: 2024
- [3] ARSALAN, Qizal: NEORV32 Hardware Accelerator (Arty A7) – Modified Core with CFU/CFS. https://github.com/QizalA/neorv32_hardware_accelerator_arty_a7, 2025
- [4] ASANOVIC, Krste: RISC-V State of the Union. <https://riscv-europe.org/summit/2025/media/proceedings/2025-05-13-RISC-V-Summit-Europe-11h30-ASANOVIC%20C4%86-slides.pdf>. Version: 2025
- [5] DAVID FLANAGAN, Adrian N.: X Toolkit Intrinsics Reference Manual: for X11 Release 4 and Release 5. The Associates, 1990
- [6] FALK, Sigurd: Ein übersichtliches Schema für die Matrizenmultiplikation. Berlin : Wiley-VCH, Zeitschrift für Angewandte Mathematik und Mechanik (ZAMM), 1951
- [7] FUJIE FAN, David C.: Stream Computing RISC-V Matrix Extension Open Source Project Upgrades to Version 0.5, Supporting Vector+Matrix Implementation. <https://riscv.org/blog/stream-computing-risc-v-matrix-extension-open-source-project-upgrades-to-version-0-5-supporting-vectormatrix-implementation/>. Version: 2024
- [8] HARRIS, Sarah L. ; HARRIS, David: Digital Design and Computer Architecture, RISC-V Edition. (2021). ISBN 0128200642, 97801282006430
- [9] IBÁÑEZ, Roger F.: Introduction to the RISC-V Vector Extension. <https://eupilot.eu/wp-content/uploads/2022/11/RISC-V-VectorExtension-1-1.pdf>. Version: 2022
- [10] INTERNATIONAL, RISC-V: Integrated Matrix Extension. <https://riscv.atlassian.net/wiki/spaces/IMEX/pages/46071925/Charter>. Version: 2024
- [11] KILLIAN, Earl A.: VME Design Considerations. <https://riscv.atlassian.net/wiki/download/attachments/663781400/DesignConsiderationsForVME.pdf?api=v2>. Version: 2025

- [12] LARABEL, Michael: Qualcomm's Xqci RISC-V Extension Now Deemed Non-Experimental For LLVM 22. <https://www.phoronix.com/news/Qualcomm-Xqci-LLVM-Stable>. Version: 2025
- [13] MOREIRA, José E. ; BARTON, Kit ; BATTLE, Steven ; BERGNER, Peter ; BERT-RAN, Ramon ; BHAT, Puneeth ; CALDEIRA, Pedro ; EDELSON, David ; FOS-SUM, Gordon C. ; FREY, Brad ; IVANOVIC, Nemanja ; KERCHNER, Chip ; LIM, Vincent ; KAPOOR, Shakti ; FILHO, Tulio M. ; MUELLER, Silvia M. ; OLSSON, Brett ; SADASIVAM, Satish ; SALEIL, Baptiste ; SCHMIDT, Bill ; SRINIVASARAGHAVAN, Rajalakshmi ; SRIVATSAN, Shricharan ; THOMPSON, Brian W. ; WAGNER, Andreas ; WU, Nelson: A matrix math facility for Power ISA(TM) processors. In: CoRR abs/2104.03142 (2021). <https://arxiv.org/abs/2104.03142>
- [14] NOLTING, Stephan: neorv32. <https://github.com/stnolting/neorv32>. Version: 2026
- [15] NOLTING, Stephan: The NEORV32 RISC-V Processor - Data Sheet. <https://stnolting.github.io/neorv32/>. Version: 2026
- [16] OSS, Stream C.: riscv-matrix-project. <https://github.com/riscv-stc/riscv-matrix-project>, 2025
- [17] THOMAS, Giles: Basic matrix maths for neural networks: the theory. <https://www.gilesthomas.com/2025/02/basic-neural-network-matrix-maths-part-1>. Version: 2025
- [18] TILO ARENS, FRANK HETTLICH, CHRISTIAN KARPfINGER, ULRICH KOCKEL-KORN, KLAUS LICHTENEGGER, HELLMUTH STACHEL: Mathematik. SpringerSpektrum, Springer-Verlag, Berlin Heidelberg 2008, 2011, 2015, 2015
- [19] VLADIMIROV, Andrey: Optimization Techniques for the INTEL MIC Architecture. Part 2 OF 3: Strip-Mining for Vectorization. (2015), June. <https://colfaxresearch.com/download/711/>
- [20] WINDECK, Christof: Bit-Rauschen, der Prozessor-Podcast: KI-Rechenwerke für CPU-Kerne, Folge 2026/3. <https://www.heise.de/meinung/Bit-Rauschen-der-Prozessor-Podcast-KI-Rechenwerke-fuer-CPU-Kerne-11136517.html>. Version: 2026
- [21] ZEE, Field G. ; GEIJN, Robert A.: BLIS: A Framework for Rapidly Instantiating BLAS Functionality. In: ACM Transactions on Mathematical Software 41 (2015), June, Nr. 3, 14:1–14:33. <https://doi.acm.org/10.1145/2764454>

Anhang

Quellcode 1: neorv32_cpu

```
--
-- =====
--
-- NEORV32 CPU - CPU Top Entity
--
--
--
--
-- HQ:          https://github.com/stnolting/neorv32
--
-- Data Sheet:  https://stnolting.github.io/neorv32
--
-- User Guide:  https://stnolting.github.io/neorv32/ug
--
-- Software Ref: https://stnolting.github.io/neorv32/sw/files.html
--
--
--
--
-- The NEORV32 RISC-V Processor - https://github.com/stnolting/
-- neorv32
-- Copyright (c) NEORV32 contributors.
--
-- Copyright (c) 2020 - 2025 Stephan Nolting. All rights reserved.
--
-- Licensed under the BSD-3-Clause license, see LICENSE for details.
--
-- SPDX-License-Identifier: BSD-3-Clause
--
--
-- =====
--
use std.textio.all;

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
--use ieee.numeric_std_unsigned.all;

library neorv32;
use neorv32.neorv32_package.all;

entity neorv32_cpu is
  generic (
    -- General --
    HART_ID          : natural range 0 to 1023;          --
      hardware thread ID
    BOOT_ADDR        : std_ulogic_vector(31 downto 0); -- CPU
      boot address
    DEBUG_PARK_ADDR   : std_ulogic_vector(31 downto 0); -- CPU
      debug mode parking loop entry address
  )
end entity;
```

```

DEBUG_EXC_ADDR      : std_ulogic_vector(31 downto 0); -- CPU
                     debug mode exception entry address
-- RISC-V ISA Extensions --
RISCV_ISA_C          : boolean;                        --
                     compressed extension
RISCV_ISA_E          : boolean;                        --
                     embedded RF extension
RISCV_ISA_M          : boolean;                        -- mul/div
                     extension
RISCV_ISA_U          : boolean;                        -- user
                     mode extension
RISCV_ISA_Zaamo      : boolean;                        -- atomic
                     read-modify-write operations extension
RISCV_ISA_Zalrsc     : boolean;                        -- atomic
                     reservation-set operations extension
RISCV_ISA_Zcb        : boolean;                        --
                     additional code size reduction instructions
RISCV_ISA_Zba        : boolean;                        -- shifted
                     -add bit-manipulation extension
RISCV_ISA_Zbb        : boolean;                        -- basic
                     bit-manipulation extension
RISCV_ISA_Zbkb       : boolean;                        -- bit-
                     manipulation instructions for cryptography
RISCV_ISA_Zbkc       : boolean;                        -- carry-
                     less multiplication instructions
RISCV_ISA_Zbkx       : boolean;                        --
                     cryptography crossbar permutation extension
RISCV_ISA_Zbs        : boolean;                        -- single-
                     bit bit-manipulation extension
RISCV_ISA_Zfinx      : boolean;                        -- 32-bit
                     floating-point extension
RISCV_ISA_Zibi       : boolean;                        -- branch
                     with immediate
RISCV_ISA_Zicntr     : boolean;                        -- base
                     counters
RISCV_ISA_Zicond     : boolean;                        -- integer
                     conditional operations
RISCV_ISA_Zihpm      : boolean;                        --
                     hardware performance monitors
RISCV_ISA_Zknd       : boolean;                        --
                     cryptography NIST AES decryption extension
RISCV_ISA_Zkne       : boolean;                        --
                     cryptography NIST AES encryption extension
RISCV_ISA_Zknh       : boolean;                        --
                     cryptography NIST hash extension
RISCV_ISA_Zksed      : boolean;                        -- ShangMi
                     hash extension
RISCV_ISA_Zksh       : boolean;                        -- ShangMi
                     block cipher extension
RISCV_ISA_Zmmul      : boolean;                        --
                     multiply-only M sub-extension
RISCV_ISA_Zxcfu      : boolean;                        -- custom
                     (instr.) functions unit
RISCV_ISA_Sdext      : boolean;                        --
                     external debug mode extension
RISCV_ISA_Sdtrig     : boolean;                        -- trigger
                     module extension
RISCV_ISA_Smpmp      : boolean;                        --
                     physical memory protection
-- Tuning Options --
CPU_TRACE_EN        : boolean;                        -- enable
                     CPU execution trace generator
CPU_CONSTT_BR_EN     : boolean;                        --
                     constant-time branches
CPU_FAST_MUL_EN      : boolean;                        -- use
                     DSPs for M extension's multiplier

```

```

CPU_FAST_SHIFT_EN : boolean; -- use
    barrel shifter for shift operations
CPU_RF_HW_RST_EN : boolean; -- enable
    full hardware reset for register file
-- Physical Memory Protection (PMP) --
PMP_NUM_REGIONS : natural range 0 to 16; -- number
    of regions (0..16)
PMP_MIN_GRANULARITY : natural; -- minimal
    region granularity in bytes, has to be a power of 2, min 4
    bytes
PMP_TOR_MODE_EN : boolean; -- enable
    TOR mode
PMP_NAP_MODE_EN : boolean; -- enable
    NAPOT/NA4 modes
-- Hardware Performance Monitors (HPM) --
HPM_NUM_CNTS : natural range 0 to 13; -- number
    of implemented HPM counters (0..13)
HPM_CNT_WIDTH : natural range 0 to 64; -- total
    size of HPM counters (0..64)
-- Trigger Module (TM) --
NUM_HW_TRIGGERS : natural range 0 to 16 -- number
    of hardware triggers
);
port (
    -- global control --
    clk_i : in std_ulogic; -- global clock
        , rising edge
    rstn_i : in std_ulogic; -- global reset
        , low-active, async
    -- status --
    trace_o : out trace_port_t; -- execution
        trace port (enabled when CPU_TRACE_EN = true)
    sleep_o : out std_ulogic; -- CPU is in
        sleep mode
    -- interrupts --
    msi_i : in std_ulogic; -- RISC-V
        machine software interrupt
    mei_i : in std_ulogic; -- RISC-V
        machine external interrupt
    mti_i : in std_ulogic; -- RISC-V
        machine timer interrupt
    firq_i : in std_ulogic_vector(15 downto 0); -- custom fast
        interrupts
    dbi_i : in std_ulogic; -- RISC-V debug
        halt request interrupt
    -- instruction bus interface --
    ibus_req_o : out bus_req_t; -- request bus
    ibus_rsp_i : in bus_rsp_t; -- response bus
    -- data bus interface --
    dbus_req_o : out bus_req_t; -- request bus
    dbus_rsp_i : in bus_rsp_t; -- response bus
);
end neorv32_cpu;

architecture neorv32_cpu_rtl of neorv32_cpu is
    -- auto-configuration --
    constant riscv_a_c : boolean := RISC_V_ISA_Zaamo and
        RISC_V_ISA_Zalrsc; -- A: atomic memory operations
    constant riscv_b_c : boolean := RISC_V_ISA_Zba and RISC_V_ISA_Zbb
        and RISC_V_ISA_Zbs; -- B: bit manipulation
    constant riscv_zcb_c : boolean := RISC_V_ISA_C and RISC_V_ISA_Zcb;
        -- Zcb: additional compressed instructions
    constant riscv_zkt_c : boolean := CPU_FAST_SHIFT_EN; -- Zkt: data-
        independent execution time for cryptography operations

```

```

constant riscv_zkn_c : boolean := RISCv_ISA_Zbkb and
    RISCv_ISA_Zbkc and RISCv_ISA_Zbkx and
        RISCv_ISA_Zkne and
        RISCv_ISA_Zknd and
        RISCv_ISA_Zknh; -- Zkn: NIST
    suite
constant riscv_zks_c : boolean := RISCv_ISA_Zbkb and
    RISCv_ISA_Zbkc and RISCv_ISA_Zbkx and
        RISCv_ISA_Zksh and
        RISCv_ISA_Zksed; -- Zks:
    ShangMi suite

-- external CSR interface read-back --
signal xcsr_tm, xcsr_cnt, xcsr_pmp, xcsr_alu, xcsr_res :
    std_ulogic_vector(XLEN-1 downto 0);

-- local signals --
signal ctrl          : ctrl_bus_t;                -- main
    control bus
signal frontend      : if_bus_t;                --
    instruction-fetch interface
signal hwtrig        : std_ulogic;                --
    hardware trigger firing
signal rf_wdata       : std_ulogic_vector(XLEN-1 downto 0); -- x
    register file write data
signal m_rf_wdata     : std_ulogic_vector(XLEN-1 downto 0); -- m
    register file write data
signal rs1            : std_ulogic_vector(XLEN-1 downto 0); -- source
    register 1
signal rs2            : std_ulogic_vector(XLEN-1 downto 0); -- source
    register 2
signal alu_res        : std_ulogic_vector(XLEN-1 downto 0); -- alu
    result
signal alu_add        : std_ulogic_vector(XLEN-1 downto 0); -- alu
    address result
signal alu_cmp        : std_ulogic_vector(1 downto 0);    --
    comparator result
signal alu_cp_done    : std_ulogic;                -- alu co
    -processor operation done
signal lsu_rdata      : std_ulogic_vector(XLEN-1 downto 0); -- lsu
    memory read data
signal lsu_mar        : std_ulogic_vector(XLEN-1 downto 0); -- lsu
    memory address register
signal lsu_err        : std_ulogic_vector(3 downto 0);    -- lsu
    alignment/access errors
signal lsu_wait       : std_ulogic;                -- wait
    for current data bus access
signal dbus_req       : bus_req_t;                -- data
    bus request
signal csr_rdata      : std_ulogic_vector(XLEN-1 downto 0); -- csr
    read data
signal pmp_fault      : std_ulogic;                -- pmp
    permission violation
signal irq_machine    : std_ulogic_vector(2 downto 0);    -- risc-v
    standard machine-level interrupts

-- [wbi] debug
signal valid          : std_ulogic;                -- bus
    signals are valid
signal instr          : std_ulogic_vector(31 downto 0);    --
    instruction

-- Matrix Extension -----
-----

-- mat mul engine state machine --

```

```

type mat_mul_engine_state_t is (
    MM_RESET,
    MM_INIT_1, MM_PROCESS_1,
    MM_INIT_2, MM_PROCESS_2,
    MM_INIT_3, MM_PROCESS_3,
    MM_INIT_4, MM_PROCESS_4,
    MM_STORE,
    MM_DONE
);
signal mat_mul_engine_state : mat_mul_engine_state_t;

signal matrix_ram_0 : ram_type;
signal matrix_ram_1 : ram_type;
signal matrix_ram_2 : ram_type;
signal head : matrix_ram_addr;
signal tail : matrix_ram_addr;
signal transfer : std_ulogic;
signal load : std_ulogic;
signal store : std_ulogic;
signal matrix_add : std_ulogic;
signal matrix_mul : std_ulogic;
signal matrix_operation_wait : std_ulogic;
signal matrix_mar_load_store : std_ulogic_vector(XLEN-1 downto 0);
signal data_out : std_ulogic_vector(XLEN-1 downto 0);
signal data_in : std_ulogic_vector(XLEN-1 downto 0);
signal matrix_immediate : std_ulogic_vector(XLEN-1 downto 0);

signal c11 : unsigned(63 downto 0); -- :=
    "0000000000000000000000000000000000000000000000000000000000000000";

signal c12 : unsigned(63 downto 0);
signal c13 : unsigned(63 downto 0);
signal c14 : unsigned(63 downto 0);

signal c21 : unsigned(63 downto 0);
signal c22 : unsigned(63 downto 0);
signal c23 : unsigned(63 downto 0);
signal c24 : unsigned(63 downto 0);

signal c31 : unsigned(63 downto 0);
signal c32 : unsigned(63 downto 0);
signal c33 : unsigned(63 downto 0);
signal c34 : unsigned(63 downto 0);

signal c41 : unsigned(63 downto 0);
signal c42 : unsigned(63 downto 0);
signal c43 : unsigned(63 downto 0);
signal c44 : unsigned(63 downto 0);

signal a11 : unsigned(31 downto 0);
signal a21 : unsigned(31 downto 0);
signal a31 : unsigned(31 downto 0);
signal a41 : unsigned(31 downto 0);

signal b11 : unsigned(31 downto 0);
signal b12 : unsigned(31 downto 0);
signal b13 : unsigned(31 downto 0);
signal b14 : unsigned(31 downto 0);

begin

    -- Configuration Info and Checks -----
    -- -----
    hello_neorv32:
    if HART_ID = 0 generate -- print only for core 0

```

```

-- CPU ISA configuration (in alphabetical order - not in
  canonical order) --
assert false report "[NEORV32]_CPU_ISA:_rv32" &
  cond_sel_string_f(RISCV_ISA_E, "e", "i") &
  cond_sel_string_f(RISCV_ISA_C, "a", "a") &
  cond_sel_string_f(RISCV_ISA_M, "b", "b") &
  cond_sel_string_f(RISCV_ISA_C, "c", "c") &
  cond_sel_string_f(RISCV_ISA_M, "m", "m") &
  cond_sel_string_f(RISCV_ISA_U, "u", "u") &
  cond_sel_string_f(true, "x", "x") & --
    always enabled
  cond_sel_string_f(RISCV_ISA_Zaamo, "_zaamo", "a") &
  cond_sel_string_f(RISCV_ISA_Zalrsc, "_zalrsc", "r") &
  cond_sel_string_f(RISCV_ISA_C, "_zca", "c") & -- Zcb
    requires Zca (=C) in the ISA string
  cond_sel_string_f(RISCV_ISA_Zcb, "_zcb", "b") &
  cond_sel_string_f(RISCV_ISA_Zba, "_zba", "a") &
  cond_sel_string_f(RISCV_ISA_Zbb, "_zbb", "b") &
  cond_sel_string_f(RISCV_ISA_Zbkb, "_zbkb", "b") &
  cond_sel_string_f(RISCV_ISA_Zbkc, "_zbkc", "c") &
  cond_sel_string_f(RISCV_ISA_Zbkx, "_zbkx", "x") &
  cond_sel_string_f(RISCV_ISA_Zbs, "_zbs", "s") &
  cond_sel_string_f(RISCV_ISA_Zibi, "_zibi", "i") &
  cond_sel_string_f(RISCV_ISA_Zicntr, "_zicntr", "c") &
  cond_sel_string_f(RISCV_ISA_Zicond, "_zicond", "c") &
  cond_sel_string_f(true, "_zicsr", "s") & --
    always enabled
  cond_sel_string_f(true, "_zifencei", "i") & --
    always enabled
  cond_sel_string_f(RISCV_ISA_Zihpm, "_zihpm", "p") &
  cond_sel_string_f(RISCV_ISA_Zfinx, "_zfinx", "x") &
  cond_sel_string_f(RISCV_ISA_Zknd, "_zknd", "d") &
  cond_sel_string_f(RISCV_ISA_Zkne, "_zkne", "e") &
  cond_sel_string_f(RISCV_ISA_Zknh, "_zknh", "h") &
  cond_sel_string_f(RISCV_ISA_Zks, "_zks", "s") &
  cond_sel_string_f(RISCV_ISA_Zksed, "_zksed", "e") &
  cond_sel_string_f(RISCV_ISA_Zksh, "_zksh", "s") &
  cond_sel_string_f(RISCV_ISA_Zkt, "_zkt", "t") &
  cond_sel_string_f(RISCV_ISA_Zmmul, "_zmmul", "m") &
  cond_sel_string_f(RISCV_ISA_Zxcfu, "_zxcfu", "f") &
  cond_sel_string_f(RISCV_ISA_Zsdx, "_zsdx", "x") &
  cond_sel_string_f(RISCV_ISA_Zsdtrig, "_sdtrig", "t") &
  cond_sel_string_f(RISCV_ISA_Zsmpmp, "_smpmp", "p") &
severity note;

-- CPU tuning options --
assert false report "[NEORV32]_CPU_tuning_options:_t" &
  cond_sel_string_f(CPU_TRACE_EN, "trace_", "t") &
  cond_sel_string_f(CPU_CONSTT_BR_EN, "constt_br_", "b") &
  cond_sel_string_f(CPU_FAST_MUL_EN, "fast_mul_", "m") &
  cond_sel_string_f(CPU_FAST_SHIFT_EN, "fast_shift_", "s") &
  cond_sel_string_f(CPU_RF_HW_RST_EN, "rf_hw_rst_", "r") &
severity note;

end generate;

-- Front-End (Instruction Fetch) -----
neorv32_cpu_frontend_inst: entity neorv32.neorv32_cpu_frontend
generic map (
  RISCV_C => RISCV_ISA_C, -- implement C ISA extension
  RISCV_ZCB => RISCV_ISA_Zcb -- implement Zcb ISA sub-extension
)
port map (

```



```
c33 <= c33 + (a31 * b13);
c34 <= c34 + (a31 * b14);

c41 <= c41 + (a41 * b11);
c42 <= c42 + (a41 * b12);
c43 <= c43 + (a41 * b13);
c44 <= c44 + (a41 * b14);

-- next state
mat_mul_engine_state <= MM_INIT_2;

when MM_INIT_2 =>

    a11 <= unsigned(matrix_ram_0(1));
    a21 <= unsigned(matrix_ram_0(5));
    a31 <= unsigned(matrix_ram_0(9));
    a41 <= unsigned(matrix_ram_0(13));

    b11 <= unsigned(matrix_ram_1(4));
    b12 <= unsigned(matrix_ram_1(5));
    b13 <= unsigned(matrix_ram_1(6));
    b14 <= unsigned(matrix_ram_1(7));

    -- next state
    mat_mul_engine_state <= MM_PROCESS_2;

when MM_PROCESS_2 =>

    c11 <= c11 + (a11 * b11);
    c12 <= c12 + (a11 * b12);
    c13 <= c13 + (a11 * b13);
    c14 <= c14 + (a11 * b14);

    c21 <= c21 + (a21 * b11);
    c22 <= c22 + (a21 * b12);
    c23 <= c23 + (a21 * b13);
    c24 <= c24 + (a21 * b14);

    c31 <= c31 + (a31 * b11);
    c32 <= c32 + (a31 * b12);
    c33 <= c33 + (a31 * b13);
    c34 <= c34 + (a31 * b14);

    c41 <= c41 + (a41 * b11);
    c42 <= c42 + (a41 * b12);
    c43 <= c43 + (a41 * b13);
    c44 <= c44 + (a41 * b14);

    -- next state
    mat_mul_engine_state <= MM_INIT_3;

when MM_INIT_3 =>

    a11 <= unsigned(matrix_ram_0(2));
    a21 <= unsigned(matrix_ram_0(6));
    a31 <= unsigned(matrix_ram_0(10));
    a41 <= unsigned(matrix_ram_0(14));

    b11 <= unsigned(matrix_ram_1(8));
    b12 <= unsigned(matrix_ram_1(9));
    b13 <= unsigned(matrix_ram_1(10));
    b14 <= unsigned(matrix_ram_1(11));

    -- next state
    mat_mul_engine_state <= MM_PROCESS_3;
```

```

when MM_PROCESS_3 =>

    c11 <= c11 + (a11 * b11);
    c12 <= c12 + (a11 * b12);
    c13 <= c13 + (a11 * b13);
    c14 <= c14 + (a11 * b14);

    c21 <= c21 + (a21 * b11);
    c22 <= c22 + (a21 * b12);
    c23 <= c23 + (a21 * b13);
    c24 <= c24 + (a21 * b14);

    c31 <= c31 + (a31 * b11);
    c32 <= c32 + (a31 * b12);
    c33 <= c33 + (a31 * b13);
    c34 <= c34 + (a31 * b14);

    c41 <= c41 + (a41 * b11);
    c42 <= c42 + (a41 * b12);
    c43 <= c43 + (a41 * b13);
    c44 <= c44 + (a41 * b14);

    -- next state
    mat_mul_engine_state <= MM_INIT_4;

when MM_INIT_4 =>

    a11 <= unsigned(matrix_ram_0(3));
    a21 <= unsigned(matrix_ram_0(7));
    a31 <= unsigned(matrix_ram_0(11));
    a41 <= unsigned(matrix_ram_0(15));

    b11 <= unsigned(matrix_ram_1(12));
    b12 <= unsigned(matrix_ram_1(13));
    b13 <= unsigned(matrix_ram_1(14));
    b14 <= unsigned(matrix_ram_1(15));

    -- next state
    mat_mul_engine_state <= MM_PROCESS_4;

when MM_PROCESS_4 =>

    c11 <= c11 + (a11 * b11);
    c12 <= c12 + (a11 * b12);
    c13 <= c13 + (a11 * b13);
    c14 <= c14 + (a11 * b14);

    c21 <= c21 + (a21 * b11);
    c22 <= c22 + (a21 * b12);
    c23 <= c23 + (a21 * b13);
    c24 <= c24 + (a21 * b14);

    c31 <= c31 + (a31 * b11);
    c32 <= c32 + (a31 * b12);
    c33 <= c33 + (a31 * b13);
    c34 <= c34 + (a31 * b14);

    c41 <= c41 + (a41 * b11);
    c42 <= c42 + (a41 * b12);
    c43 <= c43 + (a41 * b13);
    c44 <= c44 + (a41 * b14);

    -- next state
    mat_mul_engine_state <= MM_STORE;

when MM_STORE =>

```

```

        -- enable transfer in the RAM part
        transfer <= '1';

        -- next state
        mat_mul_engine_state <= MM_DONE;

    when MM_DONE =>

        transfer <= '0';

        -- all mult operations are done
        matrix_operation_wait <= '0';

        -- next state - this kills data before store!
        mat_mul_engine_state <= MM_RESET;

    when others => -- default

    end case;

end if;

end process PROC_MATRIX_MUL;

-- https://vhdlwhiz.com/ring-buffer-fifo/
PROC_RAM : process(clk_i)
    variable l : line;
begin

    if (rstn_i = '0') then

    elsif rising_edge(clk_i) then

        if (transfer = '1') then

            -- store the c temporaries into the second matrix engine
            register
            matrix_ram_0(0) <= std_ulogic_vector(c11(31 downto 0));
            matrix_ram_0(1) <= std_ulogic_vector(c12(31 downto 0));
            matrix_ram_0(2) <= std_ulogic_vector(c13(31 downto 0));
            matrix_ram_0(3) <= std_ulogic_vector(c14(31 downto 0));

            matrix_ram_0(4) <= std_ulogic_vector(c21(31 downto 0));
            matrix_ram_0(5) <= std_ulogic_vector(c22(31 downto 0));
            matrix_ram_0(6) <= std_ulogic_vector(c23(31 downto 0));
            matrix_ram_0(7) <= std_ulogic_vector(c24(31 downto 0));

            matrix_ram_0(8) <= std_ulogic_vector(c31(31 downto 0));
            matrix_ram_0(9) <= std_ulogic_vector(c32(31 downto 0));
            matrix_ram_0(10) <= std_ulogic_vector(c33(31 downto 0));
            matrix_ram_0(11) <= std_ulogic_vector(c34(31 downto 0));

            matrix_ram_0(12) <= std_ulogic_vector(c41(31 downto 0));
            matrix_ram_0(13) <= std_ulogic_vector(c42(31 downto 0));
            matrix_ram_0(14) <= std_ulogic_vector(c43(31 downto 0));
            matrix_ram_0(15) <= std_ulogic_vector(c44(31 downto 0));

            -- store the c temporaries into the second matrix engine
            register

```

```

matrix_ram_1(0) <= std_ulogic_vector(c11(31 downto 0));
matrix_ram_1(1) <= std_ulogic_vector(c12(31 downto 0));
matrix_ram_1(2) <= std_ulogic_vector(c13(31 downto 0));
matrix_ram_1(3) <= std_ulogic_vector(c14(31 downto 0));

matrix_ram_1(4) <= std_ulogic_vector(c21(31 downto 0));
matrix_ram_1(5) <= std_ulogic_vector(c22(31 downto 0));
matrix_ram_1(6) <= std_ulogic_vector(c23(31 downto 0));
matrix_ram_1(7) <= std_ulogic_vector(c24(31 downto 0));

matrix_ram_1(8) <= std_ulogic_vector(c31(31 downto 0));
matrix_ram_1(9) <= std_ulogic_vector(c32(31 downto 0));
matrix_ram_1(10) <= std_ulogic_vector(c33(31 downto 0));
matrix_ram_1(11) <= std_ulogic_vector(c34(31 downto 0));

matrix_ram_1(12) <= std_ulogic_vector(c41(31 downto 0));
matrix_ram_1(13) <= std_ulogic_vector(c42(31 downto 0));
matrix_ram_1(14) <= std_ulogic_vector(c43(31 downto 0));
matrix_ram_1(15) <= std_ulogic_vector(c44(31 downto 0));

end if;

if (load = '1') then
    -- data_out is the output of the LoadStoreUnit (LSU) and it
    -- contains the RAM/cache value
    -- which is loaded into the MatrixEngine
    if (to_integer(unsigned(matrix_immediate)) = 0) then
        matrix_ram_0(head) <= data_out;
    else
        matrix_ram_1(head) <= data_out;
    end if;
end if;

if (store = '1') then
    if (to_integer(unsigned(matrix_immediate)) = 0) then
        data_in <= matrix_ram_0(tail);
    else
        data_in <= matrix_ram_1(tail);
    end if;
end if;

if (matrix_add = '1') then
    for i in 0 to 15 loop
        matrix_ram_0(i) <= std_ulogic_vector(unsigned(matrix_ram_0
            (i)) + unsigned(matrix_immediate));
    end loop;
end if;

end if;
end process PROC_RAM;

-- Control Unit (Back-End / Instruction Execution) -----
-- -----
neorv32_cpu_control_inst: entity neorv32.neorv32_cpu_control
generic map (
    -- General --
    HART_ID          => HART_ID,          -- hardware thread ID

```

```

BOOT_ADDR      => BOOT_ADDR,          -- cpu boot address
DEBUG_PARK_ADDR => DEBUG_PARK_ADDR,    -- cpu debug mode
    parking loop entry address
DEBUG_EXC_ADDR => DEBUG_EXC_ADDR,    -- cpu debug mode
    exception entry address
-- RISC-V ISA Extensions --
RISCV_ISA_A     => riscv_a_c ,         -- atomic memory
    operations extension
RISCV_ISA_B     => riscv_b_c ,         -- bit-manipulation
    extension
RISCV_ISA_C     => RISCV_ISA_C ,       -- compressed extension
RISCV_ISA_E     => RISCV_ISA_E ,       -- embedded RF extension
RISCV_ISA_M     => RISCV_ISA_M ,       -- mul/div extension
RISCV_ISA_U     => RISCV_ISA_U ,       -- user mode extension
RISCV_ISA_Zaamo => RISCV_ISA_Zaamo ,   -- atomic read-modify-
    write operations extension
RISCV_ISA_Zalrsc => RISCV_ISA_Zalrsc , -- atomic reservation -
    set operations extension
RISCV_ISA_Zcb   => riscv_zcb_c ,       -- additional code size
    reduction instructions
RISCV_ISA_Zba   => RISCV_ISA_Zba ,     -- shifted-add bit -
    manipulation extension
RISCV_ISA_Zbb   => RISCV_ISA_Zbb ,     -- basic bit -
    manipulation extension
RISCV_ISA_Zbkb  => RISCV_ISA_Zbkb ,   -- bit-manipulation
    instructions for cryptography
RISCV_ISA_Zbkc  => RISCV_ISA_Zbkc ,   -- carry-less
    multiplication instructions
RISCV_ISA_Zbkx  => RISCV_ISA_Zbkx ,   -- cryptography crossbar
    permutation extension
RISCV_ISA_Zbs   => RISCV_ISA_Zbs ,     -- single-bit bit -
    manipulation extension
RISCV_ISA_Zfinx => RISCV_ISA_Zfinx ,   -- 32-bit floating-point
    extension
RISCV_ISA_Zibi  => RISCV_ISA_Zibi ,   -- branch with immediate
RISCV_ISA_Zicntr => RISCV_ISA_Zicntr , -- base counters
RISCV_ISA_Zicond => RISCV_ISA_Zicond , -- integer conditional
    operations
RISCV_ISA_Zihpm => RISCV_ISA_Zihpm ,   -- hardware performance
    monitors
RISCV_ISA_Zkn   => riscv_zkn_c ,       -- NIST algorithm suite
    available
RISCV_ISA_Zknd  => RISCV_ISA_Zknd ,   -- cryptography NIST AES
    decryption extension
RISCV_ISA_Zkne  => RISCV_ISA_Zkne ,   -- cryptography NIST AES
    encryption extension
RISCV_ISA_Zknh  => RISCV_ISA_Zknh ,   -- cryptography NIST
    hash extension
RISCV_ISA_Zks   => riscv_zks_c ,       -- ShangMi algorithm
    suite available
RISCV_ISA_Zksed => RISCV_ISA_Zksed ,   -- ShangMi block cipher
    extension
RISCV_ISA_Zksh  => RISCV_ISA_Zksh ,   -- ShangMi hash
    extension
RISCV_ISA_Zkt   => riscv_zkt_c ,       -- data-independent
    execution time for cryptography operations available
RISCV_ISA_Zmmul => RISCV_ISA_Zmmul ,   -- multiply-only M sub-
    extension
RISCV_ISA_Zxcfu => RISCV_ISA_Zxcfu ,   -- custom (instr.)
    functions unit
RISCV_ISA_Sdext => RISCV_ISA_Sdext ,   -- external debug mode
    extension
RISCV_ISA_Sdtrig => RISCV_ISA_Sdtrig , -- trigger module
    extension
RISCV_ISA_Smpmp => RISCV_ISA_Smpmp ,   -- physical memory
    protection

```

```

-- Tuning Options --
CPU_TRACE_EN      => CPU_TRACE_EN,      -- enable CPU execution
    trace generator
CPU_CONSTT_BR_EN  => CPU_CONSTT_BR_EN,    -- constant-time
    branches
CPU_FAST_MUL_EN   => CPU_FAST_MUL_EN,     -- use DSPs for M
    extension's multiplier
CPU_FAST_SHIFT_EN => CPU_FAST_SHIFT_EN,   -- use barrel shifter
    for shift operations
CPU_RF_HW_RST_EN  => CPU_RF_HW_RST_EN     -- enable full hardware
    reset for register file
)
port map (
-- global control --
    clk_i      => clk_i,                  -- global clock, rising edge
    rstn_i     => rstn_i,                  -- global reset, low-active,
        async
    ctrl_o     => ctrl,                    -- main control bus
-- misc --
    frontend_i => frontend,                -- front-end status and data
    pmp_fault_i => pmp_fault,              -- instruction fetch / execute
        pmp fault
    hwtrig_i   => hwtrig,                  -- hardware trigger
-- [debug] - make instruction visible
    debug_valid_i => frontend.valid,       -- bus signals are valid
    debug_instr_i => frontend.instr,       -- instruction
-- matrix extension
    matrix_add_o      => matrix_add,
    matrix_mul_o      => matrix_mul,
    matrix_immediate_o => matrix_immediate,
    matrix_mar_load_store_o => matrix_mar_load_store,
    matrix_operation_wait_i => matrix_operation_wait, -- make the
        execution engine wait for the matrix engine
-- data path interface --
    alu_cp_done_i => alu_cp_done, -- ALU iterative operation done
    alu_cmp_i     => alu_cmp,     -- comparator status
    alu_add_i     => alu_add,     -- ALU address result
    rf_rs1_i     => rs1,         -- rf source 1
    csr_rdata_o  => csr_rdata,    -- CSR read data
    xcsr_rdata_i => xcsr_res,     -- external CSR read data
-- interrupts --
    irq_dbg_i    => dbi_i,        -- debug mode (halt) request
    irq_machine_i => irq_machine, -- risc-v mei, mti, msi
    irq_fast_i   => firq_i,       -- fast interrupts
-- from/to load/store unit interface --
    lsu_wait_i   => lsu_wait,     -- make the execution engine wait
        for data bus
    lsu_mar_i    => lsu_mar,       -- memory address register
    lsu_err_i    => lsu_err,      -- alignment/access errors
);

-- RISC-V machine interrupts --
irq_machine <= mei_i & mti_i & msi_i;

-- control-external CSR read-back --
xcsr_res <= xcsr_tm or xcsr_cnt or xcsr_alu or xcsr_pmp;

-- CPU is sleeping --
sleep_o <= not ctrl.cnt_event(cnt_event_cy_c);

-- Hardware Trigger Module (Sdtrig) -----
-- -----
trigger_module_enabled:
if (RISCV_ISA_Sdtrig = true) and (NUM_HW_TRIGGERS > 0) generate
    neorv32_cpu_hwtrig_inst: entity neorv32.neorv32_cpu_hwtrig

```

```

generic map (
    NUM_TRIGGERS => NUM_HW_TRIGGERS, -- number of implemented
        hardware triggers
    RISC_V_ISA_U => RISC_V_ISA_U      -- RISC-V user-mode available
)
port map (
    -- global control --
    clk_i => clk_i, -- global clock, rising edge
    rstn_i => rstn_i, -- global reset, low-active, async
    ctrl_i => ctrl, -- main control bus
    -- data path --
    mar_i => lsu_mar, -- memory address register
    csr_o => xcsr_tm, -- CSR read data
    -- trigger firing --
    hit_o => hwtrig -- high until debug-mode is entered
);
end generate;

trigger_module_disabled:
if (RISC_V_ISA_Sdtrig = false) or (NUM_HW_TRIGGERS = 0) generate
    xcsr_tm <= (others => '0');
    hwtrig <= '0';
end generate;

-- Hardware Counters -----
-- -----
cnts_enabled:
if RISC_V_ISA_Zicntr or RISC_V_ISA_Zihpm generate
    neorv32_cpu_counters_inst: entity neorv32.neorv32_cpu_counters
    generic map (
        ZICNTR_EN => RISC_V_ISA_Zicntr, -- implement base counters
        ZIHPM_EN => RISC_V_ISA_Zihpm, -- implement hardware performance
            monitors (HPMs)
        HPM_NUM => HPM_NUM_CNTS, -- number of implemented HPM
            counters (0..13)
        HPM_WIDTH => HPM_CNT_WIDTH -- total size of HPM counters
            (0..64)
    )
    port map (
        -- global control --
        clk_i => clk_i, -- global clock, rising edge
        rstn_i => rstn_i, -- global reset, low-active, async
        ctrl_i => ctrl, -- main control bus
        -- read back --
        rdata_o => xcsr_cnt -- read data
    );
end generate;

cnts_disabled:
if (not RISC_V_ISA_Zicntr) and (not RISC_V_ISA_Zihpm) generate
    xcsr_cnt <= (others => '0');
end generate;

-- X Register File (Normal RV32) -----
-- -----
neorv32_cpu_regfile_inst: entity neorv32.neorv32_cpu_regfile
generic map (
    RST_EN => CPU_RF_HW_RST_EN, -- enable dedicated hardware reset
        ("ASIC style")
    RVE_EN => RISC_V_ISA_E -- implement embedded RF extension
)
port map (
    -- global control --
    clk_i => clk_i, -- global clock, rising edge

```

```

    rstn_i => rstn_i,    -- global reset, low-active, async
    ctrl_i => ctrl,      -- main control bus
    -- operands --
    rd_i   => rf_wdata,  -- destination operand rd
    rs1_o  => rs1,       -- source operand rs1
    rs2_o  => rs2        -- source operand rs2
);

-- signal into the x register file (normal RV32)
-- all buses are zero unless there is an according operation --
rf_wdata <= alu_res or lsu_rdata or csr_rdata or ctrl.pc_ret;

-- M Register File (Normal RV32) -----
neorv32_cpu_m_regfile_inst: entity neorv32.neorv32_m_cpu_regfile
generic map (
    RST_EN => CPU_RF_HW_RST_EN, -- enable dedicated hardware reset
    ("ASIC style")
    RVE_EN => RISCV_ISA_E       -- implement embedded RF extension
)
port map (
    -- global control --
    clk_i  => clk_i,    -- global clock, rising edge
    rstn_i => rstn_i,    -- global reset, low-active, async
    ctrl_i => ctrl,      -- main control bus
    -- operands --
    rd_i   => m_rf_wdata --, -- destination operand rd
);

-- signal into the m register file (matrix extension)
-- all buses are zero unless there is an according operation --
m_rf_wdata <= lsu_rdata;

-- Arithmetic/Logic Unit (ALU) and ALU Co-Processors -----
neorv32_cpu_alu_inst: entity neorv32.neorv32_cpu_alu
generic map (
    -- RISC-V CPU Extensions --
    RISCV_ISA_M      => RISCV_ISA_M,    -- mul/div extension
    RISCV_ISA_Zba     => RISCV_ISA_Zba,  -- address-generation
        instruction
    RISCV_ISA_Zbb     => RISCV_ISA_Zbb,  -- basic bit-manipulation
        instruction
    RISCV_ISA_Zbkb    => RISCV_ISA_Zbkb, -- bit-manipulation
        instructions for cryptography
    RISCV_ISA_Zbkc    => RISCV_ISA_Zbkc, -- carry-less
        multiplication instructions
    RISCV_ISA_Zbkx    => RISCV_ISA_Zbkx, -- cryptography crossbar
        permutation extension
    RISCV_ISA_Zbs     => RISCV_ISA_Zbs,  -- single-bit instructions
    RISCV_ISA_Zfinx   => RISCV_ISA_Zfinx, -- 32-bit floating-point
        extension
    RISCV_ISA_Zibi    => RISCV_ISA_Zibi,  -- branch with immediate
    RISCV_ISA_Zicond  => RISCV_ISA_Zicond, -- integer conditional
        operations
    RISCV_ISA_Zknd    => RISCV_ISA_Zknd,  -- cryptography NIST AES
        decryption extension
    RISCV_ISA_Zkne    => RISCV_ISA_Zkne,  -- cryptography NIST AES
        encryption extension
    RISCV_ISA_Zknh    => RISCV_ISA_Zknh,  -- cryptography NIST hash
        extension
    RISCV_ISA_Zksed   => RISCV_ISA_Zksed, -- ShangMi block cipher
        extension
    RISCV_ISA_Zksh    => RISCV_ISA_Zksh,  -- ShangMi hash extension

```

```

RISCV_ISA_Zmmul => RISCV_ISA_Zmmul, -- multiply-only M sub-
    extension
RISCV_ISA_Zxcfu => RISCV_ISA_Zxcfu, -- custom (instr.)
    functions unit
-- Tuning Options --
FAST_MUL_EN      => CPU_FAST_MUL_EN, -- use DSPs for M
    extension's multiplier
FAST_SHIFT_EN    => CPU_FAST_SHIFT_EN -- use barrel shifter for
    shift operations
)
port map (
-- global control --
clk_i  => clk_i,      -- global clock, rising edge
rstn_i => rstn_i,     -- global reset, low-active, async
ctrl_i => ctrl,       -- main control bus
-- [debug] --
debug_alu_op => ctrl.alu_op,
-- data input --
rs1_i => rs1,        -- rf source 1
rs2_i => rs2,        -- rf source 2
-- data output --
cmp_o  => alu_cmp,    -- comparator status
res_o  => alu_res,    -- ALU result
add_o  => alu_add,    -- address computation result
csr_o  => xcsr_alu,   -- CSR read data
-- status --
done_o => alu_cp_done -- iterative processing units done?
);

-- -- Load/Store Unit (LSU) -----
-- -- -----

neorv32_cpu_m_lsu_inst: entity neorv32.neorv32_cpu_m_lsu
port map (
-- global control --
clk_i      => clk_i,      -- global clock, rising edge
rstn_i     => rstn_i,     -- global reset, low-active, async
ctrl_i     => ctrl,       -- main control bus
-- debug
lsu_req_i  => ctrl.lsu_req,
lsu_en_i   => ctrl.lsu_m_en,
-- matrix extension
head_o     => head,
tail_o     => tail,
load_o     => load,
store_o    => store,
data_out_o => data_out, -- data_out is from CPU RAM to Matrix
    RAM
data_in_i  => data_in, -- data_out is from Matrix RAM to CPU
    RAM
matrix_immediate_i => matrix_immediate, -- immediate value from
    the
matrix_mar_load_store_i => matrix_mar_load_store, -- memory
    address
-- cpu data access interface --
addr_i     => alu_add,    -- access address
wdata_i    => rs2,       -- write data
rdata_o    => lsu_rdata,  -- read data
mar_o      => lsu_mar,    -- memory address register
wait_o     => lsu_wait,   -- wait for access to complete -- this
    signal tells the execution engine to wait for the load/
    store to complete before advancing
err_o      => lsu_err,    -- alignment/access errors
pmp_fault_i => pmp_fault, -- PMP read/write access fault
-- data bus --

```

```

        dbus_req_o => dbus_req, -- request
        dbus_rsp_i => dbus_rsp_i, -- response
        -- debug
        dbus_rsp_i_ack => dbus_rsp_i.ack
    );

    dbus_req_o <= dbus_req;

-- Physical Memory Protection (PMP) -----
-- -----
pmp_enabled:
if RISC_V_ISA_Smpmp generate
    neorv32_cpu_pmp_inst: entity neorv32.neorv32_cpu_pmp
    generic map (
        NUM_REGIONS => PMP_NUM_REGIONS, -- number of regions
        (0..16)
        GRANULARITY => PMP_MIN_GRANULARITY, -- minimal region
        granularity in bytes, has to be a power of 2, min 4 bytes
        TOR_EN => PMP_TOR_MODE_EN, -- enable TOR mode
        NAP_EN => PMP_NAP_MODE_EN -- enable NAPOT/NA4 modes
    )
    port map (
        -- global control --
        clk_i => clk_i, -- global clock, rising edge
        rstn_i => rstn_i, -- global reset, low-active, async
        ctrl_i => ctrl, -- main control bus
        -- operands --
        csr_o => xcsr_pmp, -- CSR read data
        rs1_i => rs1, -- data access base address
        -- access error --
        fault_o => pmp_fault -- permission violation
    );
end generate;

pmp_disabled:
if not RISC_V_ISA_Smpmp generate
    xcsr_pmp <= (others => '0');
    pmp_fault <= '0';
end generate;

-- Trace Generator -----
-- -----
trace_enabled:
if CPU_TRACE_EN generate
    neorv32_cpu_trace_inst: entity neorv32.neorv32_cpu_trace
    port map (
        -- global control --
        clk_i => clk_i, -- global clock, rising edge
        rstn_i => rstn_i, -- global reset, low-active,
        async
        ctrl_i => ctrl, -- main control bus
        -- operands --
        rs1_rdata_i => rs1, -- rs1 read data
        rs2_rdata_i => rs2, -- rs2 read data
        rd_wdata_i => rf_wdata, -- rd write data
        mem_ben_i => dbus_req.ben, -- memory byte-enable
        mem_addr_i => dbus_req.addr, -- memory address
        mem_wdata_i => dbus_req.data, -- memory write data
        -- trace port --
        trace_o => trace_o -- execution trace port
    );
end generate;

trace_disabled:

```

```

if not CPU_TRACE_EN generate
    trace_o <= trace_port_terminate_c;
end generate;

```

```

end neorv32_cpu_rtl;

```

Quellcode 2: neorv32_cpu_control.vhd

```

-- #warning-ignore-all
--
-- =====
--
-- NEORV32 CPU - Central Control Unit
--
--
-- -----
--
-- + Execute engine: Multi-cycle execution of instructions ("back-
--                   end")
-- + Trap controller: Handles interrupts and exceptions
--
-- + CSR module:      Read/write access to control and status
--                   registers
-- + Debug module:    CPU debug mode handling (on-chip debugger)
--
-- + Trigger module:  Hardware-assisted breakpoints (on-chip
--                   debugger)
--
-- -----
--
-- The NEORV32 RISC-V Processor - https://github.com/stnolting/
-- neorv32
-- Copyright (c) NEORV32 contributors.
--
-- Copyright (c) 2020 - 2025 Stephan Nolting. All rights reserved.
--
-- Licensed under the BSD-3-Clause license, see LICENSE for details.
--
-- SPDX-License-Identifier: BSD-3-Clause
--
-- =====
--
use std.textio.all; -- Imports the standard textio package

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library neorv32;
use neorv32.neorv32_package.all;

entity neorv32_cpu_control is
generic (
    -- General --
    HART_ID          : natural range 0 to 1023; -- hardware thread
    ID
    BOOT_ADDR        : std_ulogic_vector(31 downto 0); -- cpu boot
    address
    DEBUG_PARK_ADDR   : std_ulogic_vector(31 downto 0); -- cpu debug
    -mode parking loop entry address, 4-byte aligned

```

```

DEBUG_EXC_ADDR    : std_ulogic_vector(31 downto 0); -- cpu debug
                  -mode exception entry address, 4-byte aligned
-- RISC-V ISA Extensions --
RISCV_ISA_A       : boolean; -- atomic memory operations
                  extension
RISCV_ISA_B       : boolean; -- bit-manipulation extension
RISCV_ISA_C       : boolean; -- compressed extension
RISCV_ISA_E       : boolean; -- embedded-class register file
                  extension
RISCV_ISA_M       : boolean; -- mul/div extension
RISCV_ISA_U       : boolean; -- user mode extension
RISCV_ISA_Zaamo   : boolean; -- atomic read-modify-write
                  extension
RISCV_ISA_Zalrsc  : boolean; -- atomic reservation-set
                  operations extension
RISCV_ISA_Zcb     : boolean; -- additional code size reduction
                  instructions
RISCV_ISA_Zba     : boolean; -- shifted-add bit-manipulation
                  extension
RISCV_ISA_Zbb     : boolean; -- basic bit-manipulation extension
RISCV_ISA_Zbkb    : boolean; -- bit-manipulation instructions
                  for cryptography
RISCV_ISA_Zbkc    : boolean; -- carry-less multiplication
                  instructions
RISCV_ISA_Zbkx    : boolean; -- cryptography crossbar
                  permutation extension
RISCV_ISA_Zbs     : boolean; -- single-bit bit-manipulation
                  extension
RISCV_ISA_Zfinx   : boolean; -- 32-bit floating-point extension
RISCV_ISA_Zibi    : boolean; -- branch with immediate
RISCV_ISA_Zicntr  : boolean; -- base counters
RISCV_ISA_Zicond  : boolean; -- integer conditional operations
RISCV_ISA_Zihpm   : boolean; -- hardware performance monitors
RISCV_ISA_Zkn     : boolean; -- NIST algorithm suite
RISCV_ISA_Zknd    : boolean; -- cryptography NIST AES decryption
                  extension
RISCV_ISA_Zkne    : boolean; -- cryptography NIST AES encryption
                  extension
RISCV_ISA_Zknh    : boolean; -- cryptography NIST hash extension
RISCV_ISA_Zks     : boolean; -- ShangMi algorithm suite
RISCV_ISA_Zksed   : boolean; -- ShangMi block cipher extension
RISCV_ISA_Zksh    : boolean; -- ShangMi hash extension
RISCV_ISA_Zkt     : boolean; -- data-independent execution time
                  (for cryptography operations)
RISCV_ISA_Zmmul   : boolean; -- multiply-only M sub-extension
RISCV_ISA_Zxcfu   : boolean; -- custom (instr.) functions unit
RISCV_ISA_Sdext   : boolean; -- external debug mode extension
RISCV_ISA_Sdtrig  : boolean; -- trigger module extension
RISCV_ISA_Smpmp   : boolean; -- physical memory protection
-- Tuning Options --
CPU_TRACE_EN     : boolean; -- enable CPU execution trace
                  generator
CPU_CONSTT_BR_EN : boolean; -- constant-time branches
CPU_FAST_MUL_EN  : boolean; -- use DSPs for M extension's
                  multiplier
CPU_FAST_SHIFT_EN : boolean; -- use barrel shifter for shift
                  operations
CPU_RF_HW_RST_EN : boolean; -- enable full hardware reset for
                  register file
);
port (
  -- global control --
  clk_i      : in  std_ulogic;          --
              global clock, rising edge
  rstn_i     : in  std_ulogic;          --
              global reset, low-active, async

```

```

ctrl_o      : out ctrl_bus_t;           -- main
    control bus
-- misc --
frontend_i  : in  if_bus_t;           -- front
    -end status and data
pmp_fault_i : in  std_ulogic;         --
    instruction fetch / execute pmp fault
hwtrig_i    : in  std_ulogic;         --
    hardware trigger
-- [debug] - make instruction visible
debug_valid_i : in  std_ulogic;       -- bus
    signals are valid
debug_instr_i : in  std_ulogic_vector(31 downto 0); --
    instruction
-- matrix extension
matrix_add_o      : out std_ulogic;
matrix_mul_o      : out std_ulogic;
matrix_immediate_o : out std_ulogic_vector(XLEN-1 downto 0)
;
matrix_mar_load_store_o : out std_ulogic_vector(XLEN-1 downto 0)
;
matrix_operation_wait_i : in  std_ulogic; -- make the execution
    engine wait for the matrix engine
-- data path interface --
alu_cp_done_i : in  std_ulogic;       -- ALU
    iterative operation done
alu_cmp_i     : in  std_ulogic_vector(1 downto 0); --
    comparator status
alu_add_i     : in  std_ulogic_vector(XLEN-1 downto 0); -- ALU
    address result
rf_rs1_i     : in  std_ulogic_vector(XLEN-1 downto 0); -- rf
    source 1
csr_rdata_o  : out std_ulogic_vector(XLEN-1 downto 0); -- CSR
    read data
xcsr_rdata_i : in  std_ulogic_vector(XLEN-1 downto 0); --
    external CSR read data
-- interrupts --
irq_dbg_i    : in  std_ulogic;       -- debug
    mode (halt) request
irq_machine_i : in  std_ulogic_vector(2 downto 0); -- risc-
    v mei, mti, msi
irq_fast_i   : in  std_ulogic_vector(15 downto 0); -- fast
    interrupts
-- load/store unit interface --
lsu_wait_i   : in  std_ulogic;       -- wait
    for data bus
lsu_mar_i    : in  std_ulogic_vector(XLEN-1 downto 0); --
    memory address register
lsu_err_i    : in  std_ulogic_vector(3 downto 0)       --
    alignment/access errors
);
end neorv32_cpu_control;

architecture neorv32_cpu_control_rtl of neorv32_cpu_control is

    -- instruction execution engine --
    type exe_engine_state_t is (
        EX_RESTART, EX_DISPATCH, EX_TRAP_ENTER, EX_TRAP_EXIT, EX_SLEEP,
        EX_EXECUTE,
        EX_ALU_WAIT, EX_BRANCH, EX_BRANCHED, EX_SYSTEM, EX_MEM_REQ,
        EX_MATRIX_MEM_REQ,
        EX_MEM_RSP, EX_MATRIX_MEM_RSP, EX_MATRIX_OPERATION_REQ,
        EX_MATRIX_OPERATION_RSP
    );

    type exe_engine_t is record

```

```

state : exe_engine_state_t;
ir      : std_ulogic_vector(31 downto 0); -- instruction word
        being executed right now
ci      : std_ulogic; -- current instruction is de-compressed
        instruction
pc      : std_ulogic_vector(XLEN-1 downto 0); -- current PC (
        current instruction)
pc2     : std_ulogic_vector(XLEN-1 downto 0); -- next PC (next
        linear instruction)
ra      : std_ulogic_vector(XLEN-1 downto 0); -- return address
end record;
signal exe_engine, exe_engine_nxt : exe_engine_t;

-- trap controller --
type trap_ctrl_t is record
exc_buf  : std_ulogic_vector(exc_width_c-1 downto 0); --
        synchronous exception buffer (one bit per exception)
exc_fire : std_ulogic; -- set if there is a valid source in
        the exception buffer
irq_pnd  : std_ulogic_vector(irq_width_c-1 downto 0); --
        pending interrupt
irq_buf  : std_ulogic_vector(irq_width_c-1 downto 0); --
        asynchronous exception/interrupt buffer (one bit per
        interrupt source)
irq_fire : std_ulogic_vector(1 downto 0); -- set if an
        interrupt is actually kicking in
cause    : std_ulogic_vector(6 downto 0); -- trap ID for
        mcause CSR & debug-mode entry identifier
pc       : std_ulogic_vector(XLEN-1 downto 0); -- trap
        program counter
--
env_pending : std_ulogic; -- start of trap environment if
        pending
env_enter   : std_ulogic; -- enter trap environment
env_exit    : std_ulogic; -- leave trap environment
--
instr_be    : std_ulogic; -- instruction fetch bus error
instr_ma    : std_ulogic; -- instruction fetch misaligned
        address
instr_il    : std_ulogic; -- illegal instruction
ecall       : std_ulogic; -- ecall instruction
ebreak      : std_ulogic; -- ebreak instruction
end record;
signal trap_ctrl : trap_ctrl_t;

-- CPU control bus --
signal ctrl, ctrl_nxt : ctrl_bus_t;

-- control and status registers (CSRs) --
type csr_t is record
addr      : std_ulogic_vector(11 downto 0); -- physical
        access address
we, we_nxt : std_ulogic; -- write enable
re, re_nxt : std_ulogic; -- read enable
operand    : std_ulogic_vector(XLEN-1 downto 0); -- write
        operand
wdata      : std_ulogic_vector(XLEN-1 downto 0); -- write
        data
rdata      : std_ulogic_vector(XLEN-1 downto 0); -- read
        data
--
mstatus_mie : std_ulogic; -- machine-mode IRQ enable
mstatus_mpie : std_ulogic; -- previous machine-mode IRQ enable
mstatus_mpp : std_ulogic; -- machine previous privilege mode
mstatus_mprv : std_ulogic; -- effective privilege level for
        load/stores

```

```

mstatus_tw      : std_ulogic; -- do not allow user mode to
                        execute WFI instruction when set
--
mie_msi         : std_ulogic; -- machine software interrupt
                        enable
mie_mei         : std_ulogic; -- machine external interrupt
                        enable
mie_mti         : std_ulogic; -- machine timer interrupt enable
mie_firq        : std_ulogic_vector(15 downto 0); -- fast
                        interrupt enable
--
prv_level       : std_ulogic; -- current privilege level
prv_level_eff   : std_ulogic; -- current *effective* privilege
                        level
--
mepc            : std_ulogic_vector(XLEN-1 downto 0); -- machine
                        exception PC
mcause          : std_ulogic_vector(5 downto 0); -- machine trap
                        cause
mtvec           : std_ulogic_vector(XLEN-1 downto 0); -- machine
                        trap-handler base address
mtval           : std_ulogic_vector(XLEN-1 downto 0); -- machine
                        bad address or instruction
mtinst          : std_ulogic_vector(XLEN-1 downto 0); -- machine
                        trap instruction
mscratch        : std_ulogic_vector(XLEN-1 downto 0); -- machine
                        scratch register
mcounteren_cy   : std_ulogic; -- machine counter access enable:
                        cycle counter
mcounteren_ir   : std_ulogic; -- machine counter access enable:
                        instruction counter
--
dcsr_ebreakm    : std_ulogic; -- behavior of ebreak instruction
                        in m-mode
dcsr_ebreaku    : std_ulogic; -- behavior of ebreak instruction
                        in u-mode
dcsr_step       : std_ulogic; -- single-step mode
dcsr_prv        : std_ulogic; -- current privilege level when
                        entering debug mode
dcsr_cause      : std_ulogic_vector(2 downto 0); -- why was debug
                        mode entered
dcsr_rd         : std_ulogic_vector(XLEN-1 downto 0); -- debug
                        mode control and status register
dpc             : std_ulogic_vector(XLEN-1 downto 0); -- mode
                        program counter
dscratch0       : std_ulogic_vector(XLEN-1 downto 0); -- debug
                        mode scratch register 0
end record;
signal csr : csr_t;

-- debug-mode controller --
type debug_ctrl_t is record
    run, trig_hw, trig_break, trig_halt, trig_step : std_ulogic;
end record;
signal debug_ctrl : debug_ctrl_t;

-- misc/helpers --
signal if_reset : std_ulogic; -- reset instruction fetch (
                                front-end)
signal branch_taken : std_ulogic; -- fulfilled branch condition or
                                unconditional jump
signal monitor_cnt : std_ulogic_vector(alu_cp_tmo_c downto 0); --
                                execution monitor cycle counter
signal monitor_exc : std_ulogic; -- execution monitor timeout
                                exception

```

```

signal opcode      : std_ulogic_vector(6 downto 0); -- simplified
                   : opcode (2 LSBs hardwired to "11" to indicate rv32)
signal immediate   : std_ulogic_vector(XLEN-1 downto 0); --
                   : instruction's immediate
signal illegal_cmd : std_ulogic; -- illegal instruction check
signal csr_valid   : std_ulogic_vector(2 downto 0); -- CSR access
                   : [2] implemented, [1] r/w access, [0] privilege
signal cnt_event   : std_ulogic_vector(11 downto 0); -- counter
                   : events
signal ebreak_trig : std_ulogic; -- "ebreak" exception trigger

begin

--
-- *****
--
-- Instruction Execution
--
-- *****
--
-- Immediate Generator
--
-- -----

imm_gen: process(rstn_i, clk_i)
begin
  if (rstn_i = '0') then
    immediate <= (others => '0');
  elsif rising_edge(clk_i) then
    if (exe_engine.state = EX_DISPATCH) then -- prepare update of
      next PC (using ALU's PC + IMM in EX_EXECUTE state)
      if RISCV_ISA_C and (frontend_i.compr = '1') then -- is
        decompressed C instruction?
        immediate <= x"00000002";
      else
        immediate <= x"00000004";
      end if;
    else
      case opcode is
        when opcode_store_c => -- S-immediate
          immediate <= replicate_f(exe_engine.ir(31), 21) &
            exe_engine.ir(30 downto 25) & exe_engine.ir(11
              downto 7);
        when opcode_branch_c => -- B-immediate
          immediate <= replicate_f(exe_engine.ir(31), 20) &
            exe_engine.ir(7) & exe_engine.ir(30 downto 25) &
            exe_engine.ir(11 downto 8) & '0';
        when opcode_lui_c | opcode_auiipc_c => -- U-immediate
          immediate <= exe_engine.ir(31 downto 12) & x"000";
        when opcode_jal_c => -- J-immediate
          immediate <= replicate_f(exe_engine.ir(31), 12) &
            exe_engine.ir(19 downto 12) & exe_engine.ir(20) &
            exe_engine.ir(30 downto 21) & '0';
        when opcode_amo_c => -- atomic memory access
          immediate <= (others => '0');
        when others => -- I-immediate
          immediate <= replicate_f(exe_engine.ir(31), 21) &
            exe_engine.ir(30 downto 21) & exe_engine.ir(20);
        end case;
      end if;
    end if;
  end if;

```

```

end process imm_gen;

-- Branch Condition Check
-----

--
-----

branch_check: process(exe_engine, alu_cmp_i)
begin
    if (exe_engine.ir(instr_opcode_lsb_c+2) = '0') then --
        conditional branch
        if (exe_engine.ir(instr_func3_msb_c) = '0') then -- beq / bne
            branch_taken <= alu_cmp_i(cmp_equal_c) xor exe_engine.ir(
                instr_func3_lsb_c);
        else -- blt(u) / bge(u)
            branch_taken <= alu_cmp_i(cmp_less_c) xor exe_engine.ir(
                instr_func3_lsb_c);
        end if;
    else -- unconditional branch
        branch_taken <= '1';
    end if;
end process branch_check;

-- Execute Engine FSM (Micro Sequencer) Sync
-----

--
-----

execute_engine_fsm_sync: process(rstn_i, clk_i)
begin
    if (rstn_i = '0') then
        ctrl <= ctrl_bus_zero_c;
        exe_engine.state <= EX_RESTART;
        exe_engine.ir <= (others => '0');
        exe_engine.ci <= '0';
        exe_engine.pc <= BOOT_ADDR(XLEN-1 downto 2) & "00"; -- 32-
            bit-aligned boot address
        exe_engine.pc2 <= BOOT_ADDR(XLEN-1 downto 2) & "00"; -- 32-
            bit-aligned boot address
        exe_engine.ra <= (others => '0');
    elsif rising_edge(clk_i) then
        ctrl <= ctrl_nxt;
        exe_engine <= exe_engine_nxt;
    end if;
end process execute_engine_fsm_sync;

-- simplified rv32 opcode --
opcode <= exe_engine.ir(instr_opcode_msb_c downto
    instr_opcode_lsb_c+2) & "11";

-- Execute Engine FSM (Micro Sequencer) Comb
-----

--
-----

execute_engine_fsm_comb: process(
    exe_engine,
    debug_ctrl,
    trap_ctrl,
    hwtrig_i,
    opcode,
    frontend_i,

```

```

csr,
ctrl,
alu_cp_done_i,
lsu_wait_i,
alu_add_i,
branch_taken,
pmp_fault_i,
matrix_operation_wait_i -- YOU HAVE TO ADD YOUR SIGNALS TO THIS
                           LIST AS THE EXCEUTION STATE MACHINE IS NOT CLOCKED BUT ONLY
                           REACTS TO SIGNALS!
)

variable funct3_v : std_ulogic_vector(2 downto 0);
variable funct7_v : std_ulogic_vector(6 downto 0);

-- DEBUG
variable l : line;

begin
  -- shortcuts --
  funct3_v := exe_engine.ir(instr_funct3_msb_c downto
    instr_funct3_lsb_c);
  funct7_v := exe_engine.ir(instr_funct7_msb_c downto
    instr_funct7_lsb_c);

  -- arbiter defaults --
  exe_engine_nxt.state <= exe_engine.state;
  exe_engine_nxt.ir    <= exe_engine.ir;
  exe_engine_nxt.ci    <= exe_engine.ci;
  exe_engine_nxt.pc    <= exe_engine.pc;
  exe_engine_nxt.pc2   <= exe_engine.pc2;
  exe_engine_nxt.ra    <= (others => '0'); -- output zero if not a
    branch instruction
  if_reset             <= '0';
  trap_ctrl.env_enter  <= '0';
  trap_ctrl.env_exit   <= '0';
  trap_ctrl.instr_be   <= '0';
  trap_ctrl.instr_ma   <= '0';
  trap_ctrl.ecall      <= '0';
  trap_ctrl.ebreak     <= '0';
  csr.we_nxt           <= '0';
  csr.re_nxt           <= '0';
  ctrl_nxt              <= ctrl_bus_zero_c; -- all zero/off by
    default (ALU operation = ZERO, ALU.adder_out = ADD)

  -- keep the matrix engine state as it was
  ctrl_nxt.lsu_m_en    <= ctrl.lsu_m_en;

  -- ALU sign control --
  if (opcode(4) = '1') then -- ALU ops
    ctrl_nxt.alu_unsigned <= funct3_v(0); -- unsigned ALU
      operation? (SLTIU, SLTU)
  else -- branches
    ctrl_nxt.alu_unsigned <= funct3_v(1); -- unsigned branches? (
      BLTU, BGEU)
  end if;

  -- ALU operand A: is PC? --
  case opcode is
    when opcode_auipc_c | opcode_jal_c | opcode_branch_c =>
      ctrl_nxt.alu_opa_mux <= '1';
    when others =>
      ctrl_nxt.alu_opa_mux <= '0';
  end case;

  -- ALU operand B: is immediate? --

```

```

case opcode is
  -- WBI: added matrix opcode here because the mle32, mse32
    instructions contain an immediate
  -- that needs to be added to the register encoded in the
    instruction
  when opcode_matrix_c | opcode_alui_c | opcode_lui_c |
    opcode_auiipc_c | opcode_load_c | opcode_store_c |
    opcode_amo_c | opcode_branch_c | opcode_jal_c |
    opcode_jalr_c =>
    ctrl_nxt.alu_opb_mux <= '1';
  when others =>
    ctrl_nxt.alu_opb_mux <= '0';
end case;

-- (atomic) memory read/write access --
if RISC_V_ISA_Zaamo and (opcode(2) = opcode_amo_c(2)) and (
  exe_engine.ir(instr_funct5_lsb_c+1) = '0') then -- atomic
  read-modify-write operation
  ctrl_nxt.lsu_rmw <= '1'; -- read-modify-write
  ctrl_nxt.lsu_rvs <= '0';
  ctrl_nxt.lsu_rw <= '0'; -- executed as single load for the
    CPU
elsif RISC_V_ISA_Zalrsc and (opcode(2) = opcode_amo_c(2)) and (
  exe_engine.ir(instr_funct5_lsb_c+1) = '1') then -- atomic
  reservation-set operation
  ctrl_nxt.lsu_rmw <= '0';
  ctrl_nxt.lsu_rvs <= '1'; -- reservation-set
  ctrl_nxt.lsu_rw <= exe_engine.ir(instr_funct5_lsb_c);
else -- normal load/store
  ctrl_nxt.lsu_rmw <= '0';
  ctrl_nxt.lsu_rvs <= '0';
  ctrl_nxt.lsu_rw <= exe_engine.ir(instr_opcode_lsb_c+5);
end if;

-- state machine --
case exe_engine.state is

  when EX_RESTART => -- reset and restart instruction fetch at
    next PC
  --   write(l, String'("EX_RESTART"));
  --   writeline(output, l);
  --
  -----

  ctrl_nxt.rf_zero_we <= not bool_to_ulogic_f(
    CPU_RF_HW_RST_EN); -- house keeping: force writing zero
    to x0 if it's a phys. register
  if_reset <= '1';
  exe_engine_nxt.state <= EX_BRANCHED; -- delay cycle to
    restart front-end

  when EX_DISPATCH => -- wait for ISSUE ENGINE to emit a valid
    instruction word
  --
  -----

  -- debug
  --   write(l, String'("EX_DISPATCH"));
  --   writeline(output, l);

  ctrl_nxt.alu_opa_mux <= '1'; -- prepare update of next PC in
    EX_EXECUTE (opa = current PC)
  ctrl_nxt.alu_opb_mux <= '1'; -- prepare update of next PC in
    EX_EXECUTE (opb = imm = +2/4)
  --
  matrix_add_o <= '0';

```

```

matrix_mul_o <= '0';
--
if (trap_ctrl.env_pending = '1') or (trap_ctrl.exc_fire =
    '1') then -- pending trap or pending exception (fast)
    exe_engine_nxt.state <= EX_TRAP_ENTER;
elseif (frontend_i.valid = '1') and (hwtrig_i = '0') then --
    new instruction word available and no pending HW trigger
    trap_ctrl.instr_be <= frontend_i.fault or pmp_fault_i;
    -- access fault during instruction fetch
    exe_engine_nxt.ci <= frontend_i.compr; -- this is a de-
        compressed instruction
    exe_engine_nxt.ir <= frontend_i.instr; -- instruction
        word
    exe_engine_nxt.pc <= exe_engine.pc2(XLEN-1 downto 1) &
        '0'; -- PC <= next PC
    exe_engine_nxt.state <= EX_EXECUTE; -- start executing new
        instruction
end if;

when EX_TRAP_ENTER => -- enter trap environment and jump to
    trap vector
--     write(l, String'("EX_TRAP_ENTER"));
--     writeline(output, l);
--
-----

if (trap_ctrl.cause(5) = '1') and RISC_V_ISA_Sdext then --
    debug mode (re-)entry
    exe_engine_nxt.pc2 <= DEBUG_PARK_ADDR(XLEN-1 downto 2) & "
        00"; -- debug mode enter; start at "parking loop" <
        normal_entry>
elseif (debug_ctrl.run = '1') and RISC_V_ISA_Sdext then -- any
    other trap INSIDE debug mode
    exe_engine_nxt.pc2 <= DEBUG_EXC_ADDR(XLEN-1 downto 2) & "
        00"; -- debug mode enter: start at "parking loop" <
        exception_entry>
elseif (csr.mtvec(0) = '1') and (trap_ctrl.cause(6) = '1')
    then -- normal trap: vectored mode and interrupt
    exe_engine_nxt.pc2 <= csr.mtvec(XLEN-1 downto 7) &
        trap_ctrl.cause(4 downto 0) & "00"; -- PC = mtvec + 4
        * mcause
else -- normal trap: direct mode
    exe_engine_nxt.pc2 <= csr.mtvec(XLEN-1 downto 2) & "00";
    -- PC = mtvec
end if;
trap_ctrl.env_enter <= '1';
exe_engine_nxt.state <= EX_RESTART; -- restart instruction
    fetch

when EX_TRAP_EXIT => -- return from trap environment and jump
    to trap PC
--     write(l, String'("EX_TRAP_EXIT"));
--     writeline(output, l);
--
-----

if (debug_ctrl.run = '1') and RISC_V_ISA_Sdext then -- debug
    mode exit
    exe_engine_nxt.pc2 <= csr.dpc(XLEN-1 downto 1) & '0';
else -- normal end of trap
    exe_engine_nxt.pc2 <= csr.mepc(XLEN-1 downto 1) & '0';
end if;
trap_ctrl.env_exit <= '1';
exe_engine_nxt.state <= EX_RESTART; -- restart instruction
    fetch

```

```

when EX_EXECUTE => -- decode and prepare execution (FSM will
    be here for exactly 1 cycle in any case)
--   write(l, String'("EX_EXECUTE"));
--   writeline(output, l);
--
-----

exe_engine_nxt.pc2 <= alu_add_i(XLEN-1 downto 1) & '0'; --
    next PC = PC + immediate

-- decode instruction class/type; [NOTE] register file is
    read in THIS stage; due to the sync read data will be
    available in the NEXT state --
case opcode is

    -- Matrix Extension --
    when opcode_matrix_c =>

        case funct3_v is

            -- load from memory into matrix unit --
            when funct3_mle32_c =>
                -- DEBUG
                --write (l, String'("funct3_mle32_c"));
                --writeline (output, l);
                -- go to the specific states for the matrix
                -- extension.
                -- The states are inserted in order to load all the
                -- elements of a matrix in a loop
                exe_engine_nxt.state <= EX_MATRIX_MEM_REQ;

            when funct3_mse32_c =>
                exe_engine_nxt.state <= EX_MATRIX_MEM_REQ;

            when funct3_madd32_c =>
                exe_engine_nxt.state <= EX_MATRIX_OPERATION_REQ; --
                go to the state that process matrix operations

            when funct3_mmul32_c =>
                exe_engine_nxt.state <= EX_MATRIX_OPERATION_REQ; --
                go to the state that process matrix operations

            when others => -- undefined

        end case;

-- register/immediate ALU operation --
-- opcode_alu_c is true if opcode is 0110011 (see
    neorv32_package.vhd constant definition)
when opcode_alu_c | opcode_alui_c =>

    -- ALU core operation --
    case funct3_v is
        when funct3_sadd_c => ctrl_nxt.alu_op <= alu_op_add_c;
            -- ADD(I), SUB, ADDI
        when funct3_slt_c => ctrl_nxt.alu_op <= alu_op_slt_c;
            -- SLT(I)
        when funct3_sltu_c => ctrl_nxt.alu_op <= alu_op_sltu_c;
            -- SLTU(I)
        when funct3_xor_c => ctrl_nxt.alu_op <= alu_op_xor_c;
            -- XOR(I)
        when funct3_or_c => ctrl_nxt.alu_op <= alu_op_or_c;
            -- OR(I)
        when funct3_and_c => ctrl_nxt.alu_op <= alu_op_and_c;
            -- AND(I)
    end case;

```

```

        when others          => ctrl_nxt.alu_op <= alu_op_zero_c
        ;
    end case;

    -- addition/subtraction control --
    if (funct3_v(2 downto 1) = funct3_slt_c(2 downto 1)) or
        -- SLT(1), SLTU(1)
        ((funct3_v = funct3_sadd_c) and (opcode(5) = '1') and
         (exe_engine.ir(instr_funct7_msb_c-1) = '1'))
        then -- SUB
            ctrl_nxt.alu_sub <= '1';
        end if;

    -- is base rv32i/e ALU[1] instruction (excluding shifts)
    ? --
    if ((opcode(5) = '0') and (funct3_v /= funct3_sll_c) and
        (funct3_v /= funct3_sr_c)) or -- base ALUI
        instruction (excluding SLLI, SRLI, SRAI)
        ((opcode(5) = '1') and (((funct3_v = funct3_sadd_c)
            and (funct7_v = "0000000")) or -- add
            ((funct3_v = funct3_sadd_c)
                and (funct7_v = "0100000"
                )) or -- sub
            --((funct3_v = funct3_sadd_c)
                and (funct7_v =
                    "1000000")) or -- add1
            ((funct3_v = funct3_slt_c)
                and (funct7_v = "0000000"
                )) or
            ((funct3_v = funct3_sltu_c)
                and (funct7_v = "0000000"
                )) or
            ((funct3_v = funct3_xor_c)
                and (funct7_v = "0000000"
                )) or
            ((funct3_v = funct3_or_c)
                and (funct7_v = "0000000"
                )) or
            ((funct3_v = funct3_and_c)
                and (funct7_v = "0000000"
                )))) then -- base ALU
        instruction (excluding
            SLL, SRL, SRA)
            ctrl_nxt.rf_wb_en <= '1'; -- valid RF write-back (
                won't happen if exception)
            exe_engine_nxt.state <= EX_DISPATCH;
        else -- [NOTE] illegal ALU[1] instructions are handled
            as multi-cycle operations that will time-out as no
            ALU co-processor responds
            ctrl_nxt.alu_cp_alu <= '1'; -- trigger ALU[1] opcode-
                space co-processor
            exe_engine_nxt.state <= EX_ALU_WAIT;
        end if;

    -- load upper immediate --
    when opcode_lui_c =>
        ctrl_nxt.alu_op <= alu_op_movb_c; -- pass immediate
        ctrl_nxt.rf_wb_en <= '1'; -- valid RF write-back (won
            't happen if exception)
        exe_engine_nxt.state <= EX_DISPATCH;

    -- add upper immediate to PC --
    when opcode_auiipc_c =>
        ctrl_nxt.alu_op <= alu_op_add_c; -- add PC and
            immediate

```

```

ctrl_nxt.rf_wb_en    <= '1'; -- valid RF write-back (won
    't happen if exception)
exe_engine_nxt.state <= EX_DISPATCH;

-- memory access --
when opcode_load_c | opcode_store_c | opcode_amo_c =>
    exe_engine_nxt.state <= EX_MEM_REQ;

-- branch / jump-and-link (with register) --
when opcode_branch_c | opcode_jal_c | opcode_jalr_c =>
    exe_engine_nxt.state <= EX_BRANCH;

-- memory fence operations --
when opcode_fence_c =>
    if (exe_engine.ir(instr_func3_lsb_c) = '0') then --
        data fence
        ctrl_nxt.lsu_fence <= '1';
    else -- instruction fence
        ctrl_nxt.if_fence <= '1';
    end if;
    exe_engine_nxt.state <= EX_RESTART; -- reset instruction
        fetch + IPB via branch to PC+4 (actually only
        required for fence.i)

-- FPU: floating-point operations --
when opcode_fpu_c =>
    ctrl_nxt.alu_cp_fpu <= '1'; -- trigger FPU co-processor
    exe_engine_nxt.state <= EX_ALU_WAIT; -- will be aborted
        via monitor timeout if FPU is not implemented

-- CFU: custom RISC-V instructions --
when opcode_cust0_c | opcode_cust1_c =>
    ctrl_nxt.alu_cp_cfu <= '1'; -- trigger CFU co-processor
    exe_engine_nxt.state <= EX_ALU_WAIT; -- will be aborted
        via monitor timeout if CFU is not implemented

-- environment/CSR operation or ILLEGAL opcode --
when others =>
    if (funct3_v = funct3_env_c) or
        (((funct3_v = funct3_csrrw_c) or (funct3_v =
            funct3_csrrwi_c)) and (exe_engine.ir(
                instr_rd_msb_c downto instr_rd_lsb_c) = "00000"))
        then
        csr.re_nxt <= '0'; -- no read if CSRRW[1] and rd = 0
            OR if environment instruction
    else
        csr.re_nxt <= '1';
    end if;
    exe_engine_nxt.state <= EX_SYSTEM;

end case; -- EX_EXECUTE

when EX_ALU_WAIT => -- wait for multi-cycle ALU co-processor
    operation to finish or trap
--    write(l, String'("EX_ALU_WAIT"));
--    writeline(output, l);
--
-----

ctrl_nxt.alu_op    <= alu_op_cp_c;
ctrl_nxt.rf_wb_en <= alu_cp_done_i; -- valid RF write-back (
    won't happen if exception)
if (alu_cp_done_i = '1') or (or_reduce_f(trap_ctrl.exc_buf(
    exc_ialign_c downto exc_iaccess_c)) = '1') then
    exe_engine_nxt.state <= EX_DISPATCH;
end if;

```

```

when EX_BRANCH => -- update next PC on taken branches and
    jumps
--     write(l, String'("EX_BRANCH"));
--     writeline(output, l);
--
-- -----

exe_engine_nxt.ra <= exe_engine.pc2(XLEN-1 downto 1) & '0';
-- output return address
ctrl_nxt.rf_wb_en <= exe_engine.ir(instr_opcode_lsb_c+2); --
    save return address if link operation (won't happen if
    exception)
if (branch_taken = '1') then -- taken/unconditional branch
    if_reset <= '1'; -- reset instruction fetch to
        restart at modified PC
    trap_ctrl.instr_ma <= alu_add_i(1) and bool_to_ulogic_f(
        not RISC_V_ISA_C); -- branch destination misaligned?
    exe_engine_nxt.pc2 <= alu_add_i(XLEN-1 downto 1) & '0';
    exe_engine_nxt.state <= EX_BRANCHED; -- shortcut (faster
        than going to EX_RESTART)
    elsif CPU_CONSTT_BR_EN then -- constant-time branches
        if_reset <= '1';
        exe_engine_nxt.state <= EX_BRANCHED;
    else
        exe_engine_nxt.state <= EX_DISPATCH;
    end if;

when EX_BRANCHED => -- delay cycle to wait for reset of front-
    end (instruction fetch)
--     write(l, String'("EX_BRANCHED"));
--     writeline(output, l);
--
-- -----

exe_engine_nxt.state <= EX_DISPATCH;

when EX_MEM_REQ => -- trigger memory request
--
-- -----

-- -- DEBUG
--     write(l, String'("EX_MEM_REQ"));
--     writeline(output, l);

if (or_reduce_f(trap_ctrl.exc_buf(exc_ialign_c downto
    exc_iaccess_c)) = '0') then -- memory request if no
    instruction exception
    ctrl_nxt.lsu_req <= '1'; -- tell the load store unit
        to process a request
    exe_engine_nxt.state <= EX_MEM_RSP;
else
    -- -- DEBUG
    -- write(l, String'("EX_MEM_REQ->DISPATCH"));
    -- writeline(output, l);

    exe_engine_nxt.state <= EX_DISPATCH;
end if;

when EX_MATRIX_MEM_REQ => -- trigger a loop of memory requests
    for the matrix
--
-- -----

-- -- -- DEBUG

```

```

-- write(l, String '("EX_MATRIX_MEM_REQ"));
-- writeline(output, l);

ctrl_nxt.lsu_m_en    <= '1'; -- tell the matrix unit to come
                           online and process the request

if (or_reduce_f(trap_ctrl.exc_buf(exc_ialign_c downto
exc_iaccess_c)) = '0') then -- memory request if no
                           instruction exception

    -- -- DEBUG
    -- write(l, String '("LSU Process Request"));
    -- writeline(output, l);

    ctrl_nxt.lsu_req    <= '1'; -- tell the load store unit
                           to process a request

    matrix_immediate_o <= immediate; -- forward the immediate
                           from the instruction
    matrix_mar_load_store_o <= rf_rs1_i; -- rs1 (source
                           register 1) carries the address to load from / store
                           to

    exe_engine_nxt.state <= EX_MATRIX_MEM_RSP;

else

    exe_engine_nxt.state <= EX_DISPATCH;

end if;

when EX_MEM_RSP => -- wait for memory response
--
-----

-- -- DEBUG
-- write (l, String '("EX_MEM_RSP"));
-- writeline (output, l);

if (lsu_wait_i = '0') or -- bus system has completed the
                           transaction (if there was any)
    (or_reduce_f(trap_ctrl.exc_buf(exc_laccess_c downto
exc_salign_c)) = '1') then -- load/store exception

    ctrl_nxt.rf_wb_en    <= (not ctrl.lsu_rw) or ctrl.lsu_rvs
                           or ctrl.lsu_rmw; -- write-back to RF if read
                           operation (won't happen in case of exception)

    -- -- DEBUG - this is ok, a DISPATCH for MEM_RSP is normal
    -- write (l, String '("EX_MEM_RSP->EX_DISPATCH"));
    -- writeline (output, l);

    exe_engine_nxt.state <= EX_DISPATCH;
end if;

when EX_MATRIX_MEM_RSP => -- wait for memory response
--
-----

-- -- -- DEBUG
-- write(l, String '("EX_MATRIX_MEM_RSP"));
-- writeline(output, l);

if (lsu_wait_i = '0')

```

```

--or -- bus system has completed the transaction (if
      there was any)
--(or_reduce_f(trap_ctrl.exc_buf(exc_laccess_c downto
      exc_salign_c)) = '1') -- load/store exception

then

  ctrl_nxt.rf_wb_en   <= (not ctrl.lsu_rw) or ctrl.lsu_rvs
    or ctrl.lsu_rmw; -- write-back to RF if read operation
    (won't happen in case of exception)

  ctrl_nxt.lsu_m_en   <= '0'; -- tell the matrix unit to
    stay offline

  -- DEBUG - this is ok, a DISPATCH for MEM_RSP is normal
  --write(l, String'("EX_MATRIX_MEM_RSP->EX_DISPATCH"));
  --writeline(output, l);

  exe_engine_nxt.state <= EX_DISPATCH;

end if;

when EX_MATRIX_OPERATION_REQ => -- matrix add operation
--
-----

-- -- DEBUG
-- write(l, String'("EX_MATRIX_OPERATION_REQ"));
-- writeline(output, l);

if (or_reduce_f(trap_ctrl.exc_buf(exc_ialign_c downto
  exc_iaccess_c)) = '0') then

  --write(l, immediate);
  --writeline(output, l);

  matrix_add_o <= '0';
  matrix_mul_o <= '0';

  case funct3_v is

    when funct3_madd32_c =>
      matrix_add_o <= '1';

    when funct3_mmul32_c =>
      matrix_mul_o <= '1';

    when others => -- undefined

  end case;

  matrix_immediate_o <= immediate;

  -- next state
  exe_engine_nxt.state <= EX_MATRIX_OPERATION_RSP;

else
  exe_engine_nxt.state <= EX_DISPATCH;
end if;

when EX_MATRIX_OPERATION_RSP =>

  -- debug
  --write(l, String'("EX_MATRIX_OPERATION_RSP"));
  --writeline(output, l);

```

```

-- next state
if (matrix_operation_wait_i = '0') then

    -- -- debug
    -- write(l, String '("To EX_DISPATCH . . . ")');
    -- writeline(output, l);

    matrix_add_o <= '0';
    matrix_mul_o <= '0';

    exe_engine_nxt.state <= EX_DISPATCH;

end if;

when EX_SLEEP => -- sleep mode
    --write(l, String '("EX_SLEEP")');
    --writeline(output, l);
--
-----

if (or_reduce_f(trap_ctrl.irq_buf) = '1') or (debug_ctrl.run
    = '1') or (csr.dcsr_step = '1') then -- enabled pending
    IRQ, debug-mode, single-step
    exe_engine_nxt.state <= EX_DISPATCH;
end if;

when EX_SYSTEM => -- CSR/ENVIRONMENT operation; no effect if
    illegal instruction
    --write(l, String '("EX_SYSTEM")');
    --writeline(output, l);
--
-----

exe_engine_nxt.state <= EX_DISPATCH; -- default
if (funct3_v = funct3_env_c) and (or_reduce_f(trap_ctrl.
    exc_buf(exc_ialign_c downto exc_iaccess_c)) = '0') then
    -- non-illegal ENV instruction
    case exe_engine.ir(instr_imm12_lsb_c+2 downto
        instr_imm12_lsb_c) is -- three LSBs are sufficient
        here
            when "000" => trap_ctrl.ecall      <= '1'; -- ecall
            when "001" => trap_ctrl.ebreak    <= '1'; -- ebreak
            when "010" => exe_engine_nxt.state <= EX_TRAP_EXIT; --
                xret
            when "101" => exe_engine_nxt.state <= EX_SLEEP; -- wfi
            when others => exe_engine_nxt.state <= EX_DISPATCH; --
                illegal or CSR operation
        end case;
    end if;
    -- always write to CSR (if CSR instruction); ENVIRONMENT
    operations have rs1/imm5 = zero so this won't happen
    then --
    if (funct3_v = funct3_csrrw_c) or (funct3_v =
        funct3_csrrwi_c) or (exe_engine.ir(instr_rs1_msb_c
            downto instr_rs1_lsb_c) /= "00000") then
        csr.we_nxt <= '1'; -- CSRRW[1]: always write CSR; CSRR[S/C
            ][1]: write CSR if rs1/imm5 is NOT zero; won't happen
            if exception
        end if;
        -- always write to RF (even if csr.re = 0, but then we have
        rd = 0); ENVIRONMENT operations have rd = zero so this
        does not hurt --
        ctrl_nxt.rf_wb_en <= '1'; -- won't happen if exception

when others => -- undefined

```

```

--
-----

    exe_engine_nxt.state <= EX_RESTART;

end case;
end process execute_engine_fsm_comb;

-- CPU Control Bus Output
-----

--
-----

-- instruction fetch --
ctrl_o.if_fence      <= ctrl.if_fence;
ctrl_o.if_reset      <= if_reset;
ctrl_o.if_ready      <= '1' when (exe_engine.state = EX_DISPATCH)
    else '0';
-- program counter --
ctrl_o.pc_cur        <= exe_engine.pc(XLEN-1 downto 1) & '0';
ctrl_o.pc_nxt        <= exe_engine.pc2(XLEN-1 downto 1) & '0';
ctrl_o.pc_ret        <= exe_engine.ra(XLEN-1 downto 1) & '0';
-- register file --
ctrl_o.rf_wb_en      <= ctrl.rf_wb_en and -- write-back only if ...
    (not or_reduce_f(trap_ctrl.exc_buf(
        exc_ialign_c downto exc_iaccess_c)))
    and -- no instruction exception
    (not or_reduce_f(trap_ctrl.exc_buf(
        exc_laccess_c downto exc_salign_c)));
    -- no data exception
ctrl_o.rf_rs1        <= exe_engine.ir(instr_rs1_msb_c downto
    instr_rs1_lsb_c);
ctrl_o.rf_rs2        <= exe_engine.ir(instr_rs2_msb_c downto
    instr_rs2_lsb_c);
ctrl_o.rf_rd         <= exe_engine.ir(instr_rd_msb_c downto
    instr_rd_lsb_c);
ctrl_o.rf_zero_we    <= ctrl.rf_zero_we;
-- alu --
ctrl_o.alu_op        <= ctrl.alu_op;
ctrl_o.alu_sub       <= ctrl.alu_sub;
ctrl_o.alu_opa_mux   <= ctrl.alu_opa_mux;
ctrl_o.alu_opb_mux   <= ctrl.alu_opb_mux;
ctrl_o.alu_unsigned  <= ctrl.alu_unsigned;
ctrl_o.alu_imm       <= immediate;
ctrl_o.alu_cp_alu    <= ctrl.alu_cp_alu and (not or_reduce_f(
    trap_ctrl.exc_buf(exc_ialign_c downto exc_iaccess_c))); --
    trigger if no instruction exception
ctrl_o.alu_cp_cfu    <= ctrl.alu_cp_cfu and (not or_reduce_f(
    trap_ctrl.exc_buf(exc_ialign_c downto exc_iaccess_c))); --
    trigger if no instruction exception
ctrl_o.alu_cp_fpu    <= ctrl.alu_cp_fpu and (not or_reduce_f(
    trap_ctrl.exc_buf(exc_ialign_c downto exc_iaccess_c))); --
    trigger if no instruction exception
-- load/store unit --
ctrl_o.lsu_req       <= ctrl.lsu_req; -- ctrl_o.lsu_req is consumed
    inside the LSU entity
ctrl_o.lsu_rw        <= ctrl.lsu_rw;
ctrl_o.lsu_rmw       <= ctrl.lsu_rmw;
ctrl_o.lsu_rvs       <= ctrl.lsu_rvs;
ctrl_o.lsu_mo_we     <= '1' when (exe_engine.state = EX_MEM_REQ or
    exe_engine.state = EX_MATRIX_MEM_REQ) else '0'; -- write
    memory output registers (data & address)
ctrl_o.lsu_fence     <= ctrl.lsu_fence;
ctrl_o.lsu_priv      <= csr.mstatus_mpp when (csr.mstatus_mprv =

```

```

        '1') else csr.prv_level_eff; -- effective privilege level for
        loads/stores in M-mode
ctrl_o.lsu_m_en    <= ctrl.lsu_m_en;
-- control and status registers --
ctrl_o.csr_we      <= csr.we;
ctrl_o.csr_re      <= csr.re;
ctrl_o.csr_addr    <= csr.addr;
ctrl_o.csr_wdata   <= csr.wdata;
-- counter events --
ctrl_o.cnt_event   <= cnt_event;
-- instruction word bit fields --
ctrl_o.ir_funct3    <= exe_engine.ir(instr_funct3_msb_c downto
    instr_funct3_lsb_c);
ctrl_o.ir_funct12   <= exe_engine.ir(instr_imm12_msb_c downto
    instr_imm12_lsb_c);
ctrl_o.ir_opcode    <= exe_engine.ir(instr_opcode_msb_c downto
    instr_opcode_lsb_c);
-- status --
ctrl_o.cpu_priv     <= csr.prv_level_eff;
ctrl_o.cpu_trap     <= trap_ctrl.env_enter;
ctrl_o.cpu_sync_exc <= trap_ctrl.exc_fire;
ctrl_o.cpu_debug    <= debug_ctrl.run;

--
*****

-- Illegal Instruction Detection
--
*****

-- Instruction Execution Monitor (trap if multi-cycle instruction
-- does not complete) -----
--
-----

multi_cycle_monitor: process(rstn_i, clk_i)
begin
    if (rstn_i = '0') then
        monitor_cnt <= (others => '0');
    elsif rising_edge(clk_i) then
        if (exe_engine.state = EX_ALU_WAIT) then
            monitor_cnt <= std_ulogic_vector(unsigned(monitor_cnt) + 1);
        else
            monitor_cnt <= (others => '0');
        end if;
    end if;
end process multi_cycle_monitor;

-- raise illegal instruction exception if a multi-cycle
-- instruction takes longer than a bound amount of time --
monitor_exc <= monitor_cnt(monitor_cnt'left);

-- CSR Access Check
-----

--
-----

csr_check: process(exe_engine, csr, debug_ctrl)
    variable csr_addr_v : std_ulogic_vector(11 downto 0);
begin
    -- shortcut: CSR address right from the instruction word --

```

```

csr_addr_v := exe_engine.ir(instr_imm12_msb_c downto
    instr_imm12_lsb_c);

-----
-- Available at all
-----

case csr_addr_v is

    -- floating-point-unit CSRs --
    when csr_fflags_c | csr_frm_c | csr_fcsr_c =>
        csr_valid(2) <= bool_to_ulogic_f(RISCV_ISA_Zfinx); --
            available if FPU implemented

    -- machine trap setup/handling, environment/information
    registers, etc. --
    when csr_mstatus_c | csr_mstatush_c | csr_misa_c |
        csr_mie_c | csr_mtvec_c |
        csr_mscratch_c | csr_mepc_c | csr_mcause_c |
        csr_mip_c | csr_mtval_c |
        csr_mtinst_c | csr_mcountinhibit_c | csr_mvendordid_c |
        csr_marchid_c | csr_mimpid_c |
        csr_mhartid_c | csr_mconfigptr_c | csr_mxcscr_c |
        csr_mxisa_c =>
        csr_valid(2) <= '1'; -- always implemented

    -- machine-controlled user-mode CSRs --
    when csr_mcounteren_c | csr_menvcfg_c | csr_menvcfg_h_c =>
        csr_valid(2) <= bool_to_ulogic_f(RISCV_ISA_U); -- available
            if U-mode implemented

    -- physical memory protection (PMP) --
    when csr_pmpcfg0_c | csr_pmpcfg1_c | csr_pmpcfg2_c |
        csr_pmpcfg3_c | -- configuration
        csr_pmpaddr0_c | csr_pmpaddr1_c | csr_pmpaddr2_c |
        csr_pmpaddr3_c |
        csr_pmpaddr4_c | csr_pmpaddr5_c | csr_pmpaddr6_c |
        csr_pmpaddr7_c | -- address
        csr_pmpaddr8_c | csr_pmpaddr9_c | csr_pmpaddr10_c |
        csr_pmpaddr11_c |
        csr_pmpaddr12_c | csr_pmpaddr13_c | csr_pmpaddr14_c |
        csr_pmpaddr15_c =>
        csr_valid(2) <= bool_to_ulogic_f(RISCV_ISA_Smpmp); --
            available if PMP implemented

    -- hardware performance monitors (HPM) --
    when csr_mhpmcounter3_c | csr_mhpmcounter4_c |
        csr_mhpmcounter5_c | csr_mhpmcounter6_c |
        csr_mhpmcounter7_c |
        csr_mhpmcounter8_c | csr_mhpmcounter9_c |
        csr_mhpmcounter10_c | csr_mhpmcounter11_c |
        csr_mhpmcounter12_c |
        csr_mhpmcounter13_c | csr_mhpmcounter14_c |
        csr_mhpmcounter15_c | -- machine counters LOW
        csr_mhpmcounter3h_c | csr_mhpmcounter4h_c |
        csr_mhpmcounter5h_c | csr_mhpmcounter6h_c |
        csr_mhpmcounter7h_c |
        csr_mhpmcounter8h_c | csr_mhpmcounter9h_c |
        csr_mhpmcounter10h_c | csr_mhpmcounter11h_c |
        csr_mhpmcounter12h_c |
        csr_mhpmcounter13h_c | csr_mhpmcounter14h_c |
        csr_mhpmcounter15h_c | -- machine counters HIGH
        csr_mhpmevent3_c | csr_mhpmevent4_c |
        csr_mhpmevent5_c | csr_mhpmevent6_c |
        csr_mhpmevent7_c |
        csr_mhpmevent8_c | csr_mhpmevent9_c |

```

```

        csr_mhpmevent10_c | csr_mhpmevent11_c |
        csr_mhpmevent12_c |
        csr_mhpmevent13_c | csr_mhpmevent14_c |
        csr_mhpmevent15_c => -- machine event configuration
        csr_valid(2) <= bool_to_ulogic_f(RISCV_ISA_Zihpm); --
        available if Zihpm implemented

-- counter and timer CSRs --
when csr_cycle_c | csr_mcycle_c | csr_instret_c |
    csr_minstret_c | csr_cycleh_c | csr_mcycleh_c |
    csr_instreth_c | csr_minstreth_c =>
    csr_valid(2) <= bool_to_ulogic_f(RISCV_ISA_Zicntr); --
    available if Zicntr implemented

-- debug-mode CSRs --
when csr_dcsr_c | csr_dpc_c | csr_dscratch0_c =>
    csr_valid(2) <= bool_to_ulogic_f(RISCV_ISA_Sdext); --
    available if debug-mode implemented

-- trigger module CSRs --
when csr_tselect_c | csr_tdata1_c | csr_tdata2_c | csr_tinfo_c
=>
    csr_valid(2) <= bool_to_ulogic_f(RISCV_ISA_Sdtrig); --
    available if trigger module implemented

-- undefined / not implemented --
when others =>
    csr_valid(2) <= '0'; -- invalid access

end case;

-- -----
-- R/W capabilities
-- -----
if (csr_addr_v(11 downto 10) = "11") and -- CSR is read-only
    ((exe_engine.ir(instr_func3_msb_c downto instr_func3_lsb_c)
    = func3_csrrw_c) or -- will always write to CSR
    (exe_engine.ir(instr_func3_msb_c downto instr_func3_lsb_c)
    = func3_csrrwi_c) or -- will always write to CSR
    (exe_engine.ir(instr_rs1_msb_c downto instr_rs1_lsb_c) /= "
    00000")) then -- clear/set instructions: write to CSR
    only if rs1/imm5 is NOT zero
    csr_valid(1) <= '0'; -- invalid access
else
    csr_valid(1) <= '1'; -- access granted
end if;

-- -----
-- Privilege level
-- -----
if (csr_addr_v(11 downto 4) = csr_dcsr_c(11 downto 4)) and --
    debug-mode-only CSR?
    RISC_V_ISA_Sdext and (debug_ctrl.run = '0') then -- debug-mode
    implemented and not running?
    csr_valid(0) <= '0'; -- invalid access
elseif RISC_V_ISA_Zicntr and RISC_V_ISA_U and (csr.prv_level_eff =
'0') and -- any user-mode counters available and in user-
mode?
    (csr_addr_v(11 downto 8) = csr_cycle_c(11 downto 8)) and
    -- user-mode counter access
    (((csr_addr_v(1 downto 0) = csr_cycle_c(1 downto 0)) and (
    csr.mcounteren_cy = '0')) or -- illegal access to
    cycle
    ((csr_addr_v(1 downto 0) = csr_instret_c(1 downto 0)) and
    (csr.mcounteren_ir = '0')))) then -- illegal access
    to instret

```

```

        csr_valid(0) <= '0'; -- invalid access
    elsif (csr_addr_v(9 downto 8) /= "00") and (csr.prv_level_eff =
        '0') then -- invalid privilege level
        csr_valid(0) <= '0'; -- invalid access
    else
        csr_valid(0) <= '1'; -- access granted
    end if;

end process csr_check;

-- Illegal Instruction Check
-----

--
-----

illegal_check: process(exe_engine, csr, csr_valid, debug_ctrl)
-- DEBUG
    variable l : line;
begin
    illegal_cmd <= '1'; -- default: illegal
    case exe_engine.ir(instr_opcode_msb_c downto instr_opcode_lsb_c)
        is -- check entire opcode

            when opcode_lui_c | opcode_auipc_c | opcode_jal_c => -- U-
                instruction type
                illegal_cmd <= '0'; -- all encodings are valid

            when opcode_jalr_c => -- unconditional jump-and-link
                if (exe_engine.ir(instr_funct3_msb_c downto
                    instr_funct3_lsb_c) = "000") then
                    illegal_cmd <= '0';
                end if;

            when opcode_branch_c => -- conditional branch
                if (exe_engine.ir(instr_funct3_msb_c downto
                    instr_funct3_lsb_c+1) /= "01") or RISCV_ISA_Zibi then
                    illegal_cmd <= '0';
                end if;

            when opcode_load_c => -- memory load
                case exe_engine.ir(instr_funct3_msb_c downto
                    instr_funct3_lsb_c) is
                    when funct3_lb_c | funct3_lh_c | funct3_lw_c |
                        funct3_lbu_c | funct3_lhu_c => illegal_cmd <= '0';
                    when others =>
                        -- DEBUG
                        --write (l, String'("ILLEGAL_COMMAND 1"));
                        --writeline (output, l);
                        illegal_cmd <= '1';
                end case;

            when opcode_store_c => -- memory store
                case exe_engine.ir(instr_funct3_msb_c downto
                    instr_funct3_lsb_c) is
                    when funct3_sb_c | funct3_sh_c | funct3_sw_c =>
                        illegal_cmd <= '0';
                    when others => illegal_cmd <= '1';
                end case;

            -- MATRIX EXTENSION
            when opcode_matrix_c =>
                illegal_cmd <= '0';

            when opcode_amo_c => -- atomic memory operation

```

```

if (exe_engine.ir(instr_funct3_msb_c downto
    instr_funct3_lsb_c) = "010") then -- word-quantity only
case exe_engine.ir(instr_funct5_msb_c downto
    instr_funct5_lsb_c) is
when "00001" | "00000" | "00100" | "01100" | "01000" | "
    10000" | "10100" | "11000" | "11100" => illegal_cmd
    <= not bool_to_ulogic_f(RISCV_ISA_Zaamo);
when "00010" | "00011" => illegal_cmd <= not
    bool_to_ulogic_f(RISCV_ISA_Zalrsc);
when others => illegal_cmd <= '1';
end case;
end if;

when opcode_alu_c | opcode_alui_c | opcode_fpu_c |
    opcode_cust0_c | opcode_cust1_c => -- ALU[] / FPU /
    custom operations
illegal_cmd <= '0'; -- [NOTE] valid if not terminated/
    invalidated by the "instruction execution monitor"

when opcode_fence_c => -- memory ordering
if (exe_engine.ir(instr_funct3_msb_c downto
    instr_funct3_lsb_c+1) = funct3_fence_c(2 downto 1)) then
    illegal_cmd <= '0';
end if;

when opcode_system_c => -- CSR / system instruction
if (exe_engine.ir(instr_funct3_msb_c downto
    instr_funct3_lsb_c) = funct3_env_c) then -- system
    environment
if (exe_engine.ir(instr_rs1_msb_c downto instr_rs1_lsb_c)
    = "00000") and (exe_engine.ir(instr_rd_msb_c downto
    instr_rd_lsb_c) = "00000") then
case exe_engine.ir(instr_imm12_msb_c downto
    instr_imm12_lsb_c) is
when funct12_ecall_c => illegal_cmd <= '0'; -- ecall
    is always allowed
when funct12_ebreak_c => illegal_cmd <= '0'; -- ebreak
    is always allowed
when funct12_mret_c => illegal_cmd <= (not csr.
    prv_level) or debug_ctrl.run; -- mret allowed in (
    real/non-debug) M-mode only
when funct12_dret_c => illegal_cmd <= not debug_ctrl
    .run; -- dret allowed in debug mode only
when funct12_wfi_c => illegal_cmd <= (not csr.
    prv_level) and csr.mstatus_tw; -- wfi allowed in M
    -mode or if TW is zero
when others => illegal_cmd <= '1'; --
    undefined
end case;
end if;
elsif (csr_valid = "111") and (exe_engine.ir(
    instr_funct3_msb_c downto instr_funct3_lsb_c) /=
    funct3_csrril_c) then -- valid CSR operation
    illegal_cmd <= '0';
end if;

when others => -- undefined/unimplemented/illegal opcode

-- DEBUG
--write (I, String'("ILLEGAL_COMMAND 2"));
--writeline (output, I);

illegal_cmd <= '1';

end case;
end process illegal_check;

```

```
-- Illegal Operation Check
-----

--
-----

trap_ctrl.instr_il <= '1' when ((exe_engine.state = EX_EXECUTE) or
    (exe_engine.state = EX_ALU_WAIT)) and -- check in execution
    states only
        ((monitor_exc = '1') or (
            illegal_cmd = '1')) else '0';
        -- instruction timeout or
        -- illegal instruction

--
*****

-- Trap Controller
--
*****

-- Trap Buffer
-----

--
-----

trap_buffer: process(rstn_i, clk_i)
begin
    if (rstn_i = '0') then
        trap_ctrl.irq_pnd <= (others => '0');
        trap_ctrl.irq_buf <= (others => '0');
        trap_ctrl.exc_buf <= (others => '0');
    elsif rising_edge(clk_i) then

        -- Interrupt-Pending Buffer
        -----
        -- Once triggered the interrupt line should stay active until
        -- explicitly
        -- cleared by a mechanism specific to the interrupt-causing
        -- source.
        --
        -----

        trap_ctrl.irq_pnd(irq_mei_irq_c downto irq_msi_irq_c) <=
            irq_machine_i; -- RISC-V machine interrupts
        trap_ctrl.irq_pnd(irq_firq_15_c downto irq_firq_0_c) <=
            irq_fast_i(15 downto 0); -- NEORV32-specific fast
            interrupts
        trap_ctrl.irq_pnd(irq_db_halt_c) <= '0';
        -- unused debug-mode entry

        -- Interrupt Buffer
        -----
        -- Masking of interrupt request lines. Additionally, this
        -- buffer ensures
        -- that an active interrupt request line stays active (even
        -- when
        -- disabled via MIE) if the trap environment is already
        -- starting.
        --
```

```

-----

-- RISC-V machine interrupts --
trap_ctrl.irq_buf(irq_msi_irq_c) <= (trap_ctrl.irq_pnd(
    irq_msi_irq_c) and csr.mie_msi) or (trap_ctrl.env_pending
    and trap_ctrl.irq_buf(irq_msi_irq_c));
trap_ctrl.irq_buf(irq_mei_irq_c) <= (trap_ctrl.irq_pnd(
    irq_mei_irq_c) and csr.mie_mei) or (trap_ctrl.env_pending
    and trap_ctrl.irq_buf(irq_mei_irq_c));
trap_ctrl.irq_buf(irq_mti_irq_c) <= (trap_ctrl.irq_pnd(
    irq_mti_irq_c) and csr.mie_mti) or (trap_ctrl.env_pending
    and trap_ctrl.irq_buf(irq_mti_irq_c));

-- NEORV32-specific fast interrupts --
for i in 0 to 15 loop
    trap_ctrl.irq_buf(irq_firq_0_c+i) <= (trap_ctrl.irq_pnd(
        irq_firq_0_c+i) and csr.mie_firq(i)) or (trap_ctrl.
        env_pending and trap_ctrl.irq_buf(irq_firq_0_c+i));
end loop;

-- debug-mode entry (external halt request) --
trap_ctrl.irq_buf(irq_db_halt_c) <= debug_ctrl.trig_halt or (
    trap_ctrl.env_pending and trap_ctrl.irq_buf(irq_db_halt_c)
    );

-- Exception Buffer
-----

-- All requests stay pending until the trap environment is
-- started. Only
-- the highest-priority exception will kick in; others are
-- discarded.
-----

trap_ctrl.exc_buf(exc_iaccess_c) <= (trap_ctrl.exc_buf(
    exc_iaccess_c) or trap_ctrl.instr_be) and (not
    trap_ctrl.env_enter); -- instruction access error
trap_ctrl.exc_buf(exc_illegal_c) <= (trap_ctrl.exc_buf(
    exc_illegal_c) or trap_ctrl.instr_il) and (not
    trap_ctrl.env_enter); -- illegal instruction
trap_ctrl.exc_buf(exc_ialign_c) <= (trap_ctrl.exc_buf(
    exc_ialign_c) or trap_ctrl.instr_ma) and (not
    trap_ctrl.env_enter); -- instruction misaligned
trap_ctrl.exc_buf(exc_ecall_c) <= (trap_ctrl.exc_buf(
    exc_ecall_c) or trap_ctrl.ecall) and (not
    trap_ctrl.env_enter); -- environment call
trap_ctrl.exc_buf(exc_ebreak_c) <= (trap_ctrl.exc_buf(
    exc_ebreak_c) or ebreak_trig) and (not
    trap_ctrl.env_enter); -- environment break
trap_ctrl.exc_buf(exc_salign_c) <= (trap_ctrl.exc_buf(
    exc_salign_c) or lsu_err_i(2)) and (not
    trap_ctrl.env_enter); -- store address misaligned
trap_ctrl.exc_buf(exc_lalign_c) <= (trap_ctrl.exc_buf(
    exc_lalign_c) or lsu_err_i(0)) and (not
    trap_ctrl.env_enter); -- load address misaligned
trap_ctrl.exc_buf(exc_saccess_c) <= (trap_ctrl.exc_buf(
    exc_saccess_c) or lsu_err_i(3)) and (not
    trap_ctrl.env_enter); -- store access error
trap_ctrl.exc_buf(exc_laccess_c) <= (trap_ctrl.exc_buf(
    exc_laccess_c) or lsu_err_i(1)) and (not
    trap_ctrl.env_enter); -- load access error
trap_ctrl.exc_buf(exc_db_break_c) <= (trap_ctrl.exc_buf(
    exc_db_break_c) or debug_ctrl.trig_break) and (not
    trap_ctrl.env_enter); -- debug-entry: break
trap_ctrl.exc_buf(exc_db_trig_c) <= (trap_ctrl.exc_buf(

```

```

        exc_db_trig_c) or debug_ctrl.trig_hw) and (not
        trap_ctrl.env_enter); -- debug-entry: trigger
    trap_ctrl.exc_buf(exc_db_step_c) <= (trap_ctrl.exc_buf(
        exc_db_step_c) or debug_ctrl.trig_step) and (not
        trap_ctrl.env_enter); -- debug-entry: single step

end if;
end process trap_buffer;

-- environment break exception helper --
ebreak_trig <= (trap_ctrl.ebreak and (csr.prv_level) and (not
    csr.dcsr_ebreakm) and (not debug_ctrl.run)) or -- M-mode trap
    when in M-mode
        (trap_ctrl.ebreak and (not csr.prv_level) and (not
            csr.dcsr_ebreaku) and (not debug_ctrl.run));
    -- M-mode trap when in U-mode

-- Trap Priority Logic
-----

--
-----

trap_priority: process(rstn_i, clk_i)
-- DEBUG
    variable l : line;
begin
    if (rstn_i = '0') then
        trap_ctrl.cause <= (others => '0');
    elsif rising_edge(clk_i) then
        trap_ctrl.cause <= (others => '0'); -- default
        -- standard RISC-V synchronous exceptions --
        if (trap_ctrl.exc_buf(exc_iaccess_c) = '1') then trap_ctrl
            .cause <= trap_iaf_c; -- write(l, String'("exc_iaccess_c")
            ); writeline(output, l); -- instruction access fault
        elsif (trap_ctrl.exc_buf(exc_illegal_c) = '1') then trap_ctrl
            .cause <= trap_iil_c; -- write(l, String'("exc_illegal_c")
            ); writeline(output, l); -- illegal instruction
        elsif (trap_ctrl.exc_buf(exc_ialign_c) = '1') then trap_ctrl
            .cause <= trap_ima_c; -- write(l, String'("exc_ialign_c")
            ); writeline(output, l); -- instruction address misaligned
        elsif (trap_ctrl.exc_buf(exc_ecall_c) = '1') then trap_ctrl
            .cause <= trap_env_c(6 downto 2) & replicate_f(csr.
            prv_level, 2); -- write(l, String'("exc_ecall_c"));
            writeline(output, l); -- environment call
        elsif (trap_ctrl.exc_buf(exc_ebreak_c) = '1') then trap_ctrl
            .cause <= trap_brk_c; -- write(l, String'("exc_ebreak_c"))
            ; writeline(output, l); -- environment breakpoint
        elsif (trap_ctrl.exc_buf(exc_salign_c) = '1') then trap_ctrl
            .cause <= trap_sma_c; -- write(l, String'("exc_salign_c")
            ); writeline(output, l); -- store address misaligned
        elsif (trap_ctrl.exc_buf(exc_lalign_c) = '1') then trap_ctrl
            .cause <= trap_lma_c; -- write(l, String'("exc_lalign_c")
            ); writeline(output, l); -- load address misaligned
        elsif (trap_ctrl.exc_buf(exc_saccess_c) = '1') then trap_ctrl
            .cause <= trap_saf_c; -- write(l, String'("exc_saccess_c")
            ); writeline(output, l); -- store access fault
        elsif (trap_ctrl.exc_buf(exc_laccess_c) = '1') then trap_ctrl
            .cause <= trap_laf_c; -- write(l, String'("exc_laccess_c")
            ); writeline(output, l); -- load access fault
        -- standard RISC-V debug mode synchronous exceptions and
        interrupts --
        elsif (trap_ctrl.irq_buf(irq_db_halt_c) = '1') then trap_ctrl
            .cause <= trap_db_halt_c; -- write(l, String'("

```

```

        irq_db_halt_c")); writeline(output, l); -- external halt
        request
    elsif (trap_ctrl.exc_buf(exc_db_trig_c) = '1') then trap_ctrl
        .cause <= trap_db_trig_c; -- write(l, String "("
        exc_db_trig_c")); writeline(output, l); -- hardware
        trigger
    elsif (trap_ctrl.exc_buf(exc_db_break_c) = '1') then trap_ctrl
        .cause <= trap_db_break_c; -- write(l, String "("
        exc_db_break_c")); writeline(output, l); -- breakpoint
    elsif (trap_ctrl.exc_buf(exc_db_step_c) = '1') then trap_ctrl
        .cause <= trap_db_step_c; -- write(l, String "("
        exc_db_step_c")); writeline(output, l); -- single
        stepping
-- NEORV32-specific fast interrupts --
    elsif (trap_ctrl.irq_buf(irq_firq_0_c) = '1') then trap_ctrl
        .cause <= trap_firq0_c; -- write(l, String "("irq_firq_0_c
        "); writeline(output, l); -- fast interrupt channel 0
    elsif (trap_ctrl.irq_buf(irq_firq_1_c) = '1') then trap_ctrl
        .cause <= trap_firq1_c; -- write(l, String "("irq_firq_1_c
        "); writeline(output, l); -- fast interrupt channel 1
    elsif (trap_ctrl.irq_buf(irq_firq_2_c) = '1') then trap_ctrl
        .cause <= trap_firq2_c; -- write(l, String "("irq_firq_2_c
        "); writeline(output, l); -- fast interrupt channel 2
    elsif (trap_ctrl.irq_buf(irq_firq_3_c) = '1') then trap_ctrl
        .cause <= trap_firq3_c; -- write(l, String "("irq_firq_3_c
        "); writeline(output, l); -- fast interrupt channel 3
    elsif (trap_ctrl.irq_buf(irq_firq_4_c) = '1') then trap_ctrl
        .cause <= trap_firq4_c; -- write(l, String "("irq_firq_4_c
        "); writeline(output, l); -- fast interrupt channel 4
    elsif (trap_ctrl.irq_buf(irq_firq_5_c) = '1') then trap_ctrl
        .cause <= trap_firq5_c; -- write(l, String "("irq_firq_5_c
        "); writeline(output, l); -- fast interrupt channel 5
    elsif (trap_ctrl.irq_buf(irq_firq_6_c) = '1') then trap_ctrl
        .cause <= trap_firq6_c; -- write(l, String "("irq_firq_6_c
        "); writeline(output, l); -- fast interrupt channel 6
    elsif (trap_ctrl.irq_buf(irq_firq_7_c) = '1') then trap_ctrl
        .cause <= trap_firq7_c; -- write(l, String "("irq_firq_7_c
        "); writeline(output, l); -- fast interrupt channel 7
    elsif (trap_ctrl.irq_buf(irq_firq_8_c) = '1') then trap_ctrl
        .cause <= trap_firq8_c; -- write(l, String "("irq_firq_8_c
        "); writeline(output, l); -- fast interrupt channel 8
    elsif (trap_ctrl.irq_buf(irq_firq_9_c) = '1') then trap_ctrl
        .cause <= trap_firq9_c; -- write(l, String "("irq_firq_9_c
        "); writeline(output, l); -- fast interrupt channel 9
    elsif (trap_ctrl.irq_buf(irq_firq_10_c) = '1') then trap_ctrl
        .cause <= trap_firq10_c; -- write(l, String "("
        irq_firq_10_c")); writeline(output, l); -- fast interrupt
        channel 10
    elsif (trap_ctrl.irq_buf(irq_firq_11_c) = '1') then trap_ctrl
        .cause <= trap_firq11_c; -- write(l, String "("
        irq_firq_11_c")); writeline(output, l); -- fast interrupt
        channel 11
    elsif (trap_ctrl.irq_buf(irq_firq_12_c) = '1') then trap_ctrl
        .cause <= trap_firq12_c; -- write(l, String "("
        irq_firq_12_c")); writeline(output, l); -- fast interrupt
        channel 12
    elsif (trap_ctrl.irq_buf(irq_firq_13_c) = '1') then trap_ctrl
        .cause <= trap_firq13_c; -- write(l, String "("
        irq_firq_13_c")); writeline(output, l); -- fast interrupt
        channel 13
    elsif (trap_ctrl.irq_buf(irq_firq_14_c) = '1') then trap_ctrl
        .cause <= trap_firq14_c; -- write(l, String "("
        irq_firq_14_c")); writeline(output, l); -- fast interrupt
        channel 14
    elsif (trap_ctrl.irq_buf(irq_firq_15_c) = '1') then trap_ctrl
        .cause <= trap_firq15_c; -- write(l, String "("

```

```

        irq_firq_15_c")); writeline(output, 1); -- fast interrupt
        channel 15
-- standard RISC-V interrupts --
    elsif (trap_ctrl.irq_buf(irq_mei_irq_c) = '1') then trap_ctrl
        .cause <= trap_mei_c; -- write(1, String "("irq_mei_irq_c")
        ); writeline(output, 1); -- machine external interrupt (
        MEI)
    elsif (trap_ctrl.irq_buf(irq_msi_irq_c) = '1') then trap_ctrl
        .cause <= trap_msi_c; -- write(1, String "("irq_msi_irq_c")
        ); writeline(output, 1); -- machine software interrupt (
        MSI)
    elsif (trap_ctrl.irq_buf(irq_mti_irq_c) = '1') then trap_ctrl
        .cause <= trap_mti_c; -- write(1, String "("irq_mti_irq_c")
        ); writeline(output, 1); -- machine timer interrupt (MTI)
    end if;
end if;
end process trap_priority;

-- exception program counter: async. interrupt or sync. exception?
--
trap_ctrl.pc <= exe_engine.pc2 when (trap_ctrl.cause(trap_ctrl.
    cause'left) = '1') else exe_engine.pc;

-- Trap Controller
-----

--
-----

trap_controller: process(rstn_i, clk_i)
begin
    if (rstn_i = '0') then
        trap_ctrl.env_pending <= '0';
    elsif rising_edge(clk_i) then
        if (trap_ctrl.env_pending = '0') and ((trap_ctrl.exc_fire =
            '1') or (or_reduce_f(trap_ctrl.irq_fire) = '1')) then --
            trap triggered
            trap_ctrl.env_pending <= '1';
        elsif (trap_ctrl.env_pending = '1') and (trap_ctrl.env_enter =
            '1') then -- start of trap environment acknowledged by
            execute engine
            trap_ctrl.env_pending <= '0';
        end if;
    end if;
end process trap_controller;

-- any exception? --
trap_ctrl.exc_fire <= '1' when (or_reduce_f(trap_ctrl.exc_buf) =
    '1') else '0'; -- sync. exceptions CANNOT be masked

-- any system interrupt? --
trap_ctrl.irq_fire(0) <= '1' when
    ((exe_engine.state = EX_EXECUTE) or (exe_engine.state = EX_SLEEP
        )) and -- trigger system IRQ only in EX_EXECUTE state or in
        sleep mode
    (or_reduce_f(trap_ctrl.irq_buf(irq_firq_15_c downto
        irq_msi_irq_c)) = '1') and -- pending system IRQ
    ((csr.mstatus_mie = '1') or (csr.priv_level = priv_mode_u_c)) and
        -- IRQ only when in M-mode and MIE=1 OR when in U-mode
    (debug_ctrl.run = '0') and (csr.dcsr_step = '0') else '0'; -- no
        system IRQs when in debug-mode / during single-stepping

-- debug-entry halt interrupt? --
trap_ctrl.irq_fire(1) <= trap_ctrl.irq_buf(irq_db_halt_c) when
    (exe_engine.state = EX_EXECUTE) or (exe_engine.state = EX_SLEEP)

```

```

    or (exe_engine.state = EX_BRANCHED) else '0'; -- allow halt
    also after "reset" (#879)

```

```
--
```

```
*****
```

```
-- Control and Status Registers (CSRs)
```

```
--
```

```
*****
```

```
-- CSR Address Register
```

```
-----
```

```
--
```

```
-----
```

```
csr_addr_reg: process(rstn_i, clk_i)
```

```
begin
```

```
    if (rstn_i = '0') then
```

```
        csr_addr <= (others => '0');
```

```
    elsif rising_edge(clk_i) then
```

```
        if (opcode = opcode_system_c) then -- update only for actual
            CSR operations to reduce switching activity on csr_addr
            net
```

```
            csr_addr <= exe_engine.ir(instr_imm12_msb_c downto
                                     instr_imm12_lsb_c);
```

```
        end if;
```

```
    end if;
```

```
end process csr_addr_reg;
```

```
-- CSR Write-Data ALU
```

```
-----
```

```
--
```

```
-----
```

```
csr.operand <= rf_rs1_i when (exe_engine.ir(instr_funct3_msb_c) =
    '0') else (x"000000" & "000" & exe_engine.ir(19 downto 15));
```

```
-- tiny ALU to compute CSR write data --
```

```
with exe_engine.ir(instr_funct3_msb_c-1 downto instr_funct3_lsb_c)
```

```
    select csr.wdata <=
```

```
        csr.rdata or      csr.operand when "10", -- set
```

```
        csr.rdata and (not csr.operand) when "11", -- clear
```

```
        csr.operand      when others; -- write
```

```
-- CSR Write Access
```

```
-----
```

```
--
```

```
-----
```

```
csr_write_access: process(rstn_i, clk_i)
```

```
    --variable l : line;
```

```
begin
```

```
    if (rstn_i = '0') then
```

```
        csr.we <= '0';
```

```
        csr.prv_level <= priv_mode_m_c;
```

```
        csr.mstatus_mie <= '0';
```

```
        csr.mstatus_mpie <= '0';
```

```
        csr.mstatus_mpp <= '0';
```

```
        csr.mstatus_mprv <= '0';
```

```

csr.mstatus_tw    <= '0';
csr.mie_msi       <= '0';
csr.mie_mei       <= '0';
csr.mie_mti       <= '0';
csr.mie_firq      <= (others => '0');
csr.mtvec         <= (others => '0');
csr.mscratch      <= (others => '0');
csr.mepc          <= (others => '0');
csr.mcause        <= (others => '0');
csr.mtval         <= (others => '0');
csr.mtinst        <= (others => '0');
csr.mcounteren_cy <= '0';
csr.mcounteren_ir <= '0';
csr.dcsr_ebreakm  <= '0';
csr.dcsr_ebreaku  <= '0';
csr.dcsr_step     <= '0';
csr.dcsr_prv      <= '0';
csr.dcsr_cause    <= (others => '0');
csr.dpc           <= (others => '0');
csr.dscratch0     <= (others => '0');
elsif rising_edge(clk_i) then

--
-- *****
-- Software CSR access
-- *****

csr.we <= csr.we_nxt and (not or_reduce_f(trap_ctrl.exc_buf(
    exc_ialign_c downto exc_iaccess_c))); -- write if no
    instruction exception
if (csr.we = '1') then
    case csr.addr is

        -- machine status register --
        when csr_mstatus_c =>
            csr.mstatus_mie <= csr.wdata(3);
            csr.mstatus_mpie <= csr.wdata(7);
            if RISC_V_ISA_U then
                csr.mstatus_mpp <= csr.wdata(11) or csr.wdata(12); --
                    everything /= U will fall back to M
                csr.mstatus_mprv <= csr.wdata(17);
                csr.mstatus_tw <= csr.wdata(21);
            end if;

        -- machine interrupt enable register --
        when csr_mie_c =>
            csr.mie_msi <= csr.wdata(3);
            csr.mie_mti <= csr.wdata(7);
            csr.mie_mei <= csr.wdata(11);
            csr.mie_firq <= csr.wdata(31 downto 16);

        -- machine trap-handler base address --
        when csr_mtvec_c =>
            csr.mtvec <= csr.wdata(XLEN-1 downto 2) & '0' & csr.
                wdata(0); -- base + mode (vectored/direct)

        -- machine counter access enable --
        when csr_mcounteren_c =>
            if RISC_V_ISA_U and RISC_V_ISA_Zicntr then
                csr.mcounteren_cy <= csr.wdata(0);
                csr.mcounteren_ir <= csr.wdata(2);
            end if;

        -- machine scratch register --

```

```

when csr_mscratch_c =>
    csr.mscratch <= csr.wdata;

-- machine exception program counter --
when csr_mepc_c =>
    csr.mepc <= csr.wdata(XLEN-1 downto 1) & '0';
    if not RISC_V_ISA_C then -- RISC-V priv. spec.: MEPC[1]
        is masked when IALIGN = 32
        csr.mepc(1) <= '0';
    end if;

-- debug mode control and status register --
when csr_dcsr_c =>
    if (csr.addr = csr_dcsr_c) and RISC_V_ISA_Sdext then
        csr.dcsr_step <= csr.wdata(2);
        csr.dcsr_ebreakm <= csr.wdata(15);
        if RISC_V_ISA_U then
            csr.dcsr_prv <= csr.wdata(1) or csr.wdata(0); --
                everything /= U will fall back to M
            csr.dcsr_ebreaku <= csr.wdata(12);
        end if;
    end if;

-- debug mode program counter --
when csr_dpc_c =>
    if RISC_V_ISA_Sdext then
        csr.dpc <= csr.wdata(XLEN-1 downto 1) & '0';
        if not RISC_V_ISA_C then -- RISC-V priv. spec.: DPC[1]
            is masked when IALIGN = 32
            csr.dpc(1) <= '0';
        end if;
    end if;

-- debug mode scratch register 0 --
when csr_dscratch0_c =>
    if (csr.addr = csr_dscratch0_c) and RISC_V_ISA_Sdext then
        csr.dscratch0 <= csr.wdata;
    end if;

-- undefined or implemented somewhere else --
when others => NULL;

end case;

--
*****

-- Hardware CSR access: TRAP ENTER
--
*****

elsif (trap_ctrl.env_enter = '1') then

    -- NORMAL trap entry - no CSR update when in debug-mode! --
    if (not RISC_V_ISA_Sdext) or ((trap_ctrl.cause(5) = '0') and
        (debug_ctrl.run = '0')) then
        csr.mcause <= trap_ctrl.cause(trap_ctrl.cause'left) &
            trap_ctrl.cause(4 downto 0); -- trap type & identifier
        csr.mepc <= trap_ctrl.pc(XLEN-1 downto 1) & '0'; -- trap
            PC
        -- trap value (load/store trap address only, permitted by
            RISC-V priv. spec.) --
        if (trap_ctrl.cause(6) = '0') and (trap_ctrl.cause(2) =
            '1') then -- load/store misaligned/access faults [
                hacky!]

```

```

-- -- DEBUG
-- write(l, String'("MISSALIGNED"));
-- writeline(output, l);

    csr.mtval <= lsu_mar_i; -- faulting data access address
else -- everything else including all interrupts
    csr.mtval <= (others => '0');
end if;
-- trap instruction --
csr.mtinst <= exe_engine.ir;
if (exe_engine.ci = '1') and RISC_V_ISA_C then
    csr.mtinst(1) <= '0'; -- RISC-V priv. spec: clear bit 1
    if compressed instruction
end if;
-- update privilege level and interrupt-enable stack --
csr.priv_level <= priv_mode_m_c; -- execute trap in
    machine mode
csr.mstatus_mie <= '0'; -- disable interrupts
csr.mstatus_mpie <= csr.mstatus_mie; -- backup previous
    mie state
csr.mstatus_mpp <= csr.priv_level; -- backup previous
    privilege level
end if;

-- DEBUG trap entry - no CSR update when already in debug-
    mode! --
if RISC_V_ISA_Sdext and (trap_ctrl.cause(5) = '1') and (
    debug_ctrl.run = '0') then
    csr.dcsr_cause <= trap_ctrl.cause(2 downto 0); -- trap
        cause
    csr.dcsr_prv <= csr.priv_level; -- current privilege
        level when debug mode was entered
    csr.dpc <= trap_ctrl.pc(XLEN-1 downto 1) & '0'; --
        trap PC
end if;

--
    *****

-- Hardware CSR access: TRAP EXIT
--
    *****

elseif (trap_ctrl.env_exit = '1') then
    -- return from debug mode --
    if RISC_V_ISA_Sdext and (debug_ctrl.run = '1') then
        if RISC_V_ISA_U then
            csr.priv_level <= csr.dcsr_prv;
            if (csr.dcsr_prv /= priv_mode_m_c) then
                csr.mstatus_mprv <= '0'; -- clear if return to priv.
                    level less than M
            end if;
        end if;
    -- return from normal trap --
    else
        if RISC_V_ISA_U then
            csr.priv_level <= csr.mstatus_mpp; -- restore previous
                privilege level
            csr.mstatus_mpp <= priv_mode_u_c; -- set to least-
                privileged level that is supported
            if (csr.mstatus_mpp /= priv_mode_m_c) then
                csr.mstatus_mprv <= '0'; -- clear if return to priv.
                    level less than M
            end if;
        end if;
    end if;
end if;

```

```

        csr.mstatus_mie <= csr.mstatus_mpie; -- restore machine-
            mode IRQ enable flag
        csr.mstatus_mpie <= '1';
    end if;

end if;

--
    *****

-- Override - terminate unavailable registers and bits
--
    *****

-- no user-mode counters at all --
if not RISC_V_ISA_Zicntr then
    csr.mcounteren_cy <= '0';
    csr.mcounteren_ir <= '0';
end if;

-- no user mode --
if not RISC_V_ISA_U then
    csr.priv_level <= priv_mode_m_c;
    csr.mstatus_mpp <= priv_mode_m_c;
    csr.mstatus_mprv <= '0';
    csr.mstatus_tw <= '0';
    csr.dcsr_ebreaku <= '0';
    csr.dcsr_prv <= '0';
    csr.mcounteren_cy <= '0';
    csr.mcounteren_ir <= '0';
end if;

-- no debug mode --
if not RISC_V_ISA_Sdext then
    csr.dcsr_ebreakm <= '0';
    csr.dcsr_step <= '0';
    csr.dcsr_ebreaku <= '0';
    csr.dcsr_prv <= priv_mode_m_c;
    csr.dcsr_cause <= (others => '0');
    csr.dpc <= (others => '0');
    csr.dscratch0 <= (others => '0');
end if;

end if;
end process csr_write_access;

-- effective privilege level is MACHINE when in debug mode --
csr.priv_level_eff <= priv_mode_m_c when (debug_ctrl.run = '1')
    else csr.priv_level;

-- CSR Read Access
-----

--
-----

csr_read_access: process(rstn_i, clk_i)
begin
    if (rstn_i = '0') then
        csr.re <= '0';
        csr.rdata <= (others => '0');
    elsif rising_edge(clk_i) then
        csr.re <= csr.re_nxt and (not or_reduce_f(trap_ctrl.exc_buf

```

```

        (exc_ialign_c downto exc_iaccess_c)); -- read if no
        instruction exception
    csr.rdata <= (others => '0'); -- default; output all-zero if
        there is no explicit CSR read operation
    if (csr.re = '1') then
        case csr.addr is

            --
            -----

            -- machine trap setup
            --
            -----

        when csr_mstatus_c => -- machine status register, low word
            csr.rdata(3) <= csr.mstatus_mie;
            csr.rdata(7) <= csr.mstatus_mpie;
            csr.rdata(12 downto 11) <= (others => csr.mstatus_mpp);
            csr.rdata(17) <= csr.mstatus_mprv;
            csr.rdata(21) <= csr.mstatus_tw and bool_to_ulogic_f(
                RISC_V_ISA_U);

        when csr_misa_c => -- ISA and extensions
            csr.rdata(0) <= bool_to_ulogic_f(RISC_V_ISA_A);
            csr.rdata(1) <= bool_to_ulogic_f(RISC_V_ISA_B);
            csr.rdata(2) <= bool_to_ulogic_f(RISC_V_ISA_C);
            csr.rdata(4) <= bool_to_ulogic_f(RISC_V_ISA_E);
            csr.rdata(8) <= bool_to_ulogic_f(not RISC_V_ISA_E);
            csr.rdata(12) <= bool_to_ulogic_f(RISC_V_ISA_M);
            csr.rdata(20) <= bool_to_ulogic_f(RISC_V_ISA_U);
            csr.rdata(23) <= '1'; -- X CPU extension (non-standard /
                NEORV32-specific)
            csr.rdata(31 downto 30) <= "01"; -- MXL = 32

        when csr_mie_c => -- machine interrupt-enable register
            csr.rdata(3) <= csr.mie_msi;
            csr.rdata(7) <= csr.mie_mti;
            csr.rdata(11) <= csr.mie_mei;
            csr.rdata(31 downto 16) <= csr.mie_firq;

        when csr_mtvec_c => -- machine trap-handler base address
            csr.rdata <= csr.mtvec;

        when csr_mcounteren_c => -- machine counter enable
            register
            if RISC_V_ISA_U and RISC_V_ISA_Zicntr then
                csr.rdata(0) <= csr.mcounteren_cy;
                csr.rdata(2) <= csr.mcounteren_ir;
            end if;

            --
            -----

            -- machine trap handling
            --
            -----

        when csr_mscratch_c => -- machine scratch register
            csr.rdata <= csr.mscratch;

        when csr_mepc_c => -- machine exception program counter
            csr.rdata <= csr.mepc(XLEN-1 downto 1) & '0';

        when csr_mcause_c => -- machine trap cause
            csr.rdata(31) <= csr.mcause(5);
            csr.rdata(4 downto 0) <= csr.mcause(4 downto 0);

```

```

when csr_mtval_c => -- machine trap value
    csr.rdata <= csr.mtval;

when csr_mip_c => -- machine interrupt pending
    csr.rdata(3) <= trap_ctrl.irq_pnd(
        irq_msi_irq_c);
    csr.rdata(7) <= trap_ctrl.irq_pnd(
        irq_mti_irq_c);
    csr.rdata(11) <= trap_ctrl.irq_pnd(
        irq_mei_irq_c);
    csr.rdata(31 downto 16) <= trap_ctrl.irq_pnd(
        irq_firq_15_c downto irq_firq_0_c);

when csr_mtinst_c => -- machine trap instruction
    csr.rdata <= csr.mtinst;

--
-----

-- machine information
--
-----

when csr_marchid_c => csr.rdata(4 downto 0) <= "10011"; --
    architecture ID - official RISC-V open-source arch ID
when csr_mimpid_c => csr.rdata <= hw_version_c; --
    implementation ID -- NEORV32 hardware version
when csr_mhartid_c => csr.rdata(9 downto 0) <=
    std_ulogic_vector(to_unsigned(HART_ID, 10)); --
    hardware thread ID

--
-----

-- debug-mode
--
-----

when csr_dcsr_c => if RISC_V_ISA_Sdext then csr.rdata
    <= csr.dcsr_rd; end if; -- debug mode control and
    status
when csr_dpc_c => if RISC_V_ISA_Sdext then csr.rdata
    <= csr.dpc; end if; -- debug mode program
    counter
when csr_dscratch0_c => if RISC_V_ISA_Sdext then csr.rdata
    <= csr.dscratch0; end if; -- debug mode scratch
    register 0

--
-----

-- NEORV32-specific
--
-----

when csr_mxcsr_c => -- machine control and status register
    csr.rdata(25 downto 0) <= (others => '0'); --
    reserved
    csr.rdata(26) <= bool_to_ulogic_f(CPU_TRACE_EN); --
    execution trace generator
    csr.rdata(27) <= bool_to_ulogic_f(CPU_CONSTT_BR_EN); --
    constant-time branches
    csr.rdata(28) <= bool_to_ulogic_f(CPU_RF_HW_RST_EN); --
    full hardware reset of register file

```

```

csr.rdata(29) <= bool_to_ulogic_f(CPU_FAST_MUL_EN); --
    DSP-based multiplication (M extensions only)
csr.rdata(30) <= bool_to_ulogic_f(CPU_FAST_SHIFT_EN); --
    parallel logic for shifts (barrel shifters)
csr.rdata(31) <= bool_to_ulogic_f(is_simulation_c); --
    is this a simulation?

when csr_mxisa_c => -- machine extended ISA extensions
    information
    csr.rdata(0) <= '1'; --
        Zicr: CSR access (always enabled)
    csr.rdata(1) <= '1'; --
        Zifencei: instruction stream sync. (always enabled)
    csr.rdata(2) <= bool_to_ulogic_f(RISCV_ISA_Zmmul); --
        Zmmul: mul/div
    csr.rdata(3) <= bool_to_ulogic_f(RISCV_ISA_Zxcfu); --
        Zxcfu: custom instructions
    csr.rdata(4) <= bool_to_ulogic_f(RISCV_ISA_Zkt); --
        Zkt: data independent execution latency
    csr.rdata(5) <= bool_to_ulogic_f(RISCV_ISA_Zfinx); --
        Zfinx: FPU using x registers
    csr.rdata(6) <= bool_to_ulogic_f(RISCV_ISA_Zicnd); --
        Zicnd: integer conditional operations
    csr.rdata(7) <= bool_to_ulogic_f(RISCV_ISA_Zicntr); --
        Zicntr: base counters
    csr.rdata(8) <= bool_to_ulogic_f(RISCV_ISA_Smpmp); --
        Smpmp: physical memory protection
    csr.rdata(9) <= bool_to_ulogic_f(RISCV_ISA_Zihpm); --
        Zihpm: hardware performance monitors
    csr.rdata(10) <= bool_to_ulogic_f(RISCV_ISA_Sdext); --
        Sdext: external debug
    csr.rdata(11) <= bool_to_ulogic_f(RISCV_ISA_Sdtrig); --
        Sdtrig: trigger module
    csr.rdata(12) <= bool_to_ulogic_f(RISCV_ISA_Zbkx); --
        Zbkx: cryptography crossbar permutation
    csr.rdata(13) <= bool_to_ulogic_f(RISCV_ISA_Zknd); --
        Zknd: cryptography NIST AES decryption
    csr.rdata(14) <= bool_to_ulogic_f(RISCV_ISA_Zkne); --
        Zkne: cryptography NIST AES encryption
    csr.rdata(15) <= bool_to_ulogic_f(RISCV_ISA_Zknh); --
        Zknh: cryptography NIST hash functions
    csr.rdata(16) <= bool_to_ulogic_f(RISCV_ISA_Zbkb); --
        Zbkb: bit manipulation instructions for cryptography
    csr.rdata(17) <= bool_to_ulogic_f(RISCV_ISA_Zbkc); --
        Zbkc: carry-less multiplication for cryptography
    csr.rdata(18) <= bool_to_ulogic_f(RISCV_ISA_Zkn); --
        Zkn: NIST algorithm suite
    csr.rdata(19) <= bool_to_ulogic_f(RISCV_ISA_Zksh); --
        Zksh: ShangMi hash functions
    csr.rdata(20) <= bool_to_ulogic_f(RISCV_ISA_Zksed); --
        Zksed: ShangMi block ciphers
    csr.rdata(21) <= bool_to_ulogic_f(RISCV_ISA_Zks); --
        Zks: ShangMi algorithm suite
    csr.rdata(22) <= bool_to_ulogic_f(RISCV_ISA_Zba); --
        Zba: shifted-add bit-manipulation
    csr.rdata(23) <= bool_to_ulogic_f(RISCV_ISA_Zbb); --
        Zbb: basic bit-manipulation
    csr.rdata(24) <= bool_to_ulogic_f(RISCV_ISA_Zbs); --
        Zbs: single-bit bit-manipulation
    csr.rdata(25) <= bool_to_ulogic_f(RISCV_ISA_Zaamo); --
        Zaamo: atomic memory operations
    csr.rdata(26) <= bool_to_ulogic_f(RISCV_ISA_Zalrsc); --
        Zalrsc: reservation-set operations
    csr.rdata(27) <= bool_to_ulogic_f(RISCV_ISA_Zcb); --
        Zcb: additional code size reduction instructions

```

```

csr.rdata(28) <= bool_to_ulogic_f(RISCV_ISA_C);      --
    Zca: C without floating-point
csr.rdata(29) <= bool_to_ulogic_f(RISCV_ISA_Zibi);   --
    Zibi: branch with immediate-comparison
csr.rdata(31 downto 30) <= (others => '0');          --
    reserved

--
-----

-- undefined/unavailable or implemented externally
--
-----

when others => -- FPU, CFU, PMP, HPM, base counters,
    trigger module
    csr.rdata <= xcsr_rdata_i; -- return zero if accessing
        invalid external CSR

    end case;
end if;
end if;
end process csr_read_access;

-- CSR read data output (to register file mux) --
csr_rdata_o <= csr.rdata;

--
*****

-- CPU Counter Events
--
*****

-- RISC-V-compliant counter events --
cnt_event(cnt_event_cy_c) <= '0' when (exe_engine.state = EX_SLEEP
) else '1'; -- active cycle
cnt_event(cnt_event_tm_c) <= '0'; -- time: not available
cnt_event(cnt_event_ir_c) <= '1' when (exe_engine.state =
EX_EXECUTE) else '0'; -- retired (=executed) instruction

-- NEORV32-specific counter events --
cnt_event(cnt_event_compr_c) <= '1' when (exe_engine.state =
EX_EXECUTE) and (exe_engine.ci = '1') else '0';
-- executed compressed instruction
cnt_event(cnt_event_wait_dis_c) <= '1' when (exe_engine.state =
EX_DISPATCH) and (frontend_i.valid = '0') else '0';
-- instruction dispatch wait cycle
cnt_event(cnt_event_wait_alu_c) <= '1' when (exe_engine.state =
EX_ALU_WAIT) else '0';
-- multi-cycle ALU wait cycle
cnt_event(cnt_event_branch_c) <= '1' when (exe_engine.state =
EX_BRANCH) else '0';
-- executed branch instruction
cnt_event(cnt_event_branched_c) <= '1' when (exe_engine.state =
EX_BRANCHED) else '0';
-- control flow transfer
cnt_event(cnt_event_load_c) <= '1' when (ctrl.lsu_req = '1')
and ((ctrl.lsu_rw = '0') or (ctrl.lsu_rmw = '1')) else '0'; --
executed load operation
cnt_event(cnt_event_store_c) <= '1' when (ctrl.lsu_req = '1')
and ((ctrl.lsu_rw = '1') or (ctrl.lsu_rmw = '1')) else '0'; --
executed store operation
cnt_event(cnt_event_wait_lsu_c) <= '1' when (ctrl.lsu_req = '0')

```

```

        and (exe_engine.state = EX_MEM_RSP)                else '0'; --
        load/store memory wait cycle
cnt_event(cnt_event_trap_c)    <= '1' when (trap_ctrl.env_enter =
'1')                          else '0'; --
        entered trap

--
        *****

-- CPU Debug Mode
--
        *****

-- Debug Control
-----

--
-----

debug_mode_enable:
if RISC_V_ISA_Sdext generate

    debug_control: process(rstn_i, clk_i)
    begin
        if (rstn_i = '0') then
            debug_ctrl.run <= '0';
        elsif rising_edge(clk_i) then
            if (debug_ctrl.run = '0') then -- debug mode OFFLINE
                if (trap_ctrl.env_enter = '1') and (trap_ctrl.cause(5) =
                    '1') then -- waiting for entry event
                    debug_ctrl.run <= '1';
                end if;
            else -- debug mode ONLINE
                if (trap_ctrl.env_exit = '1') then -- waiting for exit
                    event
                    debug_ctrl.run <= '0';
                end if;
            end if;
        end if;
    end process debug_control;

    -- debug mode entry triggers --
    debug_ctrl.trig_hw    <= hwtrig_i and (not debug_ctrl.run); --
        enter debug-mode by HW trigger module
    debug_ctrl.trig_break <= trap_ctrl.ebreak and (debug_ctrl.run or
        -- re-enter debug mode
        ((    csr.prv_level) and csr.
            dcsr_ebreakm) or -- enabled goto-
            debug-mode in machine mode on "
            ebreak"
        ((not csr.prv_level) and csr.
            dcsr_ebreaku)); -- enabled goto-
            debug-mode in user mode on "ebreak"
    debug_ctrl.trig_halt  <= irq_dbg_i    and (not debug_ctrl.run);
        -- external halt request (if not halted already)
    debug_ctrl.trig_step  <= csr.dcsr_step and (not debug_ctrl.run)
        and cnt_event(cnt_event_ir_c); -- single-step mode

end generate;

-- Sdext ISA extension not enabled --
debug_mode_disable:
if not RISC_V_ISA_Sdext generate
    debug_ctrl.run    <= '0';

```

```

        debug_ctrl.trig_hw    <= '0';
        debug_ctrl.trig_break <= '0';
        debug_ctrl.trig_halt  <= '0';
        debug_ctrl.trig_step  <= '0';
    end generate;

    -- Debug Control and Status Register (dcsr) - Read-Back
    -----
    --
    -----

    csr.dcsr_rd(31 downto 28) <= "0100"; -- xdebugver: external debug
        support compatible to spec. version 1.0
    csr.dcsr_rd(27 downto 16) <= (others => '0'); -- reserved
    csr.dcsr_rd(15)          <= csr.dcsr_ebreakm; -- ebreakm: what
        happens on ebreak in m-mode? (normal trap OR debug-enter)
    csr.dcsr_rd(14)          <= '0'; -- reserved
    csr.dcsr_rd(13)          <= '0'; -- ebreaks: supervisor mode not
        implemented
    csr.dcsr_rd(12)          <= csr.dcsr_ebreaku when RISC_V_ISA_U
        else '0'; -- ebreaku: what happens on ebreak in u-mode? (
        normal trap OR debug-enter)
    csr.dcsr_rd(11)          <= '0'; -- stepie: interrupts are
        disabled during single-stepping
    csr.dcsr_rd(10)          <= '1'; -- stopcount: standard counters
        and HPMs are stopped when in debug mode
    csr.dcsr_rd(9)           <= '0'; -- stoptime: timers increment as
        usual
    csr.dcsr_rd(8 downto 6)  <= csr.dcsr_cause; -- debug mode entry
        cause
    csr.dcsr_rd(5)           <= '0'; -- reserved
    csr.dcsr_rd(4)           <= '1'; -- mprven: mstatus.mprv is also
        evaluated in debug mode
    csr.dcsr_rd(3)           <= '0'; -- nmip: no pending non-maskable
        interrupt
    csr.dcsr_rd(2)           <= csr.dcsr_step; -- step: single-step
        mode
    csr.dcsr_rd(1 downto 0) <= (others => csr.dcsr_prv); -- prv:
        privilege level when debug mode was entered

end neorv32_cpu_control_rtl;

```

Quellcode 3: neorv32_cpu_m_lsu.vhd

```

--
-- =====
--
-- NEORV32 CPU - Load/Store Unit
--
--
-- -----
--
-- The NEORV32 RISC-V Processor - https://github.com/stnolting/
-- neorv32
-- Copyright (c) NEORV32 contributors.
--
-- Copyright (c) 2020 - 2025 Stephan Nolting. All rights reserved.
--
-- Licensed under the BSD-3-Clause license, see LICENSE for details.
--
-- SPDX-License-Identifier: BSD-3-Clause
--

```

```
--
=====
--

use std.textio.all;

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library neorv32;
use neorv32.neorv32_package.all;

entity neorv32_cpu_m_lsu is
  port (
    -- global control --
    clk_i      : in  std_ulogic; -- global clock, rising edge
    rstn_i     : in  std_ulogic; -- global reset, low-active, async
    ctrl_i     : in  ctrl_bus_t; -- main control bus
    -- debug
    lsu_req_i  : in  std_ulogic;
    lsu_en_i   : in  std_ulogic; -- matrix enable
    -- matrix extension
    head_o     : inout matrix_ram_addr;
    tail_o     : inout matrix_ram_addr;
    load_o     : out std_ulogic;
    store_o    : out std_ulogic;
    data_out_o : out std_ulogic_vector(31 downto 0); -- data_out is
      from CPU RAM to Matrix RAM
    data_in_i  : in  std_ulogic_vector(31 downto 0); -- data_out is
      from Matrix RAM to CPU RAM
    matrix_immediate_i : in  std_ulogic_vector(XLEN-1 downto 0);
    matrix_mar_load_store_i : in std_ulogic_vector(XLEN-1 downto 0);
    -- cpu data access interface --
    addr_i     : in  std_ulogic_vector(XLEN-1 downto 0); -- access
      address
    wdata_i    : in  std_ulogic_vector(XLEN-1 downto 0); -- write
      data
    rdata_o    : out std_ulogic_vector(XLEN-1 downto 0); -- read
      data
    mar_o      : out std_ulogic_vector(XLEN-1 downto 0); -- current
      memory address register
    wait_o     : out std_ulogic; -- wait for access to complete
    err_o      : out std_ulogic_vector(3 downto 0); -- alignment/
      access errors
    pmp_fault_i : in  std_ulogic; -- PMP read/write access fault
    -- data bus --
    dbus_req_o  : out bus_req_t; -- request
    dbus_rsp_i  : in  bus_rsp_t; -- response, in neorv32_cpu.vhd:1
      .579 this signal is connected to a port of the neorv32_cpu.
      Then in neorv32_top.vhd this port is connected to
      neorv32_dcache_inst.host_rsp_o => cpu_d_rsp(i),
    -- debug
    dbus_rsp_i_ack : in std_ulogic
  );
end neorv32_cpu_m_lsu;

architecture neorv32_cpu_lsu_rtl of neorv32_cpu_m_lsu is

  signal mar      : std_ulogic_vector(XLEN-1 downto 0); -- memory
    address register
  signal misaligned : std_ulogic; -- misaligned address
  signal pending   : std_ulogic; -- pending bus request
  signal pmp_err   : std_ulogic; -- PMP access violation


```

```

signal amo_cmd      : std_ulogic_vector(3 downto 0); -- atomic
                      memory operation type
signal exc_rd       : std_ulogic; -- for exceptions: this is a read
                      operation
signal exc_wr       : std_ulogic; -- for exceptions: this is a write
                      operation

signal matrix_wait_for_load_finished : std_ulogic;
signal matrix_wait_for_store_finished : std_ulogic;

signal matrix_stb_counter_load : integer; -- for loading data from
                      CPU RAM into the matrix engine
signal matrix_stb_counter_store : integer; -- for loading data
                      from the matrix engine into CPU RAM
signal matrix_stb : std_ulogic;
signal matrix_mar_store : std_ulogic_vector(XLEN-1 downto 0);
signal matrix_mar_load : std_ulogic_vector(XLEN-1 downto 0);

signal matrix_reg_file : my_array_t(MLEN-1 downto 0, MLEN-1 downto
0);

signal test : YOUR_ARRAY_TYPE;

signal rd_data : std_ulogic_vector(31 downto 0);

signal load : std_ulogic; -- LSU currently performs a matrix LOAD
                      operation
signal store : std_ulogic; -- LSU currently performs a matrix
                      STORE operation

signal seen_001 : std_ulogic;

type matrix_load_state_t is ( ML_IDLE, ML_INIT, ML_PROCESS,
ML_INCREMENT );
signal matrix_load_state : matrix_load_state_t;

type matrix_store_state_t is (
MS_IDLE,
MS_INIT,
MS_PROCESS,
MS_INCREMENT,
MS_WAIT_FOR_STORE_DATA_1,
MS_WAIT_FOR_STORE_DATA_2,
MS_WAIT_FOR_STORE_DATA_3,
MS_WAIT_FOR_STORE_DATA_4
);
signal matrix_store_state : matrix_store_state_t;

begin

-- Access Address
-- -----

-- -----

mem_addr_reg: process(rstn_i, clk_i)
begin
    if (rstn_i = '0') then
        mar      <= (others => '0');
        misaligned <= '0';
    elsif rising_edge(clk_i) then
        if (ctrl_i.lsu_mo_we = '1') then
            mar <= addr_i; -- memory address register
            misaligned <= '0';
        end if;

```

```

    end if;
end process mem_addr_reg;

STROBE_PROCESS : process(rstn_i, clk_i)
begin
    if (rstn_i = '0') then
        matrix_stb <= '0';
    else

        matrix_stb <= ctrl_i.lsu_m_en;

    end if;
end process STROBE_PROCESS;

-- Data Output: Alignment, Byte Enable and Type Identifiers
-----
-- Maybe this part is for output towards RAM for store operations
-- such as SB, SH, SW and SD
--
-----

mem_do_reg: process(rstn_i, clk_i)
    variable l : line;
begin

    if (rstn_i = '0') then

        dbus_req_o.meta <= (others => '0');
        dbus_req_o.amo <= '0';
        dbus_req_o.amoop <= (others => '0');
        dbus_req_o.data <= (others => '0');
        dbus_req_o.ben <= (others => '0');

        tail_o <= 0;

        matrix_stb_counter_store <= 0;

        matrix_wait_for_store_finished <= '0';

        matrix_mar_store <= matrix_mar_load_store_i;

        --matrix_store_state <= MS_INIT;

        -- DEBUG
        -- write(l, String'("LSU STORE RESET"));
        -- writeline(output, l);

    elsif rising_edge(clk_i) then

        -- -- DEBUG
        -- write(l, String'("Store"));
        -- writeline(output, l);
        -- write(l, ctrl_i.lsu_m_en);
        -- writeline(output, l);

        if (ctrl_i.lsu_m_en = '0') then -- if NO matrix load command
            is executed, perform normal sb, sh, sw

            -- lsu_mo_we == write memory output registers (data &
            address)
            if (ctrl_i.lsu_mo_we = '1') then

                -- access identifiers --
                dbus_req_o.meta <= ctrl_i.cpu_debug & ctrl_i.lsu_priv &
                '0'; -- LSB: data access
            end if;
        end if;
    end if;
end process mem_do_reg;

```

```

dbus_req_o.rw      <= ctrl_i.lsu_rw; -- read/write
dbus_req_o.amo     <= ctrl_i.lsu_rmw or ctrl_i.lsu_rvs; --
    atomic memory operation
dbus_req_o.amoop   <= amo_cmd;

-- data alignment + byte-enable --
case ctrl_i.ir_funct3(1 downto 0) is
when "00" => -- byte
    dbus_req_o.data   <= wdata_i(7 downto 0) & wdata_i(7
        downto 0) & wdata_i(7 downto 0) & wdata_i(7 downto
        0);
    dbus_req_o.ben(0) <= (not addr_i(1)) and (not addr_i
        (0));
    dbus_req_o.ben(1) <= (not addr_i(1)) and (    addr_i
        (0));
    dbus_req_o.ben(2) <= (    addr_i(1)) and (not addr_i
        (0));
    dbus_req_o.ben(3) <= (    addr_i(1)) and (    addr_i
        (0));
when "01" => -- half-word
    dbus_req_o.data <= wdata_i(15 downto 0) & wdata_i(15
        downto 0);
    dbus_req_o.ben   <= addr_i(1) & addr_i(1) & (not addr_i
        (1)) & (not addr_i(1));
when others => -- word
    dbus_req_o.data <= wdata_i;
    dbus_req_o.ben   <= (others => '1');
end case;

end if;

-- MS_IDLE, MS_INIT, MS_PROCESS, MS_INCREMENT
matrix_store_state <= MS_INIT;

--matrix_wait_for_store_finished <= '0';
matrix_stb_counter_store <= 0;
tail_o <= 0;

elsif (ctrl_i.ir_funct3(2 downto 0) = "001") then -- Matrix
    store

case matrix_store_state is

when MS_IDLE => -- nop

    -- -- DEBUG
    -- write(l, String'("LSU MATRIX STORE MS_IDLE"));
    -- writeline(output, l);

    store <= '0';

when MS_INIT =>

    -- DEBUG
    -- write(l, String'("LSU MATRIX STORE MS_INIT"));
    -- writeline(output, l);

    -- set address into Memory Address Register
    matrix_mar_store <= matrix_mar_load_store_i;

    -- set values
    matrix_stb_counter_store <= 0;
    matrix_wait_for_store_finished <= '1';
    store <= '1';
    tail_o <= 0;

```

```

-- next state
matrix_store_state <= MS_WAIT_FOR_STORE_DATA_1;

when MS_WAIT_FOR_STORE_DATA_1 =>

-- next state
matrix_store_state <= MS_WAIT_FOR_STORE_DATA_2;

when MS_WAIT_FOR_STORE_DATA_2 =>

-- next state
matrix_store_state <= MS_WAIT_FOR_STORE_DATA_3;

when MS_WAIT_FOR_STORE_DATA_3 =>

-- next state
matrix_store_state <= MS_WAIT_FOR_STORE_DATA_4;

when MS_WAIT_FOR_STORE_DATA_4 =>

-- next state
matrix_store_state <= MS_PROCESS;

when MS_PROCESS =>

-- -- DEBUG
-- write(1, String'("LSU MATRIX STORE MS_PROCESS"));
-- writeline(output, 1);

-- need to wait for the cache and the memory to respond
-- before advancing in the state machine
if (dbus_rsp_i.ack = '1') then

    if (tail_o < 15) then

        -- -- DEBUG
        -- write(1, String'("LSU MATRIX STORE TAIL <15"));
        -- writeline(output, 1);

        -- -- DEBUG
        -- write(1, data_in_i);
        -- write(1, to_hstring(unsigned(data_in_i)));
        -- writeline(output, 1);

        dbus_req_o.rw <= '1';
        dbus_req_o.data <= data_in_i;
        dbus_req_o.ben <= (others => '1');

        matrix_store_state <= MS_INCREMENT;

        -- keep track of outstanding elements to strobe
        matrix_stb_counter_store <= matrix_stb_counter_store
            + 1;

    else

        matrix_wait_for_store_finished <= '0'; -- done
        waiting
        matrix_store_state <= MS_IDLE; -- GOTO IDLE state
        and remain until next activation of the MATRIX
        LSU

    end if;

-- -- debug

```

```

        -- write(l, "LSU STORE: 0x" & to_hstring(unsigned(
            matrix_mar_store));
        -- writeline(output, l);

    end if;

    when MS_INCREMENT =>

        -- advance address (aka. pointer)
        matrix_mar_store <= std_ulogic_vector(unsigned(
            matrix_mar_store) + 4);

        -- -- DEBUG
        -- write(l, String'("LSU MATRIX STORE MS_INCREMENT"));
        -- writeline(output, l);

        tail_o <= tail_o + 1;
        matrix_store_state <= MS_PROCESS;

    when others =>

        -- DEBUG
        -- write(l, String'("LSU STORE others"));
        -- writeline(output, l);

    end case;

end if;

end if;

end process mem_do_reg;

-- hardwired signals --
dbus_req_o.burst <= '0'; -- only single-access

-- out-of-band signals --
dbus_req_o.fence <= ctrl_i.lsu_fence;

-- atomic memory access operation encoding --
amo_encode: process(ctrl_i.ir_funct12)
begin
    case ctrl_i.ir_funct12(11 downto 7) is
        when "00010" => amo_cmd <= "1000"; -- Zalrsc.LR
        when "00011" => amo_cmd <= "1001"; -- Zalrsc.SC
        when "00000" => amo_cmd <= "0001"; -- Zaamo.ADD
        when "00100" => amo_cmd <= "0010"; -- Zaamo.XOR
        when "01100" => amo_cmd <= "0011"; -- Zaamo.AND
        when "01000" => amo_cmd <= "0100"; -- Zaamo.OR
        when "10000" => amo_cmd <= "1110"; -- Zaamo.MIN
        when "10100" => amo_cmd <= "1111"; -- Zaamo.MAX
        when "11000" => amo_cmd <= "0110"; -- Zaamo.MINU
        when "11100" => amo_cmd <= "0111"; -- Zaamo.MAXU
        when others => amo_cmd <= "0000"; -- Zaamo.SWAP
    end case;
end process;

-- Bus-Locking (for atomic read-modify-write operations)
--
--
-----

bus_lock: process(rstn_i, clk_i)
begin

```

```

    if (rstn_i = '0') then
        dbus_req_o.lock <= '0';
    elsif rising_edge(clk_i) then
        if (ctrl_i.lsu_mo_we = '1') and (ctrl_i.lsu_rmw = '1') and (
            ctrl_i.ir_funct12(8) = '0') then
            dbus_req_o.lock <= '1'; -- set if Zaamo instruction
        elsif (dbus_rsp_i.ack = '1') or (ctrl_i.cpu_trap = '1') then
            dbus_req_o.lock <= '0'; -- clear at the end of the bus
            access
        end if;
    end if;
end process bus_lock;

-- https://vhdlwhiz.com/ring-buffer-fifo/
--
-- When the CPU executes a matrix store instruction, the
-- mem_do_reg process in this file advances
-- the matrix_mar_store which is the address into matrix engine
-- RAM to store back into
-- CPU RAM.
--
-- The matrix engine RAM will produce the value and place it into
-- the data_in_i port
-- of this LSU entity.
--
-- The LSU entity then has to place that data into a bus store
-- request so that the data
-- is transferred into CPU RAM.
--
PROC_RAM_LSU : process(clk_i)
    variable l : line;
begin
    if rising_edge(clk_i) then

        load_o <= '0';
        store_o <= '0';

        -- load ::= from CPU RAM into matrix RAM
        if (load = '1') and (dbus_rsp_i.ack = '1') then

            -- -- debug
            -- write(l, String'("LOAD"));
            -- writeline(output, l);
            -- write(l, dbus_rsp_i.data);
            -- writeline(output, l);
            debug_data_output <= dbus_rsp_i.data;

            -- write external load signal
            load_o <= '1';
            data_out_o <= dbus_rsp_i.data;

        end if;

        -- store ::= from matrix RAM into CPU RAM
        if (store = '1') then

            -- write external read signal
            store_o <= '1';

        end if;

    end if;
end process PROC_RAM_LSU;

```

```

-- Data Input: Alignment and Sign-Extension
-----

-- LB, LH, LW, LD
--
-----

mem_di_reg : process(rstn_i, clk_i)
    variable l : line;
begin

    if (rstn_i = '0') then

        -- DEBUG
        -- write(l, String'("LSU LOAD RESET"));
        -- writeline(output, l);

        rdata_o <= (others => '0');

        matrix_stb_counter_load <= 0;

        -- the mar needs to be read from the instruction
        matrix_mar_load <= matrix_mar_load_store_i;

        matrix_wait_for_load_finished <= '0';

        head_o <= 0;

        load <= '0';

    elsif rising_edge(clk_i) then

        if (not ctrl_i.lsu_m_en = '1') then -- if NO matrix load
            command is executed, perform normal lb, lh, lw

            rdata_o <= (others => '0'); -- output zero if there is no
                memory access
            if (pending = '1') then -- pending request
                case ctrl_i.ir_funct3(1 downto 0) is
                    when "00" => -- byte
                        case mar(1 downto 0) is
                            when "00" => rdata_o <= replicate_f((not ctrl_i.
                                ir_funct3(2)) and dbus_rsp_i.data(7), 24) &
                                dbus_rsp_i.data(7 downto 0);
                            when "01" => rdata_o <= replicate_f((not ctrl_i.
                                ir_funct3(2)) and dbus_rsp_i.data(15), 24) &
                                dbus_rsp_i.data(15 downto 8);
                            when "10" => rdata_o <= replicate_f((not ctrl_i.
                                ir_funct3(2)) and dbus_rsp_i.data(23), 24) &
                                dbus_rsp_i.data(23 downto 16);
                            when others => rdata_o <= replicate_f((not ctrl_i.
                                ir_funct3(2)) and dbus_rsp_i.data(31), 24) &
                                dbus_rsp_i.data(31 downto 24);
                        end case;
                    when "01" => -- half-word
                        if (mar(1) = '0') then -- low half-word
                            rdata_o <= replicate_f((not ctrl_i.ir_funct3(2)) and
                                dbus_rsp_i.data(15), 16) & dbus_rsp_i.data(15
                                    downto 0);
                        else -- high half-word
                            rdata_o <= replicate_f((not ctrl_i.ir_funct3(2)) and
                                dbus_rsp_i.data(31), 16) & dbus_rsp_i.data(31
                                    downto 16);
                        end if;
                    when others => -- word
                        rdata_o <= dbus_rsp_i.data;
                end case;
            end if;
        end if;
    end process;

```

```

    end case;
end if;

-- ML_IDLE, ML_INIT, ML_PROCESS
matrix_load_state <= ML_INIT;

matrix_stb_counter_load <= 0;
head_o <= 0;

-- DEBUG
--write(l, String'("LSU LOAD RESET"));
--writeline(output, l);

elsif (ctrl_i.ir_funct3(2 downto 0) = "010") then -- if funct3
    is the load code

    -- load from CPU RAM into the matrix engine

    case matrix_load_state is

        when ML_IDLE => -- nop
            --write(l, String'("LSU LOAD ML_IDLE"));
            --writeline(output, l);
            load <= '0';

        when ML_INIT =>
            -- write(l, String'("LSU LOAD ML_INIT"));
            -- writeline(output, l);

            matrix_mar_load <= matrix_mar_load_store_i;
            matrix_stb_counter_load <= 0;
            matrix_wait_for_load_finished <= '1';
            head_o <= 0;
            load <= '1';

            -- next state
            matrix_load_state <= ML_PROCESS;

        when ML_PROCESS =>

            if (dbus_rsp_i.ack = '1') then -- need to wait for the
                cache and the memory to respond before advancing in
                the state machine

                -- write(l, String'("LSU LOAD ML_PROCESS"));
                -- writeline(output, l);

                if (head_o < 15) then
                    matrix_load_state <= ML_INCREMENT;
                else
                    matrix_wait_for_load_finished <= '0'; -- done
                    waiting

                    -- next state
                    matrix_load_state <= ML_IDLE; -- GOTO IDLE state and
                    remain until next activation of the MATRIX LSU
                end if;

                -- keep track of outstanding elements to strobe
                matrix_stb_counter_load <= matrix_stb_counter_load +
                    1;

                -- advance pointer
                matrix_mar_load <= std_ulogic_vector(unsigned(
                    matrix_mar_load) + 4);

```

```

-- -- debug
-- write(l, "LSU LOAD: 0x" & to_hstring(unsigned(
    matrix_mar_load)));
-- writeline(output, l);

end if;

when ML_INCREMENT =>
    head_o <= head_o + 1;
    matrix_load_state <= ML_PROCESS;

when others =>
    --write(l, String'("LSU LOAD others"));
    --writeline(output, l);

end case;

end if;

end if;

end process mem_di_reg;

-- Access Arbiter
-----

--
-----

access_arbiter: process(rstn_i, clk_i)
begin
    if (rstn_i = '0') then
        pmp_err <= '0';
        pending <= '0';
    elsif rising_edge(clk_i) then
        pmp_err <= pmp_fault_i;
        if (pending = '0') then -- idle
            pending <= ctrl_i.lsu_req;
        elsif (dbus_rsp_i.ack = '1') or (ctrl_i.cpu_trap = '1') then
            -- bus response or start of trap handling
            pending <= '0';
        end if;
    end if;
end process access_arbiter;

-- wait for bus response --
wait_o <= (matrix_wait_for_load_finished or
    matrix_wait_for_store_finished) or not dbus_rsp_i.ack;

-- filter exceptions: RMW-AMOs only cause STORE exceptions --
exc_rd <= '0' when (ctrl_i.lsu_rmw = '1') else not ctrl_i.lsu_rw;
exc_wr <= '1' when (ctrl_i.lsu_rmw = '1') else ctrl_i.lsu_rw;

-- output access/alignment errors to control unit --
err_o(0) <= pending and exc_rd and misaligned; -- misaligned load
err_o(1) <= pending and exc_rd and (dbus_rsp_i.err or pmp_err); --
    load bus access error
err_o(2) <= pending and exc_wr and misaligned; -- misaligned store
err_o(3) <= pending and exc_wr and (dbus_rsp_i.err or pmp_err); --
    store bus access error

-- access request (all source signals are driven by registers) --
dbus_req_o.stb <= matrix_stb when ctrl_i.lsu_m_en = '1'
    else ctrl_i.lsu_req and (not misaligned) and (not pmp_fault_i);

```

```
-- address output --
dbus_req_o.addr <= matrix_mar_load when ((ctrl_i.lsu_m_en = '1')
and (load = '1'))
else matrix_mar_store when ((ctrl_i.lsu_m_en = '1') and (store =
'1'))
else mar;

-- this signal goes to cpu control lsu_mar_i
mar_o <= matrix_mar_load when (ctrl_i.lsu_m_en = '1' and (load =
'1'))
else matrix_mar_store when (ctrl_i.lsu_m_en = '1' and (store =
'1'))
else mar; -- for MTVAL CSR

end neorv32_cpu_lsu_rtl;
```

Quellcode 4: neorv32_cpu_regfile.vhd

```
--
-- =====
--
-- NEORV32 CPU - Data Register File
--
-- -----
--
-- Data register file. 32 entries (= 1024 bit) for RV32I ISA (
default), 16 entries --
-- (= 512 bit) for RV32E ISA (when RISC-V "E" extension is enabled
via "RVE_EN"). --
--
--
-- By default the register file is coded to infer block RAM (for
FPGAs), that does --
-- not provide a dedicated hardware reset. For ASIC implementation
or setups that --
-- do require a dedicated hardware reset a flip-flop-based
architecture can be --
-- enabled via "RST_EN".
--
--
--
-- [NOTE] Read-during-write behavior of the register file's memory
core is --
-- irrelevant as read and write accesses are mutually
exclusive and will --
-- never happen at the same time.
--
-- -----
--
-- The NEORV32 RISC-V Processor - https://github.com/stnolting/
neorv32 --
-- Copyright (c) NEORV32 contributors.
--
-- Copyright (c) 2020 - 2025 Stephan Nolting. All rights reserved.
--
-- Licensed under the BSD-3-Clause license, see LICENSE for details.
--
-- SPDX-License-Identifier: BSD-3-Clause
--
--
```

```

=====
--

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library neorv32;
use neorv32.neorv32_package.all;

entity neorv32_cpu_regfile is
  generic (
    RST_EN : boolean; -- implement dedicated hardware reset ("ASIC
                        style")
    RVE_EN : boolean -- implement embedded RF extension
  );
  port (
    -- global control --
    clk_i : in std_ulogic; -- global clock, rising edge
    rstn_i : in std_ulogic; -- global reset, low-active, async
    ctrl_i : in ctrl_bus_t; -- main control bus
    -- operands --
    rd_i : in std_ulogic_vector(XLEN-1 downto 0); -- destination
          data rd
    rs1_o : out std_ulogic_vector(XLEN-1 downto 0); -- source data
          rs1
    rs2_o : out std_ulogic_vector(XLEN-1 downto 0) -- source data
          rs2
  );
end neorv32_cpu_regfile;

architecture neorv32_cpu_regfile_rtl of neorv32_cpu_regfile is

  -- auto-configuration --
  constant awidth_c : natural := cond_sel_natural_f(RVE_EN, 4, 5);
  -- address width

  -- register file --
  type reg_file_t is array (0 to (2**awidth_c)-1) of
    std_ulogic_vector(XLEN-1 downto 0);
  signal reg_file : reg_file_t;

  -- access logic --
  signal rf_we, rd_zero : std_ulogic;
  signal opa_addr : std_ulogic_vector(4 downto 0);

begin

  -- FPGA-Style Register File Access Logic
  -----

  --
  -----

  -- Register zero (x0) is a "normal" physical register that is set
  -- to zero by the CPU control
  -- hardware. The register file uses synchronous read accesses and
  -- a *single* multiplexed
  -- address port for writing and reading rd/rs1 and a single read-
  -- only port for reading rs2.
  -- Therefore, the whole register file can be mapped to a single
  -- true-dual-port RAM.

  rd_zero <= '1' when (ctrl_i.rf_rd = "00000") else '0';
  rf_we <= (ctrl_i.rf_wb_en and (not rd_zero)) or ctrl_i.
    rf_zero_we; -- never write to x0 unless forced

```

```
opa_addr <= "00000" when (ctrl_i.rf_zero_we = '1') else -- force
    rd = zero
    ctrl_i.rf_rd when (ctrl_i.rf_wb_en = '1') else -- rd
    ctrl_i.rf_rs1; -- rs1
```

```
-- FPGA-Style Register File (BlockRAM, no hardware reset at all)
-----
```

```
--
```

```
register_file_fpga:
if not RST_EN generate
    reg_file_inst: entity neorv32.neorv32_prim_sdpram
    generic map (
        AWIDTH => awidth_c,
        DWIDTH => XLEN,
        OUTREG => false
    )
    port map (
        -- global control --
        clk_i    => clk_i,
        -- write port --
        a_en_i    => '1',
        a_rw_i    => rf_we,
        a_addr_i  => opa_addr(awidth_c-1 downto 0),
        a_data_i  => rd_i,
        a_data_o  => rs1_o,
        -- read port --
        b_en_i    => '1',
        b_addr_i  => ctrl_i.rf_rs2(awidth_c-1 downto 0),
        b_data_o  => rs2_o
    );
    reg_file <= (others => (others => '0')); -- unused
end generate;
```

```
-- ASIC-Style Register File (individual FFs, full hardware reset)
-----
```

```
--
```

```
register_file_asic:
if RST_EN generate

    -- x0 is hardwired to zero --
    reg_file(0) <= (others => '0');

    -- individual registers --
    reg_gen:
    for i in 1 to (2**awidth_c)-1 generate
        register_file: process(rstn_i, clk_i)
        begin
            if (rstn_i = '0') then
                reg_file(i) <= (others => '0'); -- full hardware reset
            elsif rising_edge(clk_i) then

                -- ctrl_i is bus control
                if (unsigned(ctrl_i.rf_rd(awidth_c-1 downto 0)) =
                    to_unsigned(i, awidth_c))
                    and (ctrl_i.rf_wb_en = '1') then
                    reg_file(i) <= rd_i;
                end if;
            end if;
        end process register_file;
    end generate;
```

```

-- synchronous read --
rf_read: process(rstn_i, clk_i)
begin
    if (rstn_i = '0') then
        rs1_o <= (others => '0');
        rs2_o <= (others => '0');
    elsif rising_edge(clk_i) then
        rs1_o <= reg_file(to_integer(unsigned(ctrl_i.rf_rs1(awidth_c
            -1 downto 0))));
        rs2_o <= reg_file(to_integer(unsigned(ctrl_i.rf_rs2(awidth_c
            -1 downto 0))));
    end if;
end process rf_read;

end generate;

end neorv32_cpu_regfile_rtl;

```

Quellcode 5: neorv32_package.vhd

```

--
=====
--
-- NEORV32 - Main VHDL Package File
--
-----
--
-- The NEORV32 RISC-V Processor - https://github.com/stnolting/
-- neorv32
-- Copyright (c) NEORV32 contributors.
--
-- Copyright (c) 2020 - 2025 Stephan Nolting. All rights reserved.
--
-- Licensed under the BSD-3-Clause license, see LICENSE for details.
--
-- SPDX-License-Identifier: BSD-3-Clause
--
=====
--

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

package neorv32_package is

--
*****

-- Architecture Configuration and Constants
--
*****

-- Architecture Constants
-----

--
-----

```

```

constant hw_version_c : std_ulogic_vector(31 downto 0) := x"
    01120400"; -- hardware version
constant archid_c      : natural := 19; -- official RISC-V
    architecture ID
constant XLEN          : natural := 32; -- native data path width
constant MLEN          : natural := 16; -- matrix register width
constant int_bus_tmo_c : natural := 16; -- internal bus timeout
    window; has to be a power of two
constant alu_cp_tmo_c  : natural := 9;  -- log2 of max ALU co-
    processor execution cycles

-- https://stackoverflow.com/questions/72246811/best-way-to-define
-- -and-initialize -matrix-in-vhdl
-- https://stackoverflow.com/questions/63291844/vhdl-unconstrained
-- -arrays-and-size-instantiation
--type my_array_t is array(integer range <>, integer range <>) of
    integer;
type my_array_t is array(integer range <>, integer range <>) of
    std_ulogic_vector(31 downto 0);

--type YOUR_ARRAY_TYPE is array (31 downto 0) of std_ulogic_vector
    (31 downto 0);
--type YOUR_ARRAY_TYPE is array(31 downto 0) of std_logic_vector
    (31 downto 0);
type YOUR_ARRAY_TYPE is array(0 to 31) of std_logic_vector(31
    downto 0);

-- Check if we're inside the Matrix
-- -----
-- -----

constant is_simulation_c : boolean := false -- seems like we're on
    real hardware
-- pragma translate_off
-- RTL_SYNTHESIS OFF
-- or true -- this MIGHT be a simulation
-- RTL_SYNTHESIS ON
-- pragma translate_on
;

--
-- *****

-- Processor Address Space Layout
--
-- *****

-- Main Address Regions (base address must be aligned to the
-- region's size) ---
constant mem_imem_base_c : std_ulogic_vector(31 downto 0) := x"
    00000000"; -- IMEM size via top generic
constant mem_dmem_base_c : std_ulogic_vector(31 downto 0) := x"
    80000000"; -- DMEM size via top generic
constant mem_io_base_c   : std_ulogic_vector(31 downto 0) := x"
    ffe00000";
constant mem_io_size_c   : natural := 32*64*1024; -- 32 *
    iodev_size_c

-- Start of uncached memory access (256MB page / 4 MSBs only) --
constant mem_uncached_begin_c : std_ulogic_vector(31 downto 0) :=
    x"f0000000";

-- IO Address Map (base address must be aligned to the region's
-- size) --

```

```

constant iodev_size_c      : natural := 64*1024; -- size of a
    single IO device (bytes)
constant base_io_bootrom_c : std_ulogic_vector(31 downto 0) := x"
    ffe00000";
--constant base_io_???_c    : std_ulogic_vector(31 downto 0) := x"
    ffe10000"; -- reserved
--constant base_io_???_c    : std_ulogic_vector(31 downto 0) := x"
    ffe20000"; -- reserved
--constant base_io_???_c    : std_ulogic_vector(31 downto 0) := x"
    ffe30000"; -- reserved
--constant base_io_???_c    : std_ulogic_vector(31 downto 0) := x"
    ffe40000"; -- reserved
--constant base_io_???_c    : std_ulogic_vector(31 downto 0) := x"
    ffe50000"; -- reserved
--constant base_io_???_c    : std_ulogic_vector(31 downto 0) := x"
    ffe60000"; -- reserved
--constant base_io_???_c    : std_ulogic_vector(31 downto 0) := x"
    ffe70000"; -- reserved
--constant base_io_???_c    : std_ulogic_vector(31 downto 0) := x"
    ffe80000"; -- reserved
--constant base_io_???_c    : std_ulogic_vector(31 downto 0) := x"
    ffe90000"; -- reserved
constant base_io_twd_c      : std_ulogic_vector(31 downto 0) := x"
    ffea0000";
constant base_io_cfs_c     : std_ulogic_vector(31 downto 0) := x"
    ffeb0000";
constant base_io_slink_c   : std_ulogic_vector(31 downto 0) := x"
    ffec0000";
constant base_io_dma_c     : std_ulogic_vector(31 downto 0) := x"
    ffed0000";
--constant base_io_???_c    : std_ulogic_vector(31 downto 0) := x"
    ffee0000"; -- reserved
--constant base_io_???_c    : std_ulogic_vector(31 downto 0) := x"
    ffef0000"; -- reserved
constant base_io_pwm_c     : std_ulogic_vector(31 downto 0) := x"
    fff00000";
constant base_io_gptmr_c   : std_ulogic_vector(31 downto 0) := x"
    fff10000";
constant base_io_onewire_c : std_ulogic_vector(31 downto 0) := x"
    fff20000";
constant base_io_tracer_c  : std_ulogic_vector(31 downto 0) := x"
    fff30000";
constant base_io_clint_c   : std_ulogic_vector(31 downto 0) := x"
    fff40000";
constant base_io_uart0_c   : std_ulogic_vector(31 downto 0) := x"
    fff50000";
constant base_io_uart1_c   : std_ulogic_vector(31 downto 0) := x"
    fff60000";
constant base_io_sdi_c     : std_ulogic_vector(31 downto 0) := x"
    fff70000";
constant base_io_spi_c     : std_ulogic_vector(31 downto 0) := x"
    fff80000";
constant base_io_twi_c     : std_ulogic_vector(31 downto 0) := x"
    fff90000";
constant base_io_trng_c    : std_ulogic_vector(31 downto 0) := x"
    fffa0000";
constant base_io_wdt_c     : std_ulogic_vector(31 downto 0) := x"
    fffb0000";
constant base_io_gpio_c    : std_ulogic_vector(31 downto 0) := x"
    fffc0000";
constant base_io_neoled_c  : std_ulogic_vector(31 downto 0) := x"
    fffd0000";
constant base_io_sysinfo_c : std_ulogic_vector(31 downto 0) := x"
    fffe0000";
constant base_io_ocd_c     : std_ulogic_vector(31 downto 0) := x"
    ffff0000";

```

```
-- On-Chip Debugger - debug module entry points (code ROM) --
constant dm_exc_entry_c : std_ulogic_vector(31 downto 0) := x"
    fffffe00"; -- = base_io_oecd_c + code_rom_base + 0
constant dm_park_entry_c : std_ulogic_vector(31 downto 0) := x"
    fffffe04"; -- = base_io_oecd_c + code_rom_base + 4

--
    *****

-- SoC Definitions
--
    *****

-- SoC Clock Select
    -----

--
    -----

constant clk_div2_c      : natural := 0;
constant clk_div4_c      : natural := 1;
constant clk_div8_c      : natural := 2;
constant clk_div64_c     : natural := 3;
constant clk_div128_c    : natural := 4;
constant clk_div1024_c   : natural := 5;
constant clk_div2048_c   : natural := 6;
constant clk_div4096_c   : natural := 7;

-- Internal Bus Interface
    -----

--
    -----

-- bus request --
type bus_req_t is record
    meta : std_ulogic_vector(2 downto 0); -- access meta
        information
    addr : std_ulogic_vector(31 downto 0); -- access address
    data : std_ulogic_vector(31 downto 0); -- write data
    ben  : std_ulogic_vector(3 downto 0); -- byte enable
    stb  : std_ulogic; -- request strobe, single-shot
    rw   : std_ulogic; -- 0 = read, 1 = write
    amo  : std_ulogic; -- set if atomic memory operation
    amoop : std_ulogic_vector(3 downto 0); -- type of atomic memory
        operation
    burst : std_ulogic; -- set if part of burst access
    lock  : std_ulogic; -- set if exclusive access request
    -- out-of-band signals --
    fence : std_ulogic; -- set if fence(.i) operation, single-shot
end record;

-- bus source (request) termination --
constant req_terminate_c : bus_req_t := (
    meta => (others => '0'),
    addr => (others => '0'),
    data => (others => '0'),
    ben  => (others => '0'),
    stb  => '0',
    rw   => '0',
    amo  => '0',
    amoop => (others => '0'),
    burst => '0',
    lock => '0',
```

```

    fence => '0'
);

-- bus response --
type bus_rsp_t is record
    ack : std_ulogic; -- set if access acknowledge, single-shot
    err : std_ulogic; -- set if access error, valid if ack = 1
    data : std_ulogic_vector(31 downto 0); -- read data, valid if
        ack = 1
end record;

-- bus endpoint (response) termination --
constant rsp_terminate_c : bus_rsp_t := (
    ack => '0',
    err => '0',
    data => (others => '0')
);

-- Matrix Extension -----
-----

type ram_type is array (0 to MLEN-1) of std_ulogic_vector(XLEN-1
    downto 0);
subtype matrix_ram_addr is integer range 0 to MLEN-1;

-- Debug Module Interface
-----

--
-----

-- request --
type dmi_req_t is record
    op : std_ulogic_vector(1 downto 0); -- single-shot
    addr : std_ulogic_vector(6 downto 0);
    data : std_ulogic_vector(31 downto 0);
end record;

-- source (request) termination --
constant dmi_req_terminate_c : dmi_req_t := (
    op => (others => '0'),
    addr => (others => '0'),
    data => (others => '0')
);

-- request operation --
constant dmi_req_nop_c : std_ulogic_vector(1 downto 0) := "00"; --
    no operation
constant dmi_req_rd_c : std_ulogic_vector(1 downto 0) := "01"; --
    read access
constant dmi_req_wr_c : std_ulogic_vector(1 downto 0) := "10"; --
    write access

-- response --
type dmi_rsp_t is record
    data : std_ulogic_vector(31 downto 0);
    ack : std_ulogic; -- single-shot
end record;

-- endpoint (response) termination --
constant dmi_rsp_terminate_c : dmi_rsp_t := (
    data => (others => '0'),
    ack => '0'
);

```

```
-- External Bus Interface (XBUS / Wishbone)
-----

--
-----

-- xbus request --
type xbus_req_t is record
  addr : std_ulogic_vector(31 downto 0); -- access address
  data : std_ulogic_vector(31 downto 0); -- write data
  cti   : std_ulogic_vector(2  downto 0); -- cycle type
  tag   : std_ulogic_vector(2  downto 0); -- access tag
  we    : std_ulogic; -- read/write
  sel   : std_ulogic_vector(3  downto 0); -- byte enable
  stb   : std_ulogic; -- strobe
  cyc   : std_ulogic; -- valid cycle
end record;

-- source (request) termination --
constant xbus_req_terminate_c : xbus_req_t := (
  addr => (others => '0'),
  data => (others => '0'),
  cti  => (others => '0'),
  tag  => (others => '0'),
  we   => '0',
  sel  => (others => '0'),
  stb  => '0',
  cyc  => '0'
);

-- xbus response --
type xbus_rsp_t is record
  data : std_ulogic_vector(31 downto 0); -- read data, valid if
    ack=1
  ack  : std_ulogic; -- access acknowledge
  err  : std_ulogic; -- access error
end record;

-- endpoint (response) termination --
constant xbus_rsp_terminate_c : xbus_rsp_t := (
  data => (others => '0'),
  ack  => '0',
  err  => '0'
);

-- CPU Trace Port
-----

--
-----

type trace_port_t is record
  valid      : std_ulogic; -- all other signals are valid when set
  -- instruction metadata --
  order      : std_ulogic_vector(31 downto 0); -- instruction index
  insn       : std_ulogic_vector(31 downto 0); -- instruction word
  trap       : std_ulogic; -- set if the current instruction causes
    a sync exception
  halt       : std_ulogic; -- set if last instruction before
    halting
  intr       : std_ulogic; -- set if executing the first
    instruction of a trap handler
  mode       : std_ulogic_vector(1  downto 0); -- 00 = user mode, 11
    = machine mode
  ixl        : std_ulogic_vector(1  downto 0); -- XLEN; 01 = 32-bit
  debug      : std_ulogic; -- set if instruction is executed in
    debug-mode

```

```

compr      : std_ulogic; -- set if instruction is a decompressed
              instruction
-- integer register --
rs1_addr   : std_ulogic_vector(4 downto 0); -- rs1 address
rs2_addr   : std_ulogic_vector(4 downto 0); -- rs2 address
rs1_rdata  : std_ulogic_vector(31 downto 0); -- rs1 read data
rs2_rdata  : std_ulogic_vector(31 downto 0); -- rs2 read data
rd_addr    : std_ulogic_vector(4 downto 0); -- rd address
rd_rdata   : std_ulogic_vector(31 downto 0); -- rd write data
-- program counter --
pc_rdata   : std_ulogic_vector(31 downto 0); -- current
              instruction address
pc_wdata   : std_ulogic_vector(31 downto 0); -- next instruction
              address
-- control and status register --
csr_addr   : std_ulogic_vector(11 downto 0); -- csr address
csr_rdata  : std_ulogic_vector(31 downto 0); -- csr read data
csr_wdata  : std_ulogic_vector(31 downto 0); -- csr write data
-- memory access --
mem_addr   : std_ulogic_vector(31 downto 0); -- address
mem_rmask  : std_ulogic_vector(3 downto 0);  -- read-enable
mem_wmask  : std_ulogic_vector(3 downto 0);  -- write-enable
mem_rdata  : std_ulogic_vector(31 downto 0); -- read data
mem_wdata  : std_ulogic_vector(31 downto 0); -- write data
end record;

-- trace source termination --
constant trace_port_terminate_c : trace_port_t := (
  valid      => '0',
  order      => (others => '0'),
  insn       => (others => '0'),
  trap       => '0',
  halt       => '0',
  intr       => '0',
  mode       => (others => '0'),
  ixl        => "01",
  debug      => '0',
  compr      => '0',
  rs1_addr   => (others => '0'),
  rs2_addr   => (others => '0'),
  rs1_rdata  => (others => '0'),
  rs2_rdata  => (others => '0'),
  rd_addr    => (others => '0'),
  rd_rdata   => (others => '0'),
  pc_rdata   => (others => '0'),
  pc_wdata   => (others => '0'),
  csr_addr   => (others => '0'),
  csr_rdata  => (others => '0'),
  csr_wdata  => (others => '0'),
  mem_addr   => (others => '0'),
  mem_rmask  => (others => '0'),
  mem_wmask  => (others => '0'),
  mem_rdata  => (others => '0'),
  mem_wdata  => (others => '0')
);

--
*****

-- RISC-V ISA Definitions
--
*****

-- RISC-V 32-Bit Instruction Word Layout
-----

```

```
--
-----

constant instr_opcode_lsb_c : natural := 0; -- opcode bit 0
constant instr_opcode_msb_c : natural := 6; -- opcode bit 6
constant instr_rd_lsb_c     : natural := 7; -- destination
    register address bit 0
constant instr_rd_msb_c     : natural := 11; -- destination
    register address bit 4
constant instr_funct3_lsb_c : natural := 12; -- funct3 bit 0
constant instr_funct3_msb_c : natural := 14; -- funct3 bit 2
constant instr_rs1_lsb_c    : natural := 15; -- source register 1
    address bit 0
constant instr_rs1_msb_c    : natural := 19; -- source register 1
    address bit 4
constant instr_rs2_lsb_c    : natural := 20; -- source register 2
    address bit 0
constant instr_rs2_msb_c    : natural := 24; -- source register 2
    address bit 4
constant instr_funct7_lsb_c : natural := 25; -- funct7 bit 0
constant instr_funct7_msb_c : natural := 31; -- funct7 bit 6
constant instr_imm12_lsb_c  : natural := 20; -- immediate12 bit 0
constant instr_imm12_msb_c  : natural := 31; -- immediate12 bit
    11
constant instr_imm20_lsb_c  : natural := 12; -- immediate20 bit 0
constant instr_imm20_msb_c  : natural := 31; -- immediate20 bit
    21
constant instr_funct5_lsb_c : natural := 27; -- funct5 select bit
    0
constant instr_funct5_msb_c : natural := 31; -- funct5 select bit
    4

-- RISC-V Opcodes
-----

--
-----

-- matrix --
-- USING THIS OPCODE, there is no LSU processing!
--constant opcode_matrix_c : std_ulogic_vector(6 downto 0) :=
    "1110111"; -- matrix operation
--constant opcode_matrix_c : std_ulogic_vector(6 downto 0) :=
    "0000111"; -- matrix operation
constant opcode_matrix_c : std_ulogic_vector(6 downto 0) := "
    1011011"; -- matrix operation
-- alu --
constant opcode_alui_c : std_ulogic_vector(6 downto 0) := "
    0010011"; -- ALU operation with immediate
constant opcode_alu_c : std_ulogic_vector(6 downto 0) := "
    0110011"; -- ALU operation
constant opcode_lui_c : std_ulogic_vector(6 downto 0) := "
    0110111"; -- load upper immediate
constant opcode_auipc_c : std_ulogic_vector(6 downto 0) := "
    0010111"; -- add upper immediate to PC
-- control flow --
constant opcode_jal_c : std_ulogic_vector(6 downto 0) := "
    1101111"; -- jump and link
constant opcode_jalr_c : std_ulogic_vector(6 downto 0) := "
    1100111"; -- jump and link with register
constant opcode_branch_c : std_ulogic_vector(6 downto 0) := "
    1100011"; -- branch
-- memory access --
constant opcode_load_c : std_ulogic_vector(6 downto 0) := "
    0000011"; -- load
```

```

constant opcode_store_c : std_ulogic_vector(6 downto 0) := "
0100011"; -- store
constant opcode_amo_c   : std_ulogic_vector(6 downto 0) := "
0101111"; -- atomic memory access
constant opcode_fence_c : std_ulogic_vector(6 downto 0) := "
0001111"; -- fence / fence.i
-- system/csr --
constant opcode_system_c : std_ulogic_vector(6 downto 0) := "
1110011"; -- system/csr access
-- floating point operations --
constant opcode_fpu_c   : std_ulogic_vector(6 downto 0) := "
1010011"; -- dual/single operand instruction
-- official custom RISC-V opcodes - free for custom instructions
--
constant opcode_cust0_c : std_ulogic_vector(6 downto 0) := "
0001011"; -- custom-0 (NEORV32 CFU)
constant opcode_cust1_c : std_ulogic_vector(6 downto 0) := "
0101011"; -- custom-1 (NEORV32 CFU)
--constant opcode_cust2_c : std_ulogic_vector(6 downto 0) := "
"1011011"; -- custom-2 (reserved)
--constant opcode_cust3_c : std_ulogic_vector(6 downto 0) := "
"1111011"; -- custom-3 (reserved)

-- RISC-V Funct3
-----

--
-----

-- matrix
constant funct3_msetimli_c : std_ulogic_vector(2 downto 0) := "
000";
constant funct3_mse32_c   : std_ulogic_vector(2 downto 0) := "
001"; -- store words into matrix unit
constant funct3_mle32_c   : std_ulogic_vector(2 downto 0) := "
010"; -- load words out of matrix unit
constant funct3_madd32_c  : std_ulogic_vector(2 downto 0) := "
011"; -- this was only used for testing
constant funct3_mmul32_c  : std_ulogic_vector(2 downto 0) := "
100"; -- outer product
-- conditional branch --
constant funct3_beq_c     : std_ulogic_vector(2 downto 0) := "000";
-- branch if equal
constant funct3_bne_c     : std_ulogic_vector(2 downto 0) := "001";
-- branch if not equal
constant funct3_beqi_c    : std_ulogic_vector(2 downto 0) := "010";
-- branch if equal immediate (Zibi ISA ext.)
constant funct3_bnei_c    : std_ulogic_vector(2 downto 0) := "011";
-- branch if not equal immediate (Zibi ISA ext.)
constant funct3_blt_c     : std_ulogic_vector(2 downto 0) := "100";
-- branch if less than
constant funct3_bge_c     : std_ulogic_vector(2 downto 0) := "101";
-- branch if greater than or equal
constant funct3_bltu_c    : std_ulogic_vector(2 downto 0) := "110";
-- branch if less than (unsigned)
constant funct3_bgeu_c    : std_ulogic_vector(2 downto 0) := "111";
-- branch if greater than or equal (unsigned)
-- memory access --
constant funct3_lb_c      : std_ulogic_vector(2 downto 0) := "000";
-- load byte (signed)
constant funct3_lh_c      : std_ulogic_vector(2 downto 0) := "001";
-- load half word (signed)
constant funct3_lw_c      : std_ulogic_vector(2 downto 0) := "010";
-- load word (signed)
constant funct3_lbu_c     : std_ulogic_vector(2 downto 0) := "100";
-- load byte (unsigned)

```

```

constant funct3_lhu_c    : std_ulogic_vector(2 downto 0) := "101";
    -- load half word (unsigned)
constant funct3_lwu_c    : std_ulogic_vector(2 downto 0) := "110";
    -- load word (unsigned)
constant funct3_sb_c     : std_ulogic_vector(2 downto 0) := "000";
    -- store byte
constant funct3_sh_c     : std_ulogic_vector(2 downto 0) := "001";
    -- store half word
constant funct3_sw_c     : std_ulogic_vector(2 downto 0) := "010";
    -- store word
-- alu --
constant funct3_sadd_c   : std_ulogic_vector(2 downto 0) := "000";
    -- sub/add
constant funct3_sll_c    : std_ulogic_vector(2 downto 0) := "001";
    -- shift logical left
constant funct3_slt_c    : std_ulogic_vector(2 downto 0) := "010";
    -- set on less
constant funct3_sltu_c   : std_ulogic_vector(2 downto 0) := "011";
    -- set on less unsigned
constant funct3_xor_c    : std_ulogic_vector(2 downto 0) := "100";
    -- logical exclusive-or
constant funct3_sr_c     : std_ulogic_vector(2 downto 0) := "101";
    -- shift right
constant funct3_or_c     : std_ulogic_vector(2 downto 0) := "110";
    -- logical or
constant funct3_and_c    : std_ulogic_vector(2 downto 0) := "111";
    -- logical and
-- system/csr --
constant funct3_env_c    : std_ulogic_vector(2 downto 0) := "000";
    -- ecall, ebreak, xret, wfi, etc.
constant funct3_csrrw_c  : std_ulogic_vector(2 downto 0) := "001";
    -- csr r/w
constant funct3_csrrs_c  : std_ulogic_vector(2 downto 0) := "010";
    -- csr read & set
constant funct3_csrrc_c  : std_ulogic_vector(2 downto 0) := "011";
    -- csr read & clear
constant funct3_csrril_c : std_ulogic_vector(2 downto 0) := "100";
    -- undefined/illegal csr command
constant funct3_csrrwi_c : std_ulogic_vector(2 downto 0) := "101";
    -- csr r/w immediate
constant funct3_csrrsi_c : std_ulogic_vector(2 downto 0) := "110";
    -- csr read & set immediate
constant funct3_csrrci_c : std_ulogic_vector(2 downto 0) := "111";
    -- csr read & clear immediate
-- fence --
constant funct3_fence_c  : std_ulogic_vector(2 downto 0) := "000";
    -- fence - order IO/memory access
constant funct3_fencei_c : std_ulogic_vector(2 downto 0) := "001";
    -- fence.i - instruction stream sync

-- RISC-V Funct12 - SYSTEM
-----

--
-----

constant funct12_ecall_c : std_ulogic_vector(11 downto 0) := x"
    000";
constant funct12_ebreak_c : std_ulogic_vector(11 downto 0) := x"
    001";
constant funct12_wfi_c    : std_ulogic_vector(11 downto 0) := x"
    105";
constant funct12_mret_c   : std_ulogic_vector(11 downto 0) := x"
    302";
constant funct12_dret_c   : std_ulogic_vector(11 downto 0) := x"7
    b2";

```

```
-- RISC-V Floating-Point Stuff
-----

--
-----

-- number class flags --
constant fp_class_neg_inf_c    : natural := 0; -- negative
      infinity
constant fp_class_neg_norm_c   : natural := 1; -- negative normal
      number
constant fp_class_neg_denorm_c : natural := 2; -- negative
      subnormal number
constant fp_class_neg_zero_c   : natural := 3; -- negative zero
constant fp_class_pos_zero_c   : natural := 4; -- positive zero
constant fp_class_pos_denorm_c : natural := 5; -- positive
      subnormal number
constant fp_class_pos_norm_c   : natural := 6; -- positive normal
      number
constant fp_class_pos_inf_c    : natural := 7; -- positive
      infinity
constant fp_class_snan_c       : natural := 8; -- signaling NaN (
      sNaN)
constant fp_class_qnan_c       : natural := 9; -- quiet NaN (qNaN)

-- exception flags --
constant fp_exc_nx_c : natural := 0; -- inexact
constant fp_exc_uf_c : natural := 1; -- underflow
constant fp_exc_of_c : natural := 2; -- overflow
constant fp_exc_dz_c : natural := 3; -- division by zero
constant fp_exc_nv_c : natural := 4; -- invalid operation

-- special values (single-precision) --
constant fp_single_qnan_c      : std_ulogic_vector(31 downto 0) :=
      x"7fc00000"; -- quiet NaN
constant fp_single_snan_c      : std_ulogic_vector(31 downto 0) :=
      x"7fa00000"; -- signaling NaN
constant fp_single_pos_max_c   : std_ulogic_vector(31 downto 0) :=
      x"7f7fffff"; -- positive max
constant fp_single_neg_max_c   : std_ulogic_vector(31 downto 0) :=
      x"ff7fffff"; -- negative max
constant fp_single_pos_inf_c   : std_ulogic_vector(31 downto 0) :=
      x"7f800000"; -- positive infinity
constant fp_single_neg_inf_c   : std_ulogic_vector(31 downto 0) :=
      x"ff800000"; -- negative infinity
constant fp_single_pos_zero_c  : std_ulogic_vector(31 downto 0) :=
      x"00000000"; -- positive zero
constant fp_single_neg_zero_c  : std_ulogic_vector(31 downto 0) :=
      x"80000000"; -- negative zero

-- RISC-V CSRs
-----

--
-----

-- user floating-point CSRs --
constant csr_fflags_c          : std_ulogic_vector(11 downto 0) :=
      x"001";
constant csr_frm_c             : std_ulogic_vector(11 downto 0) :=
      x"002";
constant csr_fcsr_c            : std_ulogic_vector(11 downto 0) :=
      x"003";
-- machine trap setup --
constant csr_mstatus_c         : std_ulogic_vector(11 downto 0) :=
      x"300";
```

```

constant csr_misa_c      : std_ulogic_vector(11 downto 0) :=
    x"301";
constant csr_mie_c       : std_ulogic_vector(11 downto 0) :=
    x"304";
constant csr_mtvec_c     : std_ulogic_vector(11 downto 0) :=
    x"305";
constant csr_mcounteren_c : std_ulogic_vector(11 downto 0) :=
    x"306";
constant csr_mstatush_c  : std_ulogic_vector(11 downto 0) :=
    x"310";
-- machine configuration --
constant csr_menvcfg_c   : std_ulogic_vector(11 downto 0) :=
    x"30a";
constant csr_menvcfgh_c  : std_ulogic_vector(11 downto 0) :=
    x"31a";
-- machine counter setup --
constant csr_mcountinhibit_c : std_ulogic_vector(11 downto 0) :=
    x"320";
constant csr_mhpmevent3_c : std_ulogic_vector(11 downto 0) :=
    x"323";
constant csr_mhpmevent4_c : std_ulogic_vector(11 downto 0) :=
    x"324";
constant csr_mhpmevent5_c : std_ulogic_vector(11 downto 0) :=
    x"325";
constant csr_mhpmevent6_c : std_ulogic_vector(11 downto 0) :=
    x"326";
constant csr_mhpmevent7_c : std_ulogic_vector(11 downto 0) :=
    x"327";
constant csr_mhpmevent8_c : std_ulogic_vector(11 downto 0) :=
    x"328";
constant csr_mhpmevent9_c : std_ulogic_vector(11 downto 0) :=
    x"329";
constant csr_mhpmevent10_c : std_ulogic_vector(11 downto 0) :=
    x"32a";
constant csr_mhpmevent11_c : std_ulogic_vector(11 downto 0) :=
    x"32b";
constant csr_mhpmevent12_c : std_ulogic_vector(11 downto 0) :=
    x"32c";
constant csr_mhpmevent13_c : std_ulogic_vector(11 downto 0) :=
    x"32d";
constant csr_mhpmevent14_c : std_ulogic_vector(11 downto 0) :=
    x"32e";
constant csr_mhpmevent15_c : std_ulogic_vector(11 downto 0) :=
    x"32f";
-- machine trap handling --
constant csr_mscratch_c   : std_ulogic_vector(11 downto 0) :=
    x"340";
constant csr_mepc_c       : std_ulogic_vector(11 downto 0) :=
    x"341";
constant csr_mcause_c     : std_ulogic_vector(11 downto 0) :=
    x"342";
constant csr_mtval_c      : std_ulogic_vector(11 downto 0) :=
    x"343";
constant csr_mip_c        : std_ulogic_vector(11 downto 0) :=
    x"344";
constant csr_mtinst_c     : std_ulogic_vector(11 downto 0) :=
    x"34a";
-- physical memory protection - configuration --
constant csr_pmpcfg0_c    : std_ulogic_vector(11 downto 0) :=
    x"3a0";
constant csr_pmpcfg1_c    : std_ulogic_vector(11 downto 0) :=
    x"3a1";
constant csr_pmpcfg2_c    : std_ulogic_vector(11 downto 0) :=
    x"3a2";
constant csr_pmpcfg3_c    : std_ulogic_vector(11 downto 0) :=
    x"3a3";

```

```
-- physical memory protection - address --
constant csr_pmpaddr0_c      : std_ulogic_vector(11 downto 0) :=
    x"3b0";
constant csr_pmpaddr1_c      : std_ulogic_vector(11 downto 0) :=
    x"3b1";
constant csr_pmpaddr2_c      : std_ulogic_vector(11 downto 0) :=
    x"3b2";
constant csr_pmpaddr3_c      : std_ulogic_vector(11 downto 0) :=
    x"3b3";
constant csr_pmpaddr4_c      : std_ulogic_vector(11 downto 0) :=
    x"3b4";
constant csr_pmpaddr5_c      : std_ulogic_vector(11 downto 0) :=
    x"3b5";
constant csr_pmpaddr6_c      : std_ulogic_vector(11 downto 0) :=
    x"3b6";
constant csr_pmpaddr7_c      : std_ulogic_vector(11 downto 0) :=
    x"3b7";
constant csr_pmpaddr8_c      : std_ulogic_vector(11 downto 0) :=
    x"3b8";
constant csr_pmpaddr9_c      : std_ulogic_vector(11 downto 0) :=
    x"3b9";
constant csr_pmpaddr10_c     : std_ulogic_vector(11 downto 0) :=
    x"3ba";
constant csr_pmpaddr11_c     : std_ulogic_vector(11 downto 0) :=
    x"3bb";
constant csr_pmpaddr12_c     : std_ulogic_vector(11 downto 0) :=
    x"3bc";
constant csr_pmpaddr13_c     : std_ulogic_vector(11 downto 0) :=
    x"3bd";
constant csr_pmpaddr14_c     : std_ulogic_vector(11 downto 0) :=
    x"3be";
constant csr_pmpaddr15_c     : std_ulogic_vector(11 downto 0) :=
    x"3bf";
-- trigger module registers --
constant csr_tselect_c       : std_ulogic_vector(11 downto 0) :=
    x"7a0";
constant csr_tdata1_c        : std_ulogic_vector(11 downto 0) :=
    x"7a1";
constant csr_tdata2_c        : std_ulogic_vector(11 downto 0) :=
    x"7a2";
constant csr_tinfo_c         : std_ulogic_vector(11 downto 0) :=
    x"7a4";
-- debug registers --
constant csr_dcsr_c           : std_ulogic_vector(11 downto 0) :=
    x"7b0";
constant csr_dpc_c           : std_ulogic_vector(11 downto 0) :=
    x"7b1";
constant csr_dscratch0_c      : std_ulogic_vector(11 downto 0) :=
    x"7b2";
-- machine counters/timers --
constant csr_mcycle_c         : std_ulogic_vector(11 downto 0) :=
    x"b00";
constant csr_mtime_c          : std_ulogic_vector(11 downto 0) :=
    x"b01";
constant csr_minstret_c       : std_ulogic_vector(11 downto 0) :=
    x"b02";
constant csr_mhpmcounter3_c   : std_ulogic_vector(11 downto 0) :=
    x"b03";
constant csr_mhpmcounter4_c   : std_ulogic_vector(11 downto 0) :=
    x"b04";
constant csr_mhpmcounter5_c   : std_ulogic_vector(11 downto 0) :=
    x"b05";
constant csr_mhpmcounter6_c   : std_ulogic_vector(11 downto 0) :=
    x"b06";
constant csr_mhpmcounter7_c   : std_ulogic_vector(11 downto 0) :=
    x"b07";
```

```

constant csr_mhpmcounter8_c    : std_ulogic_vector(11 downto 0) :=
    x"b08";
constant csr_mhpmcounter9_c    : std_ulogic_vector(11 downto 0) :=
    x"b09";
constant csr_mhpmcounter10_c   : std_ulogic_vector(11 downto 0) :=
    x"b0a";
constant csr_mhpmcounter11_c   : std_ulogic_vector(11 downto 0) :=
    x"b0b";
constant csr_mhpmcounter12_c   : std_ulogic_vector(11 downto 0) :=
    x"b0c";
constant csr_mhpmcounter13_c   : std_ulogic_vector(11 downto 0) :=
    x"b0d";
constant csr_mhpmcounter14_c   : std_ulogic_vector(11 downto 0) :=
    x"b0e";
constant csr_mhpmcounter15_c   : std_ulogic_vector(11 downto 0) :=
    x"b0f";
constant csr_mcycleh_c         : std_ulogic_vector(11 downto 0) :=
    x"b80";
constant csr_mtimeh_c          : std_ulogic_vector(11 downto 0) :=
    x"b81";
constant csr_minstreth_c       : std_ulogic_vector(11 downto 0) :=
    x"b82";
constant csr_mhpmcounter3h_c   : std_ulogic_vector(11 downto 0) :=
    x"b83";
constant csr_mhpmcounter4h_c   : std_ulogic_vector(11 downto 0) :=
    x"b84";
constant csr_mhpmcounter5h_c   : std_ulogic_vector(11 downto 0) :=
    x"b85";
constant csr_mhpmcounter6h_c   : std_ulogic_vector(11 downto 0) :=
    x"b86";
constant csr_mhpmcounter7h_c   : std_ulogic_vector(11 downto 0) :=
    x"b87";
constant csr_mhpmcounter8h_c   : std_ulogic_vector(11 downto 0) :=
    x"b88";
constant csr_mhpmcounter9h_c   : std_ulogic_vector(11 downto 0) :=
    x"b89";
constant csr_mhpmcounter10h_c  : std_ulogic_vector(11 downto 0) :=
    x"b8a";
constant csr_mhpmcounter11h_c  : std_ulogic_vector(11 downto 0) :=
    x"b8b";
constant csr_mhpmcounter12h_c  : std_ulogic_vector(11 downto 0) :=
    x"b8c";
constant csr_mhpmcounter13h_c  : std_ulogic_vector(11 downto 0) :=
    x"b8d";
constant csr_mhpmcounter14h_c  : std_ulogic_vector(11 downto 0) :=
    x"b8e";
constant csr_mhpmcounter15h_c  : std_ulogic_vector(11 downto 0) :=
    x"b8f";
-- user counters/timers --
constant csr_cycle_c           : std_ulogic_vector(11 downto 0) :=
    x"c00";
constant csr_time_c            : std_ulogic_vector(11 downto 0) :=
    x"c01";
constant csr_instret_c         : std_ulogic_vector(11 downto 0) :=
    x"c02";
constant csr_hpmcounter3_c     : std_ulogic_vector(11 downto 0) :=
    x"c03";
constant csr_hpmcounter4_c     : std_ulogic_vector(11 downto 0) :=
    x"c04";
constant csr_hpmcounter5_c     : std_ulogic_vector(11 downto 0) :=
    x"c05";
constant csr_hpmcounter6_c     : std_ulogic_vector(11 downto 0) :=
    x"c06";
constant csr_hpmcounter7_c     : std_ulogic_vector(11 downto 0) :=
    x"c07";

```

```

constant csr_hpmcounter8_c      : std_ulogic_vector(11 downto 0) :=
    x"c08";
constant csr_hpmcounter9_c      : std_ulogic_vector(11 downto 0) :=
    x"c09";
constant csr_hpmcounter10_c     : std_ulogic_vector(11 downto 0) :=
    x"c0a";
constant csr_hpmcounter11_c     : std_ulogic_vector(11 downto 0) :=
    x"c0b";
constant csr_hpmcounter12_c     : std_ulogic_vector(11 downto 0) :=
    x"c0c";
constant csr_hpmcounter13_c     : std_ulogic_vector(11 downto 0) :=
    x"c0d";
constant csr_hpmcounter14_c     : std_ulogic_vector(11 downto 0) :=
    x"c0e";
constant csr_hpmcounter15_c     : std_ulogic_vector(11 downto 0) :=
    x"c0f";
constant csr_cycleh_c          : std_ulogic_vector(11 downto 0) :=
    x"c80";
constant csr_timeh_c           : std_ulogic_vector(11 downto 0) :=
    x"c81";
constant csr_instreth_c         : std_ulogic_vector(11 downto 0) :=
    x"c82";
constant csr_hpmcounter3h_c     : std_ulogic_vector(11 downto 0) :=
    x"c83";
constant csr_hpmcounter4h_c     : std_ulogic_vector(11 downto 0) :=
    x"c84";
constant csr_hpmcounter5h_c     : std_ulogic_vector(11 downto 0) :=
    x"c85";
constant csr_hpmcounter6h_c     : std_ulogic_vector(11 downto 0) :=
    x"c86";
constant csr_hpmcounter7h_c     : std_ulogic_vector(11 downto 0) :=
    x"c87";
constant csr_hpmcounter8h_c     : std_ulogic_vector(11 downto 0) :=
    x"c88";
constant csr_hpmcounter9h_c     : std_ulogic_vector(11 downto 0) :=
    x"c89";
constant csr_hpmcounter10h_c    : std_ulogic_vector(11 downto 0) :=
    x"c8a";
constant csr_hpmcounter11h_c    : std_ulogic_vector(11 downto 0) :=
    x"c8b";
constant csr_hpmcounter12h_c    : std_ulogic_vector(11 downto 0) :=
    x"c8c";
constant csr_hpmcounter13h_c    : std_ulogic_vector(11 downto 0) :=
    x"c8d";
constant csr_hpmcounter14h_c    : std_ulogic_vector(11 downto 0) :=
    x"c8e";
constant csr_hpmcounter15h_c    : std_ulogic_vector(11 downto 0) :=
    x"c8f";
-- machine information registers --
constant csr_mvendid_c          : std_ulogic_vector(11 downto 0) :=
    x"f11";
constant csr_marchid_c          : std_ulogic_vector(11 downto 0) :=
    x"f12";
constant csr_mimpid_c           : std_ulogic_vector(11 downto 0) :=
    x"f13";
constant csr_mhartid_c          : std_ulogic_vector(11 downto 0) :=
    x"f14";
constant csr_mconfigptr_c       : std_ulogic_vector(11 downto 0) :=
    x"f15";
-- NEORV32-specific machine registers --
constant csr_mxcsr_c            : std_ulogic_vector(11 downto 0) :=
    x"bc0";
constant csr_mxisa_c            : std_ulogic_vector(11 downto 0) :=
    x"fc0";
--constant csr_mxisah_c          : std_ulogic_vector(11 downto 0) :=
    x"fc1"; -- to be implemented...

```

```
--
*****

-- CPU Control
--
*****

-- Main CPU Control Bus
-----

--
-----

type ctrl_bus_t is record
-- instruction fetch --
if_fence      : std_ulogic;           -- fence.i
  operation
if_reset      : std_ulogic;           -- restart
  instruction fetch
if_ready      : std_ulogic;           -- ready for next
  instruction
-- program counter --
pc_cur        : std_ulogic_vector(31 downto 0); -- address of
  current instruction
pc_nxt        : std_ulogic_vector(31 downto 0); -- address of
  next instruction
pc_ret        : std_ulogic_vector(31 downto 0); -- return address
-- register file --
rf_wb_en      : std_ulogic; -- write back enable
rf_rs1        : std_ulogic_vector(4 downto 0); -- source
  register 1 address
rf_rs2        : std_ulogic_vector(4 downto 0); -- source
  register 2 address
rf_rd         : std_ulogic_vector(4 downto 0); -- destination
  register address
rf_zero_we    : std_ulogic;           -- allow/force
  write access to x0
-- alu --
alu_op        : std_ulogic_vector(2 downto 0); -- operation
  select
alu_sub       : std_ulogic;           -- addition/
  subtraction control
alu_inc       : std_ulogic;           -- increment
  during addition/subtraction control
alu_opa_mux   : std_ulogic;           -- operand A
  select (0=rs1, 1=PC)
alu_opb_mux   : std_ulogic;           -- operand B
  select (0=rs2, 1=IMM)
alu_unsigned  : std_ulogic;           -- is unsigned
  ALU operation
alu_imm       : std_ulogic_vector(31 downto 0); -- immediate
alu_cp_alu    : std_ulogic;           -- ALU.base co-
  processor trigger (one-shot)
alu_cp_cfu    : std_ulogic;           -- CFU co-
  processor trigger (one-shot)
alu_cp_fpu    : std_ulogic;           -- FPU co-
  processor trigger (one-shot)
-- load/store unit --
lsu_req       : std_ulogic;           -- trigger memory
  access request
lsu_rw        : std_ulogic;           -- 0: read access
  , 1: write access
lsu_rmw       : std_ulogic;           -- set if atomic
  read-modify-write operation
```

```

lsu_rvs      : std_ulogic;                -- set if atomic
    reservation-set operation
lsu_mo_we    : std_ulogic;                -- memory address
    and data output register write enable
lsu_fence    : std_ulogic;                -- fence
    operation
lsu_priv     : std_ulogic;                -- effective
    privilege mode for load/store
lsu_m_en     : std_ulogic;                -- LSU matrix
    unit enabled
-- control and status registers --
csr_we       : std_ulogic;                -- global write-
    enable
csr_re       : std_ulogic;                -- global read-
    enable
csr_addr     : std_ulogic_vector(11 downto 0); -- address
csr_wdata    : std_ulogic_vector(31 downto 0); -- write data
-- counter events --
cnt_event    : std_ulogic_vector(11 downto 0); -- counter
    increment events
-- instruction word --
ir_func3     : std_ulogic_vector(2 downto 0); -- funct3 bit
    field
ir_func12    : std_ulogic_vector(11 downto 0); -- funct12 bit
    field
ir_opcode    : std_ulogic_vector(6 downto 0); -- opcode bit
    field
-- status --
cpu_priv     : std_ulogic;                -- effective
    privilege mode
cpu_trap     : std_ulogic;                -- set when CPU
    is entering trap
cpu_sync_exc : std_ulogic;                -- set when CPU
    encounters a synchronous exceptions
cpu_debug    : std_ulogic;                -- set when CPU
    is in debug mode
end record;

-- control bus reset termination --
constant ctrl_bus_zero_c : ctrl_bus_t := (
    if_fence    => '0',
    if_reset    => '0',
    if_ready    => '0',
    pc_cur      => (others => '0'),
    pc_nxt      => (others => '0'),
    pc_ret      => (others => '0'),
    rf_wb_en    => '0',
    rf_rs1      => (others => '0'),
    rf_rs2      => (others => '0'),
    rf_rd       => (others => '0'),
    rf_zero_we  => '0',
    alu_op      => (others => '0'),
    alu_sub     => '0',
    alu_inc     => '0', -- do not increment during add / sub
    operations
    alu_opa_mux => '0',
    alu_opb_mux => '0',
    alu_unsigned => '0',
    alu_imm     => (others => '0'),
    alu_cp_alu  => '0',
    alu_cp_cfu  => '0',
    alu_cp_fpu  => '0',
    lsu_req     => '0',
    lsu_rw      => '0',
    lsu_rmw     => '0',
    lsu_rvs     => '0',

```

```

lsu_mo_we    => '0',
lsu_fence    => '0',
lsu_priv     => '0',
lsu_m_en     => '0',
csr_we       => '0',
csr_re       => '0',
csr_addr     => (others => '0'),
csr_wdata    => (others => '0'),
cnt_event    => (others => '0'),
ir_funct3    => (others => '0'),
ir_funct12   => (others => '0'),
ir_opcode    => (others => '0'),
cpu_priv     => '0',
cpu_trap     => '0',
cpu_sync_exc => '0',
cpu_debug    => '0'
);

-- Instruction Fetch Interface
-----

--
-----

type if_bus_t is record
    valid : std_ulogic;           -- bus signals are
        valid
    instr : std_ulogic_vector(31 downto 0); -- instruction word
    compr : std_ulogic;           -- instruction is
        decompressed
    fault : std_ulogic;           -- instruction-fetch
        error
end record;

-- Comparator Bus
-----

--
-----

constant cmp_equal_c : natural := 0;
constant cmp_less_c  : natural := 1; -- for signed and unsigned
    comparisons

-- ALU Function Codes
-----

--
-----

constant alu_op_zero_c : std_ulogic_vector(2 downto 0) := "000";
    -- result <= 0
constant alu_op_add_c  : std_ulogic_vector(2 downto 0) := "001";
    -- result <= A + (-)B
constant alu_op_cp_c   : std_ulogic_vector(2 downto 0) := "010";
    -- result <= ALU co-processor
constant alu_op_slt_c  : std_ulogic_vector(2 downto 0) := "011";
    -- result <= A < B
constant alu_op_movb_c : std_ulogic_vector(2 downto 0) := "100";
    -- result <= B
constant alu_op_xor_c  : std_ulogic_vector(2 downto 0) := "101";
    -- result <= A xor B
constant alu_op_or_c   : std_ulogic_vector(2 downto 0) := "110";
    -- result <= A or B
constant alu_op_and_c  : std_ulogic_vector(2 downto 0) := "111";
    -- result <= A and B

```

```
-- Register File Input Select
-----

--
-----

constant rf_mux_alu_c : std_ulogic_vector(1 downto 0) := "00"; --
    register file <= alu result
constant rf_mux_mem_c : std_ulogic_vector(1 downto 0) := "01"; --
    register file <= memory read data
constant rf_mux_csr_c : std_ulogic_vector(1 downto 0) := "10"; --
    register file <= CSR read data
constant rf_mux_ret_c : std_ulogic_vector(1 downto 0) := "11"; --
    register file <= link-PC (return address)

-- Trap ID Codes
-----

--
-----

-- [MSB] 1 = interrupt, 0 = sync. exception; [MSB-1] 1 = entry to
    debug mode, 0 = normal trapping
-- RISC-V compliant synchronous exceptions --
constant trap_ima_c : std_ulogic_vector(6 downto 0) := "0" &
    "0" & "00000"; -- 0: instruction misaligned
constant trap_iaf_c : std_ulogic_vector(6 downto 0) := "0" &
    "0" & "00001"; -- 1: instruction access fault
constant trap_iil_c : std_ulogic_vector(6 downto 0) := "0" &
    "0" & "00010"; -- 2: illegal instruction
constant trap_brk_c : std_ulogic_vector(6 downto 0) := "0" &
    "0" & "00011"; -- 3: environment breakpoint
constant trap_lma_c : std_ulogic_vector(6 downto 0) := "0" &
    "0" & "00100"; -- 4: load address misaligned
constant trap_laf_c : std_ulogic_vector(6 downto 0) := "0" &
    "0" & "00101"; -- 5: load access fault
constant trap_sma_c : std_ulogic_vector(6 downto 0) := "0" &
    "0" & "00110"; -- 6: store address misaligned
constant trap_saf_c : std_ulogic_vector(6 downto 0) := "0" &
    "0" & "00111"; -- 7: store access fault
constant trap_env_c : std_ulogic_vector(6 downto 0) := "0" &
    "0" & "01000"; -- 8..11: environment call
-- RISC-V compliant asynchronous exceptions (interrupts) --
constant trap_msi_c : std_ulogic_vector(6 downto 0) := "1" &
    "0" & "00011"; -- 3: machine software interrupt
constant trap_mti_c : std_ulogic_vector(6 downto 0) := "1" &
    "0" & "00111"; -- 7: machine timer interrupt
constant trap_mei_c : std_ulogic_vector(6 downto 0) := "1" &
    "0" & "01011"; -- 11: machine external interrupt
-- NEORV32-specific (RISC-V custom) asynchronous exceptions (
    interrupts) --
constant trap_firq0_c : std_ulogic_vector(6 downto 0) := "1" &
    "0" & "10000"; -- 16: fast interrupt 0
constant trap_firq1_c : std_ulogic_vector(6 downto 0) := "1" &
    "0" & "10001"; -- 17: fast interrupt 1
constant trap_firq2_c : std_ulogic_vector(6 downto 0) := "1" &
    "0" & "10010"; -- 18: fast interrupt 2
constant trap_firq3_c : std_ulogic_vector(6 downto 0) := "1" &
    "0" & "10011"; -- 19: fast interrupt 3
constant trap_firq4_c : std_ulogic_vector(6 downto 0) := "1" &
    "0" & "10100"; -- 20: fast interrupt 4
constant trap_firq5_c : std_ulogic_vector(6 downto 0) := "1" &
    "0" & "10101"; -- 21: fast interrupt 5
constant trap_firq6_c : std_ulogic_vector(6 downto 0) := "1" &
    "0" & "10110"; -- 22: fast interrupt 6
constant trap_firq7_c : std_ulogic_vector(6 downto 0) := "1" &
    "0" & "10111"; -- 23: fast interrupt 7
```

```

constant trap_firq8_c      : std_ulogic_vector(6 downto 0) := "1" &
"0" & "11000"; -- 24: fast interrupt 8
constant trap_firq9_c      : std_ulogic_vector(6 downto 0) := "1" &
"0" & "11001"; -- 25: fast interrupt 9
constant trap_firq10_c     : std_ulogic_vector(6 downto 0) := "1" &
"0" & "11010"; -- 26: fast interrupt 10
constant trap_firq11_c     : std_ulogic_vector(6 downto 0) := "1" &
"0" & "11011"; -- 27: fast interrupt 11
constant trap_firq12_c     : std_ulogic_vector(6 downto 0) := "1" &
"0" & "11100"; -- 28: fast interrupt 12
constant trap_firq13_c     : std_ulogic_vector(6 downto 0) := "1" &
"0" & "11101"; -- 29: fast interrupt 13
constant trap_firq14_c     : std_ulogic_vector(6 downto 0) := "1" &
"0" & "11110"; -- 30: fast interrupt 14
constant trap_firq15_c     : std_ulogic_vector(6 downto 0) := "1" &
"0" & "11111"; -- 31: fast interrupt 15
-- debug-mode-entry exceptions --
constant trap_db_break_c   : std_ulogic_vector(6 downto 0) := "0" &
"1" & "00001"; -- 1: break instruction
constant trap_db_trig_c    : std_ulogic_vector(6 downto 0) := "1" &
"1" & "00010"; -- 2: hardware trigger
constant trap_db_halt_c    : std_ulogic_vector(6 downto 0) := "1" &
"1" & "00011"; -- 3: external halt request
constant trap_db_step_c    : std_ulogic_vector(6 downto 0) := "1" &
"1" & "00100"; -- 4: single-stepping

-- Trap System
-----

--
-----

-- exception source list (do not change order, only append!) --
constant exc_iaccess_c    : natural := 0; -- instruction access
      fault
constant exc_illegal_c    : natural := 1; -- illegal instruction
constant exc_ialign_c     : natural := 2; -- instruction address
      misaligned
constant exc_ecall_c      : natural := 3; -- environment call
constant exc_ebreak_c     : natural := 4; -- breakpoint
constant exc_salign_c     : natural := 5; -- store address
      misaligned
constant exc_lalign_c     : natural := 6; -- load address
      misaligned
constant exc_saccess_c    : natural := 7; -- store access fault
constant exc_laccess_c    : natural := 8; -- load access fault
constant exc_db_break_c   : natural := 9; -- enter debug mode via
      break instruction
constant exc_db_trig_c    : natural := 10; -- enter debug mode via
      hardware trigger
constant exc_db_step_c    : natural := 11; -- enter debug mode via
      single-stepping
constant exc_width_c      : natural := 12; -- length of this list in
      bits
-- interrupt source list --
constant irq_msi_irq_c    : natural := 0; -- machine software
      interrupt
constant irq_mti_irq_c    : natural := 1; -- machine timer
      interrupt
constant irq_mei_irq_c    : natural := 2; -- machine external
      interrupt
constant irq_firq_0_c     : natural := 3; -- fast interrupt channel
      0
constant irq_firq_1_c     : natural := 4; -- fast interrupt channel
      1

```

Anhang

```
constant irq_firq_2_c : natural := 5; -- fast interrupt channel
2
constant irq_firq_3_c : natural := 6; -- fast interrupt channel
3
constant irq_firq_4_c : natural := 7; -- fast interrupt channel
4
constant irq_firq_5_c : natural := 8; -- fast interrupt channel
5
constant irq_firq_6_c : natural := 9; -- fast interrupt channel
6
constant irq_firq_7_c : natural := 10; -- fast interrupt channel
7
constant irq_firq_8_c : natural := 11; -- fast interrupt channel
8
constant irq_firq_9_c : natural := 12; -- fast interrupt channel
9
constant irq_firq_10_c : natural := 13; -- fast interrupt channel
10
constant irq_firq_11_c : natural := 14; -- fast interrupt channel
11
constant irq_firq_12_c : natural := 15; -- fast interrupt channel
12
constant irq_firq_13_c : natural := 16; -- fast interrupt channel
13
constant irq_firq_14_c : natural := 17; -- fast interrupt channel
14
constant irq_firq_15_c : natural := 18; -- fast interrupt channel
15
constant irq_db_halt_c : natural := 19; -- enter debug mode via
external halt request
constant irq_width_c : natural := 20; -- length of this list in
bits

-- Privilege Modes
-----

--
-----

constant priv_mode_m_c : std_ulogic := '1'; -- machine mode
constant priv_mode_u_c : std_ulogic := '0'; -- user mode

-- Counter Events
-----

--
-----

-- RISC-V-compliant base counter events --
constant cnt_event_cy_c : natural := 0; -- active cycle
constant cnt_event_tm_c : natural := 1; -- time (unused/
reserved)
constant cnt_event_ir_c : natural := 2; -- retired
instruction
-- NEORV32-specific HPM counter events --
constant cnt_event_compr_c : natural := 3; -- executed
compressed instruction
constant cnt_event_wait_dis_c : natural := 4; -- instruction
dispatch wait cycle
constant cnt_event_wait_alu_c : natural := 5; -- multi-cycle ALU
co-processor wait cycle
constant cnt_event_branch_c : natural := 6; -- executed branch
instruction
constant cnt_event_branch_c : natural := 7; -- control flow
transfer
constant cnt_event_load_c : natural := 8; -- load operation
```

```

constant cnt_event_store_c    : natural := 9; -- store operation
constant cnt_event_wait_lsu_c : natural := 10; -- load-store unit
        memory wait cycle
constant cnt_event_trap_c     : natural := 11; -- entered trap

--
        *****

-- Helper Functions
--
        *****

function index_size_f(input : natural) return natural;
function cond_sel_natural_f(cond : boolean; val_t : natural; val_f
        : natural) return natural;
function cond_sel_suv_f(cond : boolean; val_t : std_ulogic_vector;
        val_f : std_ulogic_vector) return std_ulogic_vector;
function cond_sel_string_f(cond : boolean; val_t : string; val_f :
        string) return string;
function max_natural_f(a : natural; b : natural) return natural;
function min_natural_f(a : natural; b : natural) return natural;
function bool_to_ulogic_f(cond : boolean) return std_ulogic;
function or_reduce_f(input : std_ulogic_vector) return std_ulogic;
function and_reduce_f(input : std_ulogic_vector) return std_ulogic
        ;
function xor_reduce_f(input : std_ulogic_vector) return std_ulogic
        ;
function to_hexchar_f(input : std_ulogic_vector(3 downto 0))
        return character;
function bit_rev_f(input : std_ulogic_vector) return
        std_ulogic_vector;
function is_power_of_two_f(input : natural) return boolean;
function replicate_f(input : std_ulogic; num : natural) return
        std_ulogic_vector;
function print_hex_f(data : std_ulogic_vector) return string;
function match_f(input : std_ulogic_vector; pattern :
        std_ulogic_vector) return boolean;

--
        *****

-- NEORV32 Processor Top Entity (component prototype)
--
        *****

component neorv32_top
generic (
        -- General --
        CLOCK_FREQUENCY      : natural                := 0;
        TRACE_PORT_EN        : boolean                 :=
                false;
        DUAL_CORE_EN         : boolean                 :=
                false;
        -- Boot Configuration --
        BOOT_MODE_SELECT     : natural range 0 to 2    := 0;
        BOOT_ADDR_CUSTOM     : std_ulogic_vector(31 downto 0) := x"
                00000000";
        -- On-Chip Debugger (OCD) --
        OCD_EN               : boolean                 :=
                false;
        OCD_NUM_HW_TRIGGERS  : natural range 0 to 16   := 0;
        OCD_AUTHENTICATION   : boolean                 :=
                false;

```

```

OCD_JEDEC_ID          : std_ulogic_vector(10 downto 0) := "
    00000000000";
-- RISC-V CPU Extensions --
RISCV_ISA_C           : boolean                      :=
    false;
RISCV_ISA_E           : boolean                      :=
    false;
RISCV_ISA_M           : boolean                      :=
    false;
RISCV_ISA_U           : boolean                      :=
    false;
RISCV_ISA_Zaamo       : boolean                      :=
    false;
RISCV_ISA_Zalrsc      : boolean                      :=
    false;
RISCV_ISA_Zcb         : boolean                      :=
    false;
RISCV_ISA_Zba         : boolean                      :=
    false;
RISCV_ISA_Zbb         : boolean                      :=
    false;
RISCV_ISA_Zbkb        : boolean                      :=
    false;
RISCV_ISA_Zbkc        : boolean                      :=
    false;
RISCV_ISA_Zbkx        : boolean                      :=
    false;
RISCV_ISA_Zbs         : boolean                      :=
    false;
RISCV_ISA_Zfinx       : boolean                      :=
    false;
RISCV_ISA_Zibi        : boolean                      :=
    false;
RISCV_ISA_Zicntr      : boolean                      :=
    false;
RISCV_ISA_Zicond      : boolean                      :=
    false;
RISCV_ISA_Zihpm       : boolean                      :=
    false;
RISCV_ISA_Zmmul       : boolean                      :=
    false;
RISCV_ISA_Zknd        : boolean                      :=
    false;
RISCV_ISA_Zkne        : boolean                      :=
    false;
RISCV_ISA_Zknh        : boolean                      :=
    false;
RISCV_ISA_Zksed       : boolean                      :=
    false;
RISCV_ISA_Zksh        : boolean                      :=
    false;
RISCV_ISA_Zxcfu       : boolean                      :=
    false;
-- Tuning Options --
CPU_CONSTT_BR_EN      : boolean                      :=
    false;
CPU_FAST_MUL_EN       : boolean                      :=
    false;
CPU_FAST_SHIFT_EN     : boolean                      :=
    false;
CPU_RF_HW_RST_EN      : boolean                      :=
    false;
-- Physical Memory Protection (PMP) --
PMP_NUM_REGIONS       : natural range 0 to 16        := 0;
PMP_MIN_GRANULARITY   : natural                      := 4;

```

```

PMP_TOR_MODE_EN      : boolean           :=
    false;
PMP_NAP_MODE_EN      : boolean           :=
    false;
-- Hardware Performance Monitors (HPM) --
HPM_NUM_CNTS         : natural range 0 to 13      := 0;
HPM_CNT_WIDTH        : natural range 0 to 64      := 40;
-- Internal Instruction memory (IMEM) --
IMEM_EN              : boolean           :=
    false;
IMEM_SIZE             : natural           :=
    16*1024;
IMEM_OUTREG_EN       : boolean           :=
    false;
-- Internal Data memory (DMEM) --
DMEM_EN              : boolean           :=
    false;
DMEM_SIZE             : natural           :=
    8*1024;
DMEM_OUTREG_EN       : boolean           :=
    false;
-- CPU Caches --
ICACHE_EN            : boolean           :=
    false;
ICACHE_NUM_BLOCKS    : natural range 1 to 4096    := 4;
DCACHE_EN            : boolean           :=
    false;
DCACHE_NUM_BLOCKS    : natural range 1 to 4096    := 4;
CACHE_BLOCK_SIZE     : natural range 8 to 1024    := 64;
CACHE_BURSTS_EN      : boolean           := true;
;
-- External bus interface (XBUS) --
XBUS_EN              : boolean           :=
    false;
XBUS_TIMEOUT         : natural           :=
    2048;
XBUS_REGSTAGE_EN     : boolean           :=
    false;
-- Processor peripherals --
IO_DISABLE_SYSINFO   : boolean           :=
    false;
IO_GPIO_NUM          : natural range 0 to 64      := 0;
IO_CLINT_EN          : boolean           :=
    false;
IO_UART0_EN          : boolean           :=
    false;
IO_UART0_RX_FIFO     : natural range 1 to 2**15    := 1;
IO_UART0_TX_FIFO     : natural range 1 to 2**15    := 1;
IO_UART1_EN          : boolean           :=
    false;
IO_UART1_RX_FIFO     : natural range 1 to 2**15    := 1;
IO_UART1_TX_FIFO     : natural range 1 to 2**15    := 1;
IO_SPI_EN            : boolean           :=
    false;
IO_SPI_FIFO          : natural range 1 to 2**15    := 1;
IO_SDI_EN            : boolean           :=
    false;
IO_SDI_FIFO          : natural range 1 to 2**15    := 1;
IO_TWI_EN            : boolean           :=
    false;
IO_TWI_FIFO          : natural range 1 to 2**15    := 1;
IO_TWD_EN            : boolean           :=
    false;
IO_TWD_RX_FIFO       : natural range 1 to 2**15    := 1;
IO_TWD_TX_FIFO       : natural range 1 to 2**15    := 1;
IO_PWM_NUM_CH        : natural range 0 to 16      := 0;

```

```

IO_WDT_EN          : boolean          :=
    false;
IO_TRNG_EN         : boolean          :=
    false;
IO_TRNG_FIFO       : natural range 1 to 2**15 := 1;
IO_CFS_EN          : boolean          :=
    false;
IO_NEOLED_EN       : boolean          :=
    false;
IO_NEOLED_TX_FIFO  : natural range 1 to 2**15 := 1;
IO_GPTMR_EN        : boolean          :=
    false;
IO_ONEWIRE_EN      : boolean          :=
    false;
IO_ONEWIRE_FIFO    : natural range 1 to 2**15 := 1;
IO_DMA_EN          : boolean          :=
    false;
IO_DMA_DSC_FIFO    : natural range 4 to 512   := 4;
IO_SLINK_EN        : boolean          :=
    false;
IO_SLINK_RX_FIFO   : natural range 1 to 2**15 := 1;
IO_SLINK_TX_FIFO   : natural range 1 to 2**15 := 1;
IO_TRACER_EN       : boolean          :=
    false;
IO_TRACER_BUFFER   : natural range 1 to 2**15 := 1;
IO_TRACER_SIMLOG_EN : boolean          :=
    false
);
port (
    -- Global control --
    clk_i          : in  std_ulogic;
    rstn_i         : in  std_ulogic;
    rstn_ocrd_o     : out std_ulogic;
    rstn_wdt_o     : out std_ulogic;
    -- Execution trace (available if TRACE_PORT_EN = true) --
    trace_cpu0_o    : out trace_port_t;
    trace_cpu1_o    : out trace_port_t;
    -- JTAG on-chip debugger interface (available if OCD_EN = true) --
    jtag_tck_i     : in  std_ulogic := 'L';
    jtag_tdi_i     : in  std_ulogic := 'L';
    jtag_tdo_o     : out std_ulogic;
    jtag_tms_i     : in  std_ulogic := 'L';
    -- External bus interface (available if XBUS_EN = true) --
    xbus_adr_o     : out std_ulogic_vector(31 downto 0);
    xbus_dat_o     : out std_ulogic_vector(31 downto 0);
    xbus_cti_o     : out std_ulogic_vector(2 downto 0);
    xbus_tag_o     : out std_ulogic_vector(2 downto 0);
    xbus_we_o      : out std_ulogic;
    xbus_sel_o     : out std_ulogic_vector(3 downto 0);
    xbus_stb_o     : out std_ulogic;
    xbus_cyc_o     : out std_ulogic;
    xbus_dat_i     : in  std_ulogic_vector(31 downto 0) := (others
        => 'L');
    xbus_ack_i     : in  std_ulogic := 'L';
    xbus_err_i     : in  std_ulogic := 'L';
    -- Stream Link Interface (available if IO_SLINK_EN = true) --
    slink_rx_dat_i : in  std_ulogic_vector(31 downto 0) := (others
        => 'L');
    slink_rx_src_i : in  std_ulogic_vector(3 downto 0) := (others
        => 'L');
    slink_rx_val_i : in  std_ulogic := 'L';
    slink_rx_lst_i : in  std_ulogic := 'L';
    slink_rx_rdy_o : out std_ulogic;
    slink_tx_dat_o : out std_ulogic_vector(31 downto 0);
    slink_tx_dst_o : out std_ulogic_vector(3 downto 0);

```

```

slink_tx_val_o : out std_ulogic;
slink_tx_lst_o : out std_ulogic;
slink_tx_rdy_i : in  std_ulogic := 'L';
-- GPIO (available if IO_GPIO_NUM > 0) --
gpio_o        : out std_ulogic_vector(31 downto 0);
gpio_i        : in  std_ulogic_vector(31 downto 0) := (others
=> 'L');
-- primary UART0 (available if IO_UART0_EN = true) --
uart0_txd_o   : out std_ulogic;
uart0_rxd_i   : in  std_ulogic := 'L';
uart0_rtsn_o  : out std_ulogic;
uart0_ctsn_i  : in  std_ulogic := 'L';
-- secondary UART1 (available if IO_UART1_EN = true) --
uart1_txd_o   : out std_ulogic;
uart1_rxd_i   : in  std_ulogic := 'L'; -- UART1 receive data
uart1_rtsn_o  : out std_ulogic;
uart1_ctsn_i  : in  std_ulogic := 'L';
-- SPI (available if IO_SPI_EN = true) --
spi_clk_o     : out std_ulogic;
spi_dat_o     : out std_ulogic;
spi_dat_i     : in  std_ulogic := 'L';
spi_csn_o     : out std_ulogic_vector(7 downto 0); -- SPI CS
-- SDI (available if IO_SDI_EN = true) --
sdi_clk_i     : in  std_ulogic := 'L';
sdi_dat_o     : out std_ulogic;
sdi_dat_i     : in  std_ulogic := 'L';
sdi_csn_i     : in  std_ulogic := 'H';
-- TWI (available if IO_TWI_EN = true) --
twi_sda_i     : in  std_ulogic := 'H';
twi_sda_o     : out std_ulogic;
twi_scl_i     : in  std_ulogic := 'H';
twi_scl_o     : out std_ulogic;
-- TWD (available if IO_TWD_EN = true) --
twd_sda_i     : in  std_ulogic := 'H';
twd_sda_o     : out std_ulogic;
twd_scl_i     : in  std_ulogic := 'H';
twd_scl_o     : out std_ulogic;
-- 1-Wire Interface (available if IO_ONEWIRE_EN = true) --
onewire_i     : in  std_ulogic := 'H';
onewire_o     : out std_ulogic;
-- PWM (available if IO_PWM_NUM_CH > 0) --
pwm_o         : out std_ulogic_vector(15 downto 0); -- pwm
channels
-- Custom Functions Subsystem IO --
cfs_in_i      : in  std_ulogic_vector(255 downto 0) := (
others => 'L');
cfs_out_o     : out std_ulogic_vector(255 downto 0);
-- NeoPixel-compatible smart LED interface (available if
IO_NEOLED_EN = true) --
neoled_o      : out std_ulogic;
-- Machine timer system time (available if IO_CLINT_EN = true)
--
mtime_time_o  : out std_ulogic_vector(63 downto 0);
-- CPU Interrupts --
mtime_irq_i   : in  std_ulogic := 'L';
msw_irq_i    : in  std_ulogic := 'L';
mext_irq_i    : in  std_ulogic := 'L'
);
end component;

end neorv32_package;

package body neorv32_package is
--

```

```

*****

-- Helper Functions
--
*****

-- Minimal required number of bits to represent <input> numbers
-----

--
-----

function index_size_f(input : natural) return natural is
begin
    for i in 0 to 31 loop
        if (2**i >= input) then
            return i;
        end if;
    end loop;
    return 0;
end function index_size_f;

-- Conditional select natural
-----

--
-----

function cond_sel_natural_f(cond : boolean; val_t : natural; val_f
: natural) return natural is
begin
    if cond then
        return val_t;
    else
        return val_f;
    end if;
end function cond_sel_natural_f;

-- Conditional select std_ulogic_vector
-----

--
-----

function cond_sel_suv_f(cond : boolean; val_t : std_ulogic_vector;
val_f : std_ulogic_vector) return std_ulogic_vector is
begin
    if cond then
        return val_t;
    else
        return val_f;
    end if;
end function cond_sel_suv_f;

-- Conditional select string
-----

--
-----

function cond_sel_string_f(cond : boolean; val_t : string; val_f :
string) return string is
begin
    if cond then
        return val_t;
    else
        return val_f;
    end if;
end function cond_sel_string_f;

```

```

-- Select maximal natural value
-----

--
-----

function max_natural_f(a : natural; b : natural) return natural is
begin
    if a < b then
        return b;
    else
        return a;
    end if;
end function max_natural_f;

-- Select minimal natural value
-----

--
-----

function min_natural_f(a : natural; b : natural) return natural is
begin
    if a < b then
        return a;
    else
        return b;
    end if;
end function min_natural_f;

-- Convert boolean to std_ulogic
-----

--
-----

function bool_to_ulogic_f(cond : boolean) return std_ulogic is
begin
    if cond then
        return '1';
    else
        return '0';
    end if;
end function bool_to_ulogic_f;

-- OR all bits
-----

--
-----

function or_reduce_f(input : std_ulogic_vector) return std_ulogic
is
    variable tmp_v : std_ulogic;
begin
    tmp_v := '0';
    for i in input'range loop
        tmp_v := tmp_v or input(i);
    end loop;
    return tmp_v;
end function or_reduce_f;

-- AND all bits
-----

--
-----

```

```

function and_reduce_f(input : std_ulogic_vector) return std_ulogic
is
    variable tmp_v : std_ulogic;
begin
    tmp_v := '1';
    for i in input'range loop
        tmp_v := tmp_v and input(i);
    end loop;
    return tmp_v;
end function and_reduce_f;

```

```
-- XOR all bits
```

```
--
```

```

function xor_reduce_f(input : std_ulogic_vector) return std_ulogic
is
    variable tmp_v : std_ulogic;
begin
    tmp_v := '0';
    for i in input'range loop
        tmp_v := tmp_v xor input(i);
    end loop;
    return tmp_v;
end function xor_reduce_f;

```

```
-- Convert 4-bit std_ulogic_vector to lowercase hex char
```

```
--
```

```

function to_hexchar_f(input : std_ulogic_vector(3 downto 0))
    return character is
    variable hex_v : string(1 to 16);
begin
    hex_v := "0123456789abcdef";
    if ((input(0) /= '0') and (input(0) /= '1')) or
        ((input(1) /= '0') and (input(1) /= '1')) or
        ((input(2) /= '0') and (input(2) /= '1')) or
        ((input(3) /= '0') and (input(3) /= '1')) then
        return '?';
    else
        return hex_v(to_integer(unsigned(input)) + 1);
    end if;
end function to_hexchar_f;

```

```
-- Bit reversal
```

```
--
```

```

function bit_rev_f(input : std_ulogic_vector) return
    std_ulogic_vector is
    variable tmp_v, output_v : std_ulogic_vector(input'length-1
        downto 0);
begin
    tmp_v := input;
    for i in 0 to input'length-1 loop
        output_v((input'length-1)-i) := tmp_v(i);
    end loop;
    return output_v;
end function bit_rev_f;

```

```
-- Test if input number is a power of two
-----

--
-----

function is_power_of_two_f(input : natural) return boolean is
    variable tmp_v : unsigned(31 downto 0);
begin
    tmp_v := to_unsigned(input, 32);
    if (input = 0) then
        return false;
    elsif (input = 1) then
        return true;
    elsif ((tmp_v and (tmp_v - 1)) = 0) then
        return true;
    else
        return false;
    end if;
end function is_power_of_two_f;

-- Replicate bit
-----

--
-----

function replicate_f(input : std_ulogic; num : natural) return
    std_ulogic_vector is
    variable tmp_v : std_ulogic_vector(num-1 downto 0);
begin
    tmp_v := (others => input);
    return tmp_v;
end function replicate_f;

-- Print hex value as string
-----

--
-----

function print_hex_f(data : std_ulogic_vector) return string is
    variable nibb_v : natural := data'length/4;
    variable res_v : string(1 to nibb_v);
begin
    for i in 0 to nibb_v-1 loop
        res_v(i+1) := to_hexchar_f(data(data'high - i*4 downto data'
            high - i*4 - 3));
    end loop;
    return res_v;
end function print_hex_f;

-- Check if vector matches binary pattern (skip elements compared
-- with '-') -----
--
-----

function match_f(input : std_ulogic_vector; pattern :
    std_ulogic_vector) return boolean is
    variable match_v : boolean;
begin
    if (input'length /= pattern'length) then -- no match if
        different sizes
        return false;
    else
        match_v := true;
        for i in input'length-1 downto 0 loop
```

```
        if (pattern(i) = '1') or (pattern(i) = '0') then -- valid
            pattern value, skip everything else
            match_v := match_v and boolean(pattern(i) = input(i));
        end if;
    end loop;
    return match_v;
end if;
end function match_f;

end neorv32_package;
```